

UNIVERSITY
OF OSLO

Ruben Pariente Bassa

Quantum Algorithms for Constraint Optimization and Circuit Synthesis

A NISQ journey

**Thesis submitted for the degree of
Philosophiae Doctor**

Supervisor: Franz G. Fuchs

2026



Abstract

This thesis, presented as a collection of four papers, develops quantum algorithms for combinatorial constraint optimization together with the circuit synthesis tools needed to implement them on near-term hardware. The central theme is that enforcing feasibility by construction, rather than through penalty terms, yields quantum algorithms that are both more expressive and more hardware-efficient, provided the underlying constraint-preserving operators can be compiled into shallow circuits. The first paper introduces a systematic framework for constructing QAOA mixers restricted to arbitrary feasible subspaces via the stabilizer formalism, while the second develops circuit synthesis techniques for sparse operators that provide the compilation primitives for such constraint-preserving dynamics. Building on these synthesis methods, the third paper proposes an efficient qubit encoding of the Max-k-Cut problem that reduces hardware requirements while preserving the problem structure. The final paper unifies these threads through a continuous-time quantum walk on the feasible subspace of the Minimum Vertex Cover problem, combined with a greedy iterative reduction scheme in which local quantum observables encode nonlocal graph information, yielding a penalty free, non variational hybrid algorithm that outperforms classical greedy baselines already at shallow depth.

Sammendrag

Denne avhandlingen, presentert som en samling av fire artikler, utvikler kvantealgoritmer for kombinatorisk optimering med bibetingelser, sammen med de kretssynteseverktøyene som kreves for å implementere dem på nær-fremtidig maskinvare. Det sentrale temaet er at å håndheve gyldighet ved konstruksjon, fremfor gjennom strafftermer, gir kvantealgoritmer som er både mer uttrykksfulle og mer maskinvareeffektive, forutsatt at de underliggende bibetingelsesbevarende operatorene kan kompileres til grunne kretser. Den første artikkelen introduserer et systematisk rammeverk for å konstruere QAOA-miksere begrenset til vilkårlige gyldige underrom via stabilisatorformalismen, mens den andre utvikler kretssynteseteknikker for glisne operatorer som gir kompileringsprimitivene for slik bibetingelsesbevarende dynamikk. Med utgangspunkt i disse syntesemetodene foreslår den tredje artikkelen en effektiv qubit-koding av Max-k-Cut-problemet som reduserer maskinvarekravene samtidig som problemstrukturen bevares. Den siste artikkelen forener disse trådene gjennom en kontinuerlig-tids kvantevandring på det gyldige underrommet til minimum vertex cover problemet, kombinert med et grådig iterativt reduksjonsskjema hvor lokale kvanteobservable koder ikke-lokal grafinformasjon, noe som gir en straff fri, treningsfri hybrid algoritme som overgår klassiske grådige basislinjer ved grunn kretsdybde.

Contents

Introduction	1
1 Combinatorial Optimization and Complexity	3
1.1 Problem Formulation	4
1.2 Computational Complexity	4
1.3 Approximation and Its Limits	5
1.4 Canonical Problems	5
1.4.1 Maximum Cut.	6
1.4.2 Max- k -Cut	6
1.4.3 Minimum Vertex Cover	7
1.4.4 Maximum Independent Set	8
2 Classical Methods for Combinatorial Optimization	11
2.1 Exact Methods.	11
2.1.1 Integer Linear Programming and Branch-and-Bound. .	12
2.1.2 Other Exact Methods.	14
2.2 Approximation Algorithms	15
2.2.1 Combinatorial Approximation	15
2.2.2 LP Rounding and Semidefinite Programming	16
2.2.3 Hardness of Approximation	17
2.3 Heuristic Methods	17
2.3.1 Greedy Algorithms.	18
2.3.2 Local Search and Metaheuristics.	20
3 Quantum Optimization Algorithms.	21
3.1 Preliminaries	22
3.1.1 Elementary Quantum Gates and Universality.	23
3.1.2 Pauli Exponentials.	25
3.1.3 Trotter Decomposition and Randomized Compilation .	27
3.2 Stabilizer Formalism	28
3.3 Circuit Compilation and Synthesis.	29
3.3.1 Beyond Pauli Decomposition: Alternative Synthesis Paradigms	31
3.4 Variable Encodings	32
3.4.1 Binary Variable Encoding	33
3.4.2 Integer Value Encoding.	33

Contents

3.5	Problem Formulation on Quantum Hardware	34
3.5.1	Cost function encoding	35
3.5.2	Constraint Encoding	36
3.5.3	Other approaches	39
3.6	Adiabatic Quantum Computation and Annealing	40
3.7	The Variational Quantum Eigensolver	41
3.7.1	The Ansatz Choice	43
3.7.2	The Classical Optimizer Choice	45
3.7.3	Dynamical Lie Algebras and Barren Plateaus.	46
3.8	The Quantum Approximate Optimization Algorithm.	47
3.8.1	The X Mixer	49
3.8.2	The XY Mixer	50
3.8.3	The Grover Mixer	51
3.8.4	Toward the LX Mixer	53
3.9	Quantum Walks	55
3.9.1	Continuous-Time Quantum Walks	55
3.9.2	Discrete-Time Quantum Walks.	56
3.9.3	Walk-Based Speedups and Heuristics for Optimization.	57
3.10	Iterative and Recursive Quantum Algorithms	57
3.10.1	Recursive QAOA	58
3.10.2	Quantum-Informed Recursive Optimization	58
3.10.3	A General Framework	59
Papers		61
Paper I LX-Mixers for QAOA: Optimal Mixers Restricted to Subspaces and the Stabilizer Formalism		63
Paper II Compact Circuits for Constrained Quantum Evolutions of Sparse Operators		85
Paper III Encodings of the Weighted Max-k-Cut Problem on Qubit Systems		103
Paper IV Constraint Preserving Iterative Quantum Algorithms for Minimum Vertex Cover and Maximum Independent Set		115

Conclusion

149

Contents

List of Figures

1	MaxCut on the house graph. The vertices are partitioned into two sets (circled vs. uncircled); red edges cross the partition and contribute to the cut, while black edges lie within a set and do not. The objective is to choose the partition that maximizes the number of crossing edges.	6
2	A Max-3-Cut solution. Each vertex is assigned one of three colours, and every edge here joins vertices of different colours and is therefore cut. Generalizing MaxCut, the goal is to maximize the number of bichromatic edges over all k -colourings..	7
3	Minimum Vertex Cover. The red vertices form a cover: every edge has at least one red endpoint..	8
4	Maximum Independent Set. Blue vertices form a candidate independent set. Left: a maximum independent set. Centre: not independent (two blue vertices are adjacent). Right: independent but not maximum.	9
5	Integer Linear Programming: optimize $c^T x$ over the integer points (filled red) inside the polytope $\{x : Ax \leq b\}$. The LP relaxation enlarges the feasible set to the entire polytope (yellow), yielding a tractable bound on the integer optimum..	12
6	A branch-and-bound search tree for a problem in three binary variables. Each internal edge fixes one variable; the leaves $S_1, \dots, S_5$ are the subproblems that survive pruning and for which the corresponding LP relaxations are evaluated.	13
7	First step of the maximal-matching 2-approximation for Minimum Vertex Cover. An edge is selected (highlighted) and both of its endpoints are added to the cover; all incident edges are then removed and the procedure repeats on the remaining graph.	15
8	One iteration of the two greedy heuristics for Minimum Vertex Cover on the same graph. Left: Largest-Degree-First (LDF) adds the highest-degree vertex to the cover and deletes its incident edges. Right: Smallest-Degree-First (SDF) selects the lowest-degree vertex and adds all of its neighbours to the cover.	19

9	Quantum circuit for the exponential $e^{-i\alpha P}$ of a Pauli string. The basis-change unitaries U_i rotate each non- Z Pauli factor to Z ; the descending CNOT staircase computes the parity of the support onto the bottom qubit; the rotation $R_Z(2\alpha)$ applies the phase $e^{-i\alpha(-1)^{p(z)}}$; the ascending staircase uncomputes the parity; and the inverse basis-change gates restore the original basis.	26
10	Trotter expansion of the time-evolution operator: $\exp[-iHt]$ is approximated by a product of small unitary steps generated by the local terms $H^{(k)}$	28
11	Representation of a variational algorithm. The grey area is the full Hilbert space; the green region is the set of states reachable by the ansatz $U(\vec{\theta}) \psi_0\rangle$ as the parameters $\vec{\theta}$ are varied. The yellow dot marks the initial state, fixed by the starting parameters, and the red dot marks the optimal solution (the target ground state), which the ansatz can reach only if it lies within the green region.	42
12	The VQE hybrid quantum–classical routine. At each iteration, the quantum hardware prepares the parameterized state $ \psi(\vec{\theta})\rangle = U(\vec{\theta}) 0\rangle$ and measures the energy expectation $E(\vec{\theta}) = \langle\psi(\vec{\theta}) H \psi(\vec{\theta})\rangle$ term by term in the Pauli decomposition $H = \sum_{\alpha} w_{\alpha} P_{\alpha}$. The estimate is passed to a classical optimizer, which proposes an updated parameter vector $\vec{\theta}'$ for the next iteration. The loop repeats until convergence to a local minimum of the energy landscape.	43
13	Hardware-efficient ansatz: a layer of single-qubit rotations $R_x \cdot R_y$ with independent parameters, followed by a linear chain of CNOTs that introduces entanglement.	44
14	Problem-inspired (Hamiltonian variational) ansatz: alternating unitary layers $e^{-i\gamma_k H_1}$ and $e^{-i\beta_k H_0}$ generated by the problem and mixer Hamiltonians. Because the layers inherit the symmetries of their generators, the ansatz confines the state to the relevant symmetry sector at every depth.	44
15	Schematic of the barren-plateau phenomenon. <i>Left</i> : a cost landscape exhibiting a barren plateau, in which $\text{Var}_{\vec{\theta}}[\partial_{\theta_k} E(\vec{\theta})]$ decays exponentially with the qubit count n , so that from a Haar-random initialization the landscape is effectively flat and the gradient gives no usable descent direction. <i>Right</i> : a landscape without a barren plateau, where gradients remain of order one and the optimizer can navigate to a minimum. Equation (3.52) ties the contrast to the dimension of the dynamical Lie algebra of the ansatz generators: $\text{Var}_{\vec{\theta}}[\partial_{\theta_k} E] \propto 1/\dim \mathfrak{g}$	47
16	QAOA ansatz with $p = 2$: Hadamard initialisation, alternating cost ($U(\gamma_k)$) and mixer ($R_x(\beta_k)$) layers, then measurement in the computational basis.	48
17	Circuit for one layer of the X mixer, $e^{-i\beta H_M^X} = \bigotimes_{i=1}^n R_X(2\beta)$. The mixer factorizes into independent single-qubit rotations, so a full layer is implemented in constant depth with no two-qubit gates.	50

18	Mixer graph of the complete-graph XY mixer on a one-hot-encoded k -valued variable. Restricted to the Hamming-weight-one (one-hot) subspace, the induced graph is the Johnson graph $J(k, 1)$, the complete graph K_k on the k feasible labels.	51
19	Circuit for the Grover mixer $U_M^{\text{Grover}}(\beta) = e^{-i\beta \psi_F\rangle\langle\psi_F }$. The feasible-state preparation U_S (with $U_S 0\rangle = \psi_F\rangle$) maps the reflection onto a phase on $ 0\rangle$, applied by the controlled $Z_{-\beta/\pi}$ rotation, after which $U_S^\dagger$ uncomputes the preparation. The cost of constraint preservation is thus shifted entirely into U_S	53

List of Figures

List of Tables

List of Tables

Preface

Here comes your preface, including acknowledgments and thanks.

Introduction

... if you need a separator page

Chapter 1

Combinatorial Optimization and Complexity

Combinatorial optimization sits at the intersection of mathematics, computer science, and operations research, and it underlies an incredible variety of practical problems: routing vehicles through a city, scheduling shifts in a hospital, assigning frequencies to antennas, folding proteins, laying out circuits on a chip, selecting portfolios of assets, and the list can continue indefinitely. What unites these seemingly disparate tasks is a shared mathematical framework, all these problems are said to be combinatorial. Given a finite set of candidate solutions, described implicitly by a collection of constraints, together with an objective function that assigns a real-valued cost to each candidate, the goal is to identify a solution of minimum (or maximum) cost. In principle, because the set of candidates is finite, one could simply enumerate them all and report the best. In practice, these sets, in the cases of our interest, the hard ones, grows exponentially with the size of the problem instance, and brute-force enumeration becomes hopeless after only a few dozen variables.

Optimization problems on these sets whose cardinality is nonetheless astronomical, is the central phenomenon that combinatorial optimization seeks to solve. A significant part of the field is devoted to understanding which problems admit genuinely efficient algorithms, which do not, and which lie in the vast middle ground where no efficient exact method is known but carefully designed heuristics or approximation schemes can still deliver high-quality solutions in reasonable time. The language that makes this distinction precise is computational complexity theory [4, 38], and the problems that motivate the present thesis, Maximum Cut, Max- k -Cut, Minimum Vertex Cover, and Maximum Independent Set, are on the hard side of the complexity landscape. It is precisely this hardness that makes them natural testbeds for quantum optimization: if one hopes to observe a quantum advantage, it is on problems of this kind that it must appear.

1.1 Problem Formulation

A combinatorial optimization problem can be formulated in general as

$$\min_{x \in \mathcal{F}} f(x), \quad \mathcal{F} \subseteq \mathcal{X}, \quad (1.1)$$

where $x = (x_1, \dots, x_n)$ is a vector of discrete decision variables, $\mathcal{X} = \mathcal{X}_1 \times \dots \times \mathcal{X}_n$ is a finite product space with $\mathcal{X}_i \subset \mathbb{Z}$, and $f : \mathcal{X} \rightarrow \mathbb{R}$ is the objective function. The feasible set $\mathcal{F}$ encodes the constraints of the problem and is typically specified through linear inequalities, equalities, or logical relations among the variables. The essential feature is that the search space is discrete and finite, regardless of the specific alphabet from which the variables take values.

A particularly important and widely studied subclass is obtained when the decision variables are binary, i.e., $\mathcal{X} = \{0, 1\}^n$. In this case, the problem takes the form

$$\min_{x \in \mathcal{F}} f(x), \quad \mathcal{F} \subseteq \{0, 1\}^n, \quad (1.2)$$

where each variable $x_i \in \{0, 1\}$ represents a yes/no decision. Many classical combinatorial problems admit natural formulations in this binary setting.

The feasible set $\mathcal{F}$ captures the structural requirements that any valid solution must satisfy. The objective function f encodes the quantity to be optimized, such as a total cost, the number of satisfied constraints, or the length of a tour. Together, the pair $(\mathcal{F}, f)$ fully specifies the problem, and the interplay between constraint structure and objective structure largely determines which algorithmic approaches are effective.

It is useful to distinguish three closely related computational tasks. The *decision* version asks whether there exists a feasible solution with cost at most a given threshold k . The *optimization* version asks for the minimum achievable cost, while the *search* version requires constructing an explicit optimal solution. For the classes of problems considered in this thesis, these formulations are polynomially equivalent, but the distinction clarifies what is meant by “solving” a problem.

1.2 Computational Complexity

The informal notion of a “hard” problem is made rigorous through the framework of computational complexity, developed in the 1960s and 1970s and crystallized in the seminal works of Cook [23], Levin [65], and Karp [53]. The central object of study is the scaling of the resource, typically the time, required to solve a problem as a function of the input size n . A problem is said to belong to the class **P** if it can be solved in time polynomial in n , and to the class **NP** if a candidate solution can be *verified* in polynomial time, even if no efficient algorithm is known for *finding* one. The inclusion $\mathbf{P} \subseteq \mathbf{NP}$ is immediate; whether the inclusion is strict is the celebrated **P** versus **NP** problem, one of the deepest open questions in mathematics [24].

Within **NP**, a distinguished subclass captures the hardest problems: a problem is *NP-complete* if it lies in **NP** and every problem in **NP** can be reduced to it

in polynomial time. An efficient algorithm for any single NP-complete problem would therefore yield efficient algorithms for all of them, collapsing $P = NP$. The first problem shown to be NP-complete was Boolean satisfiability (SAT), via the Cook-Levin theorem [23, 65]. Shortly afterwards, Karp demonstrated that NP-completeness is not a pathology confined to a single abstract problem but a pervasive phenomenon across combinatorial optimization: his paper [53] established the NP-completeness of twenty-one natural problems, including Vertex Cover, Independent Set, Clique, Hamiltonian Cycle, and Maximum Cut. A problem is *NP-hard* if it is at least as hard as every problem in NP, without necessarily belonging to NP itself; optimization versions of NP-complete decision problems typically fall into this category.

This means that under the widely believed assumption that $P \neq NP$, no polynomial-time algorithm exists for exactly solving problems such as Minimum Vertex Cover or Maximum Cut. Worst-case running times for exact methods therefore scale superpolynomially, and often exponentially, in the input size. This theoretical barrier motivates three complementary algorithmic responses, each of which will be developed in subsequent chapters: *exact* methods that remain exponential in the worst case but exploit problem structure to be fast in practice; *heuristics* that abandon optimality guarantees in exchange for speed; and *approximation algorithms* that guarantee solutions within a provable factor of the optimum [85, 89].

1.3 Approximation and Its Limits

For an NP-hard minimization problem, an α -*approximation algorithm* is a polynomial-time algorithm that, on every input, returns a feasible solution whose cost is at most α times the optimum, with $\alpha \geq 1$. The analogous definition for maximization requires the returned cost to be at least $1/\alpha$ times the optimum. The *approximation ratio* α quantifies the price one pays for polynomial-time tractability.

The theory of approximation algorithms reveals a rich and surprisingly delicate landscape. Some NP-hard problems admit a *polynomial-time approximation scheme* (PTAS), meaning they can be approximated within any constant factor $1 + \epsilon$ in time polynomial in the input (though typically exponential in $1/\epsilon$). Others belong to the class APX of problems with some constant approximation factor, but admit no PTAS unless $P = NP$. Still others are inapproximable within any constant factor, or even within any sublinear factor, under standard complexity assumptions [45].

1.4 Canonical Problems

Four combinatorial problems, all defined on undirected graphs, will be used in this thesis as benchmarks and as algorithmic targets. They are closely related both structurally and through polynomial reductions, and the methods developed in the included papers apply to them individually or in combination. We state

each problem formally and collect the important properties that will matter in the quantum algorithms studied in this thesis.

1.4.1 Maximum Cut

Let $G = (V, E)$ be an undirected graph with vertex set V and edge set E . A *cut* is a partition of V into two disjoint subsets S and $\bar{S} = V \setminus S$. The *size* of the cut is the number of edges with one endpoint in S and the other in $\bar{S}$. The *Maximum Cut* problem (MaxCut) asks for the cut of maximum size. Introducing binary variables $x_i \in \{0, 1\}$ indicating the side of the partition to which vertex i is assigned, MaxCut admits the compact formulation

$$\max_{x \in \{0,1\}^n} \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j). \quad (1.3)$$

The MaxCut problem was among the problems shown to be NP-complete by Karp [53]. It is also the most studied benchmark for quantum optimization: the foundational Quantum Approximate Optimization Algorithm paper introduced the approach precisely in the context of MaxCut, and a large fraction of subsequent theoretical and experimental work has used MaxCut as the reference instance on which to measure quantum performance. Its unconstrained nature, every assignment $x \in \{0, 1\}^n$ is feasible, makes it ideally suited for penalty-free variational approaches, and serves as a useful contrast to the constrained problems treated later.

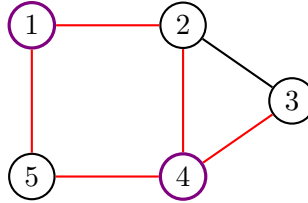


Figure 1: MaxCut on the house graph. The vertices are partitioned into two sets (circled vs. uncircled); red edges cross the partition and contribute to the cut, while black edges lie within a set and do not. The objective is to choose the partition that maximizes the number of crossing edges.

The best known classical approximation for MaxCut is the celebrated Goemans–Williamson algorithm [41], which achieves an approximation ratio of $\alpha_{GW} \approx 0.8786$ via semidefinite programming. This ratio is tight under the Unique Games Conjecture [57].

1.4.2 Max- k -Cut

The *Max- k -Cut* problem generalizes MaxCut to partitions into $k \geq 2$ parts. Given a graph $G = (V, E)$ and an integer k , one seeks a partition of V into k disjoint subsets $V_1, \dots, V_k$ maximizing the number of edges whose endpoints lie in different

subsets. Formally, assigning each vertex i a label $c_i \in \{1, \dots, k\}$, the problem reads

$$\max_{c \in \{1, \dots, k\}^n} \sum_{(i,j) \in E} \mathbb{I}[c_i \neq c_j]. \quad (1.4)$$

The case $k = 2$ recovers MaxCut. For every fixed $k \geq 2$, the problem is **NP**-hard, and it becomes harder to approximate as k increases: the best known polynomial-time approximation ratio degrades with k [37], and strong inapproximability results hold under the Unique Games Conjecture [55, 57].

From a quantum perspective, Max- k -Cut introduces a qualitatively new feature absent from MaxCut: the decision variables are no longer binary but k -valued, and efficient encoding of such higher-dimensional variables into qubits is a nontrivial design problem in itself. One of the papers included in this thesis addresses precisely this question, proposing an efficient qubit representation of Max- k -Cut that reduces the hardware requirements relative to naive one-hot encodings while preserving the combinatorial structure required for optimization.

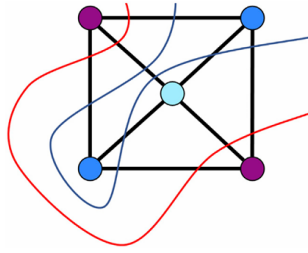


Figure 2: A Max-3-Cut solution. Each vertex is assigned one of three colours, and every edge here joins vertices of different colours and is therefore cut. Generalizing MaxCut, the goal is to maximize the number of bichromatic edges over all k -colourings.

1.4.3 Minimum Vertex Cover

A *vertex cover* of a graph $G = (V, E)$ is a subset $C \subseteq V$ such that every edge has at least one endpoint in C . The *Minimum Vertex Cover* problem (MVC) asks for a vertex cover of smallest cardinality. With $x_i \in \{0, 1\}$ indicating whether vertex i is included in the cover, the problem reads

$$\min_{x \in \{0,1\}^n} \sum_{i \in V} x_i \quad \text{subject to} \quad x_i + x_j \geq 1 \quad \forall (i,j) \in E. \quad (1.5)$$

MVC is one of Karp's original twenty-one **NP**-complete problems [53]. It admits a classical factor-2 approximation via maximal matching [38, 85], and this ratio is tight under the Unique Games Conjecture [56]. The problem's explicit constraint structure, one inequality per edge, makes it an ideal testbed for constraint-preserving quantum methods, and it is the central combinatorial target of the final paper of this thesis.

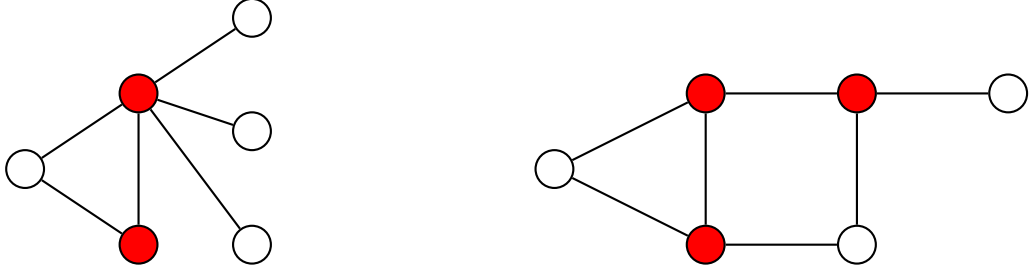


Figure 3: Minimum Vertex Cover. The red vertices form a cover: every edge has at least one red endpoint.

1.4.4 Maximum Independent Set

An *independent set* of $G = (V, E)$ is a subset $I \subseteq V$ such that no two vertices in I are connected by an edge. The *Maximum Independent Set* problem (MIS) asks for an independent set of maximum cardinality. With $y_i \in \{0, 1\}$ indicating membership, the problem reads

$$\max_{y \in \{0,1\}^n} \sum_{i \in V} y_i \quad \text{subject to} \quad y_i + y_j \leq 1 \quad \forall (i, j) \in E. \quad (1.6)$$

MIS is likewise among Karp's original NP-complete problems [53], and is far less forgiving to approximation than MVC: under standard assumptions, MIS cannot be approximated within a factor of $n^{1-\varepsilon}$ for any $\varepsilon > 0$.

MVC and MIS are related by an elementary but extremely useful duality. For any graph G , a subset $C \subseteq V$ is a vertex cover if and only if its complement $I = V \setminus C$ is an independent set. Consequently

$$|\text{MVC}(G)| + |\text{MIS}(G)| = |V|, \quad (1.7)$$

and algorithms for one problem translate directly into algorithms for the other. This duality is exploited throughout the thesis: constraint-preserving quantum dynamics on the MVC feasible subspace map, via a global bit-flip, onto dynamics on the MIS feasible subspace, so a single algorithmic framework covers both problems at once.

Chapter 2

Classical Methods for Combinatorial Optimization

The NP-hardness results discussed in the previous chapter exclude, under standard complexity assumptions, the existence of polynomial-time algorithms that solve problems such as Minimum Vertex Cover or Maximum Cut exactly for all graphs inputs. They do not, however, close the book on classical algorithm design. The classical algorithms attack such combinatorial problems with three different approaches, each accepting a different tradeoff between optimality, runtime, and generality: *exact methods* that solve the problem to proven optimality at potentially exponential worst-case cost but make use of problem structure to be fast in practice; *approximation algorithms* that sacrifice optimality by guaranteeing a provable ratio to the optimum in polynomial time; and *heuristic methods* that abandon performance guarantees entirely in exchange for speed, simplicity, and empirical effectiveness. These three paradigms are neither disjoint nor exhaustive but they provide a useful taxonomy of the classical landscape against which the quantum methods developed in later chapters will be compared.

This chapter reviews each of the three approaches with enough technical detail to support the comparisons that will appear throughout the thesis. The emphasis is on the conceptual mechanisms underlying each method and their applications to the canonical problems introduced in Chapter 1. Greedy heuristics for Minimum Vertex Cover and Maximum Independent Set receive particular attention, as they serve as the direct classical baselines for the iterative quantum algorithm developed in the last paper of this thesis.

2.1 Exact Methods

Exact methods are classical algorithms that return a provably optimal solution whenever they terminate. Of course, for NP-hard problems, this implies that their worst-case running time must be superpolynomial under $P \neq NP$. The art behind exact algorithm design lies in structuring the search so that the exponential blow-up is as far away as possible, or even avoided on specific "good" instances.

2.1.1 Integer Linear Programming and Branch-and-Bound

Many combinatorial optimization problems admit an exact formulation as an *integer linear program* (ILP),

$$\min c^\top x \quad \text{subject to} \quad Ax \leq b, \quad x \in \mathbb{Z}^n, \quad (2.1)$$

where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, and $c \in \mathbb{R}^n$. When the variables are further restricted to $x \in \{0, 1\}^n$, the formulation captures a large class of discrete selection problems on graphs, including Minimum Vertex Cover and Maximum Cut.

The feasible region of (2.1) is a discrete subset of the polyhedron $\{x \in \mathbb{R}^n : Ax \leq b\}$. This combinatorial structure is the source of both the expressive power of ILP and its computational hardness. A standard approach is therefore to consider the *linear programming relaxation* obtained by dropping integrality:

$$\min c^\top x \quad \text{subject to} \quad Ax \leq b, \quad x \in \mathbb{R}^n. \quad (2.2)$$

The feasible region of (2.2) is a convex polytope, and the problem can be solved in polynomial time using interior-point [52] or simplex-based methods [28]. Denoting by z_{LP} and z_{ILP} the optimal values of (2.2) and (2.1), respectively, one has for minimization problems

$$z_{\text{LP}} \leq z_{\text{ILP}},$$

so that the relaxation provides a lower bound on the true optimum.

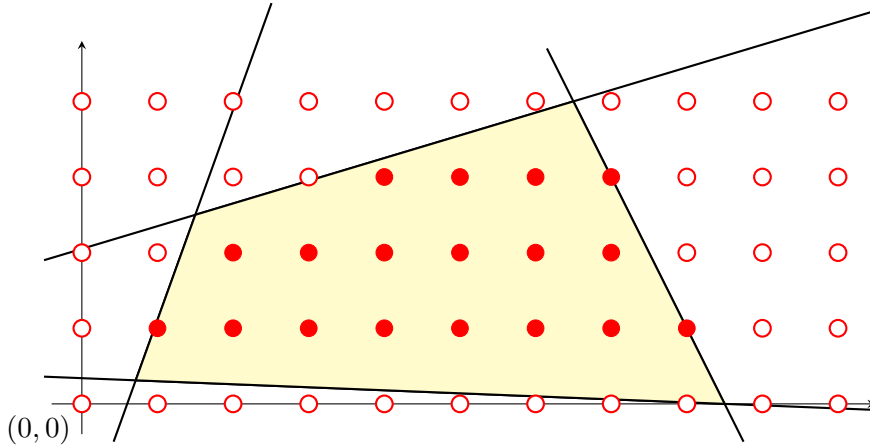


Figure 5: Integer Linear Programming: optimize $c^\top x$ over the integer points (filled red) inside the polytope $\{x : Ax \leq b\}$. The LP relaxation enlarges the feasible set to the entire polytope (yellow), yielding a tractable bound on the integer optimum.

The relaxation alone does not solve the integer problem: its optimum is generally fractional and infeasible for (2.1). To recover an integral solution while preserving optimality guarantees, ILP is solved within a *branch-and-bound* framework [61, 64], a general strategy for exact combinatorial optimization that applies whenever a problem can be cast as

$$z^* = \min_{x \in \mathcal{X}} f(x), \quad (2.3)$$

with $\mathcal{X}$ a discrete feasible set. For ILP one has $f(x) = c^\top x$ and $\mathcal{X} = \{x \in \mathbb{Z}^n : Ax \leq b\}$, and the LP relaxation supplies the bounds that drive the search.

The method organizes the search as a tree whose nodes correspond to restricted subproblems. Each node v is associated with a feasible region $\mathcal{X}_v \subseteq \mathcal{X}$ obtained by imposing additional constraints such as variable fixings. The root corresponds to the full problem, $\mathcal{X}_{\text{root}} = \mathcal{X}$, and the children of a node v are obtained by partitioning its feasible region into disjoint subsets,

$$\mathcal{X}_v = \mathcal{X}_{v_1} \dot{\cup} \mathcal{X}_{v_2}.$$

At each node, the algorithm computes a *lower bound* $\ell(v)$ satisfying

$$\ell(v) \leq \min_{x \in \mathcal{X}_v} f(x),$$

typically by solving the LP relaxation of the restricted subproblem. In parallel, the algorithm maintains the best feasible solution found so far, the *incumbent*, whose objective value

$$u = \min\{f(x) : x \in \mathcal{X}_{\text{feas}}\}$$

serves as a global upper bound on z^* . Three situations may arise at a node after its bound has been computed:

1. *Pruning by bound:* if $\ell(v) \geq u$, no solution in $\mathcal{X}_v$ can improve upon the incumbent, and the node is discarded.
2. *Pruning by feasibility:* if the relaxation yields an integral solution, the incumbent is updated and the node is closed.
3. *Branching:* otherwise, a variable x_i with fractional value $\hat{x}_i$ in the LP solution is selected, and two children are created by imposing

$$x_i \leq \lfloor \hat{x}_i \rfloor \quad \text{and} \quad x_i \geq \lceil \hat{x}_i \rceil.$$

The algorithm terminates when no open nodes remain, at which point the incumbent is provably optimal: the regions $\{\mathcal{X}_v\}$ at the leaves partition $\mathcal{X}$, and no pruned node can contain a solution better than the incumbent.

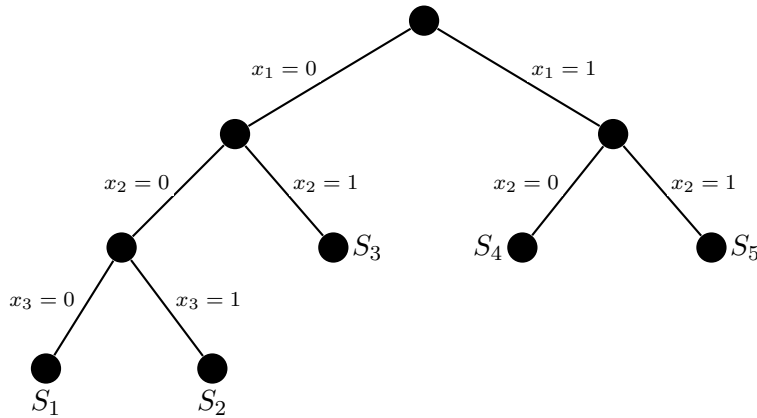


Figure 6: A branch-and-bound search tree for a problem in three binary variables. Each internal edge fixes one variable; the leaves $S_1, \dots, S_5$ are the subproblems that survive pruning and for which the corresponding LP relaxations are evaluated.

The efficiency of branch-and-bound depends on two design choices that together control the size of the explored tree. The first is the quality of the bounding procedure: tight relaxations push $\ell(v)$ toward u and enable aggressive pruning. This motivates the addition of valid inequalities (cutting planes) that tighten the LP relaxation without excluding any integer point, leading to the *branch-and-cut* paradigm [25, 74] used in modern solvers. The second is the branching rule, which determines how the feasible set is partitioned at each node; effective rules select variables whose fixing produces children with markedly different bounds, so that one branch can be discarded quickly while the other focuses the search. Additional ingredients used in practice include node selection strategies (best-first, depth-first, or hybrid traversal orders) and the early generation of high-quality incumbents via primal heuristics.

In the worst case, branch-and-bound still explores all 2^n assignments, recovering exhaustive enumeration; this is unavoidable for NP-hard problems under $P \neq NP$. What makes the framework practically successful is that, on structured instances, the combination of strong bounds and informed branching prunes the tree to a manageable size, often by many orders of magnitude relative to the trivial enumeration bound. The tree-search structure also makes branch-and-bound a natural target for quantum acceleration: quantum walk techniques can explore such trees in time roughly proportional to the square root of their size [3, 71], yielding near-quadratic improvements in the size of the explored search space without altering the underlying branching and bounding logic.

2.1.2 Other Exact Methods

Beyond integer programming and branch-and-bound, several further algorithmic frameworks are used to solve combinatorial optimization problems exactly, each exploiting a different aspect of problem structure. *Dynamic programming* [8] decomposes a problem into overlapping subproblems and tabulates their solutions, achieving polynomial-time exact algorithms on graphs of restricted structure such as trees or graphs of bounded treewidth [10, 26], where many hard problems become tractable. *Parameterized algorithms* [30] formalize the observation that hardness is sometimes controlled by a single structural parameter rather than by raw input size: a problem is *fixed-parameter tractable* with respect to a parameter k if it admits an algorithm running in time $O(f(k) \cdot \text{poly}(n))$ for some computable function f . Minimum Vertex Cover is fixed-parameter tractable in the solution size k , with running time $O(1.2738^k + kn)$ [18], allowing exact solution of large graphs whenever the optimum is small. *Boolean satisfiability solvers* (SAT and SMT) [9, 68] attack combinatorial problems by reduction to propositional logic, with modern conflict-driven clause-learning algorithms routinely solving industrial instances with millions of variables.

These methods differ in how they exploit structure but share the property that their worst-case complexity remains exponential under $P \neq NP$. Their practical effectiveness derives from problem-specific features, low treewidth, small parameters, sparse constraint graphs, or structured clause sets, that may or may not be present in the instances of interest.

2.2 Approximation Algorithms

Approximation algorithms trade optimality for polynomial runtime while retaining a provable guarantee on the quality of the solution returned. They are the branch of classical algorithm design most directly aligned with the quantum optimization programme, since both ultimately ask how close one can efficiently come to the optimum of an NP-hard problem.

2.2.1 Combinatorial Approximation

The simplest approximation algorithms exploit combinatorial structure directly, without recourse to linear programming. The paradigmatic example is the factor-2 approximation for Minimum Vertex Cover via maximal matching [38, 85]: pick an arbitrary edge, add both endpoints to the cover, remove all incident edges, and repeat. The output is a vertex cover of size at most twice the optimum, because the edges selected form a matching M , any vertex cover must contain at least one endpoint of each edge of M , and the algorithm takes both. The analysis is elementary, the algorithm runs in linear time, and the guarantee is, under the Unique Games Conjecture, asymptotically optimal [56].

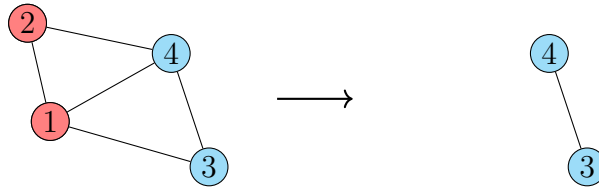


Figure 7: First step of the maximal-matching 2-approximation for Minimum Vertex Cover. An edge is selected (highlighted) and both of its endpoints are added to the cover; all incident edges are then removed and the procedure repeats on the remaining graph.

The procedure is summarized in Algorithm 1.

Algorithm 1 Maximal-matching approximation for Minimum Vertex Cover

Require: Graph $G = (V, E)$

Ensure: Vertex cover $C \subseteq V$ with $|C| \leq 2 \cdot |\text{OPT}|$

- 1: $C \leftarrow \emptyset$
 - 2: $E' \leftarrow E$
 - 3: **while** $E' \neq \emptyset$ **do**
 - 4: Pick any edge $(u, v) \in E'$
 - 5: $C \leftarrow C \cup \{u, v\}$
 - 6: $E' \leftarrow E' \setminus \{(x, y) \in E' : x \in \{u, v\} \text{ or } y \in \{u, v\}\}$
 - 7: **end while**
 - 8: **return** C
-

Analogous combinatorial arguments underpin classical approximation algorithms for MaxCut and Max- k -Cut. A random assignment that places each

vertex independently on each side of the partition with probability $1/2$ achieves a cut of expected size at least $|E|/2$, yielding a $1/2$ -approximation for MaxCut [77]. The analogous uniform random assignment on k labels achieves expected cut size $(1-1/k)|E|$, giving a $(1-1/k)$ -approximation for Max- k -Cut. These bounds, while elementary, are remarkable: they assert that even trivial randomized procedures come within a constant factor of the optimum.

2.2.2 LP Rounding and Semidefinite Programming

The integer program (2.1) is hard precisely because of its integrality constraint; if we drop it, we are left with the *linear programming relaxation*,

$$\min c^\top x \quad \text{subject to} \quad Ax \leq b, \quad x \in \mathbb{R}^n, \quad (2.4)$$

which is solvable in polynomial time, by the simplex method in practice or by interior-point methods in theory. Because every integer feasible point is also fractionally feasible, the optimum of (2.4) is a lower bound on the optimum of (2.1). The ratio between the two, the *integrality gap*, quantifies how much is lost in passing to the continuous problem, and it is the central quantity governing what LP-based approximation can hope to achieve. For Minimum Vertex Cover this gap is exactly two in the worst case [73], and as we shall see, this is no coincidence: it is precisely the barrier that the factor-2 approximation of Section 2.2.2 runs into.

The relaxation, on its own, returns a vector rather than a combinatorial object, and so cannot serve directly as a solution. The key idea of *LP rounding* is to use this vector as a guide: solve the relaxation, then convert its solution into a feasible integer one by a rule simple enough to analyse, yet faithful enough to the optimum that little is lost in the conversion [48]. For Minimum Vertex Cover the recipe is especially clean. The LP relaxation of (1.5) returns a vector $x^* \in [0, 1]^{|V|}$ satisfying $x_i^* + x_j^* \geq 1$ for every edge (i, j) ; at least one endpoint of each edge therefore has $x^* \geq 1/2$. Rounding by selecting every such vertex yields a feasible cover whose size is at most twice the LP optimum, and hence at most twice the integer optimum. We recover the same factor-2 guarantee as before, but through an entirely different route, and one that generalises: solve a continuous relaxation, round its solution, and bound the loss incurred by the rounding step. This *relax-and-round* template underlies a large fraction of modern approximation algorithms.

A substantially more powerful relaxation replaces the linear program by a *semidefinite program* (SDP), in which scalar variables are replaced by unit vectors in a high-dimensional space and constraints become inner-product inequalities. Goemans and Williamson [41] showed that an SDP relaxation of MaxCut, followed by a random-hyperplane rounding scheme, yields a $(0.878\dots)$ -approximation, where the constant is the value $\min_{0 \leq \theta \leq \pi} \frac{2\theta}{\pi(1-\cos \theta)}$. This was the first polynomial-time algorithm for MaxCut with an approximation ratio bounded away from the trivial $1/2$, and under the Unique Games Conjecture it is optimal.

The construction proceeds in three steps. First, one relaxes the MaxCut formulation $\max \frac{1}{2} \sum_{(i,j) \in E} (1 - y_i y_j)$ with $y_i \in \{-1, +1\}$ by replacing each binary label y_i with a unit vector $v_i \in \mathbb{R}^n$ on the unit sphere, yielding the semidefinite

program

$$\max \frac{1}{2} \sum_{(i,j) \in E} (1 - v_i \cdot v_j) \quad \text{subject to} \quad \|v_i\| = 1 \quad \forall i \in V. \quad (2.5)$$

This relaxation is solvable in polynomial time to any desired precision by interior-point methods. Second, given an optimal assignment of vectors $\{v_i^*\}$, one rounds them back to binary labels by sampling a random hyperplane through the origin, equivalently, a uniformly random unit vector $r \in \mathbb{R}^n$, and assigning $y_i = \text{sign}(v_i^* \cdot r)$. Vertices on the same side of the hyperplane receive the same label; vertices on opposite sides are separated. Third, one analyses the expected size of the resulting cut. The probability that an edge (i, j) is cut by a random hyperplane equals θ_{ij}/π , where θ_{ij} is the angle between v_i^* and v_j^* . Comparing this to the SDP contribution $(1 - \cos \theta_{ij})/2$ of the same edge yields the expected approximation ratio

$$\alpha_{\text{GW}} = \min_{0 \leq \theta \leq \pi} \frac{2\theta}{\pi(1 - \cos \theta)} \approx 0.878,$$

attained at $\theta \approx 2.331$ radians. The bound is tight in the sense that there exist graph instances on which the SDP-plus-rounding procedure achieves no better.

The Goemans-Williamson scheme occupies a particular place in the conceptual landscape of this thesis. Its success is an early indication that embedding combinatorial problems in a continuous, high-dimensional geometric space can expose structure invisible to purely combinatorial methods. Quantum optimization takes this idea substantially further, by embedding the feasible set not in $\mathbb{R}^d$ but in a complex Hilbert space whose dimension is exponential in the input size. Whether this exponentially richer embedding translates into correspondingly richer algorithms is one of the central open questions of the field.

2.2.3 Hardness of Approximation

The approximation ratios achieved by the algorithms above are, in many cases, provably optimal. Under $\text{P} \neq \text{NP}$, MaxCut admits no polynomial-time approximation better than $16/17$ [46, 83]; under the Unique Games Conjecture, the Goemans-Williamson bound $\alpha_{\text{GW}} \approx 0.878$ is optimal, and the factor-2 bound for Minimum Vertex Cover is likewise tight. Maximum Independent Set is far harder still [56]: it admits no polynomial-time approximation within $n^{1-\varepsilon}$ for any $\varepsilon > 0$ under standard assumptions [45]. These inapproximability results delimit the theoretically achievable performance for any polynomial-time classical algorithm, and they establish the performance floor that quantum methods would need to breach to demonstrate a rigorous advantage on approximation.

2.3 Heuristic Methods

Heuristic methods abandon performance guarantees but in return they are typically simple to implement, fast in practice, and flexible enough to incorporate domain-specific insights. On realistic instances they often produce solutions that are competitive with, or even superior to, the output of provably bounded

algorithms. They play two roles in this thesis: as the direct classical baselines against which the quantum algorithms of the included papers are benchmarked, and as conceptual precursors to the iterative and greedy quantum methods developed in the last paper.

2.3.1 Greedy Algorithms

A *greedy algorithm* constructs a solution incrementally, at each step making a locally optimal choice according to some selection rule, without revisiting earlier decisions. For Minimum Vertex Cover the two canonical greedy rules are *largest-degree-first* (LDF), which repeatedly selects the vertex of highest degree in the current graph and adds it to the cover before deleting it and all incident edges, and *smallest-degree-first* (SDF), which selects the vertex of smallest degree, adds all of its neighbours to the cover, and deletes the closed neighbourhood. The two procedures are summarized in Algorithms 2 and 3.

Algorithm 2 Largest-Degree-First (LDF) for Minimum Vertex Cover

Require: Graph $G = (V, E)$
Ensure: Vertex cover $C \subseteq V$

- 1: $C \leftarrow \emptyset$
- 2: **while** $E \neq \emptyset$ **do**
- 3: $v^* \leftarrow \arg \max_{v \in V} \deg_G(v)$
- 4: $C \leftarrow C \cup \{v^*\}$
- 5: $V \leftarrow V \setminus \{v^*\}$
- 6: $E \leftarrow E \setminus \{(v^*, u) : u \in V\}$
- 7: **end while**
- 8: **return** C

Algorithm 3 Smallest-Degree-First (SDF) for Minimum Vertex Cover

Require: Graph $G = (V, E)$
Ensure: Vertex cover $C \subseteq V$

- 1: $C \leftarrow \emptyset$
- 2: **while** $E \neq \emptyset$ **do**
- 3: $v^* \leftarrow \arg \min_{v \in V} \deg_G(v)$
- 4: $C \leftarrow C \cup N(v^*)$
- 5: $V \leftarrow V \setminus (\{v^*\} \cup N(v^*))$
- 6: $E \leftarrow E \setminus \{(x, y) \in E : x \in N(v^*) \text{ or } y \in N(v^*)\}$
- 7: **end while**
- 8: **return** C

The two rules are motivated by complementary intuitions. LDF prioritizes vertices that cover many edges at once, minimizing the number of iterations; SDF observes that a low-degree vertex has few neighbours, each of which is forced into the cover to satisfy the edge constraints, so including all of them simultaneously is efficient.

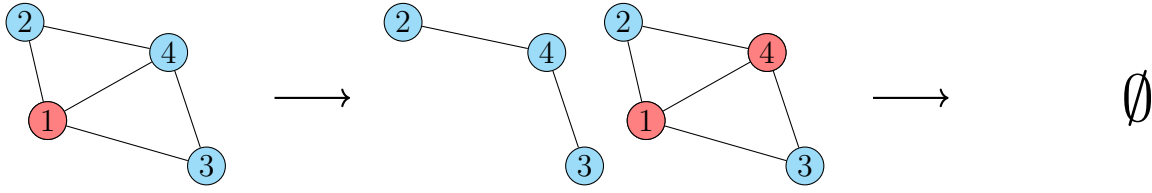


Figure 8: One iteration of the two greedy heuristics for Minimum Vertex Cover on the same graph. Left: Largest-Degree-First (LDF) adds the highest-degree vertex to the cover and deletes its incident edges. Right: Smallest-Degree-First (SDF) selects the lowest-degree vertex and adds all of its neighbours to the cover.

Neither rule offers a constant-factor approximation guarantee [50] yet both are effective in practice and both are standard baselines in the literature. For Maximum Independent Set, the analogous rules are inherited through the MVC–MIS duality relation (1.7): an LDF rule for MIS repeatedly removes the highest-degree vertex (the vertex least likely to belong to a large independent set), while an SDF rule repeatedly adds the lowest-degree vertex and removes its neighbourhood.

Similarly, we can construct several greedy algorithms for MaxCut.. For example, rather than building a single set incrementally, one maintains a partition of the assigned vertices into two sides, S and $\bar{S}$, and at each step assigns one new vertex to whichever side maximises the number of edges it cuts. Concretely, for a candidate vertex v with already-assigned neighbours, let $d_S(v) = |N(v) \cap S|$ and $d_{\bar{S}}(v) = |N(v) \cap \bar{S}|$ denote its degrees into the two sides; placing v in $\bar{S}$ contributes $d_S(v)$ cut edges, and placing it in S contributes $d_{\bar{S}}(v)$, so the locally optimal choice is the larger of the two. The two standard variants differ in the order in which vertices are considered:

- *Sequential greedy*: processes the vertices in a fixed (typically arbitrary or degree-sorted) order and commits each one immediately;
- *Adaptive greedy*: Selects, at every step, the still-unassigned vertex with the largest assigned-neighbourhood imbalance $|d_S(v) - d_{\bar{S}}(v)|$, on the grounds that this vertex has the most to gain from a careful placement and the least to lose from being decided early.

The sequential variant is summarized in Algorithm 4.

Algorithm 4 Sequential Greedy for MaxCut

Require: Graph $G = (V, E)$, ordering $v_1, v_2, \dots, v_{|V|}$ of V **Ensure:** Partition $(S, \bar{S})$ of V

```

1:  $S \leftarrow \emptyset, \bar{S} \leftarrow \emptyset$ 
2: for  $k = 1$  to  $|V|$  do
3:    $d_S \leftarrow |N(v_k) \cap S|$ 
4:    $d_{\bar{S}} \leftarrow |N(v_k) \cap \bar{S}|$ 
5:   if  $d_S \geq d_{\bar{S}}$  then
6:      $\bar{S} \leftarrow \bar{S} \cup \{v_k\}$ 
7:   else
8:      $S \leftarrow S \cup \{v_k\}$ 
9:   end if
10: end for
11: return  $(S, \bar{S})$ 

```

Unlike its vertex-cover counterparts, the sequential greedy rule for MaxCut does admit a worst-case guarantee, and a rather clean one. At the moment vertex v_k is placed, it cuts at least half of the edges connecting it to its already-assigned neighbours, since the algorithm chooses the better of the two sides. Summing over vertices, the final cut contains at least $|E|/2$ edges, which is at least half of the optimum, recovering the same $1/2$ -approximation as the trivial random assignment. The point of the greedy procedure is therefore not an improved worst-case ratio but its typical-case behaviour: on most instances the cut it returns is substantially larger than $|E|/2$ and competitive with the output of the considerably more expensive Goemans–Williamson algorithm of Section 2.2.2.

These greedy methods, for all their simplicity, constitute the baseline against which the iterative quantum algorithm of the final paper is compared. A central observation of that paper, developed in detail in later chapters, is that greedy selection rules informed by non-local quantum measurements can outperform their purely classical counterparts precisely because the quantum observables encode information beyond the immediate neighbourhood of each vertex.

2.3.2 Local Search and Metaheuristics

A richer class of heuristics constructs an initial solution and then iteratively improves it by local modifications. *Local search* [1] defines a neighbourhood structure on the feasible set and repeatedly moves from the current solution to a better neighbour until a local optimum is reached. More elaborate *metaheuristics* introduce mechanisms for escaping local optima: *simulated annealing* [58] accepts worsening moves with a temperature-dependent probability that gradually decreases; *tabu search* [39, 40] forbids recently visited configurations to force exploration; *evolutionary and genetic algorithms* [42, 49] maintain populations of candidate solutions and combine them through crossover and mutation.

Chapter 3

Quantum Optimization Algorithms

The central question of quantum combinatorial optimization is whether the computational resources of a quantum processor, like superposition or interference, can be used to solve combinatorial optimization problems like the ones introduced in Chapter 1 more efficiently than the classical methods reviewed in Chapter 2. Answering this question in full generality is presently out of reach: rigorous separations between classical and quantum complexity classes remain an open research area, and near-term quantum hardware is still too noisy and small to study it experimentally on problems of industrial scale. What the field has produced instead is a rich variety of *quantum algorithmic paradigms*, like adiabatic evolution, variational optimization, quantum walk-based approaches, iterative, and non-variational algorithms, each encoding a different intuition about how quantum dynamics might be harnessed for optimization, and each making different tradeoffs between theoretical guarantees, hardware requirements, and practical performance.

The intuitions behind these algorithms come from several distinct areas. Some are inspired by Grover’s quantum search algorithm [44], which establishes a quadratic speedup for unstructured search. Others are inspired by analogue quantum simulation: condensed-matter physics has long studied the dynamics of interacting spin systems, and combinatorial cost functions can be expressed as Ising Hamiltonians, which inherit a wealth of physical intuition about phase transitions, ground-state preparation, and adiabatic evolution. Still others derive their power from *interference*, the distinctively quantum phenomenon by which probability amplitudes from distinct computational paths combine constructively or destructively like in quantum walk based algorithms. Each algorithmic family selects a different mix of these ingredients, and each makes a different bet on which of them provides the decisive advantage over classical computation.

This chapter reviews these approaches in more technical depth than the previous chapters, because the papers collected in this thesis contribute directly to several of the topics developed below: constraint-preserving mixers and their stabilizer-theoretic structure, synthesis of the sparse operators, efficient problem encodings, and iterative quantum walk-based algorithms operating on restricted feasible subspaces. The chapter is organized in two parts. The first part establishes the technical groundwork: a preliminary section on quantum gates and circuit basics, followed by sections on the stabilizer formalism and on circuit synthesis.

The second part is devoted to quantum optimization algorithms proper, organized around two recurring concerns. The first is the *encoding* of the problem on quantum hardware: how the classical objective and its constraints are translated into quantum operators, and in particular how constraints are enforced, either through Hamiltonian penalty terms that make infeasible configurations costly, or through structural restrictions that render them unreachable. The second is the *dynamics*: how the quantum state is evolved to concentrate amplitude on low-cost feasible configurations, how the corresponding circuits can be implemented efficiently, and whether this evolution is continuous (adiabatic, walk-based), parameterized and classically optimized (variational), or structured by iterative classical reduction (recursive, greedy). The chapter closes with a synthesis that locates each of the four papers of this thesis along both axes.

3.1 Preliminaries

Before turning to quantum optimization algorithms proper, we fix notation and recall the minimal quantum formalism used throughout the remainder of this thesis.

The state of an isolated quantum system is described by a unit vector in a complex Hilbert space. For a system of n qubits, the elementary quantum two-level systems on which all the algorithms discussed here are formulated, the Hilbert space is

$$\mathcal{H} = (\mathbb{C}^2)^{\otimes n}, \quad (3.1)$$

of complex dimension 2^n . The *computational basis* $\{|z\rangle : z \in \{0, 1\}^n\}$ is obtained as the tensor product of the single-qubit basis $\{|0\rangle, |1\rangle\}$, and provides a one-to-one correspondence between classical bitstrings and basis vectors of $\mathcal{H}$. A generic pure state is a complex linear combination

$$|\psi\rangle = \sum_{z \in \{0,1\}^n} \alpha_z |z\rangle, \quad \sum_z |\alpha_z|^2 = 1, \quad (3.2)$$

and the squared amplitudes $|\alpha_z|^2$ give the probability of observing the bitstring z upon a computational-basis measurement. Observables are represented by Hermitian operators on $\mathcal{H}$. The most fundamental are the single-qubit Pauli operators

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad (3.3)$$

each with eigenvalues ± 1 . Together with the identity I , they form a basis of the 2×2 Hermitian matrices, and their tensor products

$$P = \sigma_1 \otimes \sigma_2 \otimes \cdots \otimes \sigma_n, \quad \sigma_i \in \{I, X, Y, Z\}, \quad (3.4)$$

called *Pauli strings*, form an orthonormal basis of the 4^n -dimensional space of Hermitian operators on $\mathcal{H}$ with respect to the Hilbert–Schmidt inner product. The number of non-identity factors in a Pauli string is its *weight*, and we adopt the standard convention of writing X_i, Y_i, Z_i for the single-qubit Pauli acting on

qubit i and identity elsewhere. Any Hermitian operator H on n qubits can be expanded uniquely as a real linear combination of Pauli strings,

$$H = \sum_P c_P P, \quad c_P \in \mathbb{R}, \quad (3.5)$$

a decomposition that will reappear later for Hamiltonian encodings of optimization problems discussed later in the chapter.

The time evolution of a closed quantum system is generated by a Hamiltonian H , a Hermitian operator whose eigenvalues represent the energy levels of the system through the Schrödinger equation, whose formal solution is the unitary

$$U(t) = e^{-iHt}. \quad (3.6)$$

A particularly important class of operators on $\mathcal{H}$ consists of *projectors*. A projector is a Hermitian operator $\Pi : \mathcal{H} \rightarrow \mathcal{H}$ satisfying the idempotency condition $\Pi^2 = \Pi$. Its spectrum consists of only two eigenvalues, $\{0, 1\}$, so Π acts as the identity on the subspace $\Pi\mathcal{H}$ it projects onto and as zero on its orthogonal complement. Projectors play a dual role in this thesis: they describe measurement outcomes via the Born rule $\Pr[\text{outcome } k] = \langle \psi | \Pi_k | \psi \rangle$, and they encode *constraints* by selecting the subspace of feasible configurations of a combinatorial problem. The elementary single-qubit projectors

$$|0\rangle\langle 0| = \frac{I + Z}{2}, \quad |1\rangle\langle 1| = \frac{I - Z}{2}, \quad (3.7)$$

which we will use repeatedly to translate classical binary variables into quantum operators, are the building blocks of more elaborate projector constructions that will appear throughout the thesis. In particular, products of single-qubit projectors of the form

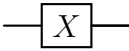
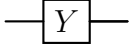
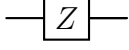
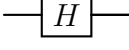
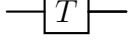
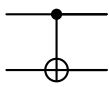
$$\Pi = \bigotimes_{i \in S} |b_i\rangle\langle b_i|, \quad b_i \in \{0, 1\}, \quad (3.8)$$

project onto the subspace in which a specified subset S of qubits is fixed to prescribed classical values, and lie at the heart of the constraint-preserving mixers developed in Chapter ??.

3.1.1 Elementary Quantum Gates and Universality

Quantum algorithms are expressed as sequences of elementary unitary operations, called *gates*, drawn from a small fixed set. The question of which sets of gates suffice to implement, or at least to approximate, every unitary on n qubits is the question of *universality*, and it is the gate-level analogue of the classical fact that any Boolean function can be implemented from $\{\text{AND}, \text{OR}, \text{NOT}\}$ or even from NAND alone.

A gate set is called *universal* if any unitary on n qubits can be approximated to arbitrary precision in operator norm by finite sequences drawn from the set. A foundational result of Barenco et al. [5] shows that the set consisting of all single-qubit unitaries together with any two-qubit entangling gate, most commonly

Operator	Gate(s)	Matrix
Pauli-X (X)		$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)		$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)		$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)		$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
$\pi/8$ (T)		$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
Controlled Not (CNOT, CX)		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
SWAP		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

the controlled-NOT, is universal in the exact sense. Standard single-qubit gates include the Pauli operators X , Y , Z , the Hadamard gate

$$H_{\text{gate}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad (3.9)$$

which maps the computational basis to the Hadamard basis $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$, the phase gates $S = \text{diag}(1, i)$ and $T = \text{diag}(1, e^{i\pi/4})$, and the parameterized rotation gates

$$R_{\sigma}(\theta) = e^{-i\theta\sigma/2} = \cos(\theta/2) I - i \sin(\theta/2) \sigma, \quad \sigma \in \{X, Y, Z\}, \quad (3.10)$$

which generate continuous rotations of the qubit on the Bloch sphere. Two-qubit entangling gates are required for universality; the canonical choice is the controlled-NOT

$$\text{CNOT} = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X, \quad (3.11)$$

which flips the target qubit conditional on the control qubit being in $|1\rangle$. For *discrete* universality, approximate, rather than exact, simulation of arbitrary unitaries from a finite gate set, the standard reference is the Solovay-Kitaev theorem [60], which states that any single-qubit unitary can be approximated to precision ε by a sequence of length $\mathcal{O}(\log^c(1/\varepsilon))$ drawn from any finite gate set that generates a dense subgroup of $\text{SU}(2)$ and is closed under inversion. A particularly important such set is the *Clifford+T* gate set $\{H_{\text{gate}}, S, \text{CNOT}, T\}$, which underlies fault-tolerant quantum computing: the Clifford subgroup $\{H_{\text{gate}}, S, \text{CNOT}\}$ admits highly efficient fault-tolerant implementations on most quantum error-correcting codes, while the non-Clifford T gate, which is comparatively expensive

to realize fault-tolerantly, supplies the resource needed for universality. The T -count of a circuit, the number of T gates it uses, is therefore the dominant cost metric in the fault-tolerant regime.

The choice of native gate set, however, depends strongly on the underlying hardware platform, and this has direct consequences for circuit compilation and resource estimation. Superconducting qubit architectures, such as those developed by IBM, Google, and Rigetti, typically expose single-qubit rotations together with a two-qubit entangling gate of the controlled- Z , CNOT, or iSWAP family, with restricted connectivity dictated by the chip layout. Trapped-ion devices, by contrast, benefit from all-to-all connectivity and natively realize the Mølmer-Sørensen gate $e^{-i\theta X_i X_j}$ [80][**<empty citation>**], which directly implements a two-qubit Pauli exponential and is therefore particularly well suited to Hamiltonian-simulation workloads. In each case, the abstract universal gate set must be *compiled* into the native one, with overheads that vary substantially across platforms and that influence which algorithmic primitives are most natural to use.

3.1.2 Pauli Exponentials

Most quantum optimization algorithms, and more generally most Hamiltonian-simulation routines, reduce to the implementation of unitaries of the form $e^{-i\alpha P}$, where P is a Pauli string on n qubits and $\alpha \in \mathbb{R}$ is an angle. Such *Pauli exponentials* admit a canonical decomposition into one- and two-qubit gates that is worth describing in detail, since its structure recurs throughout the thesis.

The construction relies on three observations. First, for a single-qubit Pauli $P = \sigma_i$, the exponential is precisely the rotation gate $R_\sigma(2\alpha)$ of (3.10), so the elementary case is solved by a single one-qubit rotation. Second, the Pauli operators X and Y are related to Z by single-qubit basis changes,

$$X = H_{\text{gate}} Z H_{\text{gate}}, \quad Y = (S H_{\text{gate}}) Z (S H_{\text{gate}})^\dagger, \quad (3.12)$$

so that any Pauli string can be conjugated, qubit by qubit, into a string of Z 's and identities by applying appropriate basis-change unitaries. Third, for a Pauli string $P = Z_{i_1} Z_{i_2} \cdots Z_{i_w}$ supported on w qubits, the eigenvalues of P on a computational-basis state $|z\rangle$ are ± 1 according to the parity $z_{i_1} \oplus z_{i_2} \oplus \cdots \oplus z_{i_w}$ of the relevant bits, which means that

$$e^{-i\alpha P} |z\rangle = e^{-i\alpha(-1)^{p(z)}} |z\rangle, \quad p(z) = z_{i_1} \oplus \cdots \oplus z_{i_w}. \quad (3.13)$$

The exponential of an all- Z Pauli string is therefore diagonal and applies a phase that depends only on the parity of the support.

These three observations combine into the well-known *CNOT staircase* construction. To implement $e^{-i\alpha P}$ for a general Pauli string of weight w :

1. For each qubit in the support of P on which the Pauli factor is X (respectively Y), prepend a Hadamard (respectively $S^\dagger H_{\text{gate}}$) gate. After this layer, the effective Pauli string in the rotated basis is a string of Z 's on the same support.

2. Choose one qubit in the support as the *target*, and apply a ladder of $w - 1$ CNOT gates that successively compute the parity of the support qubits into the target. After the ladder, the target qubit holds the value $p(z)$ and the others have been reset to their original values up to entanglement structure encoded in the ladder.
3. Apply a single-qubit rotation $R_Z(2\alpha)$ on the target qubit, which imprints the desired phase $e^{-i\alpha(-1)^{p(z)}}$.
4. Uncompute the parity by applying the CNOT ladder in reverse, restoring the qubits to their pre-ladder configuration.
5. Apply the inverse basis-change gates of step (1) to return to the original basis.

The resulting circuit, displayed in Figure 9, uses $2(w-1)$ CNOTs, a single one-qubit Z -rotation, and at most $2w$ basis-change gates. The CNOT count grows linearly with the weight of the Pauli string, and since each Pauli exponential dominates the cost of one term in a Trotterized Hamiltonian simulation, this scaling is the principal driver of the gate budget of QAOA-style implementations.

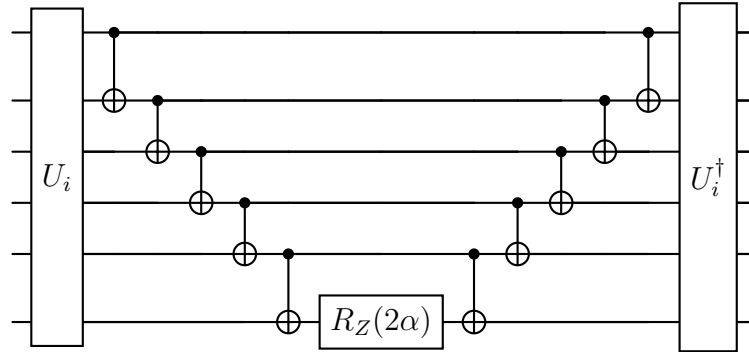


Figure 9: Quantum circuit for the exponential $e^{-i\alpha P}$ of a Pauli string. The basis-change unitaries U_i rotate each non- Z Pauli factor to Z ; the descending CNOT staircase computes the parity of the support onto the bottom qubit; the rotation $R_Z(2\alpha)$ applies the phase $e^{-i\alpha(-1)^{p(z)}}$; the ascending staircase uncomputes the parity; and the inverse basis-change gates restore the original basis.

A useful generalization of the Pauli exponential, which will reappear throughout this thesis, is the *projected Pauli operator*. Given a Pauli string P and a projector Π onto some subspace $\mathcal{S} \subseteq \mathcal{H}$, the projected operator

$$\hat{P} = \Pi P \Pi \quad (3.14)$$

acts as P on the subspace $\mathcal{S}$ and trivially on its orthogonal complement. When P maps states inside $\mathcal{S}$ to states outside it, the surrounding projectors annihilate the spurious amplitude, and the result is a constraint-preserving operator by construction: $[\hat{P}, \Pi] = 0$. When Π factorizes as a product of mutually commuting elementary projectors, as in (3.8), the projected operator can often be written more

compactly as a multi-controlled Pauli rotation, in which the controls implement the projector and the target carries the single-qubit Pauli action. This construction is the algebraic backbone of the constraint-preserving mixers introduced later in this chapter and is developed in full generality, together with the corresponding circuit synthesis techniques, in Chapter ??.

3.1.3 Trotter Decomposition and Randomized Compilation

When a Hamiltonian is a sum of non-commuting terms, $H = \sum_{j=1}^L H_j$, the unitary e^{-iHt} does not factorize as a product of single-term exponentials, because $[H_j, H_k] \neq 0$ in general. The *Trotter–Suzuki decomposition* [84] provides controlled approximations to such evolutions by alternating short evolutions under each summand. The first-order Trotter formula reads

$$e^{-iHt} = \left(\prod_{j=1}^L e^{-iH_j t/p} \right)^p + \mathcal{O}\left(\frac{t^2}{p} \sum_{j < k} \|[H_j, H_k]\| \right), \quad (3.15)$$

with error controlled by the Trotter step $\Delta t = t/p$ and the magnitudes of the commutators between summands. The error per step is $\mathcal{O}(\Delta t^2)$ and aggregates to $\mathcal{O}(t^2/p)$ over the full evolution.

This first-order formula is the most economical but also the least accurate. Higher-order formulae are therefore exponentially more expensive per step but require correspondingly fewer steps to reach a target accuracy, and the choice of order is a non-trivial optimization that depends on the structure of the Hamiltonian and the target precision.

A different line of work avoids deterministic Trotter sequences altogether and exploits *randomization* to suppress error. The *quantum stochastic drift protocol* (qDRIFT) [15] samples each elementary exponential independently from the distribution $p_j = \|H_j\| / \sum_k \|H_k\|$ and applies it for a fixed effective time. The number of samples required to approximate e^{-iHt} to precision ε in diamond norm is

$$N_{\text{qDRIFT}} = \mathcal{O}\left(\frac{\left(\sum_j \|H_j\| \right)^2 t^2}{\varepsilon} \right), \quad (3.16)$$

which is independent of the number of summands L , a striking improvement over Trotter formulae when H is the sum of many low-norm terms, as is the case for typical molecular electronic-structure Hamiltonians.

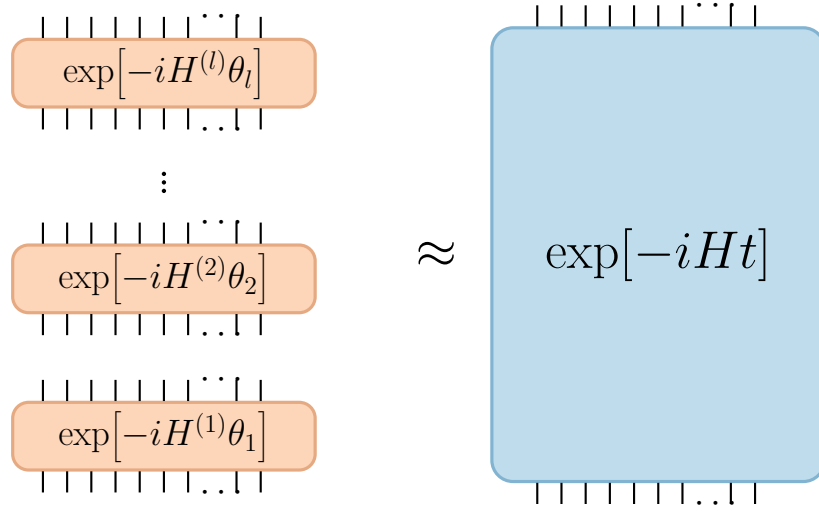


Figure 10: Trotter expansion of the time-evolution operator: $\exp[-iHt]$ is approximated by a product of small unitary steps generated by the local terms $H^{(k)}$.

3.2 Stabilizer Formalism

Several of the algorithmic constructions developed in this thesis, in particular the LX-mixer of the first paper, can be naturally formulated in the language of the *stabilizer formalism*. Originally introduced by Gottesman [43] as the algebraic backbone of quantum error correction, the formalism provides a compact way to describe subspaces of the n -qubit Hilbert space $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$ through their symmetries rather than through an explicit basis. We collect here the minimal notation and the key facts needed in the remainder of the chapter.

The *Pauli group* on n qubits is

$$\mathcal{P}_n = \{\alpha P_1 \otimes \cdots \otimes P_n : P_i \in \{I, X, Y, Z\}, \alpha \in \{\pm 1, \pm i\}\}, \quad (3.17)$$

that is, tensor products of single-qubit Paulis decorated with a global phase drawn from the fourth roots of unity. The phases are necessary for $\mathcal{P}_n$ to be closed under multiplication, since for example $XY = iZ$. Any two elements $P, Q \in \mathcal{P}_n$ either commute ($PQ = QP$) or anticommute ($PQ = -QP$), a property that turns the analysis of operator relations in $\mathcal{P}_n$ into a problem in linear algebra over $\mathbb{F}_2$ and is the foundation of the efficient algorithms associated with the formalism.

A *stabilizer group* $\mathcal{S}$ is an abelian subgroup of $\mathcal{P}_n$ that does not contain $-I$. Any stabilizer group admits a minimal generating set $\mathcal{S} = \langle g_1, \dots, g_k \rangle$ of k independent generators, where $|\mathcal{S}| = 2^k$, and the corresponding *stabilizer subspace* (or *code space*) is

$$\mathcal{C}(\mathcal{S}) = \{|\psi\rangle \in \mathcal{H} : g|\psi\rangle = |\psi\rangle \text{ for all } g \in \mathcal{S}\}, \quad (3.18)$$

the simultaneous $(+1)$ -eigenspace of all stabilizer generators. $\mathcal{C}(\mathcal{S})$ has dimension 2^{n-k} . The orthogonal projector onto it admits two equivalent and useful expressions:

$$\Pi_{\mathcal{S}} = \prod_{j=1}^k \frac{I + g_j}{2} = \frac{1}{|\mathcal{S}|} \sum_{g \in \mathcal{S}} g, \quad (3.19)$$

where the first form is the product over generators and the second is the average over all 2^k group elements. The sum form is what is related to the cost analysis of the mixer circuits, since each term $g \in \mathcal{S}$ is a Pauli string that must be exponentiated separately in the Trotterized circuit.

Two further subgroups of $\mathcal{P}_n$ play a central role in the algebraic theory of mixers. The *centralizer* of $\mathcal{S}$ in $\mathcal{P}_n$ is the set of Pauli operators that commute with every stabilizer element,

$$C(\mathcal{S}) = \{P \in \mathcal{P}_n : [P, g] = 0 \ \forall g \in \mathcal{S}\}, \quad (3.20)$$

and the *normalizer* is the set of Pauli operators under whose conjugation $\mathcal{S}$ is preserved as a set,

$$N(\mathcal{S}) = \{P \in \mathcal{P}_n : P g P^\dagger \in \mathcal{S} \ \forall g \in \mathcal{S}\}. \quad (3.21)$$

For stabilizer groups, a standard result is that $C(\mathcal{S}) = N(\mathcal{S})$, a coincidence specific to the fact that $-I \notin \mathcal{S}$. Operators in $C(\mathcal{S}) \setminus \mathcal{S}$ act non-trivially on the code space while preserving it, and are the *logical operators* of the code: they implement encoded gates on the qubits virtually stored in $\mathcal{C}(\mathcal{S})$.

For the purposes of this thesis, the stabilizer formalism plays a different role from its original error-correction setting: it provides the algebraic substrate for designing *constraint-preserving mixers*. A feasible subspace $\text{span}(B) \subset \mathcal{H}$, defined by a set $B \subset \{0, 1\}^n$ of feasible computational basis states, can be partitioned into smaller subspaces that themselves admit stabilizer descriptions. We can define the set of pairs sharing the same logical X :

$$V_X = \{|x\rangle \in B : \exists |y\rangle \in B \text{ with } X|x\rangle = |y\rangle\} \quad (3.22)$$

and the corresponding Hamiltonian term takes the form

$$H_X = X \Pi_{\text{span}(V_X)}, \quad (3.23)$$

a single logical X followed by a projection onto the subspace $\text{span}(V_X)$. Crucially, all non-zero Pauli strings in the expansion of H_X commute with one another, they all lie in the centralizer of the underlying stabilizer group, and therefore the Trotterised exponential of H_X is *exact*: no Trotter error is incurred within a single logical- X block. In conclusion we have that constraints define a feasible subspace; the stabilizer formalism encodes that subspace algebraically through its symmetries; and the centralizer of the resulting stabilizer group identifies, which operators are admissible as constraint-preserving mixers.

3.3 Circuit Compilation and Synthesis

The Hamiltonians produced by the encoding constructions of the next sections are typically not in a form that can be executed directly on quantum hardware. The Trotter formula (3.15) reduces the problem of simulating e^{-iHt} to that of implementing a sequence of single-term exponentials, and the Pauli-exponential decomposition reduces each single-term exponential to a sequence of CNOTs

and rotations. What remains is a *circuit synthesis* or *circuit compilation* problem: given a target operator, produce an explicit circuit of elementary gates implementing its evolution at minimal cost. The second paper collected in this thesis addresses this problem for the specific class of operators that arise in the constraint-preserving constructions of the previous section, and we collect here the relevant background.

The cost metrics that govern circuit synthesis are heterogeneous, and the right metric depends on the hardware target. The principal ones that appear repeatedly in the synthesis literature are the following.

- *Two-qubit gate count.* Typically the CNOT count is the dominant fidelity bottleneck on near-term superconducting devices, where two-qubit gate errors are roughly an order of magnitude larger than single-qubit ones. On trapped-ion and neutral-atom devices the relevant primitive may be the Mølmer-Sørensen or Rydberg-blockade gate rather than the CNOT, but the same metric, entangling operations per circuit, governs the fidelity budget.
- *Circuit depth.* The longest sequential chain of gates controls total runtime and the cumulative effect of decoherence. Depth and total gate count can decouple substantially when gates on disjoint qubits can be scheduled in parallel, and a circuit with low depth but high total gate count may be preferable on a device whose coherence time, rather than its gate-error rate, is the biggest constraint.
- *Ancilla qubit count.* The auxiliary register size required by the synthesis is constrained on near-term devices where the number of qubits is limited. Many synthesis techniques trade ancilla qubits for reduced two-qubit gate count or depth, and the right operating point depends on whether the bottleneck is qubits, fidelity, or runtime.
- *T-gate count.* On fault-tolerant architectures, the cost metric shifts from physical-gate fidelity to logical resource cost. Clifford operations (CNOT, Hadamard, phase S) admit transversal implementations on standard stabilizer codes and are essentially free in the logical layer; the non-Clifford T gate, by contrast, requires magic-state distillation, whose cost per gate is several orders of magnitude greater than that of a transversal Clifford. The *T-count* (total number of T gates) and *T-depth* (longest sequential chain of T gates) are therefore the canonical resource estimates for fault-tolerant circuit synthesis, and arbitrary single-qubit rotations $R_\sigma(\theta)$ must be decomposed into a sequence of Clifford and T gates at a cost of $\mathcal{O}(\log(1/\epsilon))$ T gates per rotation to achieve precision ϵ , via the Solovay-Kitaev theorem. The second paper of this thesis tracks T-count alongside the other metrics, since the relative ranking of synthesis strategies changes substantially depending on whether the target architecture is NISQ (CNOT-dominated) or fault-tolerant (T-dominated).

Different synthesis techniques optimize different combinations of these metrics, and the choice among them depends on which constraint dominates on the target hardware.

For a single Pauli exponential $e^{-i\alpha P}$ on a Pauli string P of weight w , the canonical synthesis discussed in Section 3.1 requires $2(w - 1)$ CNOTs, one single-qubit rotation, and zero ancillas. For sums of Pauli exponentials the synthesis literature develops a plethora of optimisation techniques including *phase polynomial* representations, *commuting block* reorderings, *phase gadget* synthesis [27], and *Gray code* traversals of the Pauli string set; each offers different tradeoffs between depth and CNOT count.

These Pauli-based pipelines are well suited to operators whose Pauli decomposition is short. They become unattractive, however, when the same operator admits a different, more structured representation in which the number of non-Clifford resources is much smaller than its Pauli weight suggests. A telling example is the class of constrained evolutions

$$U(t) = e^{-it\sigma\Pi_B}, \quad (3.24)$$

where σ is a Pauli string, Π_B is the projector onto a subspace spanned by a set B of computational basis states, and $[\sigma, \Pi_B] = 0$. These operators appear pervasively in the algorithms studied in this thesis: the constraint-preserving mixers of Section 3.2 are sums of exactly this form, the phase-separating Hamiltonians of QAOA reduce to it when the cost function is supported on a feasible subspace, and the fermionic excitation operators that underpin variational quantum eigensolver ansätze take this form after a Jordan-Wigner transformation [90]. Expanding $\sigma\Pi_B$ in the Pauli basis and synthesising each term independently is straightforward, but the resulting Pauli expansion can have exponentially many non-zero terms even when the subspace B is small, so the number of single-qubit rotations, and hence T gates after Solovay–Kitaev decomposition, grows exponentially with the qubit count. The underlying operator has rank at most $|B|$, and one would expect a synthesis whose cost is controlled by $|B|$ rather than by 4^n .

This expectation is the starting point of the second paper of this thesis. The key observation is that an evolution of the form (3.24) can be re-interpreted as a *subspace-controlled unitary*: a transformation that acts as a Pauli rotation on $\text{span}(B)$ and as the identity on its orthogonal complement, in direct analogy with the way an ordinary multi-controlled gate acts as a unitary on the subspace where its controls are satisfied and trivially elsewhere. The promise of this reformulation is that the resource cost can be made to scale with the dimension of the controlled subspace rather than with the weight of the underlying Pauli decomposition, which translates into a substantial reduction in non-Clifford resources whenever $|B|$ is small compared to 2^n — precisely the sparse regime in which most constrained optimization Hamiltonians are.

3.3.1 Beyond Pauli Decomposition: Alternative Synthesis Paradigms

The Pauli-exponential viewpoint covers the bulk of the synthesis problems that arise from QAOA-style algorithms, but it does not exhaust the synthesis toolbox. Two further paradigms have become standard in the literature, each replacing

the Pauli decomposition with a different representation of the target unitary and exposing different optimisation opportunities.

ZX-Calculus and Diagrammatic Synthesis

An alternative synthesis paradigm represents circuits not as ordered sequences of gates but as graphical diagrams equipped with a complete set of local rewrite rules: the *ZX-calculus* [22, 31]. Circuit synthesis in this setting becomes a problem of *diagram simplification*, and ZX-based pipelines have been shown to expose CNOT and T -count reductions that are difficult to obtain from gate-level optimisation alone, in particular through the work of Kissinger and van de Wetering on the reduction of Clifford+ T circuits [59]. We do not develop the calculus here, and refer to the literature for the full graphical formalism and its application to Clifford+ T optimisation.

Recursive KAK Decompositions

A further line of synthesis results exploits the Cartan decomposition of the unitary group, known in the quantum-information literature as the *KAK decomposition*. The Cartan theorem factors any element U of a semisimple Lie group into a product $U = K_1 A K_2$, where K_1, K_2 lie in a chosen subgroup and A lies in a maximal abelian subgroup transverse to it; specialised to the unitary groups arising from n -qubit Hilbert spaces, this factors out an algebraically tractable subgroup of local operations from a residual non-local block. Applying the decomposition recursively, with a sequence of Cartan involutions tailored to the hierarchical tensor structure of n qubits, produces a synthesis scheme that breaks an arbitrary n -qubit unitary into a tree of local rotations and structured non-local blocks of decreasing size, terminating at two-qubit and single-qubit primitives. Recursive KAK synthesis [87] provides a unified framework that recovers the quantum Shannon [79] and cosine–sine decompositions as special cases, and is asymptotically optimal in two-qubit gate count for fully generic unitaries.

3.4 Variable Encodings

Before any quantum optimization algorithm can be applied, the classical variables and cost function must be expressed in terms of qubits and Hermitian operators. The encoding of the variables that characterize our problems are the subject of the present section. The starting point is almost always a *diagonal* Hamiltonian, one whose eigenbasis coincides with the computational basis:

$$H_C |z\rangle = f(z) |z\rangle, \quad z \in \{0, 1\}^n, \quad (3.25)$$

for some classical objective $f : \{0, 1\}^n \rightarrow \mathbb{R}$. Such Hamiltonians are polynomials in the Pauli- Z operators, they act by multiplying each computational basis state by the corresponding objective value, and minimising f is equivalent to finding the ground state of H_C .

The first question we have to handle so is how to encode a single classical variable. When the variable is itself binary the natural representation is one qubit per variable, with the two computational basis states standing for the two possible values. The interesting case arises when a variable takes more than two values, an integer, a label, a color, or a position in a permutation, since a register of two-level systems has no canonical way to store a k -valued symbol. Each encoding makes a different tradeoff between three quantities: the number of qubits required to represent a variable, the locality and gate-complexity of the cost Hamiltonian that results once the objective is rewritten in terms of those qubits, and the structure of the resulting state space, in particular whether the encoding introduces spurious computational basis states with no classical counterpart and, if so, how those states are handled.

3.4.1 Binary Variable Encoding

The map from classical objective to quantum Hamiltonian is achieved by identifying binary variables $x_i \in \{0, 1\}$ with the projectors

$$x_i \longleftrightarrow \frac{I \pm Z_i}{2}, \quad (3.26)$$

so that $x_i = 1$ corresponds to $\frac{I-Z_i}{2} = |1\rangle\langle 1|$, the eigenstate $|1\rangle$ of Z_i with eigenvalue -1 , and $x_i = 0$ to $\frac{I+Z_i}{2} = |0\rangle\langle 0|$, so the Z_i eigenvector $|0\rangle$ with eigenvalue $+1$. Under this substitution any polynomial objective in the x_i becomes a polynomial in the Z_i , which acts diagonally on the computational basis. This encoding is qubit-optimal, one qubit per binary variable, and the resulting Hamiltonians inherit the locality structure of the classical objective: a k -local classical polynomial yields a k -local Pauli- Z Hamiltonian.

3.4.2 Integer Value Encoding

The interesting case arises when a variable takes values in a discrete set of more than two elements, $c_i \in \{0, 1, \dots, k-1\}$, as for the vertex colors of Max- k -Cut with $k > 2$. The two extreme strategies for representing such a variable, compact in qubits but complex in Hamiltonian structure on the one hand, profligate in qubits but simple in structure on the other, are described below, and they bracket the design space within which the third paper operates.

Integer Binary Encoding

For variables taking values in a discrete set $c_i \in \{0, 1, \dots, k-1\}$, a natural choice is the *integer* (or *binary*) encoding, in which each variable is represented by $b = \lceil \log_2 k \rceil$ qubits whose joint computational basis state encodes the integer value in binary. For Max- k -Cut on a graph $G = (V, E)$, each vertex $i \in V$ is assigned b qubits whose state $|c_i\rangle = |c_i^{(b-1)}, \dots, c_i^{(1)}, c_i^{(0)}\rangle$ encodes the label c_i of the partition to which the vertex belongs. The total qubit count is therefore $b|V|$, logarithmic in the alphabet size k and linear in the number of vertices.

The qubit cost of this encoding is the most economical of the constructions considered in this section. The price is a more complex cost Hamiltonian: compiling the exponential of such a cost Hamiltonian into elementary gates becomes a nontrivial circuit synthesis problem of exactly the kind introduced in Section 3.3, and is precisely the question addressed by the third paper in this thesis in the specific case of the max- k -cut problem. A problem in particular arises when k is not a power of two: the encoding includes $2^b - k$ spurious labels with no classical counterpart, and these must be either explicitly excluded by additional penalty or constraint terms or rendered harmless by careful design of the mixer Hamiltonian or the definition of equivalence classes.

One-Hot Encoding

The *one-hot* encoding sits at the opposite extreme of the qubit-cost spectrum. Each variable taking k possible values is assigned k qubits, exactly one of which is in the $|1\rangle$ state, with the index of that qubit indicating the label. For Max- k -Cut, each vertex i receives k qubits $\{q_{i,\ell}\}_{\ell=1}^k$, and the constraint

$$\sum_{\ell=1}^k q_{i,\ell} = 1 \quad (3.27)$$

enforces the one-hot structure on each vertex. The total qubit count is $k|V|$, linear in both k and the number of vertices.

The one-hot encoding trades qubit count for cost-Hamiltonian simplicity. Its principal drawback is the introduction of a hard constraint, the cardinality condition (3.27), which must be enforced by penalty terms, constraint-preserving mixers, or by restricting the dynamics to the appropriate Hamming-weight subspace. The XY-mixer family discussed later in this chapter is particularly well suited to this last strategy and turns the qubit-cost overhead into a source of structural simplicity in the mixer dynamics.

3.5 Problem Formulation on Quantum Hardware

Once the classical decision variables have been encoded as quantum states, the next step is to translate the optimization problem itself into a Hamiltonian whose ground state encodes the solution. This translation has two distinct components, treated in turn in the subsections below. The *cost function encoding* expresses the classical objective f as a diagonal Hamiltonian H_C , whose ground states are the optimal classical configurations. The *constraint encoding* extends this construction to constrained problems by ensuring that infeasible configurations are either energetically suppressed or structurally excluded from the quantum dynamics. Together, these two pieces define the quantum formulation of a combinatorial optimization problem, and the choices made at each stage determine which algorithmic paradigms are applicable and how efficiently they can be executed on hardware.

3.5.1 Cost function encoding

The most common quantum-compatible formulation of a combinatorial optimization problem is the *quadratic unconstrained binary optimization* (QUBO) model,

$$\min_{x \in \{0,1\}^n} x^\top Q x, \quad (3.28)$$

where $Q \in \mathbb{R}^{n \times n}$ is a real symmetric matrix. Under the substitution (3.26) the QUBO (3.28) becomes the *Ising Hamiltonian*

$$H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j + \text{const}, \quad (3.29)$$

with coefficients h_i and J_{ij} determined by the entries of Q . Finding a ground state of (3.29) is equivalent to solving the QUBO, and every NP-hard problem with a polynomial-size classical formulation can be cast in this form, at most at the cost of introducing auxiliary variables [66].

For the four canonical problems introduced in Chapter 1, the corresponding cost Hamiltonians take particularly transparent forms. For MaxCut on a graph $G = (V, E)$,

$$H_{\text{MaxCut}} = \frac{1}{2} \sum_{(i,j) \in E} (Z_i Z_j - I), \quad (3.30)$$

whose ground states are bit strings that maximize the number of edges with differing endpoint assignments. For Minimum Vertex Cover, minimizing the cover size corresponds to maximizing

$$H_{\text{MVC}}^{\text{obj}} = \sum_{i \in V} \frac{I + Z_i}{2}, \quad (3.31)$$

since $x_i = 0$ (excluded from the cover) has Z_i -eigenvalue $+1$. For Maximum Independent Set the analogous objective reverses the sign through the duality relation (1.7). The Max- k -Cut problem, as discussed in Section 3.4, is more delicate: the decision variables are k -valued, and the choice of encoding determines the structure of the resulting cost Hamiltonian.

Classical objectives that are polynomials of degree higher than two over the binary variables yield *higher-order unconstrained binary optimization* (HUBO) formulations,

$$\min_{x \in \{0,1\}^n} \sum_{S \subseteq \{1, \dots, n\}} c_S \prod_{i \in S} x_i, \quad (3.32)$$

which under the substitution (3.26) produce Pauli- Z Hamiltonians containing terms of weight greater than two:

$$H_C^{\text{HUBO}} = \sum_{S \subseteq \{1, \dots, n\}} \tilde{c}_S \prod_{i \in S} Z_i + \text{const}, \quad (3.33)$$

with $\tilde{c}_S$ determined by the coefficients c_S . The locality of the resulting Hamiltonian is set by the maximum monomial degree of the classical objective: a k -local HUBO becomes a k -local Ising-type Hamiltonian. HUBO formulations can be

reduced to QUBOs by introducing auxiliary variables that linearize the high-degree monomials, but this reduction increases the variable count and is often less efficient than working with the higher-locality Hamiltonian directly.

A particularly important case in which HUBO Hamiltonians arise naturally is the *integer encoding* of multi-valued classical variables. When a classical variable $c_i \in \{0, 1, \dots, k-1\}$ is encoded into $b = \lceil \log_2 k \rceil$ qubits, the cost function on c_i becomes a polynomial in the encoding qubits whose degree can reach b .

The structure of (3.29) suggests an immediate quantum algorithmic strategy: prepare a ground state of H_C , measure in the computational basis, and return the observed bit string. Every quantum optimization algorithm discussed in this chapter can be read as a specific proposal for how to perform, or approximate, this ground state preparation. Once we have encoded the problem cost function into a Hamiltonian the next important challenge is how to deal with the constraints of our problem, like for example in MVC and MIS. Here two main strategies are discussed: penalty-based and constraint-preserving approaches, but other techniques have been recently proposed like IF-QAOA.

3.5.2 Constraint Encoding

For unconstrained problems the cost Hamiltonian alone fully defines the problem: every computational basis state is a valid candidate solution, and the algorithm's task reduces to ground-state preparation. Constrained problems introduce a second problem. The classical formulation imposes one or more conditions $g_\ell(x) = 0$ or $g_\ell(x) \leq 0$ on the decision variables, restricting the space of valid solutions to a subset $\mathcal{F} \subseteq \{0, 1\}^n$ typically much smaller than the full space of 2^n bit strings. A quantum algorithm operating on $\mathcal{H}$ has no automatic mechanism to restrict its dynamics to the corresponding feasible subspace $\mathcal{H}_{\text{feas}} = \text{span}\{|z\rangle : z \in \mathcal{F}\}$, and the choice of how to enforce this confinement is one of the defining algorithmic choices in quantum combinatorial optimization. Two principal strategies have been considered in literature: *penalty-based encodings*, which extend the cost Hamiltonian with a term that raises the energy of infeasible configurations, and *constraint-preserving encodings*, which design the dynamics themselves to remain within $\mathcal{H}_{\text{feas}}$ at every point in the algorithm's execution. We treat each in turn.

Penalty-Based Encodings

Penalty-based encoding is the direct quantum analogue of *Lagrangian relaxation* in classical optimization, where a constrained problem $\min_x f(x)$ subject to $g_\ell(x) \leq 0$ is replaced by an unconstrained objective $f(x) + \sum_\ell \lambda_\ell g_\ell(x)^2$. The Lagrange multipliers become *penalty weights*, and the objective and constraints are combined into a single Hamiltonian

$$H = H_C + \lambda H_P, \quad H_P \geq 0, \quad \ker H_P = \mathcal{H}_{\text{feas}}, \quad (3.34)$$

where H_P is a nonnegative penalty Hamiltonian annihilating precisely the feasible computational basis states, and $\lambda > 0$ is chosen large enough that no infeasible configuration lies below the lowest-energy feasible one. Inequality constraints are

typically converted to equalities with slack variables before penalizing; a linear equality $\sum_i a_i x_i = b$ yields the canonical quadratic penalty $(\sum_i a_i x_i - b)^2$, which under (3.26) produces a two-local or higher Ising term. A systematic catalogue of such formulations for most of Karp’s NP-complete problems was compiled by Lucas [66] and remains the standard reference; for Minimum Vertex Cover, for instance, the term $\lambda \sum_{(i,j) \in E} (1 - x_i)(1 - x_j)$ added to $\sum_i x_i$ penalizes every uncovered edge and recovers a QUBO.

The appeal of the approach is that it reduces any constrained problem to an Ising ground-state problem, making the full toolbox of quantum optimization methods immediately available. Its drawbacks, however, are equally well known, and fall into three categories.

Parameter tuning. The weight λ must satisfy the separation condition

$$\lambda \min_{z \notin \mathcal{F}} \langle z | H_P | z \rangle > \max_{z \in \mathcal{F}} \langle z | H_C | z \rangle - \min_{z \in \mathcal{F}} \langle z | H_C | z \rangle, \quad (3.35)$$

whose right-hand side depends on the optimal feasible value, the very quantity the algorithm is trying to compute. One therefore substitutes a worst-case bound, typically yielding a λ far larger than necessary. The result is a tradeoff between two failure modes: too small a λ admits infeasible low-energy states and the algorithm may return an invalid solution, while too large a λ flattens the feasible landscape, so that meaningful cost differences within $\mathcal{H}_{\text{feas}}$ become negligible against the penalty scale. The optimal choice is instance-dependent and rarely available in closed form [66].

Spectral distortion. Even when λ is well chosen, the penalty introduces a dense block of infeasible eigenvalues at energies of order λ , within which the feasible spectrum, on which success ultimately depends, becomes embedded. For adiabatic evolution this can produce avoided crossings at the feasible-subspace boundary whose gaps close exponentially in system size, forcing exponentially long schedules even when the restricted problem has a constant gap. For variational methods the landscape acquires sharp ridges along the feasible-infeasible boundary, with gradients vanishing on penalty plateaus or diverging at penalty walls, geometry that standard optimizers struggle to navigate.

Locality inflation. The weight of the penalty Pauli strings is set by the constraint width. A cardinality constraint $\sum_{i=1}^k x_i = 1$, for example, produces a fully connected graph of two-local $Z_i Z_j$ terms, requiring $\binom{k}{2}$ two-qubit gates per Trotter layer; for one-hot encodings this cost recurs for every variable, and nonlinear constraints can push the locality higher still.

These limitations have motivated two complementary responses. One retains the penalty framework but mitigates its pathologies through warm-starting with classical feasible solutions [32], adaptive schedules, or hybrid procedures that restrict to feasible subspaces after an initial exploration. The other abandons penalties altogether, enforcing feasibility structurally through dynamics that cannot leave the feasible subspace, and is the subject of the next section.

Constraint-Preserving Encodings

A constraint defines a subset $\mathcal{F} \subseteq \{0,1\}^n$ of feasible configurations and, correspondingly, a feasible subspace

$$\mathcal{H}_{\text{feas}} = \text{span}\{|z\rangle : z \in \mathcal{F}\} \subseteq \mathcal{H}. \quad (3.36)$$

A *constraint-preserving* quantum algorithm is one whose dynamics restrict to $\mathcal{H}_{\text{feas}}$: the state remains a superposition of feasible basis states at every point in the algorithm's execution. Formally, this requires that every unitary U applied in the algorithm commute with the projector Π_{feas} onto the feasible subspace, or equivalently that every Hamiltonian generator H satisfy $[H, \Pi_{\text{feas}}] = 0$. Under this condition the problem reduces to ground-state preparation of the *unconstrained restriction*

$$H_C|_{\mathcal{H}_{\text{feas}}}, \quad (3.37)$$

on a Hilbert space whose dimension is $|\mathcal{F}|$ rather than 2^n . Both the penalty parameter and the entire spurious spectrum of infeasible states disappear from the problem.

The commutation condition $[H, \Pi_{\text{feas}}] = 0$ is the defining algebraic property of constraint-preserving dynamics and deserves a closer look. Equivalently, it requires that the Hamiltonian preserves the block structure $\mathcal{H} = \mathcal{H}_{\text{feas}} \oplus \mathcal{H}_{\text{feas}}^\perp$, with no matrix elements connecting the two blocks: $\langle z|H|z'\rangle = 0$ whenever $|z\rangle \in \mathcal{H}_{\text{feas}}$ and $|z'\rangle \in \mathcal{H}_{\text{feas}}^\perp$. The contrast with the penalty-based approach of the previous section is sharp on three points. First, the search space is fundamentally smaller: constraint-preserving dynamics live on a Hilbert space of dimension $|\mathcal{F}|$, whereas penalty-based dynamics live on the full 2^n -dimensional Hilbert space and use energy gradients to suppress the $2^n - |\mathcal{F}|$ infeasible directions. For problems with small feasible subspaces, the dimensional reduction is exponential. The ground state of the restricted Hamiltonian $H_C|_{\mathcal{H}_{\text{feas}}}$ is automatically the optimal feasible solution, with no risk of an infeasible configuration sneaking into the low-energy spectrum. Second, no penalty parameter has to be tuned, no separation condition has to be verified, and no instance-dependent calibration is needed before the algorithm can be deployed. Third, the spectrum of H_C on $\mathcal{H}_{\text{feas}}$ is exactly the spectrum that the algorithm must optimize, with no spurious clusters of infeasible eigenstates and no avoided crossings introduced at the feasible-infeasible boundary. The spectral gap that controls adiabatic and variational performance is the natural one, set by the structure of the cost function on the feasible set rather than by an artificial penalty-induced one.

These advantages come at a cost. The constraint-preserving paradigm trades a simple cost Hamiltonian for a complex mixer, since the generators that commute with Π_{feas} are typically more elaborate than in the unconstrained. The resulting tradeoff between mixer depth and Hamiltonian simplicity is one of the central design tradeoffs in the field. On the simple end, single-qubit transverse-field generators yield very shallow circuits but make no use of constraint structure. On the complex end, full constraint preserving Hamiltonian can implement transitions on arbitrarily restricted feasible subspaces but at correspondingly greater circuit cost. Which choice to take depends on the structure of the constraints, the

depth budget of the target hardware, and the available ancilla resources, and the performance of the optimization and a substantial portion of the constraint-preserving literature is devoted to study this tradeoffs for specific problem classes.

The constraint-preserving approach has deep roots in quantum simulation, where the same structural ideas have long been used to enforce conserved quantities by construction. In quantum chemistry, Hamiltonians preserving total particle number are essential: the electronic Hamiltonian commutes with the number operator $\hat{N} = \sum_i n_i$, and ansatzes such as the unitary coupled cluster (UCC) construction are designed so that their generators preserve $\hat{N}$ automatically. The fixed-particle-number subspace plays exactly the role of $\mathcal{H}_{\text{feas}}$ in the present setting, and many of the algebraic tools later applied to combinatorial optimization, in particular the systematic construction of generators commuting with a given symmetry, were developed first in the chemistry literature. Spin-conservation, projection onto fixed total angular momentum, and related symmetry-respecting ansatzes are further examples of the same ideas.

In quantum optimization the corresponding constraint-preservation techniques take the form of dedicated mixer designs for QAOA. The XY mixer preserves Hamming-weight subspaces [86], which arise naturally for cardinality constraints and one-hot encodings; the Grover mixer delegates feasibility to the initial state and uses a feasibility-preserving reflection as mixer [6] ; the LX mixer of Fuchs and Pariente Bassa provides a systematic stabilizer-theoretic construction of optimal subspace-preserving mixers for arbitrary feasible subspaces, exploiting the commutant structure introduced in Section 3.2. Each of these will be discussed in detail later in this chapter, after the QAOA framework that gives them their algorithmic context has been introduced.

3.5.3 Other approaches

Beyond mixer design, constraint preservation admits several further implementations. *Quantum Zeno dynamics* enforces feasibility by frequent measurement of a projector onto $\mathcal{H}_{\text{feas}}$: the Zeno effect suppresses transitions out of the feasible subspace without requiring a penalty term in the Hamiltonian [47]. *Penalty-free* formulations of a related kind have been developed by Bucher, Stein, Feld, and Linnhoff-Popien [13], who show that constrained optimization can be accelerated by modifying the evolution itself rather than the cost Hamiltonian. *Filtering and indicator-function approaches*, such as the IF-QAOA (*indicator-function QAOA*) framework, enforce feasibility in post-processing or through projective filters applied during the variational loop, occupying an intermediate position between pure penalty and pure subspace restriction [14]. *Biased quantum search* for constrained combinatorial optimization [88] further generalizes the Grover-mixer idea to non-uniform feasible distributions, and *quantum tree generators* prepare superpositions over feasible configurations by walking a decision tree whose branches respect the constraint structure [21]. Each of these proposals reflects a distinct position on the penalty-versus-feasibility spectrum, and several will reappear as comparators to the constraint-preserving approach developed in the final paper of this thesis.

3.6 Adiabatic Quantum Computation and Annealing

The earliest quantum optimization algorithm proposed, and the conceptually simplest, is *adiabatic quantum computation* (AQC), introduced by Farhi, Goldstone, Gutmann, and Sipser [35]. The idea is to encode the problem as the ground state of a Hamiltonian and prepare that ground state by slow interpolation from an easily prepared initial state.

Let H_B be a *driver* Hamiltonian whose ground state $|\psi_0\rangle$ is easy to prepare, the canonical choice is $H_B = -\sum_i X_i$, with ground state $|\psi_0\rangle = |+\rangle^{\otimes n}$, and let H_C be the diagonal cost Hamiltonian whose ground state encodes the solution to the combinatorial problem. Define the time-dependent Hamiltonian

$$H(s) = (1 - s)H_B + sH_C, \quad s \in [0, 1], \quad (3.38)$$

and evolve the system under the schedule $s(t) = t/T$ for $t \in [0, T]$. The *adiabatic theorem* [54] guarantees that if the schedule is sufficiently slow, specifically, if T exceeds a quantity controlled by the inverse square of the minimum spectral gap

$$\Delta_{\min} = \min_{s \in [0, 1]} (E_1(s) - E_0(s)) \quad (3.39)$$

between the ground and first-excited states of $H(s)$, then the final state is close to the ground state of H_C , and measurement in the computational basis yields the optimal solution with high probability.

Its applicability to optimization is bounded by two factors. First, the runtime T required to maintain adiabaticity scales as $\mathcal{O}(\Delta_{\min}^{-2})$, and for many hard instances $\Delta_{\min}$ closes exponentially in the problem size, reducing the adiabatic speedup to at most polynomial factors, and in some cases eliminating it altogether. Second, a perfectly coherent adiabatic evolution is experimentally demanding, and no existing hardware realizes it at scale.

Quantum annealing is a finite-temperature, open-system variant of AQC that abandons the strict adiabatic ideal in exchange for implementability on contemporary hardware [51]. The system is initialized in a superposition governed by a dominant transverse field and slowly cooled as H_C is ramped up, with thermal fluctuations playing a role broadly analogous to that of stochastic moves in classical simulated annealing. Commercial annealing devices manufactured by D-Wave Systems implement this paradigm at the scale of several thousand physical qubits.

The adiabatic and annealing algorithms also provide the conceptual foundation for the variational algorithms introduced in the next section. The Quantum Approximate Optimization Algorithm, that will be introduced in the next section, in particular, can be interpreted as a Trotterized and parameterized approximation of the adiabatic evolution (3.38), in which the continuous schedule $s(t)$ is replaced by a finite sequence of angles that are optimized classically rather than fixed by the adiabatic prescription. The connection runs deep enough that several performance analyses of QAOA proceed by comparison with the underlying adiabatic path, and the $p \rightarrow \infty$ limit of QAOA recovers exact ground-state preparation under the same spectral-gap conditions that control AQC.

3.7 The Variational Quantum Eigensolver

The variational quantum eigensolver (VQE) is one of the first examples of variational quantum algorithms proposed in literature. It use the Rayleigh-Ritz variational principle to produce upper bounds on the ground-state energy of a Hamiltonian H , parameterizing the trial state by a quantum circuit $U(\boldsymbol{\theta})$ and using a classical optimizer to minimize the energy expectation value over the parameters $\boldsymbol{\theta}$. Originally formulated for quantum chemistry, the same algorithm applies to combinatorial optimization by identifying H with the cost Hamiltonian H_C introduced in Section 3.5.1. This type of algorithms are typically called hybrid in literature because it use both the quantum computer and the classical in a combined way: the quantum hardware is responsible only for state preparation and measurement of expectation values, while the variational parameter updates are performed classically. Its weaknesses, principally the choice of ansatz and the difficulty of the classical optimization, are precisely the topics this section develops. We begin with the variational principle and the algorithmic description, and then turn to ansatz design, classical optimization, expressive power as captured by the dynamical Lie algebra, and the trainability obstructions known as barren plateaus.

Let H be a Hermitian operator acting on the Hilbert space $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$. By the spectral theorem, there exists an orthonormal basis of eigenvectors $\{|\psi_i\rangle\}_{i=1}^{2^n}$ and corresponding real eigenvalues $\{\lambda_i\}_{i=1}^{2^n}$ such that

$$H = \sum_i \lambda_i |\psi_i\rangle \langle \psi_i|, \quad \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_{2^n}. \quad (3.40)$$

The ground-state energy is $E_0 = \lambda_1$, and any associated eigenvector $|\psi_1\rangle$ is called a ground state.

Given a quantum state $|\phi\rangle \in \mathcal{H}$, the expectation value of H is

$$\langle H \rangle_\phi := \langle \phi | H | \phi \rangle = \sum_i \lambda_i |\langle \psi_i | \phi \rangle|^2. \quad (3.41)$$

Since $\sum_i |\langle \psi_i | \phi \rangle|^2 = 1$, it follows that

$$\langle \phi | H | \phi \rangle \geq \lambda_1 = E_0, \quad (3.42)$$

which is the Rayleigh–Ritz variational principle. Equality holds if and only if $|\phi\rangle$ lies entirely in the ground-state eigenspace.

The variational quantum eigensolver (VQE), introduced by Peruzzo *et al.* [75], restricts the search for the ground state to a parameterized family of states

$$|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta}) |0\rangle, \quad \boldsymbol{\theta} \in \Theta \subset \mathbb{R}^p,$$

where $U(\boldsymbol{\theta})$ is a quantum circuit. The associated cost function is

$$E(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | H | \psi(\boldsymbol{\theta}) \rangle = \langle 0 | U^\dagger(\boldsymbol{\theta}) H U(\boldsymbol{\theta}) | 0 \rangle \geq E_0. \quad (3.43)$$

The VQE problem is therefore the nonlinear optimization

$$E_{\text{VQE}} = \min_{\boldsymbol{\theta} \in \Theta} E(\boldsymbol{\theta}), \quad (3.44)$$

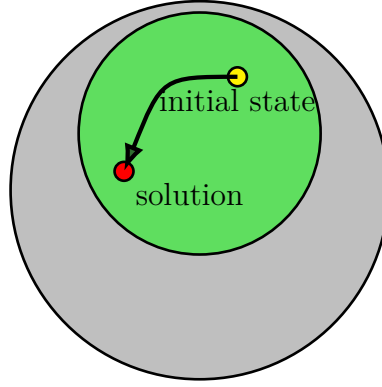


Figure 11: Representation of a variational algorithm. The grey area is the full Hilbert space; the green region is the set of states reachable by the ansatz $U(\vec{\theta})|\psi_0\rangle$ as the parameters $\vec{\theta}$ are varied. The yellow dot marks the initial state, fixed by the starting parameters, and the red dot marks the optimal solution (the target ground state), which the ansatz can reach only if it lies within the green region.

which seeks the best approximation to the ground-state energy within the ansatz class.

In practice, the Hamiltonian is decomposed typically as a linear combination of Pauli operators,

$$H = \sum_{\alpha} w_{\alpha} P_{\alpha}, \quad P_{\alpha} \in \{I, X, Y, Z\}^{\otimes n}, \quad (3.45)$$

so that the cost function can be estimated as

$$E(\boldsymbol{\theta}) = \sum_{\alpha} w_{\alpha} \langle 0 | U^{\dagger}(\boldsymbol{\theta}) P_{\alpha} U(\boldsymbol{\theta}) | 0 \rangle. \quad (3.46)$$

Each term is measured on a quantum device, and the resulting estimate of $E(\boldsymbol{\theta})$ is passed to a classical optimizer that proposes a parameter update $\boldsymbol{\theta} \rightarrow \boldsymbol{\theta}'$; the cycle of quantum state preparation, measurement, and classical update repeats until convergence.

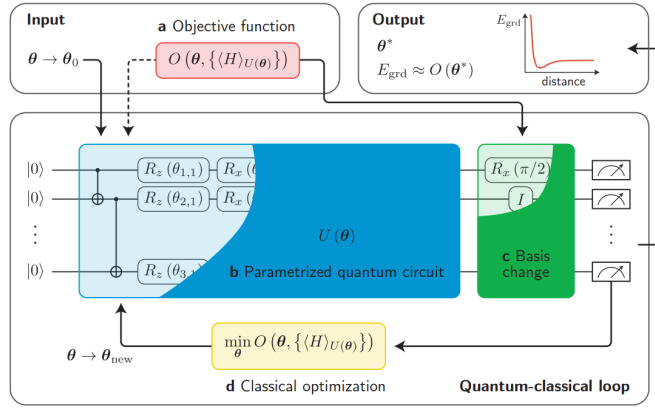


Figure 12: The VQE hybrid quantum–classical routine. At each iteration, the quantum hardware prepares the parameterized state $|\psi(\vec{\theta})\rangle = U(\vec{\theta})|0\rangle$ and measures the energy expectation $E(\vec{\theta}) = \langle\psi(\vec{\theta})|H|\psi(\vec{\theta})\rangle$ term by term in the Pauli decomposition $H = \sum_{\alpha} w_{\alpha} P_{\alpha}$. The estimate is passed to a classical optimizer, which proposes an updated parameter vector $\vec{\theta}'$ for the next iteration. The loop repeats until convergence to a local minimum of the energy landscape.

3.7.1 The Ansatz Choice

The central design choice is the *ansatz*, the family of circuits $U(\theta)$, which must be expressive enough to contain a good approximation of the ground state yet shallow enough to run coherently on available hardware. The central design choice in a variational algorithm is the *ansatz*, the family of circuits $U(\theta)$ over which the energy is minimized. A good ansatz must balance two competing demands. The first is *expressivity*: the circuit must be rich enough that its reachable set of states contains a good approximation of the target ground state, since the variational principle can only return the best state within that set. The second is *trainability*: the energy landscape must be navigable by a classical optimizer, so that a descent direction can actually be found from a reasonable starting point using a feasible number of measurements. These two demands pull in opposite directions. Increasing expressivity by adding layers or unstructured entangling gates enlarges the reachable set, but it also tends to flatten the cost landscape, eventually producing the *barren-plateau* phenomenon in which gradients vanish exponentially with the qubit count. The tension is not incidental: as we discuss in Section ??, both properties are governed by the same underlying object, the *dynamical Lie algebra* of the ansatz generators, so that the very richness that makes an ansatz expressive is what undermines its trainability. The choice of ansatz is therefore a choice of where to sit on this tradeoff, and the two dominant families in the literature occupy opposite ends of it. Two ansatz families dominate the literature: *hardware-efficient* ansatzes and *problem-inspired* ansatzes.

A hardware-efficient ansatz is built as a sequence of L repeated layers, each consisting of a layer of single-qubit rotations followed by a layer of two-qubit entangling gates matched to the device’s native connectivity:

$$U_{\text{HE}}(\theta) = \prod_{\ell=1}^L \left(W_{\ell} \cdot \prod_{i=1}^n R(\theta_{i,\ell}) \right), \quad (3.47)$$

where $R(\theta_{i,\ell})$ is a single-qubit rotation parameterized by one or more angles in $\theta_{i,\ell}$, and W_ℓ is a fixed entangling block, typically a brick-wall arrangement of CNOT or CZ gates over the device's coupling graph. The total number of parameters scales as $\mathcal{O}(nL)$, and the structure makes no reference to the problem Hamiltonian. Hardware-efficient ansatzes are easy to implement and broadly expressive at sufficient depth, but they pay for this generality with poor trainability, they suffer from the barren-plateau phenomenon that we will discuss later.

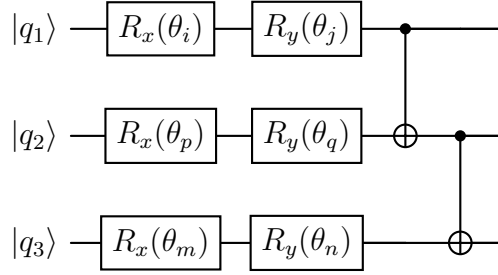


Figure 13: Hardware-efficient ansatz: a layer of single-qubit rotations $R_x \cdot R_y$ with independent parameters, followed by a linear chain of CNOTs that introduces entanglement.

Problem-inspired ansatzes derive their structure from physical or algorithmic considerations specific to the Hamiltonian at hand. The *Hamiltonian variational ansatz* is the canonical example: given a splitting $H = \sum_i H_i$ of the target Hamiltonian into non commuting parts, the ansatz is a sequence of alternating exponentials,

$$U_{\text{HVA}}(\boldsymbol{\alpha}, \boldsymbol{\beta}) = \prod_{\ell=1}^p \prod_i e^{-i\beta_{i,\ell} H_i} \quad (3.48)$$

which as we will see later is precisely the structure of QAOA when $H_A = H_C$ is the cost Hamiltonian and $H_B = H_M$ is a mixer.

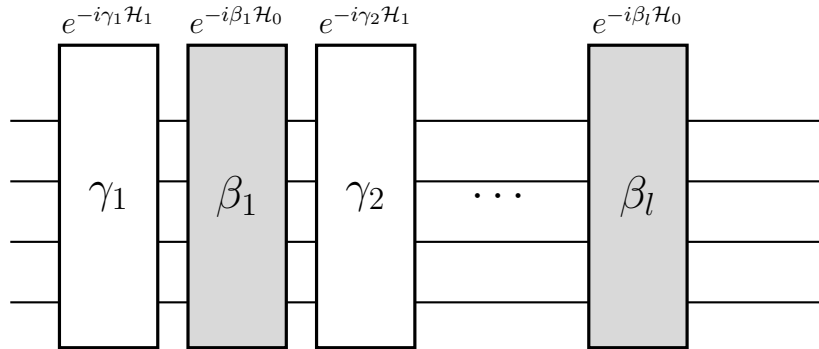


Figure 14: Problem-inspired (Hamiltonian variational) ansatz: alternating unitary layers $e^{-i\gamma_k H_1}$ and $e^{-i\beta_k H_0}$ generated by the problem and mixer Hamiltonians. Because the layers inherit the symmetries of their generators, the ansatz confines the state to the relevant symmetry sector at every depth.

The ansatz inherits the symmetries of H : if H_i commutes with some symmetry operator S , and $|\psi_0\rangle$ lies in a definite S -eigenspace, then the variational state

remains in that eigenspace at every depth, the Dynamical Lie algebra obtained from this generator are problem related. This makes Hamiltonian variational ansatzes natural carriers of constraint-preserving structure, and it underlies the connection between QAOA, the Quantum Alternating Operator Ansatz, and the constraint-preserving mixers of this thesis.

3.7.2 The Classical Optimizer Choice

The variational loop closes through a classical optimizer that update the parameters of the ansatz wavefunction using the noisy estimates of $E(\boldsymbol{\theta})$. Three families of optimizers are commonly used in the VQE literature, and the choice among them depends on whether and how gradients of the cost function are available, and on the noise of the underlying device.

Gradient-based optimizers: use exact analytical gradients of the cost function with respect to each variational parameter. For ansatzes built from single-parameter rotations, these gradients can be evaluated on quantum hardware via the *parameter-shift rule*: for a rotation $e^{-i\theta P/2}$ by a Pauli generator P , the partial derivative is

$$\frac{\partial E}{\partial \theta} = \frac{1}{2} \left[E\left(\theta + \frac{\pi}{2}\right) - E\left(\theta - \frac{\pi}{2}\right) \right], \quad (3.49)$$

expressing the derivative as a finite difference of two cost-function evaluations at shifted parameters. Each gradient component therefore costs two additional circuit evaluations, and the full gradient costs $2p$ evaluations for an ansatz with p parameters. For Projected Pauli operator, that have a spectrum composed of 3 eigenvalues the parameter shift formula is composed by 4 different evaluations at $\theta \pm \frac{\pi}{2}$ and $\theta \pm \frac{\pi}{4}$ [70, 78]. Once gradients are available, classical methods such as gradient descent, Adam, or quasi-Newton schemes (BFGS, L-BFGS) can drive the parameter optimization. Their strength is rapid local convergence; their weakness is sensitivity to the barren-plateau phenomenon discussed below, since vanishing gradients make gradient-based methods ineffective.

Stochastic gradient methods: they are characterized by treating the gradient estimate itself as a stochastic quantity. Each parameter-shift evaluation produces an empirical estimate of the gradient component drawn from a distribution centered on the true value, with variance set by the shot noise in the ideal case. *Stochastic gradient descent* updates the parameters using this noisy estimate at each step, and the variance of the update is controlled by tuning the shot budget per circuit. More elaborate variants such as *Stochastic Perturbation Stochastic Approximation* (SPSA) [81] reduce the per-step circuit cost by estimating the gradient with randomly sampled directions rather than evaluating each component independently, trading per-step accuracy for fewer circuit calls.

Gradient-free optimizers avoid the parameter-shift overhead entirely by extracting parameter updates directly from sampled cost values. Methods such as *COBYLA* (Constrained Optimization by Linear Approximation), *Nelder-Mead simplex search*, and *Bayesian optimization* fit local surrogate models to the cost landscape and propose updates from those surrogates. They are robust to moderate measurement noise and require no analytical structure on the ansatz, but

they typically scale less favorably with the number of parameters than gradient-based methods. They are particularly useful when the parameter-shift rule is unavailable or expensive, for example when the ansatz contains continuous gates outside the single-parameter-Pauli family.

The choice among these three families is governed by the same trainability considerations that motivate the dynamical Lie algebra and barren-plateau analyses below.

3.7.3 Dynamical Lie Algebras and Barren Plateaus

The expressive power of a parameterized ansatz is captured by the algebra of Hamiltonians its generators can produce under iterated commutation. Given a parameterized circuit $U(\boldsymbol{\theta}) = \prod_{\ell} e^{-i\theta_{\ell} H_{\ell}}$ with Hermitian generators $\{H_{\ell}\}_{\ell=1}^r$, the *dynamical Lie algebra* (DLA) is

$$\mathfrak{g} = \text{span}_{\mathbb{R}} \langle iH_1, \dots, iH_r \rangle_{\text{Lie}}, \quad (3.50)$$

the smallest real subalgebra of $\mathfrak{u}(\dim \mathcal{H})$ containing the generators $\{iH_{\ell}\}$ and closed under the Lie bracket. Its exponential $\mathcal{G} = e^{\mathfrak{g}}$ is the *dynamical Lie group*, and the set of unitaries reachable by the circuit at sufficient depth coincides with $\mathcal{G}$ [63, 76]; correspondingly, the states reachable from a reference $|\psi_0\rangle$ form the orbit $\mathcal{O}_{|\psi_0\rangle} = \{U|\psi_0\rangle : U \in \mathcal{G}\}$.

The DLA controls two important properties of the ansatz. The first is *controllability*: the ansatz reaches every unitary on $\mathcal{H}$ if and only if $\mathfrak{g} = \mathfrak{u}(\dim \mathcal{H})$ (or $\mathfrak{su}(\dim \mathcal{H})$ modulo global phase), while a proper subalgebra restricts reachability accordingly. In particular, when $\mathfrak{g}$ commutes with the feasibility projector Π_{feas} , the dynamics confine themselves to $\mathcal{H}_{\text{feas}}$ by construction, and full reachability of the feasible subspace amounts to $\mathfrak{g}|_{\mathcal{H}_{\text{feas}}}$ acting transitively on it.

The second property, *trainability*, is where the DLA connects to the central scaling obstacle of variational algorithms. An ansatz exhibits a *barren plateau* (BP) if the variance of the cost-function gradient, taken over a Haar-random parameter distribution, decays exponentially with the qubit count,

$$\text{Var}_{\boldsymbol{\theta}} [\partial_{\theta_k} E(\boldsymbol{\theta})] \in \mathcal{O}(b^{-n}), \quad b > 1. \quad (3.51)$$

By concentration of measure, an exponentially small variance forces the gradient itself to be exponentially small, so the landscape looks effectively flat from a random starting point and locating a descent direction requires either exponentially many samples or a structured initialization that avoids the plateau [62]. The comprehensive review of Larocca et al. [62] identifies five distinct sources of barren plateaus, each tied to a different ingredient of the variational pipeline: ansatz expressibility (the original mechanism of [69], in which unitary distributions approaching the Haar measure suppress gradients), hardware noise (which induces plateaus even in shallow circuits), initial-state entanglement, observable globality, and choice of loss function. A unifying thread through several

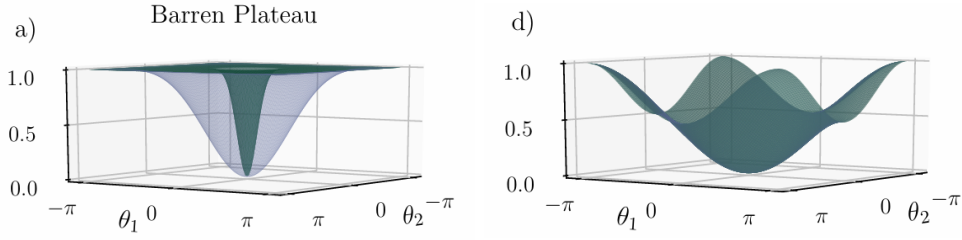


Figure 15: Schematic of the barren-plateau phenomenon. *Left:* a cost landscape exhibiting a barren plateau, in which $\text{Var}_{\vec{\theta}}[\partial_{\theta_k} E(\vec{\theta})]$ decays exponentially with the qubit count n , so that from a Haar-random initialization the landscape is effectively flat and the gradient gives no usable descent direction. *Right:* a landscape without a barren plateau, where gradients remain of order one and the optimizer can navigate to a minimum. Equation (3.52) ties the contrast to the dimension of the dynamical Lie algebra of the ansatz generators: $\text{Var}_{\vec{\theta}}[\partial_{\theta_k} E] \propto 1/\dim \mathfrak{g}$.

of these sources was provided by Ragone et al. [76], who showed that the gradient variance is controlled by the dimension of the dynamical Lie algebra,

$$\text{Var}_{\vec{\theta}}[\partial_{\theta_k} E(\vec{\theta})] \propto \frac{1}{\dim \mathfrak{g}}, \quad (3.52)$$

modulo factors depending on the projections of the initial state and observable into the algebra. This closes the loop with the expressibility picture: a DLA large enough to make the ansatz highly expressive is large enough to flatten its gradients. The result carries a troubling corollary, however. Keeping the variance polynomially bounded requires a DLA of polynomial dimension, and most ansatzes with a polynomial-dimensional DLA admit classical simulation algorithms of polynomial cost [16, 29, 62]. Whether a trainable variational ansatz can offer a genuine quantum advantage at all is therefore an open question, and one of the field’s central ones [62].

Different strategies have been proposed to mitigate this gradient problem, most of which act by controlling $\dim \mathfrak{g}$ or by avoiding the random-initialization regime. Structured initializations, such as identity-block strategies that begin where the ansatz acts trivially, guarantee a large initial gradient; layer-wise training optimizes one layer at a time, exploiting the bounded DLA of each partial ansatz; and problem-informed schedules, such as those derived from adiabatic-path heuristics, similarly steer clear of the flat random regime. Most relevant to this thesis, symmetry and constraint-preserving ansatzes restrict the DLA to a problem-relevant subalgebra by construction, sidestepping the bound entirely.

3.8 The Quantum Approximate Optimization Algorithm

The *Quantum Approximate Optimization Algorithm* (QAOA), introduced by Farhi, Goldstone, and Gutmann [33], is the most studied variational algorithm for combinatorial optimization. Its ansatz can be interpreted as a Trotterized

approximation of the adiabatic interpolation (3.38), in which the continuous schedule is discretized into p layers, each consisting of a cost-driven evolution followed by a mixer-driven evolution, or equivalently as the Hamiltonian Ansatz obtained from the Adiabatic Hamiltonian.

Formally, let H_C be the cost Hamiltonian of the optimization problem and H_M the *mixer Hamiltonian*, whose role is to generate transitions between computational basis states. The QAOA ansatz of depth p is the state

$$|\gamma, \beta\rangle = \prod_{\ell=1}^p e^{-i\beta_\ell H_M} e^{-i\gamma_\ell H_C} |\psi_0\rangle, \quad (3.53)$$

where $\gamma = (\gamma_1, \dots, \gamma_p)$ and $\beta = (\beta_1, \dots, \beta_p)$ are real parameters, and $|\psi_0\rangle$ is the ground state of H_M . For unconstrained binary optimization problems the standard choice is the *transverse-field mixer* $H_M = -\sum_i X_i$, with $|\psi_0\rangle = |+\rangle^{\otimes n}$, which generates single-qubit bit flips independently on each qubit and admits a uniform superposition over all 2^n computational basis states as its ground state. The expectation value

$$F_p(\gamma, \beta) = \langle \gamma, \beta | H_C | \gamma, \beta \rangle \quad (3.54)$$

is minimized over $2p$ parameters by a classical optimizer.

As $p \rightarrow \infty$ the QAOA ansatz can approximate the adiabatic evolution arbitrarily well, so in the infinite-depth limit it inherits the performance guarantees of AQC.

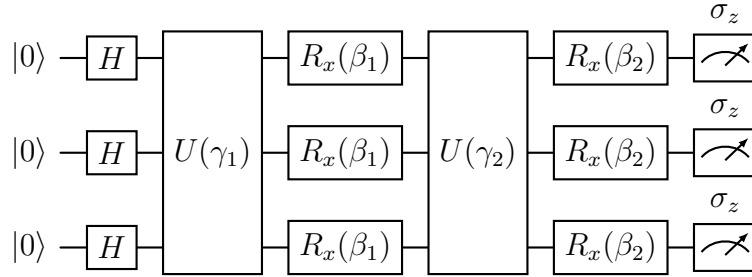


Figure 16: QAOA ansatz with $p = 2$: Hadamard initialisation, alternating cost ($U(\gamma_k)$) and mixer ($R_x(\beta_k)$) layers, then measurement in the computational basis.

Mixer Design and the Quantum Alternating Operator Ansatz

The structure of (3.53) reveals that the mixer Hamiltonian H_M is not a fixed ingredient but a design choice, with consequences for both the feasible region explored and the cost optimization landscape. The *Quantum Alternating Operator Ansatz* [hadfield_quantum_2019] generalizes QAOA by allowing H_M to be any Hermitian operator whose induced evolution is appropriate for the problem at hand. Several design criteria organize the literature on mixer construction.

The most useful conceptual framework for mixer design is the identification of the mixer with a graph on the solution space. Given a mixer Hamiltonian H_M and a target subspace $\mathcal{H}_{\text{feas}}$, define the *mixer graph* $G_M = (\mathcal{F}, E_M)$ whose vertices are the feasible configurations and whose edges connect configurations

3.8. The Quantum Approximate Optimization Algorithm

$z, z' \in \mathcal{F}$ whenever the matrix element $\langle z | H_M | z' \rangle$ is nonzero. The mixer thereby induces a continuous-time quantum walk on G_M : the unitary $e^{-i\beta H_M}$ propagates amplitude along the edges of the mixer graph, with phases and weights set by the matrix elements. Properties of the algorithm translate cleanly into properties of this graph. Constraint preservation requires G_M to be confined to $\mathcal{F}$; reachability requires G_M to be connected; mixing rate is controlled by the spectral gap of H_M restricted to G_M . Designing a good mixer thus reduces to designing a good graph on the feasible set, and the criteria below correspond to specific properties of this graph.

Feasibility. For constrained problems, the mixer should preserve the feasible subspace:

$$[H_M, \Pi_{\text{feas}}] = 0. \quad (3.55)$$

When the initial state $|\psi_0\rangle$ lies in $\mathcal{H}_{\text{feas}}$, condition (3.55) guarantees that the entire evolution remains feasible, eliminating the need for penalty terms.

Connectivity. The mixer must connect feasible configurations to one another. Formally, viewing the feasible subspace as a graph whose vertices are feasible bit strings and whose edges are nonzero matrix elements of H_M between them, connectivity of this graph is necessary for the ansatz to be able to prepare any feasible state from $|\psi_0\rangle$ in the $p \rightarrow \infty$ limit. In Lie-algebraic terms, the *dynamical Lie algebra* $\text{Lie}\{iH_M, iH_C\}$ must act transitively on $\mathcal{H}_{\text{feas}}$.

Implementability. The mixer must be expressible as a circuit of reasonable depth on the target hardware. A mixer that satisfies (3.55) and is connectivity-complete is useless if its Trotterization requires circuits deeper than the coherence budget. This consideration motivates the search for mixers whose terms are individually simple and can be scheduled in parallel.

The constraint-preserving viewpoint was systematized by Hadfield, Wang, O’Gorman, Rieffel, Venturelli, and Biswas [hadfield_quantum_2019] in the *Quantum Alternating Operator Ansatz* framework: the design problem shifts from tuning a penalty coefficient to constructing a mixer whose action is confined to the feasible subspace and whose algebra is nonetheless rich enough to connect every feasible configuration to every other. We discuss the three most prominent constructions in detail.

3.8.1 The X Mixer

The simplest and most widely used mixer for unconstrained binary optimization is the *transverse-field mixer*, or *X mixer*, already encountered as the driver Hamiltonian of the adiabatic algorithm in Section 3.6. Its generator is a sum of single-qubit Pauli- X operators acting independently on each qubit of the register,

$$H_M^X = \sum_{i=1}^n X_i, \quad (3.56)$$

and the corresponding unitary at angle β factorizes as a tensor product of single-qubit rotations,

$$e^{-i\beta H_M^X} = \bigotimes_{i=1}^n R_X(2\beta), \quad (3.57)$$

which is implementable in a single circuit layer with no two-qubit gates and no ancilla qubits. The ground state of H_M^X is the uniform superposition $|\psi_0\rangle = |+\rangle^{\otimes n}$ over all 2^n computational basis states, providing a natural initial state that assigns equal amplitude to every candidate solution.

The mixer graph associated with (3.56) is the n -dimensional *hypercube* Q_n : its vertices are the 2^n bit strings of length n , and two vertices are connected by an edge whenever they differ in exactly one bit. Each generator X_i implements transitions $|z\rangle \leftrightarrow |z \oplus e_i\rangle$ along the i -th coordinate direction, and the full mixer Hamiltonian is the adjacency matrix of Q_n up to an additive constant.

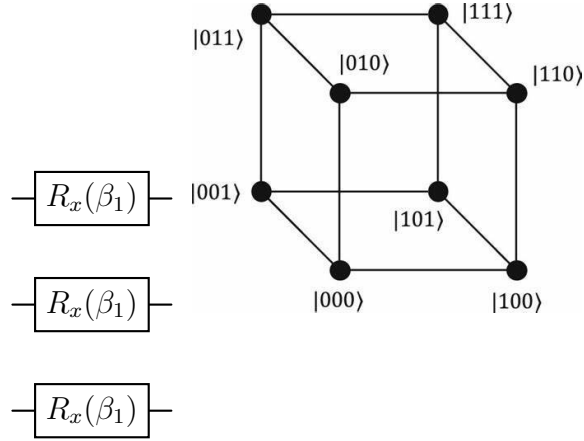


Figure 17: Circuit for one layer of the X mixer, $e^{-i\beta H_M^X} = \bigotimes_{i=1}^n R_X(2\beta)$. The mixer factorizes into independent single-qubit rotations, so a full layer is implemented in constant depth with no two-qubit gates.

The X mixer is the canonical choice for unconstrained QUBO and Ising problems, and the original analytical results of Farhi, Goldstone, and Gutmann [33] on MaxCut were obtained with this mixer at depth one. Its strengths are its simplicity, its parallel implementability, and the symmetry of the induced walk on the hypercube, all of which make it amenable to analytical treatment. Its limitation, when constraints are present, is that the hypercube connects every computational basis state to every other through a sequence of single-bit flips, including transitions that move into infeasible regions of the configuration space. For constrained problems, the X mixer must therefore be combined with a penalty term or replaced by a constraint-preserving alternative, motivating the constructions developed in the remainder of this section.

3.8.2 The XY Mixer

The XY mixer of Wang, Rubin, Dominy, and Rieffel [86] preserves Hamming-weight subspaces and is suited to cardinality and one-hot constraints. Its elementary generators are the two-qubit operators

$$H_{ij}^{XY} = \frac{1}{2}(X_i X_j + Y_i Y_j), \quad (3.58)$$

3.8. The Quantum Approximate Optimization Algorithm

which act on the two-qubit subspace by the swap $|01\rangle \leftrightarrow |10\rangle$ and annihilate $|00\rangle$ and $|11\rangle$. A direct computation shows that $[H_{ij}^{XY}, \sum_\ell Z_\ell] = 0$, so the generator preserves the total Hamming weight of any state on which it acts, and the induced unitary $e^{-i\beta H_{ij}^{XY}}$ maps Hamming-weight- k configurations to Hamming-weight- k configurations.

The full XY mixer Hamiltonian is obtained by summing these generators over an *interaction graph* E_M that specifies which pairs of qubits are coupled:

$$H_M^{XY} = \sum_{(i,j) \in E_M} H_{ij}^{XY}. \quad (3.59)$$

The choice of E_M involves a tradeoff between the connectivity of the induced walk on the feasible subspace and the depth of the implementation circuit. Three canonical choices are common in the literature. The *ring mixer* couples qubits in a one-dimensional cycle and offers minimal gate count per Trotter layer at the cost of slow mixing across the feasible subspace, since amplitude must propagate through all intermediate qubits between any two non-adjacent ones. The *complete-graph mixer* couples every pair and offers maximal connectivity at the cost of gate count quadratic in the number of qubits per constrained variable. The *hardware-matched mixer* chooses E_M to coincide with the native connectivity of the target device, eliminating the need for SWAP overhead in compilation. The XY mixer is particularly well suited to one-hot encodings of k -valued variables, where it preserves the cardinality constraint (3.27) at every step of the algorithm without recourse to penalty terms.

GEOMETRY OF COMPLETE IS JOHNSON GRAPH SHOW CIRCUIT AND EXPLAIN

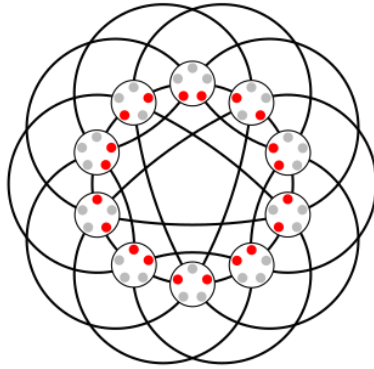


Figure 18: Mixer graph of the complete-graph XY mixer on a one-hot-encoded k -valued variable. Restricted to the Hamming-weight-one (one-hot) subspace, the induced graph is the Johnson graph $J(k, 1)$, the complete graph K_k on the k feasible labels.

3.8.3 The Grover Mixer

The *Grover mixer* of Bärtschi and Eidenbenz [7] adopts a complementary strategy: rather than constructing a mixer whose dynamics preserve feasibility, it delegates feasibility to the initial state preparation and uses a mixer whose effect is to reflect

the state about the uniform feasible superposition. Concretely, letting

$$|\psi_F\rangle = \frac{1}{\sqrt{|\mathcal{F}|}} \sum_{z \in \mathcal{F}} |z\rangle \quad (3.60)$$

denote the uniform superposition over feasible configurations, the Grover mixer is the unitary

$$U_M^{\text{Grover}}(\beta) = e^{-i\beta |\psi_F\rangle\langle\psi_F|} = I + (e^{-i\beta} - 1) |\psi_F\rangle\langle\psi_F|, \quad (3.61)$$

whose action is the identity on the orthogonal complement of $|\psi_F\rangle$ and a phase rotation by β on $|\psi_F\rangle$ itself. Because $|\psi_F\rangle$ is by construction a superposition of feasible basis states, the mixer trivially preserves the feasible subspace.

The cost of constraint preservation in the Grover-mixer approach is shifted from the mixer dynamics to the state-preparation step. Implementing the reflection (3.61) requires a subroutine for preparing $|\psi_F\rangle$, reflecting about it via ancilla-assisted phase kickback, and uncomputing the preparation; the total cost is dominated by the preparation circuit, which depends sensitively on the combinatorial structure of $\mathcal{F}$. For feasible subspaces with efficient sampling oracles, such as Hamming-weight subspaces or independent sets in restricted graph classes, the preparation can be done in polynomial depth; for general $\mathcal{F}$ it can be exponentially expensive.

The mixer graph induced by (3.61) on the feasible subspace is the *complete graph* $K_{|\mathcal{F}|}$: every feasible configuration is connected to every other through a single application of the mixer, since the rank-one projector $|\psi_F\rangle\langle\psi_F|$ has nonzero matrix elements between every pair of feasible basis states.

$$|\psi_F\rangle\langle\psi_F| = \frac{1}{|\mathcal{F}|} \sum_{i,j \in \mathcal{F}} |i\rangle\langle j| \quad (3.62)$$

The induced walk therefore has diameter one, and the connectivity criterion of Section 3.8 is satisfied trivially. The price for this maximal connectivity is paid in the spectral structure: the mixer Hamiltonian has only two distinct eigenvalues (zero on $|\psi_F\rangle^\perp$ and one on $|\psi_F\rangle$), and the induced dynamics provides no mechanism for amplifying low-cost configurations beyond the uniform-rotation scheme of Grover search itself. The Grover-mixer QAOA inherits the asymptotic limitations of amplitude amplification: any black-box use of the cost function through this mixer requires $\Omega(\sqrt{|\mathcal{F}|})$ depth to identify the optimum in the worst case.

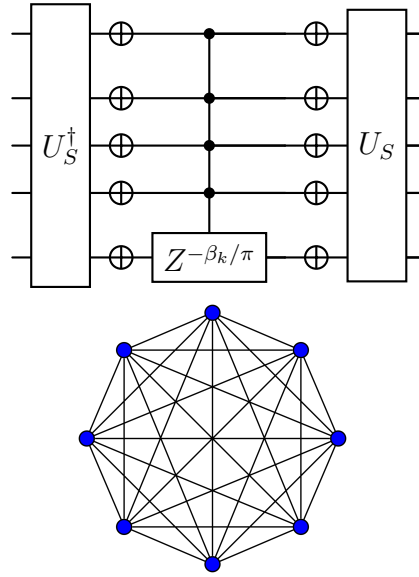


Figure 19: Circuit for the Grover mixer $U_M^{\text{Grover}}(\beta) = e^{-i\beta|\psi_F\rangle\langle\psi_F|}$. The feasible-state preparation U_S (with $U_S|0\rangle = |\psi_F\rangle$) maps the reflection onto a phase on $|0\rangle$, applied by the controlled $Z_{-\beta/\pi}$ rotation, after which $U_S^\dagger$ uncomputes the preparation. The cost of constraint preservation is thus shifted entirely into U_S .

The Grover mixer thus sits at the opposite end of the design spectrum from the X mixer. Where the X mixer is local, shallow, and unaware of constraints, the Grover mixer is fully connected, structurally non-local, and feasibility-aware by construction; where the X mixer induces a walk on the hypercube of all bit strings, the Grover mixer induces a walk on the complete graph of feasible configurations only. The two constructions delimit the extremes of the constraint-handling spectrum, and the natural question is whether intermediate constructions exist that combine the locality and shallow circuits of the X mixer with the feasibility-by-construction guarantee of the Grover mixer. This is precisely the question taken up by the first paper of this thesis, which we summarize next.

3.8.4 Toward the LX Mixer

The X mixer, the XY mixer, and the Grover mixer are three examples in a much larger design space, and they leave open the question of how to construct constraint-preserving mixers for feasible subspaces that do not admit one of these three canonical structures.

The first paper collected in this thesis [fuchs2024] addresses a more general setting by providing a systematic construction of optimal subspace-preserving mixers for arbitrary enumerable feasible subspaces using the stabilizer formalism. The construction of a constraint-preserving mixer therefore reduces to the selection of a minimal generating subset whose induced graph on $\mathcal{H}_{\text{feas}}$ is connected.

The LX mixer construction provides the algebraic foundation for the constraint-preserving algorithms developed in the remainder of this thesis. The constraint-

preserving CTQW of the fourth paper, in particular, can be read as an LX mixer specialized to the vertex-cover stabilizer subspace: the projected Pauli operators $\widehat{X}_i = \Pi_{N(i)} X_i$ introduced in Section 3.1 are exactly the Pauli strings that commute with the vertex-cover feasibility projector and have minimal weight, and their sum is the LX mixer generator for the MVC subspace. The walk-theoretic perspective developed in the next section and the iterative algorithms of Section 3.10 can therefore be viewed as a study of the dynamics induced by a specific LX mixer on a specific stabilizer subspace, with the algebraic structure of the mixer translating into the geometric structure of the feasible configuration graph on which the walk propagates.

Warm-Starting

QAOA and VQE performance depends strongly on initialization, and *warm-starting* incorporates classical information into the quantum starting point so as to inherit classical approximation guarantees while refining them quantum-mechanically. In the scheme of Egger, Mareček, and Woerner [32] for example, the uniform state $|+\rangle^{\otimes n}$ is replaced by a product state biased toward a classically computed fractional solution $x^* \in [0, 1]^n$ of the LP relaxation, $|\psi_0^{\text{ws}}\rangle = \bigotimes_i (\sqrt{1-x_i^*}|0\rangle + \sqrt{x_i^*}|1\rangle)$, with the mixer modified so that this state is its ground state. Depth-0 QAOA then reproduces the classical solution exactly and any added depth can only improve on it, so warm-started QAOA is provably at least as good as the classical algorithm that seeded it, and empirically better at shallow depth. Warm starting has been shown to help even in the penalize approach by preparing as initial state a superposition of feasible solutions for example.

Other Constraint-Handling Approaches

Beyond the mixer and warm-start-based approaches discussed above, several further proposals occupy distinct positions in the design space. The *indicator-function* QAOA (IF-QAOA) framework enforces feasibility by inserting an indicator operator $\chi_{\mathcal{F}}$ into the cost evaluation, effectively projecting the variational state onto the feasible subspace at measurement time without requiring constraint-preserving dynamics [14]. This approach retains the simplicity of transverse-field mixers while guaranteeing that the reported expectation value reflects only feasible configurations; its weakness is that the indicator projection can amplify sampling noise when the feasible subspace is small relative to $\mathcal{H}$. *Filter-based QAOA* variants apply related ideas through mid-circuit measurement or ancilla-assisted projection. At the other extreme, *hard-constraint embeddings* compile the constraint structure directly into the circuit topology, so that infeasible states cannot be represented at all; this is the strategy underlying the quantum tree generator approach of [21], in which the feasible set is traversed by a sequential circuit whose branching structure mirrors a decision-tree enumeration of solutions, and of the constraint-oriented biased quantum search framework of [88], which generalizes the Grover-mixer idea to non-uniform feasible distributions. Each of these approaches trades off differently between circuit depth, state-preparation cost, and sample efficiency,

and the choice among them depends on the constraint structure of the specific problem.

Analytical Results

QAOA admits rigorous analysis at low depth, providing reference points for heuristic methods. The foundational result is the depth-1 analysis of MaxCut on 3-regular graphs by Farhi, Goldstone, and Gutmann [33], who proved an approximation ratio of at least 0.6924, a bound independent of graph size, depending only on local neighborhood structure through the light-cone locality of the depth-1 ansatz. Wurtz and Love extended this to 0.7559 at depth 2 and 0.7924 at depth 3, a monotone hierarchy expected to approach the MaxCut optimum as $p \rightarrow \infty$. Conversely, Bravyi, Kliesch, Koenig, and Tang [12] proved a *negative* result: constant-depth QAOA cannot beat the Goemans–Williamson bound on high-girth graphs, a structural limitation that motivates the recursive variants discussed later. For the Grover mixer, Bärtzsch and Eidenbenz [7] established a depth-1 lower bound of $(f_{\min} + (|\mathcal{F}| - 1)\bar{f})/|\mathcal{F}|$, interpolating between the random-feasible expectation $\bar{f}$ and the optimum $f_{\min}$ on every instance; on the upper-bound side it inherits the $\Omega(\sqrt{|\mathcal{F}|})$ worst-case depth of amplitude amplification. These bounds together delimit a band within which Grover-mixer performance is constrained, with the actual value set by the structure of the cost function on $\mathcal{F}$.

3.9 Quantum Walks

A quantum walk is the unitary analogue of a classical random walk: an evolution on a graph whose vertices are basis states and whose edges carry nonzero transition amplitudes [2, 34]. The two frameworks share a common setting but differ at the level of dynamics: where classical probabilities can only add, quantum amplitudes can interfere. This interference is the source of the most rigorous quantum speedups known from walk dynamics, and the framework is central to the fourth paper of this thesis, which reinterprets a subspace-preserving mixer as a continuous-time quantum walk on a feasible configuration graph.

3.9.1 Continuous-Time Quantum Walks

The *continuous-time quantum walk* (CTQW) on a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is the evolution generated by a Hamiltonian $H_{\mathcal{G}}$ whose matrix elements in the vertex basis $\{|v\rangle\}$ encode the adjacency structure,

$$\langle u | H_{\mathcal{G}} | v \rangle \neq 0 \iff (u, v) \in \mathcal{E}, \quad (3.63)$$

the canonical choices being the adjacency matrix A or the graph Laplacian $L = D - A$. A state evolves as $|\psi(t)\rangle = e^{-iH_{\mathcal{G}}t}|\psi(0)\rangle$. The contrast with the classical walk is clearest in the path-sum expansion of the propagator,

$$\langle u | e^{-iH_{\mathcal{G}}t} | v \rangle = \sum_{m=0}^{\infty} \frac{(-it)^m}{m!} N_m(v, u), \quad (3.64)$$

where $N_m(v, u)$ counts walks of length m from v to u . The classical walk weights each such walk by a positive probability and the contributions add; the quantum walk weights it by a phase and the contributions interfere.

The simplest illustration is the walk on the infinite line. A classical random walker spreads diffusively, with variance growing as $\langle x^2 \rangle \sim t$, so that the typical displacement scales as $\sqrt{t}$. The corresponding CTQW spreads *ballistically*, with $\langle x^2 \rangle \sim t^2$ and a typical displacement linear in t : constructive interference along the leading edge produces a wavefront that propagates at constant group velocity, exactly as a free quantum particle would. This quadratic gap in spreading rate is the prototype of every walk-based speedup, and the same ballistic signature reappears in the layered feasible-state walk analyzed in the fourth paper. On structured graph families the separation can be far sharper: Childs et al. [20] exhibited a glued-trees graph on which the CTQW crosses in polynomial time while any classical algorithm requires exponential time, and Childs and Goldstone [19] established polynomial speedups for spatial search on lattices in dimension $d > 4$.

One structural feature of CTQWs is used directly in this thesis. When $\mathcal{G}$ is bipartite, with bipartition $\mathcal{V} = \mathcal{V}_0 \cup \mathcal{V}_1$, the walk counts $N_m(v, u)$ vanish unless m has the same parity as the graph distance $d(v, u)$. The path sum (3.64) then collapses to a fixed-parity series,

$$\langle u | e^{-iH_{\mathcal{G}}t} | v \rangle = (-i)^{d(v,u)} \sum_{n=0}^{\infty} \frac{(-1)^n t^{d(v,u)+2n}}{(d(v,u) + 2n)!} N_{d(v,u)+2n}(v, u), \quad (3.65)$$

so that the amplitude on a vertex at distance d is governed only by walk counts of lengths $d, d+2, d+4, \dots$. The feasible configuration graph of the fourth paper is bipartite with respect to Hamming-weight parity, and this parity-selected expansion is the technical engine behind its layer-resolved probability analysis.

The observation that ties this section to the mixer framework of Section 3.8 is that a constraint-preserving evolution $U(\alpha) = e^{-i\alpha H_M}$ with a subspace-preserving mixer H_M is *itself* a CTQW, not on the original problem graph G , but on the induced *feasible configuration graph* G_{feas} , whose vertices are the feasible bit strings and whose edges connect configurations related by an allowed mixer move. The mixer is the walk generator, and the feasibility structure is encoded in the walk geometry.

3.9.2 Discrete-Time Quantum Walks

The *discrete-time quantum walk* (DTQW) replaces continuous evolution with alternating *coin* and *shift* operations. The walker occupies an extended space $\mathcal{H}_{\text{walker}} \otimes \mathcal{H}_{\text{coin}}$, and each step is the unitary $W = S \cdot (I \otimes C)$, where C acts on the coin register and S translates the walker conditioned on the coin state. The coin provides an additional design degree of freedom, and many walk-based algorithms are most naturally formulated in this framework.

The general bridge from classical to quantum walks is the construction of Szegedy [82], which maps any reversible Markov chain P to a quantum walk $W(P)$ whose spectral gap is the square root of the classical one, $\Delta(W(P)) = \Theta(\sqrt{\Delta(P)})$. Since hitting times scale inversely with the gap, this amplification yields a

quadratic speedup for any algorithm dominated by mixing or hitting time, and it underlies most near-quadratic walk speedups in the literature. The same structure appears in Grover’s algorithm [44], which is a DTQW on the complete graph whose coin reflects about the marked vertex; the Grover mixer of Section 3.8 inherits exactly this form, clarifying why Grover-mixer QAOA shares the limitations of unstructured search.

3.9.3 Walk-Based Speedups and Heuristics for Optimization

Several rigorous quantum speedups for optimization follow a uniform recipe: identify a classical search procedure with a random walk on an appropriate search graph, then replace it with a quantum walk to obtain a near-quadratic improvement. Montanaro [72] established such a speedup over classical branch-and-bound by encoding the search tree as a walk and applying amplitude amplification; Chakrabarti et al. [17] extended this to branch-and-cut and to the broad class of tree-search heuristics that dominate modern combinatorial solvers. Related techniques have been applied to 0–1 knapsack via quantum tree generators [21], in which the feasible set is built in superposition by walking a constraint-respecting decision tree, and the biased-search framework of Wilkening [88] generalizes the Grover-mixer idea to non-uniform feasible distributions.

A complementary line of work, closer in spirit to this thesis, uses short-time CTQW evolution as a source of *structural signal* inside a classical reduction loop rather than as a route to worst-case speedups. Luiz et al. [67] introduced a scalable walk-based heuristic for Minimum Vertex Cover in which the graph Laplacian of G generates a CTQW from $|+\rangle^{\otimes n}$; per-vertex marginals extracted by single-qubit measurements then drive a greedy reduction that fixes vertices and prunes neighbors, recursing on the residual graph. This anticipates two features of the fourth paper: the use of CTQW dynamics to convert global structure into local selection scores, and the embedding of short-time evolution in a recursive classical reduction. The methods differ in where feasibility lives. The heuristic of [67] walks on the *problem* graph and recovers feasibility through classical post-processing, so its state has support on every bit string. The method developed in this thesis walks on the *feasible configuration* graph: the generator $H_{\text{MVC}} = \sum_i \hat{X}_i$ is built from projected Pauli operators whose support is intrinsically confined to the vertex-cover subspace, so the walk propagates amplitude only among feasible configurations.

3.10 Iterative and Recursive Quantum Algorithms

Many of the most successful classical heuristics for combinatorial optimization are greedy: rather than constructing a full solution in one step, they build it incrementally, committing a single variable at each round according to a locally computed score and recursing on the residual instance. Iterative and recursive quantum algorithms can be understood as a transposition of this greedy paradigm into the quantum setting: the role of the classical scoring heuristic is played by a quantum subroutine, which prepares a state encoding the problem structure and

returns a measured observable that scores the available decisions. Under this view the quantum processor is not asked to output a solution. It is asked to output structural information: one piece of evidence sufficient to commit one variable correctly. A classical reduction step fixes a variable, reduce the dimension of the instance, and the procedure recurses on the smaller residual problem. The total quantum resource is thereby distributed across many shallow circuits, each invoked as a structural primitive, rather than concentrated in a single circuit tasked with the entire optimization. The premise is precisely that of a greedy algorithm: even where a single subroutine cannot solve the instance, it can still rank decisions reliably enough to commit one variable, and iterating this turns a sequence of shallow circuits into a complete algorithm. This quantum greedy approaches are called in literature Iterative Quantum Algorithms (IQA) framework of Brady and Hadfield [11], under which the methods discussed below are most naturally described.

3.10.1 Recursive QAOA

Recursive QAOA (RQAOA), introduced by Bravyi et al. [12], is the first example of iterative quantum algorithm proposed in literature. In the original paper the formulation is specific for MaxCut and it can be described as follows:

1. Run a shallow QAOA on the current problem instance and measure the two-point correlators $\langle Z_i Z_j \rangle$ on every edge (i, j) .
2. Select the edge of largest correlator magnitude and impose the corresponding hard constraint: $x_i = x_j$ if the correlator is positive, $x_i \neq x_j$ if negative.
3. Substitute this relation into the cost function, contracting the two variables into one and producing an instance with one fewer variable.
4. Repeat on the contracted instance until it is small enough to solve by direct enumeration, then back-substitute to recover the full assignment.

Bravyi et al. [12] showed that, at constant depth, RQAOA provably outperforms standard QAOA on high-girth D -regular graphs, where the depth- p light cone caps single-shot QAOA but the recursive contraction propagates information from the full graph. The effective depth of the algorithm thus scales with the iteration count rather than the circuit depth. A subsequent insight is that the contraction rule of step 2 is itself a design choice—thresholding correlators, fixing several per round, or weighting by edge betweenness are all viable—and the optimal rule is problem-dependent. The fourth paper of this thesis contributes three further selection rules (marginal probabilities, local energies, and echo fidelities) to this catalogue.

3.10.2 Quantum-Informed Recursive Optimization

Quantum-Informed Recursive Optimization (QIRO), developed by Finžgar et al. [36], generalizes RQAOA by letting the reduction step exploit richer information than a single correlator. The expectation values of a QAOA state form a structured

data set, vertex marginals $\langle Z_i \rangle$, edge correlators $\langle Z_i Z_j \rangle$, higher-order correlators, and conditional values such as $\langle Z_i Z_j \mid Z_k = +1 \rangle$, and QIRO uses problem-specific rules that combine several of these to fix each variable. The trade-off is characteristic: QIRO exploits problem structure that pure RQAOA cannot.

3.10.3 A General Framework

Brady and Hadfield [11] unified RQAOA, QIRO, and related methods under the heading of *Iterative Quantum Algorithms* (IQA), providing the vocabulary in which the algorithms of the fourth paper are most naturally described. The framework decouples three design choices that are entangled in any single instantiation:

- *The quantum primitive*: which state the subroutine prepares, and hence which observables are accessible. Standard RQAOA uses the QAOA ansatz with transverse-field mixer; the fourth paper uses a constraint-preserving CTQW.
- *The measurement protocol*: which observables are extracted and at what cost. Vertex marginals cost $\mathcal{O}(n)$ per state preparation, edge correlators $\mathcal{O}(|E|)$, and conditional values $\mathcal{O}(n^2)$ or more, with correspondingly different noise sensitivities.
- *The classical reduction rule*: which variable is fixed, and to what value, given the measured observables.

The fourth paper can be described in this framework with the following choices: the primitive is a constraint-preserving CTQW on the feasible MVC subspace, the protocol extracts per-vertex scores of three types (probabilities, energies, fidelities), and the reduction rules implement largest- and smallest-degree-first selection in the quantum-score space.

Papers

Paper I

LX-Mixers for QAOA: Optimal Mixers Restricted to Subspaces and the Stabilizer Formalism

Franz G. Fuchs
Ruben Pariente Bassa

Abstract: We present a novel formalism to both understand and construct mixers that preserve a given subspace. The method connects and utilizes the stabilizer formalism that is used in error correcting codes. This can be useful in the setting where the quantum approximate optimization algorithm (QAOA), a popular meta-heuristic for solving combinatorial optimization problems, is applied in the case where the constraints of the problem lead to a feasible subspace that is large but easy to specify. The proposed method gives a systematic way to construct mixers that are resource efficient in the number of controlled-not gates and can be understood as a generalization of the well-known X- and XY-mixers and a relaxation of the Grover mixer: given a basis of any subspace, a resource-efficient mixer can be constructed that preserves the subspace. We call our approach logical X-mixer or logical X QAOA (**LX-QAOA**), since it can be understood as dividing the subspace into code spaces of stabilizers S and consecutively applying logical rotational X gates associated with these code spaces.

Published in Quantum **8**, 1535 (2024)

LX-mixers for QAOA: Optimal mixers restricted to subspaces and the stabilizer formalism

Franz G. Fuchs^{1,2} and Ruben Pariente Bassa¹

¹SINTEF AS, Department of Mathematics and Cybernetics, Oslo, Norway

²Department of Mathematics, University of Oslo, Norway

14.11.2024

We present a novel formalism to both understand and construct mixers that preserve a given subspace. The method connects and utilizes the stabilizer formalism that is used in error correcting codes. This can be useful in the setting when the quantum approximate optimization algorithm (QAOA), a popular meta-heuristic for solving combinatorial optimization problems, is applied in the setting where the constraints of the problem lead to a feasible subspace that is large but easy to specify. The proposed method gives a systematic way to construct mixers that are resource efficient in the number of controlled not gates and can be understood as a generalization of the well-known X- and XY-mixers and a relaxation of the Grover mixer: Given a basis of any subspace, a resource efficient mixer can be constructed that preserves the subspace. The numerical examples provided show a dramatic reduction of CX gates when compared to previous results. We call our approach logical X-Mixer or logical X QAOA (**LX-QAOA**), since it can be understood as dividing the subspace into code spaces of stabilizers S and consecutively applying logical rotational X gates associated with these code spaces. Overall, we hope that this new perspective can lead to further insight into the development of quantum algorithms.

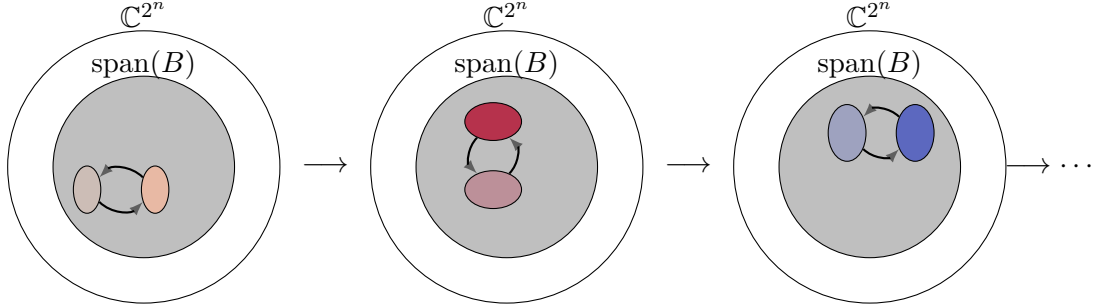


Figure 1: Schematic view of the proposed method to construct efficient mixers for the quantum alternating operator ansatz (QAOA) constrained to a subspace. The method can be understood as dividing the subspace into code spaces of stabilizers S and consecutively applying logical rotational X gates associated with the code spaces.

1 Introduction

The quantum approximate optimization algorithm (QAOA) [7, 10] is a popular meta-heuristic for solving combinatorial optimization problems in the form of a quadratic (unconstrained) binary optimization (QUBO) problem. There is both evidence that points towards an advantage over classical algorithms [4] as well as against it [19], particularly in the NISQ era. Given an objective function $f : \{0, 1\}^n \rightarrow \mathbb{R}$ one defines the problem Hamiltonian through $H_P |x\rangle = f(x) |x\rangle$. Consequently, ground states of H_P minimize the objective function. The general form of QAOA is to prepare a parametrized quantum state given by

$$|\gamma, \beta\rangle = U_M(\beta_p) U_P(\gamma_p) \cdots U_M(\beta_1) U_P(\gamma_1) |\phi_0\rangle. \quad (1)$$

That is, starting with an initial state $|\phi_0\rangle$ one alternates the application of a phase separating operator $U_P(\gamma)$ and mixing operator $U_M(\beta)$. The goal of QAOA is then to find optimal parameters $\gamma, \beta \in \mathbb{R}$, such that $\langle \gamma, \beta | H_P | \gamma, \beta \rangle$ is minimized. Several extensions and variants of

QAOA have been proposed since its inception. One such variant is ADAPT-QAOA [23], which is an iterative version tailored to specific hardware constraints. Another variant is R-QAOA [3], which recursively removes variables from the Hamiltonian until the remaining instance can be solved classically. A third variant is WS-QAOA [6], which takes into account solutions of classical algorithms to warm-start QAOA.

In this article we consider the case where solutions are constrained to a feasible subspace $\text{span}(B) \subset \mathcal{H}$ of the full Hilbert space $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$. The subspace is described by a subset of all computational basis states, i.e.,

$$B = \{|z_j\rangle, \quad 1 \leq j \leq J, \quad z_j \in \{0,1\}^n\}. \quad (2)$$

There are multiple ways to take constraints into account with QAOA. One popular approach is to penalize unsatisfied constraints in the objective function by formulating them as another QUBO. Although it is straightforward to define a phase separating Hamiltonian for the resulting QUBO, this approach has some downsides. In particular, the probability of obtaining infeasible solutions is typically rather high, which reduces efficiency. Additionally, the quality of the results is very sensitive to the penalization weight, which must be carefully balanced between optimality and feasibility [22]. This can be a challenging task since the optimal weight depends on the specific problem and there is a trade-off between optimality and feasibility in the objective function. Applications of this approach include graph coloring problems, the traveling salesperson problem [11], the Max-k-cut problem [8], portfolio optimization [2], and the tail-assignment problem [21]. Other approaches have been proposed recently that do not rely on the penalization of the cost function but for example on postprocessing the unfeasible states [17]. Solving combinatorial optimizations problems with hard constraints can also be done by combining the quantum variational framework with a genetic algorithm for multiobjective optimization [5]. Another alternative is to perform the optimization considering only in-constraint samples obtained from a non-constraint preserving ansatz [12]. Finally, quantum Zeno dynamics can be employed to restrict the wave function to the feasible subspace via repeated projective measurements using ancillary qubits [14].

The approach presented in this work to model constraints with QAOA is to design mixers that have zero probability to transition from a feasible state to an infeasible one, thereby rendering the penalization weight from the previous approach unnecessary. However, creating mixers that account for constraints is generally more difficult. In this work, we define the mixer for feasible states using logical operations, drawing an analogy to quantum error correction. This approach allows us to create a structure termed the logical X-Mixer (**LX-Mixer**). The schematic representation of our construction is illustrated in Figure 1, where each logical X rotation facilitates mixing exclusively between codespaces within the feasible subspace. The XY-mixer [11, 22] is a well-known example in the literature which confines the evolution to the subspace of “k-hot” states, i.e., subspace spanned by computational basis states with exactly k entries equal to one. This mixer, which can be described in terms of logical X operations, has been successfully implemented for constrained optimization problems on real quantum devices [16].

One notable example is GM-QAOA [1] which uses an ansatz of the form in Equation (5) with $T_{j,k} = 1, \forall j, k$, i.e. it is special case of our approach, providing all-to-all mixing of states. An implementation can be based on the Grover-like form $U_M(\beta) = U_S (I + (e^{-i\beta} - 1) |0\rangle\langle 0|) U_S^\dagger$, where U_S is a circuit creating equal superposition of all feasible states, and $I + (e^{-i\beta} - 1) |0\rangle\langle 0|$ is implemented with a rotational gate with $n - 1$ controls. A similar mixer construction, based on continuous quantum walks, that provides mixing for cases of circulant adjacency matrices T_{jk} is proposed in [15]. However, it might be advantageous to use the full flexibility of choosing T to create valid mixers. In general, a mixer is called valid if the following conditions hold [10].

Definition 1 (Valid mixer)

A mixer is called valid if it preserves the feasible subspace, i.e.,

$$U_M(\beta) |v\rangle \in \text{span}(B), \quad \forall |v\rangle \in \text{span}(B), \forall \beta \in \mathbb{R}, \quad (3)$$

and if it provides transitions between all pairs of feasible states, i.e., for each pair of computational basis states $|x\rangle, |y\rangle \in B$ there exist $\beta^* \in \mathbb{R}$ and $r \in \mathbb{N} \cup \{0\}$, such that

$$|\langle x | \underbrace{U_M(\beta^*) \cdots U_M(\beta^*)}_{r \text{ times}} | y \rangle| > 0. \quad (4)$$

However, in this article we choose a different ansatz of the form [9]:

$$U_M(\beta) = e^{-i\beta H_M}, \quad H_M = \sum_{j < k} (T)_{j,k} H_{z_j \leftrightarrow z_k}, \quad H_{z_j \leftrightarrow z_k} = |z_j\rangle\langle z_k| + |z_k\rangle\langle z_j|. \quad (5)$$

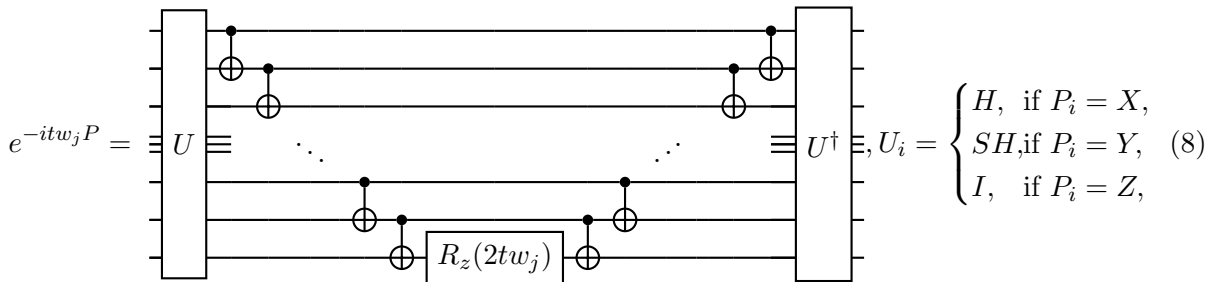
The space of bounded linear operators $\mathcal{B}(\mathcal{H})$ is endowed with the Hilbert-Schmidt inner product $(A, B) = \text{tr}(A^\dagger B)$ [18]. The operator H_M in Equation (5) is expressed on the standard/computational basis $\{|i\rangle\langle j|\}$, $i, j = 0, \dots, 2^n - 1$. In order to allow for a practical implementation we express H_M in the Pauli basis $P_j \in \{I, X, Y, Z\}^n$, which can be achieved via

$$H_M = \sum_j w_j P_j, \quad w_j = \frac{1}{2^n} \text{tr}(P_j^\dagger H_M) \in \mathbb{R}. \quad (6)$$

After using a standard Trotterization scheme [13, 20] (which is exact for commuting Pauli-strings),

$$U_M(t) = e^{-itH_M} \approx \prod_{\substack{j \\ |w_j| > 0}} e^{-itw_j P_j}, \quad (7)$$

each term with non-zero weight can be implemented using a standard circuit consisting of $2(\text{weight}(P_j) - 1)$ controlled not gates CX , see Equation (8). Lower depth variants to realize the circuit are possible, but the number of CX gates is the same. Here, $0 \leq \text{weight}(P_j) \leq n$ is the weight of a Pauli string P_j , which is the number of qubits that P_j acts on by a nontrivial Pauli matrix, i.e. X, Y or Z .



where S is the phase gate and H is the Hadamard gate. Hence, a good indicator for the cost of a quantum algorithm is given by the number of required CX gates, defined as

$$\text{Cost}(U_M) = \text{Cost}(H_M) = \sum_{\substack{P_j \neq I \\ |w_j| \neq 0}} 2(\text{weight}(P_j) - 1). \quad (9)$$

In this article we develop algorithms to find constraint preserving mixers that minimize the number of CX gates required. Given B there is a generally a lot of design freedom for U_M to be a valid mixer, which can heavily impact the cost. There are many matrices T that lead to valid mixers. In Section 2 we discuss how an optimal strategy to obtain the lowest cost is related to solving a graph optimization problem. In Section 3 we make a connection between mixers and the stabilizer formalism. This can be utilized to a potentially large reduction of the cost. A further reduction of the cost comes when taking into account that the operators only have to act on the feasible subspace, see Section 4. Finally, we present examples of our approach in Section 5.

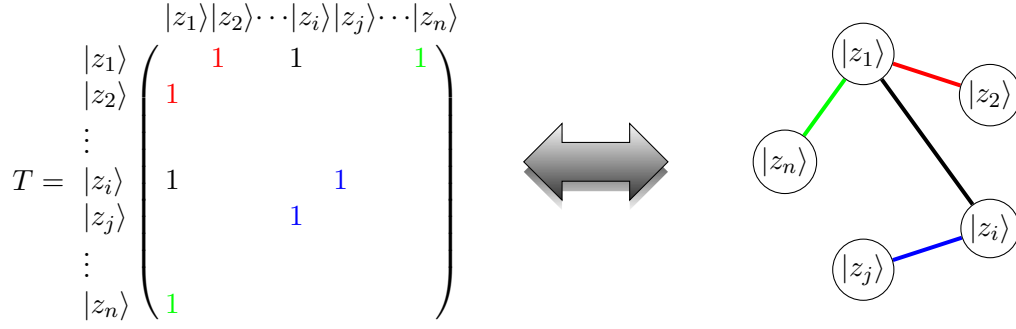


Figure 2: The matrix T in Equation (5) can be interpreted as the adjacency matrix of a graph G_T and vice versa. For a mixer to be valid the graph must be connected.

2 Optimal Trotterization and the adjacency matrix

Mixers of the form given in Equation (5) are valid if for $T \in \mathbb{R}^{J \times J}$ we have that T is symmetric and $\forall 1 \leq j, k \leq J, \exists r_{j,k} \in \mathbb{N} \cup \{0\}$, s.t. $(T^{r_{j,k}})_{j,k} \neq 0$ [9, Theorem 1]. This condition is not very intuitive, is cumbersome to check and it is not clear how one would practically find optimal mixers using it. In this section we will describe a practical way to construct a valid Trotterization along with a condition of optimality.

2.1 Valid Trotterization

The matrix T in Equation (5) can be interpreted as the adjacency matrix of a graph G_T , see Figure 2. A mixer provides transitions between all pairs of feasible states if G_T is a connected graph.

Theorem 1 (Valid mixer)

The mixer Hamiltonian H_M defined in Equation (5) is valid according to Definition 1 if the graph G_T of the adjacency matrix T is undirected and connected.

Proof: We show equivalence with [9, Theorem 1]. By definition, T is symmetric for an undirected graph. If T is the adjacency matrix of the undirected graph G_T , then the entry $(T^k)_{i,j}$ gives the number of undirected walks of length k from vertex i to vertex j . Hence, the non-zero entries of T^k gives all pairs of nodes that are connected by a path of length k . From this we see that the conditions in [9, Theorem 1] are fulfilled if T is a connected graph. ■

In general, the terms in Equation (5) will not commute and one needs to perform a Trotterization by decomposing T into a sum of symmetric matrices T_q , where for each q all non-zero Pauli-operators of $\sum_{j < k} (T_q)_{j,k} H_{z_j \leftrightarrow z_k}$ have to commute. In particular, a Trotterized mixer has to fulfill that for all $1 \leq j, k \leq |J|$ there exist $r_m \in \mathbb{N} \cup \{0\}$ (possibly depending on the pair) such that $\left(\prod_{q \in Q}^M T_q^{r_m} \right)_{j,k} \neq 0$ [9, Theorem 2]. From the proof of Theorem 1 it follows that this condition is equivalent to the following.

Corollary 1 (Valid Trotterized mixer)

The mixer Hamiltonian H_M defined in Equation (5) is valid according to Definition 1 if the undirected graph $G_T = \cup_q G_{T_q}$ is connected and for each q all non-zero Pauli strings of the mixer $\sum_{j < k} (T_q)_{j,k} H_{z_j \leftrightarrow z_k}$ commute.

It is straight forward to check if a graph is connected. What is still missing is a way to find a family of graphs where all non-zero Pauli strings commute.

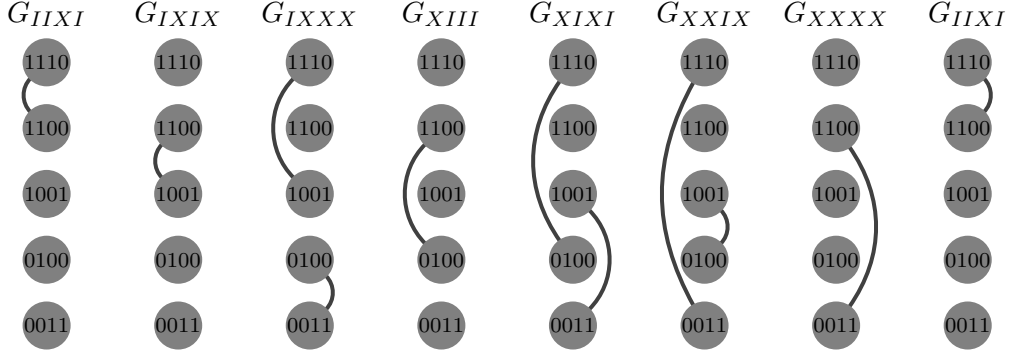


Figure 3: Family of graphs for $B = \{|1110\rangle, |1100\rangle, |1001\rangle, |0100\rangle, |0011\rangle\}$. A graph has an edge between two vertices v_1, v_2 if $\bar{X}|v_1\rangle = |v_2\rangle$. Each graph has edges for the same operator $\bar{X}$, which ensures that the Pauli-strings of the associated Hamiltonian all commute, see Equation (13).

2.2 Optimal Trotterization

How one can one identify all pairs of B such that the corresponding operators $H_{v_k \leftrightarrow v_l}$ all commute? We start by defining a *logical X* operator $\bar{X}$ of a pair $|x\rangle \neq |y\rangle \in B$ through

$$\bar{X}_i = \begin{cases} X, & \text{if } x_i \neq y_i \\ I, & \text{otherwise.} \end{cases} \quad (10)$$

The reason why we call it "logical" will become apparent when we make the connection to the stabilizer formalism in Section 3. For all $\bar{X} \in \{I, X\}^n \setminus \{I\}^n$ we define two sets: the set of all states in B that are connected by applying a particular $\bar{X}$, and pairs of all states that are connected, i.e.,

$$\begin{aligned} V_{\bar{X}} &= \{ |x\rangle \in B \mid \exists |y\rangle \in B \text{ s.t. } \bar{X}|x\rangle = |y\rangle \}, \\ E_{\bar{X}} &= \{ \{|x_i\rangle, |y_i\rangle\} \in B \times B \mid \bar{X}|x\rangle = |y\rangle \}. \end{aligned} \quad (11)$$

Note that $E_{\bar{X}}$ is a set of *unordered* pairs. For the example given in Figure 3 we have $V_{IXXX} = \{|1110\rangle, |1001\rangle, |0100\rangle, |0011\rangle\}$ and $E_{IXXX} = \{ \{|1110\rangle, |1001\rangle\}, \{|0100\rangle, |0011\rangle\} \}$.

Definition 2 (Family of valid Mixers)

Given a feasible set B we define the family of graphs $(G_{\bar{X}})_{\bar{X} \in \{I, X\}^n \setminus \{I\}^n}$ where $G_{\bar{X}} = (B, E_{\bar{X}})$. This gives rise to a family of mixers $(H_{\bar{X}})_{\bar{X} \in \{I, X\}^n \setminus \{I\}^n}$

$$H_{\bar{X}} = \sum_{j < k} (T_{\bar{X}})_{j,k} H_{z_j \leftrightarrow z_k} = \sum_{\{|x\rangle, |y\rangle\} \in E_{\bar{X}}} H_{x \leftrightarrow y}, \quad (12)$$

where $T_{\bar{X}}$ is the adjacency matrix of the graph $G_{\bar{X}}$. For each $\bar{X}$ all non-vanishing Pauli-strings of the mixer $H_{\bar{X}}$ commute, see Corollary 2.

For each logical X operators we can write

$$\sum_{\{|x\rangle, |y\rangle\} \in E_{\bar{X}}} H_{x \leftrightarrow y} = \sum_{\{|x\rangle, |y\rangle\} \in E_{\bar{X}}} (|x\rangle \langle y| + |y\rangle \langle x|) = \bar{X} \sum_{|x\rangle \in V_{\bar{X}}} |x\rangle \langle x| = \bar{X} \prod_{\text{span}(V_{\bar{X}})}. \quad (13)$$

The action of the Hamiltonian can therefore be understood as a projection onto a feasible subspace followed by a logical X operation. Furthermore, observe that for any computational basis state $|x\rangle$ the operator $|x\rangle \langle x|$ has basis in $\{I, Z\}^n$.

All non-empty sets $E_{\bar{X}}$ can be identified in time $\mathcal{O}(|B|^2)$, by going through all pairs of states in B and grouping them together according to the logical X operator between them. An example of such a set of graphs is given in Figure 3.

Definition 3 (Optimal Trotterization)

Let $(G_i)_i$ a set of graphs that contains all graphs according to Definition 2, together with all possible subgraphs. For each graph G_i we define the cost $c_i = \sum_{\{|x\rangle, |y\rangle\} \in E_i} \text{Cost}(H_{x \leftrightarrow y})$. An optimal mixer is given by $\text{argmin}\{c^T x \mid G = (B, \bigcup_{i|x_i=1} E_i) \text{ is a connected graph}\}$.

In the next section we describe a way to efficiently calculate the mixer in the Pauli basis and a way to potentially reduce the cost of the projection operator by restricting its action to the feasible subspace B .

3 Stabilizer formalism to construct mixers

In this section we show that mixers have a useful connection to the stabilizer formalism. This becomes very convenient since it allows us to work with generating sets, instead of all elements of the group, meaning we can work with exponentially fewer elements.

3.1 Stabilizer formalism for a single pair of states

For $|x\rangle \neq |y\rangle \in B$ one can interpret $C = \text{span}(|x\rangle, |y\rangle)$ as the code space of a stabilizer S . The stabilizer is defined as the commutative subgroup of $\mathcal{P}_n$, such that $S = \{g \in \mathcal{P}_n \mid g|z\rangle = |z\rangle \ \forall |z\rangle \in C\}$. Here, $\mathcal{P}_n$ is the Pauli group on n qubits. Because $\dim(C) = 2$ the stabilizer has a minimal generating set of $n - 1$ elements, i.e., $S = \langle g_1, \dots, g_{n-1} \rangle$. The centralizer $\mathcal{C}$ of the stabilizer group S in $\mathcal{P}_n$ is the set of operators such that $\mathcal{C} = \{p \in \mathcal{P}_n \mid ps = sp \ \forall s \in S\}$. The normalizer $\mathcal{N}$ of a stabilizer group consists of the set of operators such that $\mathcal{N} = \{p \in \mathcal{P}_n \mid psp^\dagger \in S \ \forall s \in S\}$. Due to the fact that the stabilizer group does not contain $-I$, it turns out that the centralizer is equal to the normalizer. Since B consists of only computational basis states, i.e., strings of zeros and ones, it is very easy to find a generator for the stabilizer through the $\mathcal{O}(n)$ algorithm 1. As an example, for the pair $|x\rangle = |10010\rangle, |y\rangle = |01011\rangle$ a minimal generating set is given by $S = \langle Z_3, -Z_4, -Z_1Z_2, Z_2Z_5 \rangle$. The sum of all elements of the stabilizer group acts as a projector onto the code subspace.

Proposition 1 (Stabilizer subspace projector)

The stabilizer subspace projector $\Pi_S := \text{Proj}(C)$ given by

$$\Pi_{C(S)} = \frac{1}{|S|} \sum_{g \in S} g, \quad (14)$$

is the projector onto the code space C .

Proof: Observe that $g \Pi_S = \Pi_S \forall g \in S$ since it acts on S by permuting its elements. Therefore $\Pi_S^2 = \Pi_S$. Hence Π_S is a projector and has eigenvalues 0 or 1. It is also a projector onto the code space, i.e., $\text{Im}(\Pi_S) = C$. Since $g \Pi_S = \Pi_S \forall g \in S$ we have that $\text{Im}(\Pi_S) \subseteq C$. Furthermore, if $|\phi\rangle \in C$, then $\Pi_S |\phi\rangle = |\phi\rangle$ and therefore $C \subseteq \text{Im}(\Pi_S)$ ■

For the above example, i.e., the pair $|x\rangle = |10010\rangle, |y\rangle = |01011\rangle$, we have that $\bar{X} = X_1X_2X_5$. However, there are more logical X operators, which are all elements of $N \setminus S$ where N is the normalizer of S . All logical X operators are given by the set $\bar{X}S = \{\bar{X}g \mid g \in S\}$. Indeed, for any $\hat{X} \in \bar{X}S$ we have that $\hat{X}|x\rangle = \bar{X}g|x\rangle = \bar{X}|x\rangle = |y\rangle$.

Now we have all we need to show that $H_{x \leftrightarrow y}$ consists of all logical X operators in the following sense.

Theorem 2 (Mixer as sum of logical X operators)

Let $x \neq y \in B$, S be the stabilizer of the code space $C = \text{span}(|x\rangle, |y\rangle)$, and $\bar{X}$ be as defined

in Equation (10). Then for $H_{x \leftrightarrow y} = |x\rangle \langle y| + |y\rangle \langle x|$ we have that

$$H_{x \leftrightarrow y} = \frac{1}{|S|} \sum_{\widehat{X} \in \overline{X}S} \widehat{X}. \quad (15)$$

This means $H_{x \leftrightarrow y}$ is the sum of all logical X operators.

Proof: To show equality of the linear operators, it is enough to show equality for all computational basis states. We start by observing that

$$\frac{1}{|S|} \sum_{\widehat{X} \in \overline{X}S} \widehat{X} = \frac{1}{|S|} \sum_{g \in S} \overline{X}g = \overline{X} \prod_{C(S)}, \quad (16)$$

which means that the action of the right hand side in equation (15) is a projection into the code space followed by a logical X operation. Both sides in equation (15) are zero for $|\psi\rangle \in (\mathbb{C}^2)^{\otimes n} \setminus C$. Furthermore, we have that $\overline{X} \prod_S |x\rangle = \overline{X} |x\rangle = |y\rangle$ and equivalently maps $|y\rangle$ to $|x\rangle$. Hence, also in this case both sides in equation (15) are equal, which proves the assertion. ■

Corollary 2 (All non-vanishing Pauli strings commute)

Theorem 3 in [8] now follows as a simple consequence by the construction via the stabilizer formalism. From equation (15) we can see that all non-vanishing Pauli strings of $H_{x \leftrightarrow y}$ commute, since they are in the normalizer of S , which in this case is equal to the centralizer of S .

3.2 Stabilizer formalism for multiple pairs of states

When there is more than one pair of states associated with the same logical operator, i.e. when $|E_{\overline{X}}| = |V_{\overline{X}}|/2 > 1$, we can in general no longer apply the stabilizer formalism directly for all states. One can still express the projector $\prod_{\text{span}(V_{\overline{X}})} = \sum_{|x\rangle \in V_{\overline{X}}} |x\rangle \langle x|$ in Equation (13) in the Pauli-basis, e.g. by using Equation (6). Since this is not necessarily optimal, we show how one can utilize the structure of the space subspace $V_{\overline{X}}$ to construct mixers with as few Pauli-strings as possible.

We begin with the following observation. Let $\{s_1, \dots, s_l\}$ a minimal generating set of a stabilizer group $S = \langle s_1, \dots, s_l \rangle$. This defines the code space $C(S) = \text{span}(|z\rangle : s|z\rangle = |z\rangle \forall s \in S)$. Let us now assume that there is a Pauli-string E such that $E|z\rangle \notin C(S)$ for $|z\rangle \in C(S)$. In the error correction lingo E would be interpreted as a detectable error. Then we define $S^E = \langle s_1^E, \dots, s_l^E \rangle$ with

$$s_i^E = \begin{cases} s_i, & \text{if } [E, s_i] = 0, \\ -s_i, & \text{otherwise.} \end{cases} \quad (17)$$

(Remember that Pauli strings either commute or anti-commute). It follows that S^E is a stabilizer group that defines the code space $C(S^E) = \text{span}(E|z\rangle : |z\rangle \in C(S))$. This is easy to see, since for any $|z\rangle \in C(S)$ we have that

$$s_i^E E|z\rangle = \begin{cases} s_i E|z\rangle, & \text{if } [s_i^E, E] = 0, \\ -s_i E|z\rangle, & \text{otherwise} \end{cases} = \begin{cases} E s_i |z\rangle, & \text{if } [s_i^E, E] = 0, \\ -E(-s_i) |z\rangle, & \text{otherwise} \end{cases} = E|z\rangle. \quad (18)$$

We observe that the terms in S^E and S only differ by sign. Additionally, we know that $\exists s_i \in \{s_1, \dots, s_l\}$ s.t. $\{s_i, E\} = 0$, i.e. $\exists s_i^E = -s_i$. We define the index set

$$I^c = \{1 \leq i \leq l \mid s_i = s_i^E\}, \quad (19)$$

i.e. the index set where the generators coincide, and $I^d = \{1 \leq i \leq l\} \setminus I^c$, where they differ. Let

$$I^{d,2} = \{(i, j) : i \neq j, i, j \in I^d\}, \quad (20)$$

i.e., $s_i = -s_i^E \wedge s_j = -s_j^E$ for $(i, j) \in I^{d,2}$. These observations lead to the following

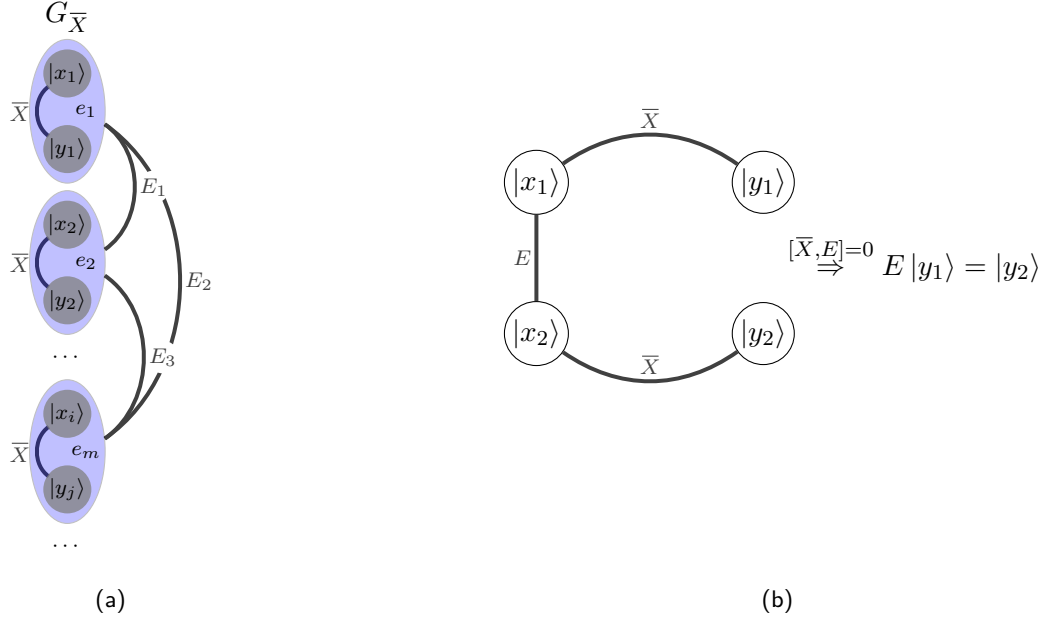


Figure 4: For each graph $G_{\bar{X}}$ one can specify operators $\{E_i\}_i$ that map the states from one subspace spanned by the elements of $E_{\bar{X}}$ to another. (a) An illustration, where individual subspaces of $E_{\bar{X}}$ are highlighted in light blue and E_i map between them. (b) An operator E maps one subspace onto another, as long as $\bar{X}$ and E commute. Indeed, we have that $E|y_1\rangle = E\bar{X}|x_1\rangle = \bar{X}E|x_1\rangle = \bar{X}|x_2\rangle = |y_2\rangle$.

Theorem 3 (Stabilizers for sum of orthogonal subspaces)

Let $\{s_1, \dots, s_l\}$ be a minimal generating set of the stabilizer group $S = \langle s_1, \dots, s_l \rangle$ defining the code subspace $C(S)$. Let E be a Pauli-string with $E|z\rangle \notin C(S)$ for all $|z\rangle \in C(S)$.

Then the stabilizer group $S^{(E)} = \langle s_1^{(E)}, \dots, s_{l-1}^{(E)} \rangle$ with the minimal generating set

$$\{s_1^{(E)}, \dots, s_{l-1}^{(E)}\} = \bigcup_{i \in I^c} \{s_i\} \bigcup_{(i,j) \in J} \{s_i s_j\}, \quad (21)$$

for any $J \subset I^{d,2}$ with $|J| = l - |I^c| - 1$, stabilizes the code space $C(S^{(E)}) = C(S) \oplus C(S^E)$.

Proof: For all $i \in I^c$ we have that all states in $C(S)$ and $C(S^E)$ are a plus one eigenstate of s_i . For $(i, j) \in I^{d,2}$ we have that $s_i s_j = s_i^E s_j^E$ and therefore all states in $C(S)$ and $C(S^E)$ are a plus one eigenstates. Notice that $\dim(C(S^{(E)})) = \dim(C(S)) + \dim(C(S^E)) = 2^{n-l} + 2^{n-l} = 2^{n-(l-1)}$, and therefore a minimal generating set has $l - 1$ elements. ■

The relationship $|S^{(E)}| = |S|/2$ means that the number of Pauli strings to realize the projector onto $C(S^{(E)})$ is halved as compared to $C(S)$. This inspires the following algorithm to efficiently find a minimal generator set when the code space has an underlying group structure. In the following we interpret $\{g_1, \dots, g_k\}|\phi\rangle$ as $\{g_1|\phi\rangle, \dots, g_k|\phi\rangle\}$.

Algorithm 1 Minimal generator set of stabilizer of subspace $V = \text{span}(\langle E_1, \dots, E_k \rangle |z\rangle)$

Given: $E_i \in \{I, X\}^n$, computational basis state $|z\rangle$

Result: Minimal generating set $G_k = \{g_1, \dots, g_{n-k}\}$ such that $C(\langle g_1, \dots, g_{n-k} \rangle) = V$

$G_0 = \{g_{0,1}, \dots, g_{0,n}\}$ with $g_{0,i} = (-1)^{z_i} Z_i$

for $1 \leq i \leq k$ **do**

 Apply Theorem 3 with $E = E_i$ to get $G_i = \{g_{i,1}, \dots, g_{i,n-i}\}$

end for

We illustrate Algorithm 1 for $V = \{|1011\rangle, |1100\rangle, |0111\rangle, |0000\rangle, |1110\rangle, |1001\rangle, |0010\rangle, |0101\rangle\} = \langle X_2 X_3 X_4, X_1 X_2, X_1 X_4 \rangle |1011\rangle$. In the first step we get that $G_0 = \{-Z_1, Z_2, -Z_3, -Z_4\}$ stabilizes the state $|1011\rangle$. Applying Theorem 3 recursively

- with $E = X_2X_3X_4$ gives $G_1 = \{-Z_1, -Z_2Z_3, Z_3Z_4\}$,
- with $E = X_1X_2$ gives $G_2 = \{Z_1Z_2Z_3, Z_3Z_4\}$,
- and with $E = X_1X_4$ gives $G_3 = \{Z_1Z_2Z_4\}$.

It is easy to check that $C(\langle Z_1Z_2Z_4 \rangle) = \text{span}(V)$ and therefore the projector to that subspace is given by $\Pi_{\text{span}(V)} = \frac{1}{2}(IIII + ZZIZ)$.

Definition 4 (Stabilizer subgraphs)

For every graph $G_{\bar{X}}$ we take the set of operators $\{E_i\}_i$ between the subspaces spanned by the elements of $E_{\bar{X}}$. Note that $E_i \in \{I, X\}^n$ by construction. Let F be a subset of operators $F = \{E_{i_1}, \dots, E_{i_q}\} \subset \{E_i\}_i$. We call $G_{\bar{X}, F}$ a stabilizer subgraph of $G_{\bar{X}}$ if for an $|x\rangle$ associated with a pair connected by F , we have that

$$e|x\rangle \in V_{\bar{X}}, \quad \forall e \in \langle E_{i_1}, \dots, E_{i_q} \rangle. \quad (22)$$

An illustration of the set of operators $\{E_i\}_i$ is given in Figure 4. Algorithm 1 can be applied to all stabilizer subgraphs. The larger the set F , the smaller the number of Pauli-strings required to form the projector to the subspace, since the stabilizer group has $2^n - |\langle F \rangle|$ elements.

4 Restricting projectors to the feasible subspace

We have seen in the previous sections mixers can be written as a sum of projection operators to certain subspaces followed by a logical X operator, i.e., $\bar{X}_a \Pi_{\text{span}(V)}$ for some some subspace $V \subseteq V_{\bar{X}}$. In this section we discuss the possibility to reduce the cost of a mixer further by restricting the projector to the feasible subspace $\text{span}(B)$. The essential properties of the projector $P_{V,B} := \Pi_{\text{span}(V)} \big|_{\text{span}(B)}$ that we need to fulfill are that

$$P_{V,B} |x\rangle = |x\rangle, \quad |x\rangle \in V, \quad (23a)$$

$$P_{V,B} |x\rangle = \mathbf{0}, \quad |x\rangle \in B \setminus V_{\bar{X}}. \quad (23b)$$

Since $\bar{X}|x\rangle \in \text{span}(V_{\bar{X}})$ for all $|x\rangle \in V_{\bar{X}}$, it does not matter whether $P_{V,B}$ projects the states in the (possibly empty) set $V_{\bar{X}} \setminus V$ to zero or themselves as long as $P_{V,B}^2 = P_{V,B}$. We present two ways to achieve this goal.

4.1 Linear algebra approach

From Section 3 we saw that the projector can be written using the stabilizer formalism as $\Pi_{\text{span}(V)} = \frac{1}{|S|} \sum_{s \in S} s$, where S is a stabilizer group with $C(S) = \text{span}(V)$. Let s_j be an enumeration of the elements in the group S . One can now construct the matrix $A : \mathbb{R}^{B \setminus V_{\bar{X}} \times |S|} \rightarrow \mathbb{R}$ with entries given by

$$A_{i,j} = \lambda_i^{s_j}, \quad s_j \in S, |x_i\rangle \in B \setminus V_{\bar{X}}, \quad (24)$$

where $s_j |x_i\rangle = \lambda_i^{s_j} |x_i\rangle$. In other words, $\lambda_i^{s_j} \in \pm 1$ indicates whether $|x_i\rangle$ is in the ± 1 eigenspace of $s_j \in S$. This gives the possibility to reduce the number of Pauli-strings as follows.

Theorem 4 (Projectors restricted to a feasible subspace)

Let S be a stabilizer group with $C(S) = \text{span}(V)$ and A as defined Equation (24). If $v \in \mathbb{R}^{|S|}$ such that $v \in \ker(A)$ and $\mathbf{1}^T v = \sum_{i=1}^{|S|} v_i \neq 0$, then

$$P = \frac{1}{\sum_{i=1}^{|S|} v_i} \sum_{i=1}^{|S|} v_i s_i \quad (25)$$

fulfills Conditions (23).

Proof: First, we observe that P is well-defined since $\sum_{i=1}^{|S|} v_i \neq 0$. Let $|x\rangle \in \text{span}(V) = C(S)$ then by definition $s_i |x\rangle = |x\rangle$ and we have that

$$P|x\rangle = \frac{1}{\sum_{i=1}^{|S|} v_i} \sum_{i=1}^{|S|} v_i s_i |x\rangle = \frac{1}{\sum_{i=1}^{|S|} v_i} \sum_{i=1}^{|S|} v_i |x\rangle = \frac{1}{\sum_{i=1}^{|S|} v_i} \underbrace{\left(\sum_{i=1}^{|S|} v_i \right)}_{\neq 0} |x\rangle = |x\rangle, \quad (26)$$

so Condition (23a) is fulfilled.

If $|x_i\rangle \in B \setminus V_{\overline{X}}$ we have that

$$\left(\sum_{i=1}^{|S|} v_i\right) P|x_i\rangle = \sum_{j=1}^{|S|} v_j s_j |x_i\rangle = \sum_{j=1}^{|S|} v_j A_{i,j} |x_i\rangle = (Av)_i |x_i\rangle = \mathbf{0}, \quad (27)$$

where the last equality holds because $v \in \ker(A)$. This shows Condition (23b) is fulfilled.

Since $P_{V,B}$ is a projector onto V this implies that every row of A has to have equally many plus one and minus one entries. Therefore, a trivial solution of Theorem 4 is $v = \mathbf{1}$. However, in general there are other solutions, which can be found by bringing A into *reduced row echelon form*.

An example is given for $B = \{|10010\rangle, |01010\rangle, |10011\rangle, |11101\rangle, |001110\rangle, |01010\rangle\}$ and $C_1 = \text{span}(|10010\rangle, |01110\rangle)$, where A becomes the following matrix.

$$A = \begin{pmatrix} |11101\rangle \\ |01010\rangle \\ |10011\rangle \\ |00110\rangle \end{pmatrix} \begin{pmatrix} +ZIZZ & -ZZIZ & -ZIZI & +ZZIZ & -IZZZI & -IZZZI & -IZIZ & -IIIZI & +IZZIZ & +ZZIZZ & +IIIII & +IZZIZ & -IIIZI & -IIIZI & +IZZIZ & +IIIII \\ -1 & +1 & +1 & +1 & -1 & +1 & -1 & -1 & +1 & -1 & -1 & -1 & +1 & +1 & +1 & +1 \\ -1 & +1 & -1 & +1 & -1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 & +1 & +1 & -1 & +1 \\ -1 & -1 & -1 & +1 & +1 & -1 & +1 & +1 & +1 & -1 & -1 & -1 & +1 & -1 & +1 & +1 \\ +1 & -1 & +1 & -1 & -1 & -1 & +1 & +1 & +1 & -1 & +1 & -1 & -1 & +1 & -1 & +1 \end{pmatrix} \quad (28)$$

The rank of A is 4, and therefore the kernel of A has dimension 12, after the rank-nullity theorem. The subspace of the kernel where also $\mathbf{1}^T v \neq 0$ has dimension 10. Two solutions with the lowest cost are given by $v = (0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0)$ and $v = (0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0)$ with $H_{XXII} = XXII(-ZZII - ZIZI)$ and $H_{XXII} = XXII(-ZIZI - ZZII)$, respectively. Both reduced mixers bring down the cost from 96 to 10.

4.2 Stabilizer formalism approach

An alternative approach can be derived using the stabilizer formalism. Theorem 2 and Equation (16) gives us a way to construct valid mixers with fewer Pauli terms. We start by observing that for any $|\psi\rangle \notin C$ there exists at least one g_i in the minimal generating set of S with $g_i |\psi\rangle = -|\psi\rangle$. But since the eigenvalues of Pauli strings are ± 1 and $\prod_S |\phi\rangle = \mathbf{0}$ this means that half of the terms in S must be $+1$ and the other half -1 eigenvectors of $|\psi\rangle$, otherwise they would not cancel out. If we can find a subgroup $H < S$ that still has this characteristic for all $|\psi\rangle \in \text{span}(B) \setminus C$ then we need fewer Pauli-strings to realize $H_{x \leftrightarrow y}$ on the feasible subspace. In general, we seek the following.

Problem 1 (Optimal projector for subspace $V = \text{span}(\langle \bar{X}_1, \dots, \bar{X}_k \rangle |z\rangle)$)

Let S be the stabilizer of the code space $V = \text{span}(\langle \bar{X}_1, \dots, \bar{X}_k \rangle |z\rangle)$. Then we want to find a smallest subgroup $H < S$ such that

$$\frac{1}{|H|} \sum_{h \in H} h \Big|_{\text{span}(B)} = \Pi_s \Big|_{\text{span}(B)}. \quad (29)$$

Then by construction $P = \frac{1}{|H|} \sum_{h \in H} h$ fulfills Conditions (23).

The next question is how to find such a smallest subgroup. Let $S = \langle g_1, \dots, g_{n-k} \rangle$. One can now construct the matrix $M : \mathbb{R}^{|B \setminus V_{\bar{X}}| \times (n-k)} \rightarrow \mathbb{R}$ with entries given by

$$M_{i,j} = \lambda_i^{g_j}, \quad g_j \in S, |x_i\rangle \in B \setminus V_{\bar{X}}, \quad (30)$$

where $g_j |x_i\rangle = \lambda_i^{g_j} |x_i\rangle$. In other words, $M_{i,j} \in \pm 1$ indicates whether $|x_i\rangle$ is in the ± 1 eigenspace of g_j . Note, that in contrast to A in Equation (28), g_j is only taken from a minimal generating set of the group, not all elements of the group. Since $|x_i\rangle \in B \setminus V_{\bar{X}}$ is outside the code space of $S = \langle g_1, \dots, g_{n-k} \rangle$ is a generating set, this means that there has to be at least one -1 entry in each row. Otherwise $|x_i\rangle$ would belong to the code space.

One can create all minimal generating sets of S by successively multiplying one element by another as long as they remain linearly independent, e.g., $\{g_1, \dots, g_j, \dots, g_l, \dots, g_{n-k}\} \rightarrow \{g_1, \dots, g_j g_l, \dots, g_l, \dots, g_{n-k}\}$. For the matrix M this changing g_j to $g_j g_l$ means multiplying column j with l . To solve Problem 1 becomes then equivalent to finding the smallest subset of columns that have at least one -1 entry in each row, taking into account all possible multiplications of columns.

For the same example that was used in Section 4.1 we have

$$M = \begin{matrix} & \begin{matrix} IIZZ \\ -IIIZ \\ -IIIZ \\ ZZIII \\ +IZZII \end{matrix} \\ & \begin{matrix} = \\ = \\ = \\ = \end{matrix} \\ & \begin{matrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{matrix} \\ \begin{matrix} |11101\rangle \\ |01010\rangle \\ |10011\rangle \\ |00110\rangle \end{matrix} & \begin{pmatrix} +1 & -1 & +1 & +1 \\ -1 & -1 & -1 & +1 \\ +1 & +1 & -1 & -1 \\ +1 & +1 & +1 & -1 \end{pmatrix} \end{matrix}. \quad (31)$$

In this case, multiplying columns g_2 and g_4 gives a column with all entries equal to -1 . Therefore $H = \langle g_2 g_4 \rangle = \langle IZZIZ \rangle$ is a solution to Problem 1 leading to $H_{XXII} = XXXII(IIIII + IZZIZ)$. This reduces the cost from 96 to 10.

4.3 Comparison

The method in Section 4.2 is based on a minimal generating set of the stabilizer group, whereas the one presented in Section 4.1 uses all elements of the group. Therefore, the matrix M in Equation (31) is exponentially larger in the number of group elements than the matrix A in Equation (28). However, the computational complexity for finding the projector restricted to the feasible subset with the lowest cost seems to be lower for the approach in Section 4.1, as it only involves bringing the matrix A into reduced row echelon form. An optimal solution to Problem 1 seems to be computationally more complex (at least without further investigation). There seems to be a trade-off between size and complexity.

Furthermore, the method in Section 4.1 is more general, as it in principle works for any projector $P = \sum_i w_i P_i$, $P_i \in \{I, Z\}$ and the resulting solution does not need to be a group. This can also be seen in the example above, and it does not seem a-priorily obvious which of the two methods gives the better solution. An example of the reduction in cost is given in Table 1.

5 Numerical results of the algorithm

In this section we provide examples of the proposed approach. The overall approach is sketched in Figure 5, which allows to work with minimal generating sets until the last step, where the final mixer is generated. The overall complexity of the algorithm in Figure 5 is as follows:

- Step (1) generates the family of valid mixers of at most $\mathcal{O}(|B|^2)$ graphs according to Definition 2 by going through all pairs of states in B . It should be noted that this is an upper bound: The larger the set B , the more likely it is for pairs to be in the same graph.

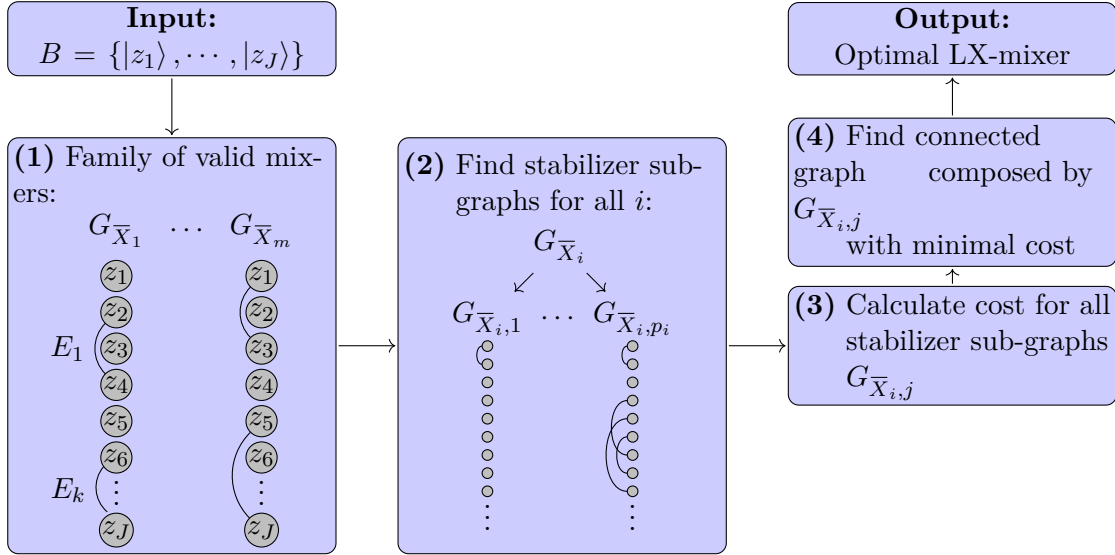


Figure 5: The steps of the overall algorithm. An optimal mixer is found by generating all possible stabilizer subgraphs and finding the connected graph composed by a subset of these subgraphs that has minimal cost.

- Step (2) generates stabilizer subgraphs according to Definition 4. A necessary condition for this is that there are power of two edges, so we can upper bound this by $\sum_{i=0}^{\lfloor \log_2(|B|) \rfloor} \binom{|B|}{2^i} \leq \mathcal{O}(2^{|B|})$. This is a very rough upper bound.
- Step (3) consists of calculating the cost for all graphs, e.g., by solving Problem 1. A brute-force solver scales exponentially with $B \setminus V_{\bar{X}}$ so the runtime can be upper bounded by $\mathcal{O}(2^{|B|})$.
- Step (4) consists of finding the combination of stabilizer subgraphs that leads to a connected graph and has minimal cost, see Definition 3. A brute-force implementation looking through all possible combinations scales as $2^{|G|}$, where the number of graphs can be upper bounded by $|G| \leq |B|2^{|B|}$.

We remark, that heuristics might be developed for better scaling that still lead to good solutions.

Figure 6 shows how the proposed approach compares for randomly selected B . We observe that the proposed method reduces the cost of the mixer that connects states linearly as in a chain. This corresponds to a T in Equation 5 that has 1 in the first off-diagonal entry and 0 in the rest of the entries. The Figure shows that finding the optimal Trotterization reduces the cost (red line vs. blue line), particularly when $|B|$ is large. For the case when $|B| = 2^n$, the proposed algorithm reproduces the standard X -mixer which has zero cost. Figure 7 shows the optimal graph for the X -mixer, i.e., the unconstrained case. Furthermore, the approach to restrict the projector to the feasible subspace, as described in Section 4, can further reduce the required number of CNOT gates in a wide range of cases.

The proposed method also reproduces the case which is sometimes referred to as "k-hot" or "XY"-mixer[22]. The feasible subspace is defined by all computational basis states with exactly k "1"s, i.e., $B = \{|x\rangle, x_j \in \{0,1\}^n, \text{ s.t. } \sum x_j = k\}$. As depicted in Figure 8 for the 6 qubits 4-hot case each term in the mixer is of the form $XX + YY$ when restricted to the feasible subspace B . An extension of this is shown in Figure 11, where B consists of k -hot states for a range of different values k . The approach shows that a low-cost mixer can be constructed as a combination of XY- and X-mixers.

Finally, we illustrate the family of graphs for the case when $B = \{|1010\rangle, |0111\rangle, |1110\rangle, |1001\rangle, |0010\rangle, |0000\rangle, |1101\rangle\}$ in Figure 9. Comparing the "chain" mixer to the optimal solutions results in a roughly 70% fewer CNOT gates. Restricting the mixers to the feasible subspace further reduces the number of CNOT gates by 75% and 51%, respectively.

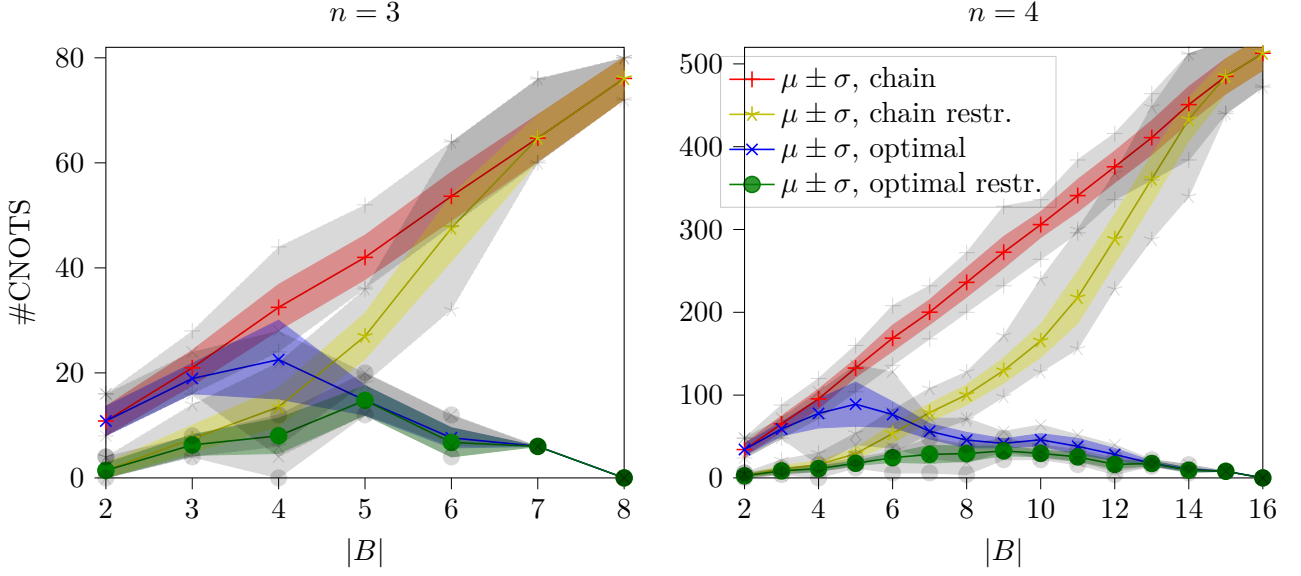


Figure 6: Statistics of the number of controlled not gates required depending on the number of feasible states, i.e. $|B|$. For each $|B|$ 100 randomly selected states are drawn all computational basis states of $(C^2)^{\otimes n}$ and the corresponding restricted mixers are generated. The mean and one standard deviation of the cost is plotted in color. The gray shaded area is the range between the minimum and maximum obtained. We can see that the mixer corresponding to a linear chain of consecutive states is easily outperformed by finding an optimal Trotterization. We observe also that restricting the mixers to the feasible subspace reduces the cost.

	standard	ref.[9]	LX-Mixer
$C_1 = \{ 10010\rangle, 01110\rangle\}$	96	40	10
$C_2 = \{ 10010\rangle, 10011\rangle\}$	64	24	4
$C_3 = \{ 10010\rangle, 11101\rangle\}$	112	48	14
$C_4 = \{ 10010\rangle, 00110\rangle\}$	80	32	10
$C_5 = \{ 10010\rangle, 01010\rangle\}$	80	32	10
$C_6 = \{ 01110\rangle, 10011\rangle\}$	112	48	14
$C_7 = \{ 01110\rangle, 11101\rangle\}$	96	48	12
$C_8 = \{ 01110\rangle, 00110\rangle\}$	64	24	4
$C_9 = \{ 01110\rangle, 01010\rangle\}$	64	24	4
$C_{10} = \{ 10011\rangle, 11101\rangle\}$	96	40	10
$C_{11} = \{ 10011\rangle, 00110\rangle\}$	96	40	10
$C_{12} = \{ 10011\rangle, 01010\rangle\}$	96	40	10
$C_{13} = \{ 11101\rangle, 00110\rangle\}$	112	48	12
$C_{14} = \{ 11101\rangle, 01010\rangle\}$	112	48	12
$C_{15} = \{ 00110\rangle, 01010\rangle\}$	80	32	4

Table 1: Comparing the cost of the mixers for $B = \{|10010\rangle, |01010\rangle, |10011\rangle, |11101\rangle, |00110\rangle, |01010\rangle\}$. When compared to the unrestricted mixer or the brute-force method in Fuchs et al. [9], the cost can be considerably reduced with the approach described in Section 4.

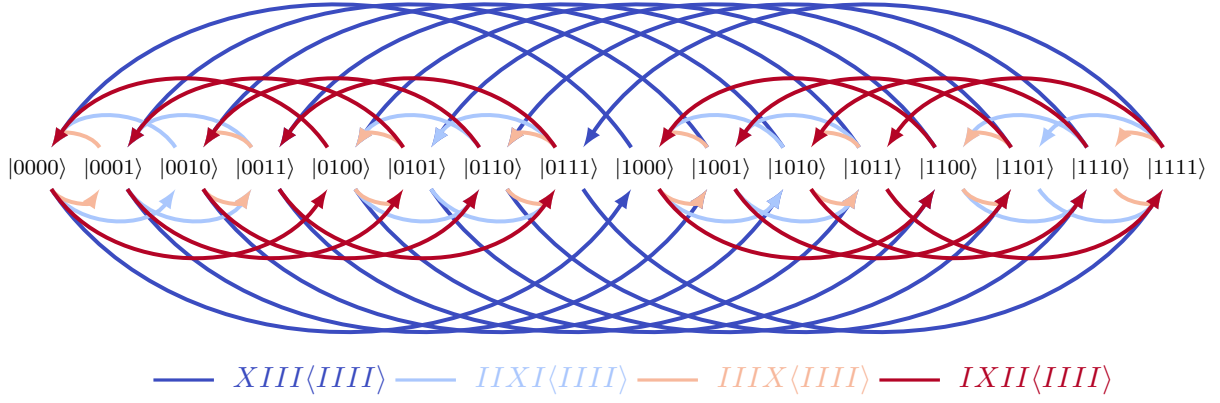


Figure 7: The proposed algorithm reproduces the standard "X"-mixer for the unconstrained case.

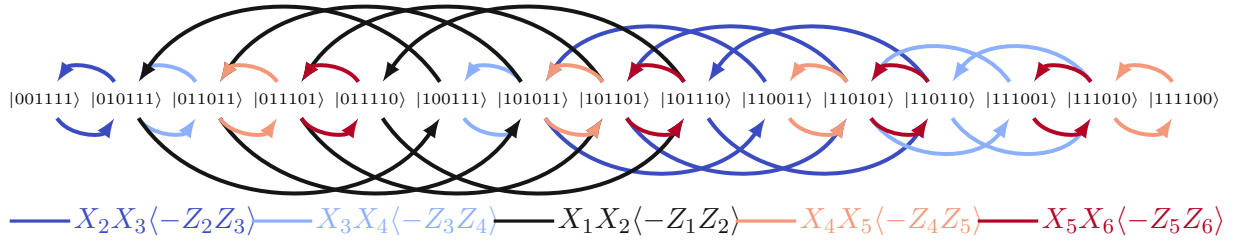


Figure 8: The proposed algorithm reproduces the standard "XY"-mixer for k-hot states. Here illustrated for $k = 4$ with 6 qubits.

6 Applications and practical considerations

The numerical results discussed previously demonstrate a promising reduction in cost, specifically in terms of CNOTs, through the implementation of a general algorithm for optimal mixer circuits. Despite this advancement the algorithm will in general scale prohibitively with n when applied to general unstructured set B . The reason for this is that the family of graphs scale exponentially with respect to the number of vertices that must be evaluated. This restricts the feasibility of this approach for unstructured problems, limiting its application to small-scale cases. Nevertheless, the introduced formalism holds promise for discovering (near-)optimal mixers in terms of CNOTs when a structured feasible subset is provided by the problem at hand. In this section, we show one way the proposed method scales favourably with the number of qubits when the bitstrings of the feasible subspace admit a Cartesian product structure. Furthermore, we show a relaxation of the k-hot case to the multi-k-hot case. Finally, we show a numerical simulation of a small case combining the two approaches employing the LX-QAOA ansatz on a 10 variables optimization problem.

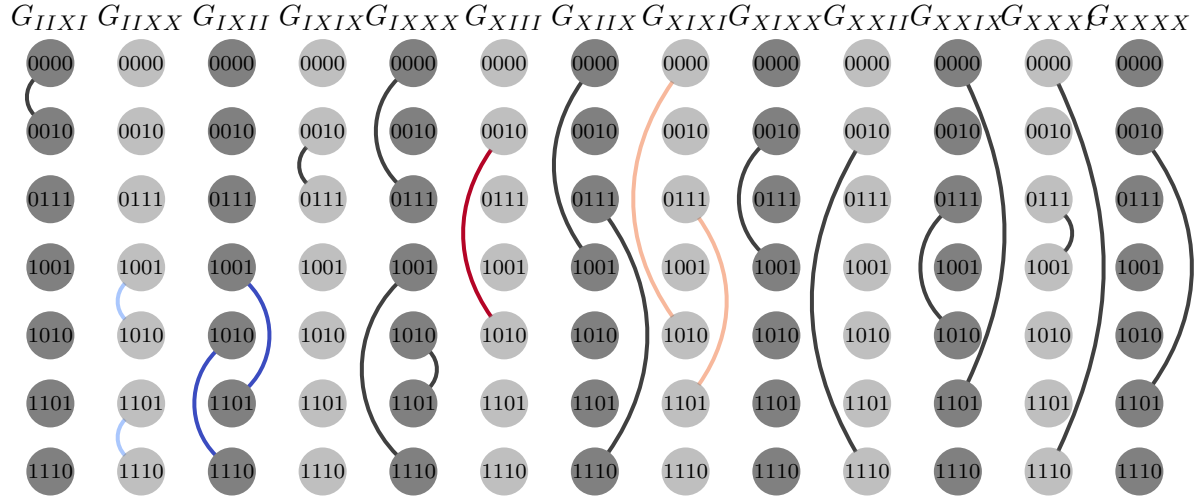
6.1 Feasible sets with Cartesian product structure

When the feasible bitstrings admit a Cartesian product structure, the feasible subspace becomes a tensor product of the form

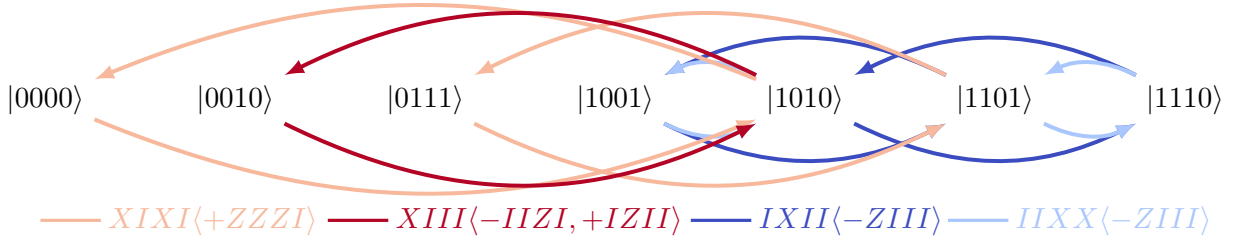
$$B = B^1 \otimes \cdots \otimes B^L. \quad (32)$$

For this case it is possible to find an optimal mixer by employing the proposed algorithm independently on each subspace B_i . The algorithm remains scalable if there are a fixed number of different cases B_i when L is increased. Given the independence of the constraints, we can then define the following mixer Hamiltonian

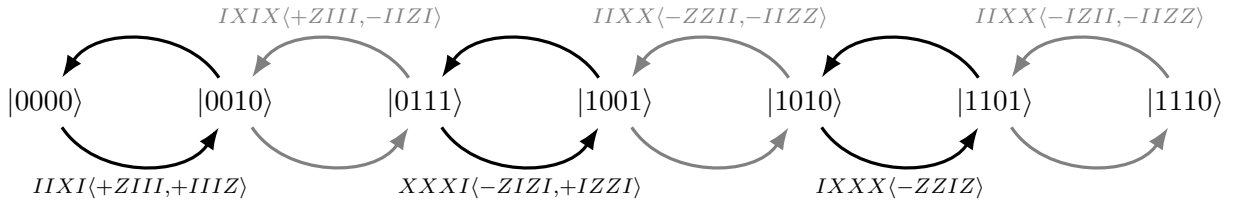
$$H_M = \sum_{i=1}^L H_i, \quad (33)$$



(a)



(b)



(c)

Figure 9: (a) Example of family of graphs with commuting pairs for $B = \{|1010\rangle, |0111\rangle, |1110\rangle, |1001\rangle, |0010\rangle, |0000\rangle, |1101\rangle\}$. (b) The optimal Trotterization as proposed in this work has cost $64/22$ (with/without restriction to feasible subspace). (c) The linear "chain" mixer has cost $200/98$ (with/without restriction to feasible subspace).

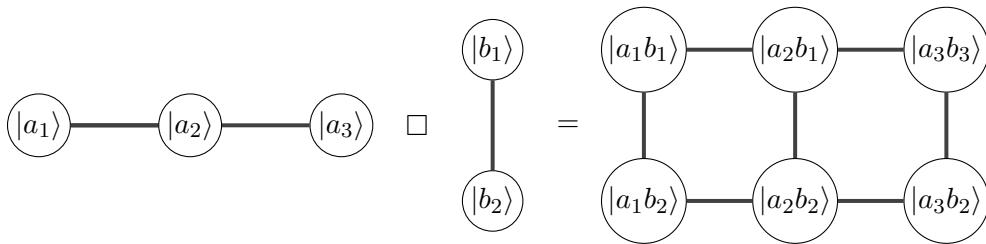


Figure 10: An example of the box product construction for $B = \{|s_1 s_2\rangle, s_1 \in \{10, 01\}, s_2 \in \{100, 010, 001\}\}$. For this case $a_1 = 100, a_2 = 010, a_3 = 001, b_1 = 10, b_2 = 01$, with optimal mixers $H = H_1 + H_2$, where $H_1 = X_1 X_2 \langle -Z_1 Z_2 \rangle + X_2 X_3 \langle -Z_2 Z_3 \rangle$ and $H_2 = X_1 X_2$.

where H_i acts non-trivially on the subspace according to B_i . From algebraic graph theory, it is known that the Kronecker sum of adjacency matrices corresponds to what is commonly referred to as the Cartesian product of graphs, see Figure 10.

Definition 5 (Cartesian product of graphs)

Let $G(V, E)$ and $H(V', E')$ be two graphs, the Cartesian product $G \square H$, usually called box product, is a graph such that:

- The vertex set of $G(V, E) \square H(V', E')$ is composed by the Cartesian product of $V \times V'$
- Two vertex (u, u') and (v, v') are adjacent in $G \square H$ if
 - $u = v$ and u' and v' are adjacent in H
 - $u' = v'$ and u and v are adjacent in G

By construction, each distinct H_i defines a valid LX-mixer for B_i represented by a connected adjacency matrix. Therefore we need to verify that the adjacency matrix of the graph associated to the new Hamiltonian H_M is connected too or more specifically that the Cartesian product of connected graphs is connected.

Theorem 5 (Cartesian product Connectivity)

Let $G(V, E)$ and $H(V', E')$ be connected graphs then $G \square H$ is connected.

Proof: Consider two connected graphs G and H . Let $V(G) = \{v_1, v_2, \dots, v_n\}$ and $V(H) = \{u_1, u_2, \dots, u_m\}$ be the vertex sets of G and H , respectively. For any vertex $v_i \in V(G)$, consider the set of vertices $V_{v_i} = \{(v_i, u_j) | u_j \in V(H)\}$ in the Cartesian product graph $G \square H$. Since H is connected, for each v_i , there exists a path in H connecting any two vertices u_a and u_b . Therefore, for any pair of vertices (v_i, u_a) and (v_i, u_b) in V_{v_i} , there exists a path in $G \square H$ connecting them, following the Cartesian product definition. The same argument can be used for any vertex $u_i \in V(H)$, consider the set of vertices $V_{u_i} = \{(v_j, u_i) | v_j \in V(G)\}$ in the Cartesian product graph $G \square H$ and using the Cartesian product definition.

Now consider any pair of vertices (v_i, u_a) and (v_k, u_b) in $G \square H$. We can construct a path between (v_i, u_a) and (v_k, u_b) as follows:

1. Move from (v_i, u_a) to (v_i, u_b) along a path in V_{v_i} , which exists as shown earlier since G is connected.
2. Move from (v_i, u_b) to (v_k, u_b) along the path in V_{u_b} , which exists as shown earlier since H is connected.

Therefore, we have constructed a path in $G \square H$ connecting (v_i, u_a) and (v_k, u_b) for any pair of vertices, proving that $G \square H$ is connected. ■

An example of the construction is given in Figure 10. The commutativity property of the Cartesian product correspond to the fact that the terms in the Hamiltonian commute with each other since they are applied to different subsets of qubits. This means that no Trotterization is needed for each term of the Hamiltonian and the total cost $Cost(H_M) = \sum_i Cost(H_i)$ is the sum of the optimal costs for each subsets of bits. In conclusion, if the constraints for each independent subset of the bit string are small and unstructured, we can still use the logical X algorithm to find the optimal mixers for each set of qubits and then tensorize them.

6.2 A mixer for the multi-k-hot case

We consider scenarios where the number of ones is not constrained to a single value, but to for instance to a range $0 \leq k_1 \leq k \leq k_2 \leq n$. We define the feasible set B_{k_1, k_2} as

$$B_{k_1, k_2} = \left\{ |z\rangle, z \in \{0, 1\}^n, k_1 \leq \sum_i^n z_i \leq k_2 \right\}. \quad (34)$$

We already know that an appropriate XY-mixer provides an undirected and connected graph that connects all the states in a fixed B_k . This means we need to look for a new set of logical

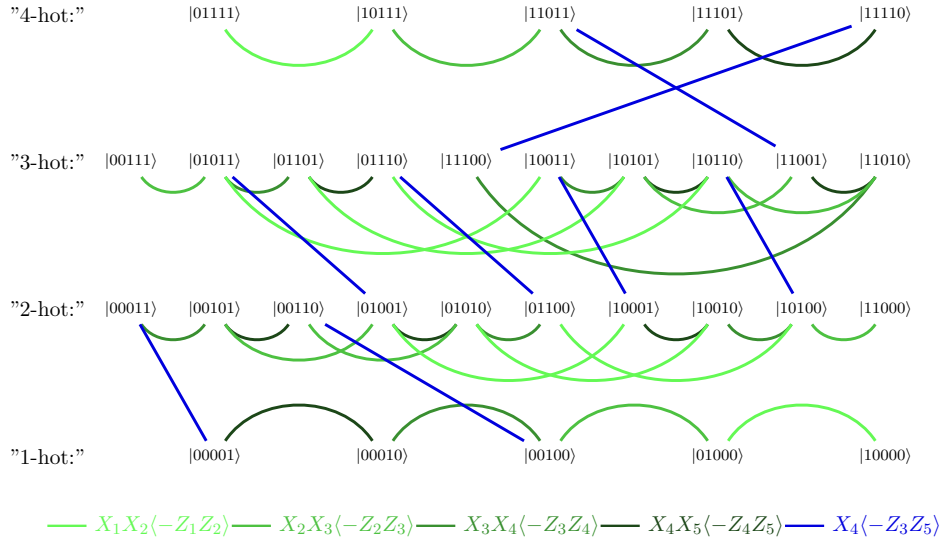


Figure 11: Example graph for $B = \{ |z\rangle, z \in \{0,1\}^5, 1 \leq \sum_j z_j \leq 4 \}$, i.e. B consist of all computational basis states except $|00000\rangle$ and $|11111\rangle$. One of the optimal mixers is shown in the graph. It consist of the XY-mixer which mixes all states with the same number of ones (green arrows), plus a restricted X -mixer, which connects sates with different number of ones (blue arrows).

X s that provides transitions between different B_k sets. Although identifying an optimal solution becomes more challenging in this context, as will be clear later, with the stabilizer formalism we can still construct an efficient mixer.

For this construction, we utilize a logical X operator comprised solely of a single Pauli X_i applied to a specific qubit i . By design the operator flips the i -th qubit providing transitions between B_k and $B_{k\pm 1}$. To ensure the condition that defines B_{k_1, k_2} we still need to avoid transition from B_{k_1} to B_{k_1-1} and B_{k_2} to B_{k_2+1} . Let's start by constructing the sets of states in $|v\rangle \in B_{k_1} \cup B_{k_2}$ such that $X|v\rangle \notin B_{k_1, k_2}$.

$$V_{k_1, k_2}^{X_i} = \{ |v\rangle \in B_{k_1} \cup B_{k_2} \quad s.t \quad X_i |v\rangle \notin B_{k_1+1} \cup B_{k_2-1} \} \quad (35)$$

Next, we construct the projector restricted to the feasibel subspace, i.e.,

$$\Pi = \mathbb{I} - \Pi_{V_{k_1, k_2}^{X_i}} \Big|_{\text{span} B_{k_1, k_2}}. \quad (36)$$

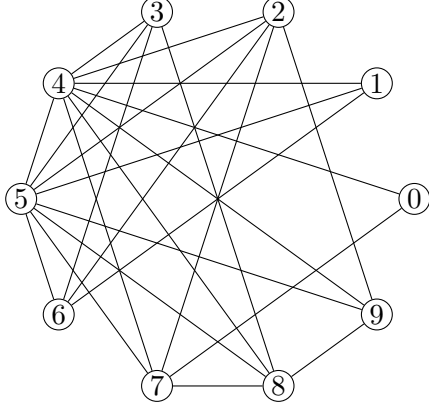
By implementing these strategy, we constructed an efficient LX-mixer capable of providing transitions between different B_k sets while respecting the specified constraints. In this case many different projectors can be constructed and so to find the optimal mixer we are suppose to look for the projector that minimizes the CNOT cost function. Figure 11 shows the construction for the case of $B_{1,4}$.

6.3 Simulation example for a constrained MAXCUT instance

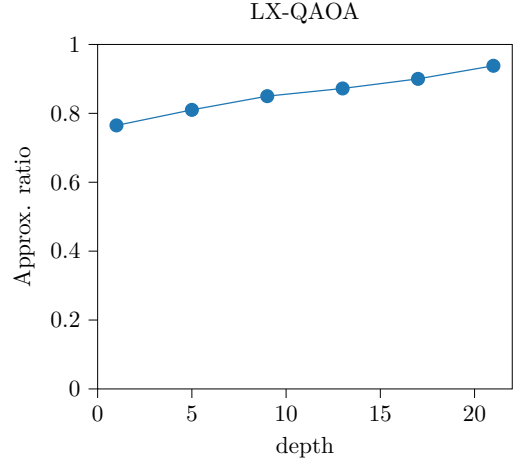
Finally, we provide a numerical example for the specific case of a constrained MAXCUT as a proof-of-principle for LX-QAOA. The problem has constraints with the Cartesian product structure of Section 6.1, where each subspace is a multi- k -hot state, more specifically $B = B_{0,1} \otimes B_{0,1}$. The optimization problem is formulated as follows:

$$\max_{\{x_{i,j}\}_{i,j}} \sum_{i,k=1}^2 \sum_{j,l=1}^5 x_{i,j}(1-x_{k,l})C_{ijkl}, \quad x_{i,j} \in \{0,1\}, \quad \sum_{j=1}^5 x_{i,j} \leq 1, \forall i. \quad (37)$$

The quadratic cost function is encoded in the weights of the graph, as illustrated in Figure 12 (a). For this constraint MAXCUT problem, we introduce a 5 vertices constraint for each of the 2



(a) 10 vertex Barabási-Albert graph instance. The weights for this specific case are distributed uniformly between (0,1) and are available at <https://github.com/OpenQuantumComputing>.



(b) Approximation ratio α using the LX-QAOA ansatz with the COBYLA optimizer, with $\alpha = 1$ indicating that the QAOA achieves the optimal solution.

Figure 12: The LX-QAOA ansatz has been employed up to depth $p=21$ using the gradient free COBYLA optimizer for the optimization of the cost function (37). The optimal solution has been computed brute-force and compared to the QAOA results using the approximation ratio defined as $\alpha = \frac{C}{C_{\text{opt}}}$, where C represents the expectation value for QAOA and C_{opt} denotes the cost of the optimal solution.

subsets, ensuring that the state for each set of vertices is either $|0\rangle$ or a 1-hot state. Figure 12 shows the simulation results, indicating a good convergence of the approximation.

Starting in the uniform superposition of all feasible states, we run the QAOA algorithm with the LX-mixer given by $H = I^5 \otimes H_{0,1} + H_{0,1} \otimes I^5$, with

$$H_{0,1} = X_1 X_2 \langle -Z_1 Z_2 \rangle + X_2 X_3 \langle -Z_2 Z_3 \rangle + X_3 X_4 \langle -Z_3 Z_4 \rangle + X_4 X_5 \langle -Z_4 Z_5 \rangle + X_1 \langle Z_2 Z_3 Z_4 Z_5 \rangle. \quad (38)$$

Simulation results show that the approximation ratio increases with depth, see Figure 12 (b). By design the approach only gives feasible solutions. The approach can be easily scaled to larger instances by design.

7 Availability of Data and Code

All data and the python/jupyter notebook source code for reproducing the results obtained in this article are available at <https://github.com/OpenQuantumComputing> in the repositories LogicalXMixer, and QAOA.

8 Author contributions

Franz G. Fuchs formulated the concept, developed the methodology, wrote the software, made the formal analysis and investigation, wrote the article, made the visualizations. Ruben Pariente Bassa, contributed with the section on applications, including concepts, analysis, software simulations, writing and visualizations.

9 Acknowledgment

We would like to thank for funding of the work by the Research Council of Norway through project number 332023 (80%) and 324944 (20%). The authors wish to acknowledge the helpful discussions with Aleksandra Glesaaen, Giorgio Sartor, Kjetil Olsen Lye, Alexander Müller-Hermes, Alexander Stasik, Carlo Mannino, and Erlend Syljuåsen. We would also like to thank the reviewers for taking the time and effort necessary to review the manuscript.

10 Conclusion and Future Work

We have presented a useful connection between the construction of mixers that act only on subspaces of the full Hilbert space and the stabilizer formalism. The proposed method provides a general approach to construct resource efficient mixers for a given feasible subspace. The numerical results show a large reduction of the number of CNOT gates required. One limitation of this work is that the feasible subspace has to be known beforehand, so the method will for instance not be useful if it is NP-hard to even find a feasible solution. The proposed method is only applicable to problem domains that have an associated indexing function, which efficiently assigns a unique integer index to each feasible solution. Another limitation is that we do not expect our approach to scale to large dimensions of the Hilbert space. For an unstructured feasible set B , the presented algorithm has a worst case scaling of $\mathcal{O}(2^{2|B|})$. However, the method might still be useful in the following case: When the size of the problem increases, the structure of the subspace has to be such that there is a systematic way to describe the resulting constrained mixers for larger instances. An example of this would be the "XY"-mixer, which is straight forward to generalize to any number of qubits, or along the lines of Section 6.1. In that sense, our approach can be seen as a discovery method for efficient constrained mixers for structured problems. Future work includes deriving efficient methods to prepare initial states within the feasible subspace for QAOA, as well as a mathematical analysis of the complexity of the graph problems for optimal Trotterization. Even though it might turn out to be NP-hard to find an optimal Trotterization, efficient heuristics to lower the scaling of the algorithm can be applied to reduce the cost of the circuits of the mixers considerably, without necessarily finding the optimal solution.

References

- [1] Andreas Bärtschi and Stephan Eidenbenz. "Grover mixers for QAOA: Shifting complexity from mixer design to state preparation". In: *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE. 2020, pp. 72–82. DOI: <https://doi.org/10.1109/QCE49297.2020.00020>.
- [2] Sebastian Brandhofer et al. "Benchmarking the performance of portfolio optimization with QAOA". In: *Quantum Information Processing* 22.1 (2023), pp. 1–27. DOI: <https://doi.org/10.1007/s11128-022-03766-5>.
- [3] Sergey Bravyi et al. "Obstacles to Variational Quantum Optimization from Symmetry Protection". In: *Phys. Rev. Lett.* 125 (26 Dec. 2020), p. 260505. DOI: <https://doi.org/10.1103/PhysRevLett.125.260505>.
- [4] Pablo Díez-Valle, Diego Porras, and Juan José García-Ripoll. "Quantum Approximate Optimization Algorithm Pseudo-Boltzmann States". In: *Physical Review Letters* 130.5 (2023), p. 050601. DOI: <https://doi.org/10.1103/PhysRevLett.130.050601>.
- [5] Pablo Díez-Valle et al. "Multiobjective variational quantum optimization for constrained problems: an application to cash handling". In: *Quantum Science and Technology* 8.4 (July 2023), p. 045009. ISSN: 2058-9565. DOI: <https://doi.org/10.1088/2058-9565/ace474>.
- [6] Daniel J. Egger, Jakub Mareček, and Stefan Woerner. "Warm-starting quantum optimization". en. In: *Quantum* 5 (June 2021), p. 479. ISSN: 2521-327X. DOI: <https://doi.org/10.22331/q-2021-06-17-479>.
- [7] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. "A quantum approximate optimization algorithm". In: *arXiv preprint arXiv:1411.4028* (2014). DOI: <https://doi.org/10.48550/arXiv.1411.4028>.
- [8] Franz G Fuchs et al. "Efficient Encoding of the Weighted MAX k-CUT on a Quantum Computer Using QAOA". In: *SN Computer Science* 2.2 (2021), pp. 1–14. DOI: <https://doi.org/10.1007/s42979-020-00437-z>.

- [9] Franz Georg Fuchs et al. “Constraint Preserving Mixers for the Quantum Approximate Optimization Algorithm”. In: *Algorithms* 15.6 (2022), p. 202. DOI: <https://doi.org/10.3390/a15060202>.
- [10] Stuart Hadfield et al. “From the quantum approximate optimization algorithm to a quantum alternating operator ansatz”. In: *Algorithms* 12.2 (2019), p. 34. DOI: <https://doi.org/10.3390/a12020034>.
- [11] Stuart Hadfield et al. “Quantum approximate optimization with hard and soft constraints”. In: *Proceedings of the Second International Workshop on Post Moores Era Supercomputing*. 2017, pp. 15–21. DOI: <https://doi.org/10.1145/3149526.3149530>.
- [12] Tianyi Hao et al. “Exploiting In-Constraint Energy in Constrained Variational Quantum Optimization”. In: *2022 IEEE/ACM Third International Workshop on Quantum Computing Software (QCS)*. IEEE, Nov. 2022. DOI: <https://doi.org/10.1109/qcs56647.2022.00017>.
- [13] Naomichi Hatano and Masuo Suzuki. “Finding exponential product formulas of higher orders”. In: *Quantum annealing and other optimization methods*. Springer, 2005, pp. 37–68. DOI: https://doi.org/10.1007/11526216_2.
- [14] Dylan Herman et al. “Constrained optimization via quantum Zeno dynamics”. In: *Communications Physics* 6.1 (Aug. 2023). ISSN: 2399-3650. DOI: <https://doi.org/10.1038/s42005-023-01331-9>.
- [15] S. Marsh and J. B. Wang. “Combinatorial optimization via highly efficient quantum walks”. In: *Phys. Rev. Res.* 2 (2 2020), p. 023302. DOI: <https://doi.org/10.1103/PhysRevResearch.2.023302>.
- [16] Pradeep Niroula et al. “Constrained quantum optimization for extractive summarization on a trapped-ion quantum computer”. In: *Scientific Reports* 12.1 (Oct. 2022). ISSN: 2045-2322. DOI: <https://doi.org/10.1038/s41598-022-20853-w>.
- [17] Tatsuhiko Shirai and Nozomu Togawa. “Postprocessing Variationally Scheduled Quantum Algorithm for Constrained Combinatorial Optimization Problems”. In: *IEEE Transactions on Quantum Engineering* 5 (2024), pp. 1–14. DOI: <https://doi.org/10.1109/TQE.2024.3376721>.
- [18] Jens Siewert. “On orthogonal bases in the Hilbert-Schmidt space of matrices”. In: *Journal of Physics Communications* 6.5 (2022), p. 055014. DOI: <https://doi.org/10.1088/2399-6528/ac6f43>.
- [19] Daniel Stilck França and Raul Garcia-Patron. “Limitations of optimization algorithms on noisy quantum devices”. In: *Nature Physics* 17.11 (2021), pp. 1221–1227. DOI: <https://doi.org/10.1038/s41567-021-01356-3>.
- [20] Hale F Trotter. “On the product of semi-groups of operators”. In: *Proceedings of the American Mathematical Society* 10.4 (1959), pp. 545–551. DOI: <https://doi.org/10.2307/2033649>.
- [21] Pontus Vikstål et al. “Applying the Quantum Approximate Optimization Algorithm to the Tail-Assignment Problem”. In: *Phys. Rev. Applied* 14 (3 Sept. 2020), p. 034009. DOI: <https://doi.org/10.1103/PhysRevApplied.14.034009>.
- [22] Zhihui Wang et al. “XY mixers: Analytical and numerical results for the quantum alternating operator ansatz”. In: *Physical Review A* 101.1 (Jan. 2020). DOI: <https://doi.org/10.1103/physreva.101.012320>.
- [23] Linghua Zhu et al. “Adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer”. In: *Phys. Rev. Res.* 4 (3 July 2022), p. 033029. DOI: <https://doi.org/10.1103/PhysRevResearch.4.033029>.

Paper II

Compact Circuits for Constrained Quantum Evolutions of Sparse Operators

Franz G. Fuchs
Ruben Pariente Bassa

Abstract: We introduce a general framework for constructing compact quantum circuits that implement the real-time evolution of Hamiltonians of the form $H = \sigma P_B$, where σ is a Pauli string commuting with a projection operator P_B onto a subspace of the computational basis. Such Hamiltonians frequently arise in quantum algorithms, including constrained mixers in QAOA, fermionic and excitation operators in VQE, and lattice gauge theory applications. Additionally, we construct transposition gates that scale more efficiently than the best known constructions in literature. Our method emphasizes the minimization of non-transversal gates, particularly T-gates, critical for fault-tolerant quantum computing. We construct circuits requiring $\mathcal{O}(n|B|)$ CX gates and $\mathcal{O}(n|B| + \log(|B|)\log(1/\epsilon))$ T-gates, where n is the number of qubits, $|B|$ the dimension of the projected subspace, and ϵ the desired approximation precision. For subspaces generated by Pauli X-orbits we further reduce complexity to $\mathcal{O}(n \log |B|)$ CX gates and $\mathcal{O}(n + \log(1/\epsilon))$ T-gates.

Preprint

Compact Circuits for Constrained Quantum Evolutions of Sparse Operators

Franz G. Fuchs[†] and Ruben P. Bassa^{*†}

[†]SINTEF AS, Department of Mathematics and Cybernetics, Oslo, Norway,

^{*}Corresponding Author: ruben.bassa@sintef.no

May 6, 2026

Abstract

We introduce a general framework for constructing compact quantum circuits that implement the real-time evolution of Hamiltonians of the form $H = \sigma P_B$, where σ is a Pauli string commuting with a projection operator P_B onto a subspace of the computational basis. Such Hamiltonians frequently arise in quantum algorithms, including constrained mixers in QAOA, fermionic and excitation operators in VQE, and lattice gauge theory applications. Additionally, we construct transposition gates, widely used in quantum computing, that scale more efficiently than the best known constructions in literature. Our method emphasizes the minimization of non-transversal gates, particularly T-gates, critical for fault-tolerant quantum computing. We construct circuits requiring $\mathcal{O}(n|B|)$ CX gates and $\mathcal{O}(n|B| + \log(|B|) \log(1/\epsilon))$ T-gates, where n is the number of qubits, $|B|$ the dimension of the projected subspace, and ϵ the desired approximation precision. For subspaces that are generated by Pauli X-orbits we further reduce complexity to $\mathcal{O}(n \log |B|)$ CX gates and $\mathcal{O}(n + \log(\frac{1}{\epsilon}))$ T gates. Our constructive proofs yield explicit algorithms and include several applications, such as improved transposition circuits, efficient implementations of fermionic excitations, and oracle operators for combinatorial optimization. In the sparse case, i.e. when $|B|$ is small, the proposed algorithms scale favourably when compared to direct Pauli evolution.

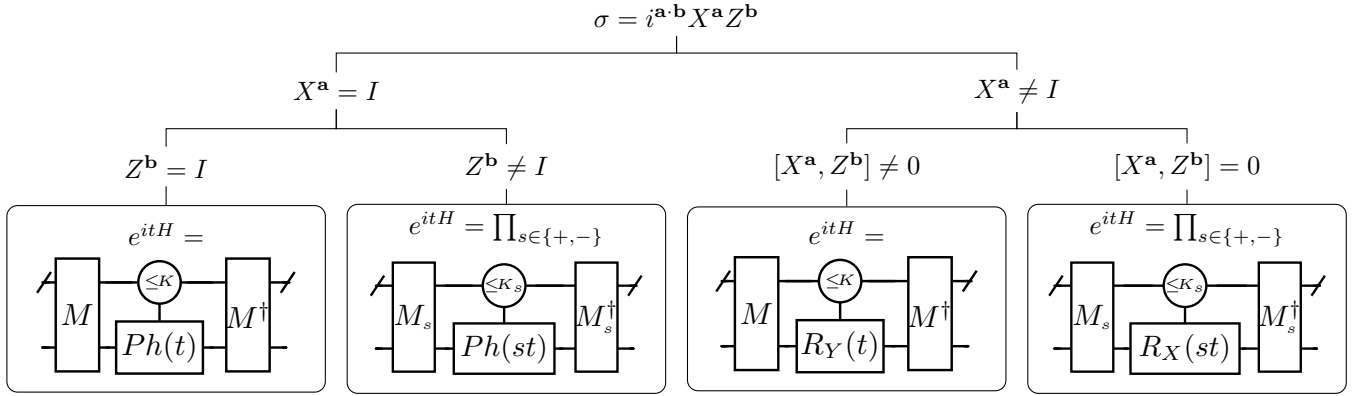


Figure 1: Given a Pauli string σ and a commuting projection operator P_B , the real time evolution of $H = \sigma P_B$ can be realized efficiently for small $|B|$ with permutation operators M described in Section 3.2 and low-pass controlled unitary gates with $K_{(s)} \leq |B|$, which are introduced in Section 2.2. The specific unitary can be classified into four distinct cases depending on the relation of the X and Z terms in σ . Here, $Ph(t) = |0\rangle\langle 0| + e^{it}|1\rangle\langle 1|$ is the phase shift gate and R_X, R_Y is the rotational X, Y gate.

1 Introduction and related work

In this article we propose a general method to construct quantum circuits that minimize the number of non-transversal gates for realizing the real time evolution of a Hamiltonian of the form $H = \sigma P_B$, where σ is a Pauli string that commutes with the projection operator P_B , onto a subspace of the Hilbert space. Evolution operators e^{-itH} of this form show up in several important applications. In VQE [16] fermionic excitation operators in second quantization take this form after applying the Jordan-Wigner transform. Similarly, in QAOA [5, 9], both phase separating operators [8] and constrained mixers acting on subspaces [7] can exhibit this structure. Other examples are the trace gate for lattice gauge theory [1], and the transposition gate that permutes two computational basis state, which is widely found in quantum computing.

gate	#CX	#T=depth	#anc
R_X, R_Y, R_Z	0	$\mathcal{O}(\log(\frac{1}{\epsilon}))$	0
$C^n X, T_{x,y}$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	1
$C^n U, U \in SU(2)$	$\mathcal{O}(n)$	$\mathcal{O}(n + \log(1/\epsilon))$	0
$C^{n-k} e^{it\sigma}, \sigma \in S_k$	$\mathcal{O}(n)$	$\mathcal{O}(n + \log(1/\epsilon))$	0
$C_{\leq K} U, U \in SU(2)$	$\mathcal{O}(n \log(K))$	$\mathcal{O}((n + \log(1/\epsilon)) \log(K))$	0

Table 1: Asymptotic resource requirements for common gates in terms of #CX and T gates, as well as number of ancilla qubits (#anc). The parameter ϵ denotes the target approximation precision. $T_{x,y}$ is the transposition gate between two computational basis states $|x\rangle$ and $|y\rangle$, $C^n X$ is the n -controlled NOT gate, $C^n U$ is a multi-controlled unitary, and S_k is the Pauli group on k qubits. In addition, we include the scaling of the low-pass controlled gate introduced in Section 2.2.

A generic way of approximating the real time evolution of any Hamiltonian on a gate based quantum computer is to decompose it in the Pauli basis and then realize the evolution of each term of the weighted sum through a circuit given in Figure 2. This is exact if the terms commute, otherwise we have to employ a Trotterization. There are two arguments against using this construction to create a circuit. Firstly, the decomposition requires in general to evaluate 4^n Hilbert-Schmidt inner products, since one needs to express P as a weighted sum of Pauli strings. Secondly, evaluating the circuit depth on NISQ device, which is dominated by CX-gate count, is vastly different to the fault tolerant setting, where also rotational gates can have a dominant footprint.

Fault tolerant quantum computers typically use a discrete set of gates, with one of the most common choices being Clifford+T gates. A 2×2 unitary matrix can be exactly expressed using Clifford+T gates if and only if its entries belong to the ring $D[\omega]$ [13], where $D[\omega]$ is the set of dyadic unitary numbers generated by $\omega = e^{i\pi/4}$. If an operation such as $R_z(t)$ has matrix entries outside this ring, it must be approximated by a sequence of Clifford + T gates. This result is fundamental to quantum compiling techniques such as the Gridsynth algorithm [18] and other Solovay-Kitaev-like methods, which approximate arbitrary single-qubit rotations using Clifford+T circuits with a given precision ϵ . As a typical example, the approximation of $R_z(\pi/128)$ up to $\epsilon = 10^{-10}$ by ancilla-free Clifford+T circuits requires at least 102 T-gates [18]. In the typical case, ancilla-free circuit approximations require $\log(1/\epsilon)$ T-gates. On a fault-tolerant quantum device using the surface code performing a logical CX gate is much faster (μs) than magic state distillation (ms) in most practical scenarios [6, 14].

This motivates to derive a method to construct compact circuits for realizing $e^{-it\sigma P_B}$ described above. Here, compact means using as few rotational gates as possible. Related work include circuit constructions for Hamiltonian simulation [21] and phase gadget synthesis [3]. One can also use the QR factorization for gate decomposition in certain settings [15]. Another central tool for unitary synthesis are recursive Cartan decompositions [20], which provide a way to exactly factorize quantum circuits into smaller components. The fundamental routine of state preparation is a closely related topic, for which many algorithms have been proposed. There exist methods that have linear gate and qubit complexity in the number of non-zero amplitudes [17].

Since our algorithm is expressed in terms of MCX, transposition, and MCU gates it is useful to know their resource costs in terms of Clifford+T gates, as summarized in Table 1. In particular, the *multi-controlled NOT (MCX)* gate, also called the n -Toffoli gate, is defined as

$$C^n X = |1\rangle\langle 1|^{\otimes n} \otimes X + (\mathbb{I} - |1\rangle\langle 1|^{\otimes n}) \otimes \mathbb{I}.$$

Different implementations optimize circuit complexity with at least one ancilla. A linear-depth and size approach ($\mathcal{O}(n)$) is given in [10], while a recursive decomposition in [2] achieves $\mathcal{O}(\log(n)^3)$ depth at the cost of $\mathcal{O}(n \log(n)^4)$ size and a higher T-count. A *transposition gate* swaps two computational basis states:

$$T_{\mathbf{x},\mathbf{y}} = |\mathbf{x}\rangle\langle \mathbf{y}| + |\mathbf{y}\rangle\langle \mathbf{x}| + \sum_{\mathbf{w} \neq \mathbf{x},\mathbf{y}} |\mathbf{w}\rangle\langle \mathbf{w}|.$$

A near-optimal construction requires $\Theta(n)$ X, CX gates, two $C^{n+1} X$ gates, and one clean ancilla [11]. Our method achieves $T_{\mathbf{x},\mathbf{y}}$ without ancillas, using $\mathcal{O}(n)$ X, CX gates, $\mathcal{O}(\log(n))$ depth, and one $C^n X$, based on Theorem 3. A *multi-controlled unitary (MCU)* gate is given by

$$C_{\mathbf{b}}^{\mathbf{c}} U_{\mathbf{t}} = (I_{\mathbf{c}} - |\mathbf{b}\rangle\langle \mathbf{b}|_{\mathbf{c}}) \otimes I_{\mathbf{t}} + |\mathbf{b}\rangle\langle \mathbf{b}|_{\mathbf{c}} \otimes U_{\mathbf{t}}, \quad (1)$$

where $\mathbf{c}, \mathbf{t}$ index control and target qubits, and $\mathbf{b}$ specifies the control condition.

- For $U \in SU(2)$ and n control qubits, a decomposition with $\mathcal{O}(n)$ depth and CX gates exists [19]. The T-count scales as $\mathcal{O}(n + \log(1/\epsilon))$.

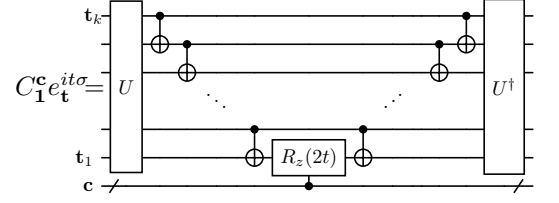


Figure 2: Standard circuit to realize the MCU gate for $U = e^{-it\sigma}$, with control state $\mathbf{1} = 1 \dots 1$ on control register $\mathbf{c} = (1, \dots, n-k)$ and target register $\mathbf{t} = (n-k+1, \dots, n)$. The unitary $U = \bigotimes_{i=1}^k U_i$ realizes the basis change $Z = U_i \sigma_i U_i^\dagger$. Note that the same circuit holds for the uncontrolled version.

- For $U = e^{it\sigma}$, $\sigma \in S_k$, i.e. a length k Pauli string and $n - k$ control qubits, a decomposition with $\mathcal{O}(n)$ depth and CX gates and $\mathcal{O}(n + \log(1/\epsilon))$ T-gates exists. The circuit for this case is shown in Figure 2. Let U_σ be the basis change and M the CNOT stairs from the Figure, we have that $\sigma = U_\sigma^\dagger M \mathbb{I} \otimes Z M^\dagger U_\sigma$ and consequently $U = e^{it\sigma} = U_\sigma^\dagger M^\dagger C_n R_Z(t) M U_\sigma$.

The main contribution is presented in Section 3 providing an efficient construction of the real time evolution of Hamiltonians of the form $H = \sigma P_B$ for $[\sigma, P_B] = 0$ in the sparse case, i.e., if $|B|$ is small. The proof for the special case of Pauli X-orbit generated subspaces is provided in Section 3.1, and for the general case in Section 3.2. To be able to prove the general case we introduce the concept of subspace-controlled unitary gates in Section 2. In particular, we introduce a gate in Section 2.2 that applies a unitary conditioned on the reference state being in one of the first/last K computational basis states. We show that this gate, which we dub low/high-pass controlled unitary gate, admits an efficient realization. In Section 4 we provide several examples, where our method can be applied. One example is the transposition gate, where, to the best of our knowledge, the most efficient explicitly constructed algorithm for transposing any computational basis state requires $\mathcal{O}(n)$ X and CX gates, two MCX gates, and one clean ancilla qubit [11], whereas we achieve a realization with $\mathcal{O}(n)$ X, CX gates, only one MCX gate (with one less control), and no ancilla qubit.

2 Subspace-controlled unitary gates

Central to this paper is the concept of subspace-controlled unitary operations, which can be understood as a generalization of multi-controlled unitary gates. We start by defining the general concept, before we introduce unitary gate, controlled by the first or last k computational basis states, which serves as a building block for our main theorem.

2.1 Theory

In this paper a (sub)set of *computational basis states* of the Hilbert space of n qubits is denoted by

$$B = \{ |z_j\rangle \mid 1 \leq j \leq J, \quad z_j \in \{0, 1\}^n, \quad z_j \neq z_i \text{ for } i \neq j \}, \quad (2)$$

and the *projector* onto the subspace spanned by a B is given by

$$P_B := \sum_{|z\rangle \in B} |z\rangle \langle z|. \quad (3)$$

This allows us to generalize the notion of controlled gates through the following definition.

Definition 1 (*Subspace-controlled Unitary*)

Given a unitary $U : \mathcal{H} \rightarrow \mathcal{H}$ and a projector $P_B : \mathcal{H} \rightarrow \mathcal{H}$ with $[U, P_B] = 0$, we define a unitary operator controlled by the subspace B as

$$C_B U := (I - P_B) + P_B U P_B,$$

i.e. it acts as a unitary U on the subspace $\text{span}(B)$ and as the identity on $\text{span}(B)^\perp$.

We remark that this is well defined; Assuming $[U, P_B] = 0$, we compute the product of $C_B U$ with its adjoint. Using that U is unitary, P_B is a projection and the commutation property it follows directly that

$$C_B U (C_B U)^\dagger = I - P_B + P_B U P_B U^\dagger P_B = I,$$

and similarly for $(C_B U)^\dagger C_B U$ follows. So $C_B U$ is indeed a unitary operator. It is also easy to show the other direction that if $C_B U$ is unitary then $[P_B, U] = 0$. Note that the definition is consistent with multi-controlled unitary gates given in Equation (1), since

$$\begin{aligned} C_{\{|b\rangle_c \otimes I_t\}}(I_c \otimes U_t) &= (I - |b\rangle \langle b|_c \otimes I_t) + (|b\rangle \langle b|_c \otimes I_t)(I_c \otimes U_t)(|b\rangle \langle b|_c \otimes I_t) \\ &= (I_c - |b\rangle \langle b|_c) \otimes I_t + |b\rangle \langle b|_c \otimes U_t = C_b^c U_t. \end{aligned}$$

A transposition gate can also be interpreted as a subspace controlled unitary

$$T_{x,y} = C_{\{|x\rangle, |y\rangle\}} X^{x \oplus y},$$

where $x \oplus y$ is component-wise addition modulo 2.

As a reminder, the set of all *Pauli strings* of length n is given by

$$S_n := \left\{ i^{\mathbf{a} \cdot \mathbf{b}} X^{\mathbf{a}} Z^{\mathbf{b}} \mid \mathbf{a}, \mathbf{b} \in \mathbb{Z}_2^n, \quad \mathbf{a} \cdot \mathbf{b} = \sum_{j=1}^n a_j b_j \pmod{4} \right\}, \quad (4)$$

where $X^{\mathbf{a}} = X^{a_1} \otimes \dots \otimes X^{a_n}$, $Z^{\mathbf{b}} = Z^{b_1} \otimes \dots \otimes Z^{b_n}$. We can now interpret the time evolution of $H = \sigma P_B$ as a subspace-controlled unitary if σ and P_B commute.

Lemma 1 (Time evolution is subspace-controlled rotation)

Let P_B be a projector onto a subspace B and $\sigma \in S_n$ be a Pauli string with $[\sigma, P_B] = 0$. Then for the real time evolution of $H = \sigma P_B$ we have that

$$e^{it\sigma P_B} = C_B e^{it\sigma},$$

i.e. it is a subspace controlled Pauli evolution.

Proof. From the commutation relations and the properties of a projector and Pauli strings we obtain

$$(\sigma P_B)^{2j} = P_B, \quad (\sigma P_B)^{2j+1} = \sigma P_B. \quad (5)$$

Expanding the matrix exponential of H and using the results above, we get:

$$\begin{aligned} e^{it\sigma P_B} &= \sum_{j=0}^{\infty} \frac{(it)^j}{j!} (\sigma P_B)^j = I + \sum_{j=1}^{\infty} \frac{(it)^{2j}}{(2j)!} (\sigma P_B)^{2j} + \sum_{j=0}^{\infty} \frac{(it)^{2j+1}}{(2j+1)!} (\sigma P_B)^{2j+1} \\ &\stackrel{(5)}{=} I + (\cos(t) - 1)P_B + i \sin(t)\sigma P_B = (I - P_B) + (\cos(t) + i \sin(t)\sigma)P_B = C_B e^{it\sigma}, \end{aligned}$$

showing the assertion. $\square$

2.2 High- and low-pass controlled unitary gates

Essential for our construction is the efficient realization of a gate that applies a unitary conditioned on the reference state being in one of the first K computational basis states, which we dub low-pass controlled unitary gate. It is defined as follows

$$C_{\leq K} U := \sum_{i < K} |i\rangle \langle i|_{\mathbf{c}} \otimes U_t + \sum_{i \geq K} |i\rangle \langle i|_{\mathbf{c}} \otimes \mathbb{I}_t = (\oplus_K U)(\oplus_{2^n - K} I) = \begin{bmatrix} U & O & \dots & O \\ O & \ddots & & \\ & & U & \ddots & \vdots \\ \vdots & & \ddots & I & \\ O & \dots & O & O & I \end{bmatrix},$$

where $\oplus$ denotes the direct sum of matrices.

Theorem 1 (Low-pass controlled unitary gate)

The low-pass controlled unitary gate can be expressed as a product of $\mathcal{O}(\log(K))$ multi-controlled unitaries, i.e.

$$C_{\leq K}^{\mathbf{c}} U_{\mathbf{t}} = \prod_{i=0}^{p-1} C_{c_i(K)}^{\mathbf{c}} U_{\mathbf{t}},$$

where $c_i(K)$ is a bitstring given in Equation (6).

Proof. First, we express K in its binary representation

$$K = 2^{k_1} + 2^{k_2} \dots + 2^{k_p}, \text{ s.t. } k_1 > k_2 > \dots > k_p,$$

which means that k_i are the indices of the binary strings that are 1 in big-endian byte encoding. We partition the integer interval $[0, K - 1]$ into successive segments with dimension that are powers of two determined by the presence of 1s in the binary expansion of K . Given a number $1 \leq K \leq 2^n$ we define an auxiliary sequence that represents the cumulative sum of powers of two from 0 to i given by

$$K_j = \sum_{i=1}^j 2^{k_i}, \quad 0 \leq j \leq p.$$

It is easy to check that the binary representation of K_j to $K_{j+1} - 1$ has the first $n - k_j$ entries fixed, while the remaining indices go through all possible combinations of bitstrings from 0 to $K_{j+1} - 1$. The bits in common for the interval $[K_j, K_{j+1} - 1]$ are given by the binary representation of

$$c_j(K) = \sum_{i=1}^{j-1} 2^{k_i - k_j}, \quad (6)$$

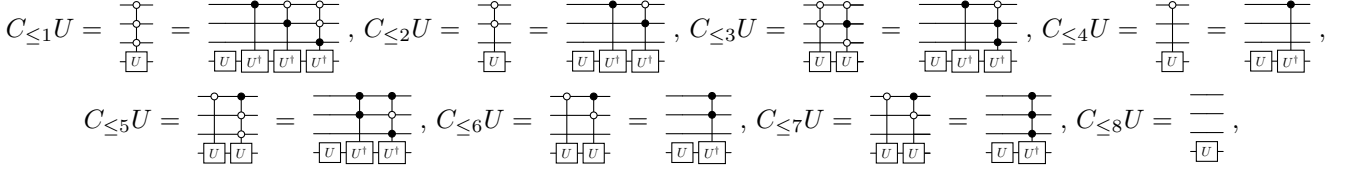


Figure 3: All possible low-pass controlled gates $C_{\leq K}^{\mathbf{c}} U_{\mathbf{t}}$ for $n = 3$ with control register $\mathbf{c} = (1, 2, 3)$ and target register $\mathbf{t} = (4)$. Note, that they can also be realized with specific high-pass controlled gates, namely $(\mathbb{I}^{\mathbf{c}} \otimes U_{\mathbf{t}})(C_{\geq K+1}^{\mathbf{c}} U_{\mathbf{t}}^{\dagger})$.

which allows us to express

$$\sum_{i=K_j}^{K_{j+1}-1} |i\rangle \langle i| = |c_j(K)\rangle \langle c_j(K)| \otimes \mathbb{I}^{\otimes k_j}$$

Using this notation, the first K -state controlled unitary operator can be decomposed as follows:

$$\begin{aligned} C_{\leq K}^{\mathbf{c}} U_{\mathbf{t}} &= \sum_{i < K} |i\rangle \langle i|_{\mathbf{c}} \otimes U_{\mathbf{t}} + \sum_{i \geq K} |i\rangle \langle i|_{\mathbf{c}} \otimes \mathbb{I}_{\mathbf{t}} \\ &= \prod_{j=0}^{p-1} \left(\sum_{i=K_j}^{K_{j+1}-1} |i\rangle \langle i|_{\mathbf{c}} \otimes U_{\mathbf{t}} + \sum_{i \notin (K_j, K_{j+1}]} |i\rangle \langle i|_{\mathbf{c}} \otimes \mathbb{I}_{\mathbf{t}} \right) \\ &= \prod_{j=0}^{p-1} \left(\sum_{i=K_j}^{K_{j+1}-1} (|c_j(K)\rangle \langle c_j(K)| \otimes \mathbb{I}^{\otimes k_j})_{\mathbf{c}} \otimes U_{\mathbf{t}} + \sum_{i \notin (K_j, K_{j+1}]} |i\rangle \langle i|_{\mathbf{c}} \otimes \mathbb{I}_{\mathbf{t}} \right) \\ &= \prod_{j=0}^{p-1} C_{c_j(K)}^{\mathbf{c}} U_{\mathbf{t}}. \end{aligned}$$

where each term in the product can be interpreted as a block matrix acting on the subspace spanned by basis states between K_j and K_{j+1} , while the identity operation applies elsewhere. $\square$

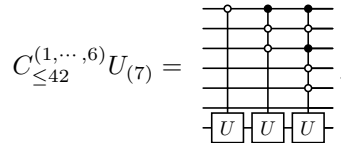
Remark 1

In the special case where $U \in SU(2)$, the number of CX gates scales as $\mathcal{O}(n \log(K))$ and the depth and number of T gates scales as $\mathcal{O}((n + \log(\frac{1}{\epsilon})) \log(K))$. The second term related to ϵ comes from the overhead from approximating single-qubit rotational gates.

As an example $K = 42 = 2^5 + 2^3 + 2^1$, i.e., $\mathbf{k} = (5, 3, 1)$. The intervals are given by

$$\begin{aligned} [K_0, K_1 - 1] &= [\underline{000000}, \underline{011111}], \\ [K_1, K_2 - 1] &= [\underline{100000}, \underline{100111}], \\ [K_2, K_3 - 1] &= [\underline{101000}, \underline{101001}], \end{aligned}$$

where the common bits, $c_1(K) = 0$, $c_2(K) = 100$, and $c_3(K) = 10100$, in the ranges are underlined. Overall, this leads to the circuit



In conclusion, the circuit consists of a sequential application of multi-controlled unitaries, where the number of control qubits increases according to the binary representation of K . Since the cost of $C^n U$ gate scales linearly in both depth and CX size with respect to the number of controls $\mathcal{O}(n)$. Summing over all required control levels up to $\log(K)$ the total cost scales as:

$$\sum_i^{\log(K)} \mathcal{O}(n - i) = \mathcal{O}(n \log(K)).$$

In the specific case of $U \in SU(2)$ the T count has the same scaling plus a contribution coming from the single qubit approximation and since we have $\log(K)$ multicontrol U this overhead scales as $\mathcal{O}(\log(k) \log(\frac{1}{\epsilon}))$.

Corollary 1 (*High-pass controlled unitary gate*)

We define the conditional gate that applies a unitary U to a target register t if the control register c is in one of the last K computational basis states as

$$C_{\geq K}^c U_t = \sum_{i \geq K} |i\rangle \langle i|_c \otimes U_t + \sum_{i < K} |i\rangle \langle i|_c \otimes \mathbb{I}_t = \prod_{i=0}^{p-1} C_{c_i(2^n - K + 1) + 1}^c U_t,$$

where $c_i(\cdot)$ is a bitstring given in Equation (6). The depth and size scale the same as the low-pass controlled gate.

The result follow from Theorem 1 by observing that counting from 0 to $K - 1$ in binary is equivalent to counting from 2^n down to $(2^n - K - 1)$ and adding $11 \dots 1$ to the strings modulo 2.

Note, that one can always write $C_{\leq K}^c U_t = (\mathbb{I}^c \otimes U_t)(C_{\geq K+1}^c U_t^\dagger)$, which sometimes allows for a more efficient realization. Figure 3 shows an example for $n = 3$, where we can see that for $K \in 3, 6, 7$ this is indeed the case.

Remark 2 (*Low-/high-pass controlled phase gates*)

The $C_{\leq K}^c \text{Ph}(t)$ gate applies a phase to the first K states and consequently has no target qubit. An example is provided in Section 4.4.

3 Main theorem

We begin by introducing an important class of subspaces that will be central to the results that follow. Let a group G act on a set X , then the *orbit* of an element $x \in X$ under this action is defined as

$$G \cdot x = \{g \cdot x \mid g \in G\}.$$

In the following we will consistently drop the $\cdot$ in the notation. We define the following class of subspaces.

Definition 2 (*Pauli X-orbit subspaces*)

Let $\bar{X}_1, \dots, \bar{X}_k$ with $\bar{X}_i \in I, X^n$ be a minimal generating set of the group $G_k = \langle \bar{X}_1, \dots, \bar{X}_k \rangle$. Then the subspace generated by an element of the set of computational basis states, $|z\rangle$, is defined by the linear span of the orbit under G_k as $\text{span}(G_k |z\rangle)$.

Note, that $|G_k |z\rangle| = 2^k$ and that $G_k |z\rangle$ consists entirely of computational basis states. Moreover, the structure of these orbits satisfies the recursion

$$G_{k+1} |z\rangle = G_k |z\rangle \oplus G_k \bar{X}_{k+1} |z\rangle, \quad (7)$$

where $\oplus$ denotes the direct sum of subspaces. With this definition in place, we are now ready to state our main theorem.

Theorem 2

Let P_B be the projector onto a subspace given by set of computational basis states B , and let $\sigma \in S_n$ be a Pauli string fulfilling $[\sigma, P_B] = 0$. Then we can realize e^{itH} for $H = \sigma P_B$ with a circuit consisting of

- at most two operators M that permute states in B to the first K states together with $M^\dagger$, and
- at most two low-pass controlled unitaries $C_{\leq K}^c U_t$ for $K \leq |B|$ and $U_t \in SU(2)$,

with the structure shown in Figure 1.

For a **general subspace** the permutation M can be realized with $\mathcal{O}(|B|)$ transposition, so in total this means we need $\mathcal{O}(n|B|)$ CX gates and $\mathcal{O}(n|B| + \log(|B|) \log(\frac{1}{\epsilon}))$ T gates, where ϵ is a given tolerance for approximating R_Z gates.

For a **Pauli X-orbit subspace** the permutation M can be realized with $\mathcal{O}(n \log(|B|))$ CX gates with depth of $\mathcal{O}(\log(n) \log(|B|))$ and one $C_x^c R_{\sigma, t}(\pm t)$, which can be realized with $\mathcal{O}(n)$ CX gates and $\mathcal{O}(n + \log(\frac{1}{\epsilon}))$ T gates.

In both cases, the circuit depth scales as the number of T gates.

The idea of the construction/proof of Theorem 2 is to partition B into sets B_i , where σ acts as a unitary $U \in SU(2)$ on each set B_s . The circuit to realize e^{itH} consist of first applying a basis change M_s for each of the sets B_s that maps to the first $|B_s|$ computational basis states, apply the unitary U , conditional on being in the first $|B_s|$ states, and apply the inverse basis change $M_s^\dagger$, as depicted in Figure 1. The proof for the case when B is a Pauli X-orbit is presented in Section 3.1, and the general case in Section 3.2.

Remark 3 (*Direct sum of Pauli X-generated subspaces*)

Even though $\text{span}(B)$ might not be a Pauli X-orbit subspace, it can be decomposed into a direct sum of Pauli

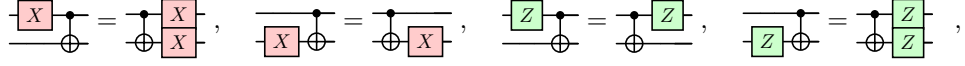


Figure 4: Permutation rules for the CX gate and Pauli operators, often presented in the context of error propagation.

X-orbit subspaces. As long as the number of those subspaces stays sufficiently small, this can still allow for an efficient construction.

3.1 Proof for Pauli X-orbit generated subspaces

An important special case, where the subspace B is given by a Pauli X-orbit of an $|z\rangle \in B$, we proof that there exists a permutation matrix M consisting only of X and CX gates.

Theorem 3

Let $B = G_k |z\rangle$ be a Pauli X-orbit of $G_k = \langle \bar{X}_1, \dots, \bar{X}_k \rangle$ with $\text{span}(B) = 2^k$ and P_B the projector onto B .

1. Then there exists a permutation operator M consisting of $\mathcal{O}(nk)$ X, CX gates such that

$$MP_B M^\dagger = I_k \otimes |0\rangle\langle 0|_{n-k}, \quad (8)$$

where I_k is the identity operator on k qubits, and $|0\rangle\langle 0|_{n-k} = \bigotimes_{i=1}^{n-k} |0\rangle\langle 0|$.

2. Let σ be a Pauli string that commutes with P_B . Then $\exists \hat{\sigma} \in S_k$ such that

$$\sigma P_B = \pm M \hat{\sigma} \otimes |0\rangle\langle 0|_{n-k} M^\dagger, \quad (9)$$

i.e. one can propagate σ through M consisting only of X, CX gates such that it acts at most as a sign change on the last $n - k$ qubits.

The proof is constructive and leads directly to an efficient algorithm to construct the circuit for both (8) and (9).

Proof. We will start by proving the first assertion by induction.

Base case $k = 1$. The subspace is given by $G_1 |z\rangle = \{|z\rangle, |w\rangle = \bar{X}_1 |z\rangle\}$. Applying a Pauli X gate on the indices where z is 1 we are left with the states $|0 \dots 0\rangle$ and $|z \oplus w\rangle$. Looking at the indices where $z \oplus w$ is 1, we see that we can use the “CX-stairs” from the GHZ state preparation circuit to map $|0 \dots 0\rangle$ to itself and $|z \oplus w\rangle$ to $|10 \dots 0\rangle$, where we relabel the indices in case the 1 in $z \oplus w$ is not in the first index. In total, we get the form shown in Equation (8) with the identity operator on the first qubit. We need at most $\mathcal{O}(n)$ X, CX gates to realize M .

Induction step $k \rightarrow k + 1$. Let the assumption hold for k , i.e. $\exists M_k$ constructed by X, CX gates, such that for $G_k = \langle \bar{X}_1, \dots, \bar{X}_k \rangle$ we have

$$M_k \sum_{|w\rangle \in G_k |z\rangle} |w\rangle\langle w| M_k^\dagger = I_k \otimes |x\rangle\langle x|. \quad (10)$$

Since M_k can be realized with only X, CX gates, we know that we can propagate Pauli-X gates through M_k . From this it follows trivially that

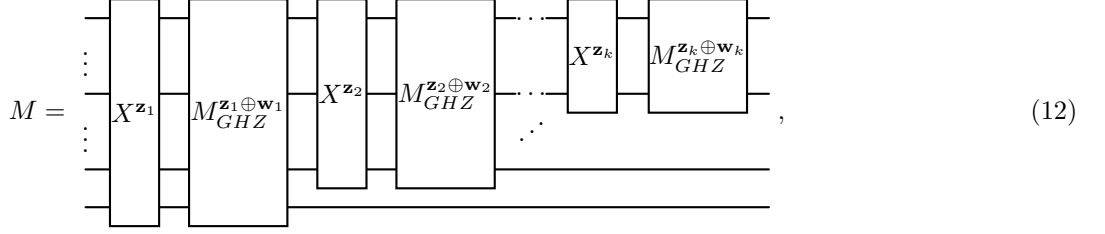
$$M_k \sum_{|w\rangle \in G_k |z\rangle} (\bar{X}_{k_1} |w\rangle\langle w| \bar{X}_{k_1}) M_k^\dagger = \hat{X}_{k_1} I_k \otimes |x\rangle\langle x| \hat{X}_{k_1} = I_k \otimes |\hat{x}\rangle\langle \hat{x}|, \quad (11)$$

where $\hat{x}$ is a bit-string. We can add Equations (10) and (11), using Equation (7) to see that

$$M_k \sum_{|w\rangle \in G_{k+1} |z\rangle} |w\rangle\langle w| M_k^\dagger = I_k \otimes (|x\rangle\langle x| + |\hat{x}\rangle\langle \hat{x}|).$$

Using the base case, we know there exists an $\widetilde{M}$ which can be realized with only X and CX gates, such that $\widetilde{M} I_k \otimes (|x\rangle\langle x| + |\hat{x}\rangle\langle \hat{x}|) \widetilde{M}^\dagger = I_{k+1} \otimes |y\rangle\langle y|$. Setting $M_{k+1} = M_k \widetilde{M}$ shows the existence of the permutation operator. Overall,

the circuit for M can be realized with



where $M_{GHZ}^{\mathbf{z}}$ is the circuit realizing the GHZ state for the states where $\mathbf{z}$ is one, *without* the Hadamard gate. Note, that each individual circuit for $M_{GHZ}^{\mathbf{z}}$ can be realized with logarithmic depth in the number of qubits [4]. Of course, all X gates can be permuted to the front. In total, the number of X, CX gates scales as $\sum_{j=0}^{k-1} (n-j) = k \cdot n - \frac{k(k-1)}{2}$, and the depth scales as $\sum_{j=0}^{k-1} \log(n-j) = \log\left(\frac{n!}{(n-k)!}\right)$.

We will now proof the second assertion. Since M is realized with X, CX gates, there exists a Pauli string $\tilde{\sigma}$ such that $\sigma M = M \tilde{\sigma}$. See Figure 4 showing the rules that can be applied for the CX-gate. Using the commutation relation, we have that

$$(I_k \otimes |x\rangle \langle x|_B) \tilde{\sigma} = M^\dagger P \sigma M = M^\dagger \sigma P M = \tilde{\sigma} (I_k \otimes |x\rangle \langle x|_B).$$

Since $|x\rangle$ is not the zero vector, it follows directly that $\tilde{\sigma}|_B |x\rangle \langle x| = \pm |x\rangle \langle x|$. $\square$

Corollary 2

Let $B = G_k |z\rangle$ so that $\text{span}(B)$ is a Pauli X -orbit generated subspace of dimension 2^k and P_B its projector. Let σ be a Pauli string that commutes with P_B . Then there exists a permutation operator M and a Pauli string $\sigma \in S_k$ such that

$$e^{it\sigma P_B} = M C_x^{\mathbf{c}} R_{\sigma, \mathbf{t}}(\pm t) M^\dagger,$$

with control $\mathbf{c} = (k+1, \dots, n)$ and target $\mathbf{t} = (1, \dots, k)$. The permutation M can be realized with $\mathcal{O}(n \log(|B|))$ X, CX gates with circuit depth $\mathcal{O}(\log(n) \log(|B|))$. The multi-controlled σ gate can be realized with $\mathcal{O}(n)$ X, CX gates with circuit depth $\mathcal{O}(n)$ and $\mathcal{O}(\log(1/\epsilon))$ T gates.

Proof. The assertion follows directly from Theorem 3, which gives us the existence of M consisting of $\mathcal{O}(n \log(|B|))$ X, CX gates with depth $\mathcal{O}(\log(n) \log(|B|))$ depth such that

$$e^{it\sigma P_B} = M e^{it(\pm \hat{\sigma}) \otimes |\mathbf{x}\rangle \langle \mathbf{x}|} M^\dagger,$$

where σ is a Pauli string acting on k qubits. The exponential of $(\pm \hat{\sigma}) \otimes |\mathbf{x}\rangle \langle \mathbf{x}|$ can be realized with a σ rotation controlled by the state $|\mathbf{x}\rangle$. $\square$

3.2 Proof for general subspaces

We divide the proof of Theorem 2 for a general subspace into four cases as follows.

3.2.1 Case $\sigma = I$

We start by mapping the (indexed) states of B to the first $|B|$ computational basis states which can be achieved by

$$M = \prod_{j=0}^{|B|-1} T_{j, z_j}.$$

It follows directly that

$$M P_B M^\dagger |j\rangle = M P_B |z_j\rangle = \begin{cases} M |z_j\rangle = |j\rangle, & \text{for } j < |B|, \\ \mathbf{0}, & \text{otherwise.} \end{cases}$$

Therefore, we have that $M P_B M^\dagger = \sum_{j < |B|} |j\rangle \langle j|$, from which it follows that the exponential has the form

$$e^{itP_B} = M^\dagger e^{it \sum_{j < |B|} |j\rangle \langle j|} M = M^\dagger C_{\leq |B|}^{\mathbf{c}} Ph(t) M$$

where $\mathbf{c} = (1, \dots, n)$.

3.2.2 Case $\sigma = Z^{\mathbf{b}}$

For $s \in \{+, -\}$ we define

$$B_s := \{|\mathbf{z}\rangle \in B \mid Z^{\mathbf{b}}|\mathbf{z}\rangle = s|\mathbf{z}\rangle\}.$$

Furthermore, after indexing the states in B_s we define the permutation matrices

$$M_s = \prod_{\substack{j=0, \\ |\mathbf{z}_j\rangle \in B_s}}^{|B_s|-1} T_{j, z_j}.$$

It follows that

$$M_s Z^{\mathbf{b}} P_{B_s} M_s^\dagger |j\rangle = M_s Z^{\mathbf{b}} P_{B_s} |z_j\rangle = \begin{cases} M_s Z^{\mathbf{b}} |z_j\rangle = s |j\rangle, & \text{for } j < |B_s|, \\ \mathbf{0}, & \text{otherwise.} \end{cases}$$

Write $P_B = P_{B_+} + P_{B_-}$ and using that $[Z^{\mathbf{b}} P_{B_-}, Z^{\mathbf{b}} P_{B_+}] = 0$ since it involves diagonal matrices, we have

$$e^{itZ^{\mathbf{b}} P_B} = \prod_{s \in \{+, -\}} e^{itZ^{\mathbf{b}} P_{B_s}} = \prod_{s \in \{+, -\}} M_s^\dagger e^{itP_{B_s}} M_s = \prod_{s \in \{+, -\}} M_s^\dagger C_{\leq |B_s|}^{\mathbf{c}} Ph(st) M_s,$$

which proofs the assertion.

3.2.3 Case $X^{\mathbf{a}} \neq I, [X_\sigma^\alpha, Z_\sigma^\beta] \neq 0$

We define the set of ordered pairs

$$E := \{(|x\rangle, |y\rangle) \in B \times B \mid X^{\mathbf{a}}|x\rangle = |y\rangle, Z^{\mathbf{b}}|x\rangle = +|x\rangle\}.$$

For $(|x\rangle, |y\rangle) \in E$ we have that $Z^{\mathbf{b}}|y\rangle = Z^{\mathbf{b}}X^{\mathbf{a}}|x\rangle = -X^{\mathbf{a}}Z^{\mathbf{b}}|x\rangle = X^{\mathbf{a}}|x\rangle = -|y\rangle$ which is the defining property for the ordering of the pairs in E . The matrix σ acts on $(|\mathbf{x}\rangle, |\mathbf{y}\rangle) \in E$ in the following way

$$\begin{aligned} \sigma|x\rangle &= i^{\mathbf{a} \cdot \mathbf{b}} X^{\mathbf{a}} Z^{\mathbf{b}}|x\rangle = i^{\mathbf{a} \cdot \mathbf{b}} X^{\mathbf{a}}|x\rangle = i^{\mathbf{a} \cdot \mathbf{b}}|y\rangle, \\ \sigma|y\rangle &= i^{\mathbf{a} \cdot \mathbf{b}} X^{\mathbf{a}} Z^{\mathbf{b}}|y\rangle = -i^{\mathbf{a} \cdot \mathbf{b}} X^{\mathbf{a}}|x\rangle = -i^{\mathbf{a} \cdot \mathbf{b}}|x\rangle. \end{aligned}$$

Again for $(|\mathbf{x}\rangle, |\mathbf{y}\rangle) \in E$ we have $|x\rangle = \sigma^2|x\rangle = \sigma i^{\mathbf{a} \cdot \mathbf{b}}|y\rangle = -i^{2(\mathbf{a} \cdot \mathbf{b})}|x\rangle$, from which it follows that $\mathbf{a} \cdot \mathbf{b} = 1 \pmod{4}$ or $\mathbf{a} \cdot \mathbf{b} = 3 \pmod{4}$, which means that $i^{\mathbf{a} \cdot \mathbf{b}} = \pm i$. After indexing the pairs in E we define the permutation matrix

$$M = \prod_{\substack{j=0, \\ (|\mathbf{x}_j\rangle, |\mathbf{y}_j\rangle) \in E}}^{|E|-1} T_{2j, \mathbf{x}_j} T_{2j+1, \mathbf{y}_j}.$$

From this it follows that

$$M \sigma P_B M^\dagger |j\rangle = \begin{cases} M \sigma |x_j\rangle = \pm i M |y_j\rangle = \pm i |j+1\rangle, & \text{for } j < |E|, j \pmod{2} = 0, \\ M \sigma |y_j\rangle = \mp i M |x_j\rangle = \mp i |j-1\rangle, & \text{for } j < |E|, j \pmod{2} = 1, \\ \mathbf{0}, & \text{for } j \geq |E|, \end{cases}$$

i.e. $M \sigma P_B M^\dagger = \pm \sum_{j=0}^{|E|-1} i (|2j\rangle \langle 2j+1| - |2j+1\rangle \langle 2j|) = \pm \sum_{j < |B|/2} |j\rangle \langle j|_{n-1} \otimes Y$. Hence, the exponential has the form

$$e^{it\sigma P_B} = M^\dagger e^{it \sum_{j < |B|/2} |j\rangle \langle j|_{n-1} \otimes (\pm Y)} M = M^\dagger C_{\leq |B|/2}^{\mathbf{c}} R_{Y, \mathbf{t}}(\pm t) M$$

where $\mathbf{c} = (1, \dots, n-1)$, and $\mathbf{t} = (n)$.

3.2.4 Case $X^{\mathbf{a}} \neq I, [X_\sigma^\alpha, Z_\sigma^\beta] = 0$

We define the set of unordered pairs

$$E_s := \{\{|x\rangle, |y\rangle\} \in B \times B \mid X^{\mathbf{a}}|x\rangle = |y\rangle, Z^{\mathbf{b}}|x\rangle = s|x\rangle\},$$

where $s \in \{+, -\}$. The matrix σ acts on $(|\mathbf{x}\rangle, |\mathbf{y}\rangle) \in E_s$ in the following way

$$\sigma|x\rangle = i^{\mathbf{a} \cdot \mathbf{b}} X^{\mathbf{a}} Z^{\mathbf{b}}|x\rangle = s i^{\mathbf{a} \cdot \mathbf{b}}|y\rangle,$$

and likewise for $\sigma|y\rangle = si^{\mathbf{a}\cdot\mathbf{b}}|x\rangle$. Furthermore, for $(|x\rangle, |y\rangle) \in E_s$ we have $|x\rangle = \sigma^2|x\rangle = s\sigma i^{\mathbf{a}\cdot\mathbf{b}}|y\rangle = i^{2(\mathbf{a}\cdot\mathbf{b})}|x\rangle$, from which it follows that $\mathbf{a}\cdot\mathbf{b} = 0 \pmod{4}$ or $\mathbf{a}\cdot\mathbf{b} = 2 \pmod{4}$, which means that $i^{\mathbf{a}\cdot\mathbf{b}} = \pm 1$. After indexing the pairs in E we define the permutation matrices

$$M_s = \prod_{\substack{j=0, \\ \{|x_j\rangle, |y_j\rangle\} \in E_s}}^{|E_s|-1} T_{2j, \mathbf{x}_j} T_{2j+1, \mathbf{y}_j}.$$

From this it follows that

$$M_s \sigma P_{B_s} M_s^\dagger |j\rangle = \begin{cases} M\sigma|x_j\rangle = \pm s M|y_j\rangle = \pm s|j+1\rangle, & \text{for } j < |E|, j \pmod{2} = 0, \\ M\sigma|y_j\rangle = \pm s M|x_j\rangle = \pm s|j-1\rangle, & \text{for } j < |E|, j \pmod{2} = 1, \\ \mathbf{0}, & \text{for } j \geq |E|, \end{cases}$$

i.e. $M_s \sigma P_{B_s} M_s^\dagger = \pm s \sum_{j=0}^{|E_s|-1} (|2j\rangle\langle 2j+1| + |2j+1\rangle\langle 2j|) = \pm s \sum_{j < |B_s|/2} |j\rangle\langle j|_{n-1} \otimes X$. Define $B_\pm = \{|x\rangle | \{|x\rangle, |y\rangle\} \in E_\pm\}$. and write $P_B = P_{B_+} + P_{B_-}$. Note, that $B_+ \cap B_- = \{\}$. Observe that

$$\sigma P_{B_s} = \sigma \sum_{\{|x\rangle, |y\rangle\} \in E_s} (|x\rangle\langle x| + |y\rangle\langle y|) = \pm s \sum_{\{|x\rangle, |y\rangle\} \in E_s} (|y\rangle\langle x| + |x\rangle\langle y|) = P_{B_s} \sigma,$$

which leads to $[\sigma P_{B_-}, \sigma P_{B_+}] = \sigma[P_{B_-}, \sigma]P_{B_+} + \sigma\sigma[P_{B_-}, P_{B_+}] + [\sigma, \sigma]P_{B_+}P_{B_-} + \sigma[\sigma, P_{B_+}]P_{B_-} = 0$. Therefore, we have that

$$e^{it\sigma P_B} = \prod_{s \in \{+, -\}} e^{it\sigma P_{B_s}} = \prod_{s \in \{+, -\}} M_s^\dagger C_{\leq |B_s|/2}^{\mathbf{c}} R_{X, \mathbf{t}}(\pm st) M_s,$$

where $\mathbf{c} = (1, \dots, n-1)$, and $\mathbf{t} = (n)$.

This concludes the proof and we continue with example applications.

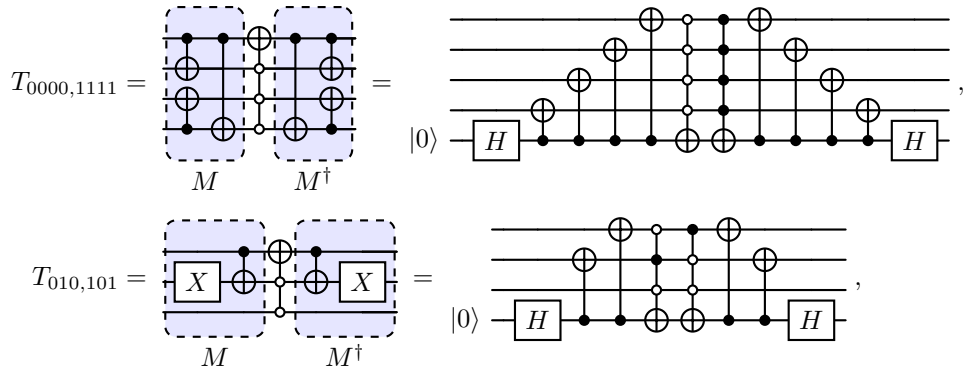
4 Examples

4.1 Transposition gates

As an example we can apply Theorem 3 to the transposition gate which can be written as,

$$T_{\mathbf{x}, \mathbf{y}} = X^{\mathbf{x} \oplus \mathbf{y}} P_{\{\mathbf{x}, \mathbf{y}\}},$$

i.e. we can apply Theorem 3 for $\sigma = X^{\mathbf{x} \oplus \mathbf{y}}$ and $B = \langle X^{\mathbf{x} \oplus \mathbf{y}} | \mathbf{x} \rangle$. Since the subspace is generated by a Pauli X-orbit, according to Equation (12) $M = X^{\mathbf{x}} M_{GHZ}^{\mathbf{x} \oplus \mathbf{y}}$ consists of at most $\mathcal{O}(n)$ X and CX gates and has depth $\mathcal{O}(\log(n))$. Compared to the method proposed in [11] this approach reduces the resources to realize the transposition gate, by not requiring an ancilla qubit and using only one multi-controlled Toffoli gate (with one less control). We present two typical examples for 4 and 3 qubits realizing a transposition gate between two computational basis states through



where the left is the proposed method, and the right is the one from [11].

4.2 Fermionic excitations

The unitary coupled cluster ansatz [12] for simulating molecules requires to realize fermionic excitation operators defined by the exponential of the skew-Hermitian operators

$$T_i^k = a_k^\dagger a_i - a_i^\dagger a_k, \quad T_{i,j}^{k,l} = a_k^\dagger a_l^\dagger a_i a_j - a_i^\dagger a_j^\dagger a_k a_l, \quad \dots, \quad T_{i_1, \dots, i_n}^{k_1, \dots, k_n} = \prod_{j=1}^n a_{k_j}^\dagger a_{i_j} - \prod_{j=1}^n a_{i_j}^\dagger a_{k_j}$$

related to single, double, and higher excitation operators, respectively. Here, $a_i^\dagger$ and a_i refer to the fermionic creation and annihilation operators. The Jordan-Wigner mapping is given by $a_i = Q_i \prod_{r=0}^{i-1} Z_r$, $a_i^\dagger = Q_i^\dagger \prod_{r=0}^{i-1} Z_r$ with the qubit creation and annihilation operators defined as $Q_i^\dagger = \frac{1}{2}(X_i - iY_i)$ and $Q_i = \frac{1}{2}(X_i + iY_i)$. In this case, the n qubit excitation operators can be expressed as

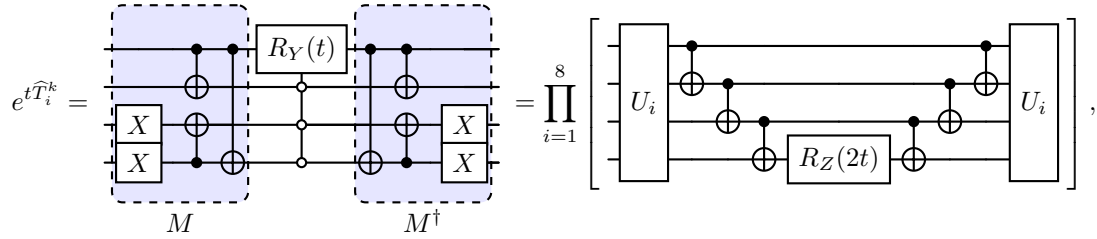
$$\hat{T}_{i_1, \dots, i_n}^{k_1, \dots, k_n} = \prod_{j=1}^n Q_{k_j}^\dagger Q_{i_j} - \prod_{j=1}^n Q_{i_j}^\dagger Q_{k_j} = -iZ_{i_1} \prod_{j=1}^n X_{i_j} X_{k_j} G, \quad (13)$$

where G is the group generated by the minimal set of Pauli operators $\{Z_{i_j} Z_{i_{j+1}}, Z_{k_j} Z_{k_{j+1}} \mid 1 \leq j \leq n-1\} \cup \{-Z_{i_n} Z_{k_1}\}$, and we define $\sigma H := \frac{1}{|H|} \sum_{h \in H} \sigma h$ for a group $H = \langle \sigma_1, \dots, \sigma_k \rangle$. Interpreting this through the lens of stabilizer codes used in quantum error correction, it is easy to see that the subspace of the projector P_B is given by the stabilizer subspace $B = \{|0 \dots 01 \dots 1\rangle, |1 \dots 10 \dots 0\rangle\}$, and σ is any logical Y -operator times i , e.g. for $X_\sigma^\alpha = X \dots X$ and $Z_\sigma^\beta = Z I \dots I$ in symplectic form.

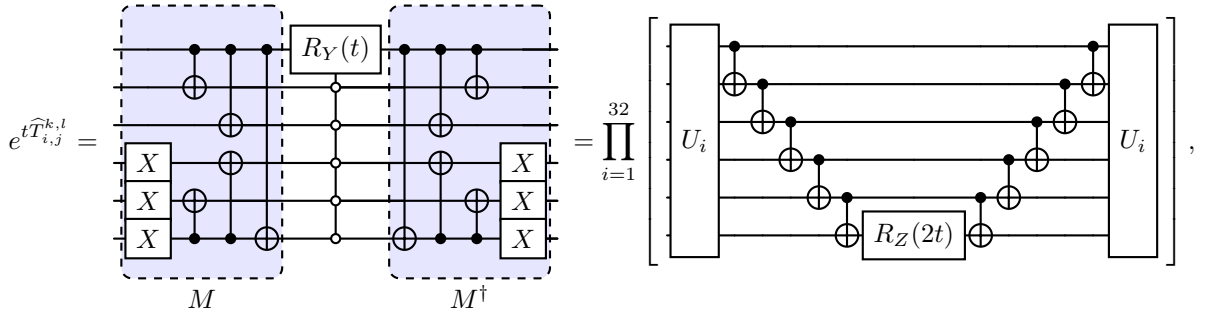
The **n-qubit excitation operator** expressed in the Pauli basis as in Equation (13) has 2^{2n-1} non-zero Pauli strings, meaning one needs an exponential number of R_Z and therefore also T gates when the circuit is realized through Pauli evolution. On the other hand, since the subspace is generated by a Pauli X-orbit, applying the proposed method yields a circuit composed of $M = X^\mathbf{x} M_{GHZ}^1$ for $\mathbf{x} = 0 \dots 01 \dots 1$ according to Equation (12) and one $n-1$ controlled R_Y gate. This leads to a circuit with $\mathcal{O}(n)$ X, CX gates and $\mathcal{O}(n + \log(\frac{1}{\epsilon}))$ T-gates for realizing the n-qubit excitation operator.

Note, that the fermionic unitaries can be efficiently obtain from the unitary evolution of qubit excitation operators [21], using the circuit realization depicted in Figure 2, so that it is sufficient to be able to realize qubit excitation operators efficiently.

As an example, one can realize the parametrized first qubit excitation operator through the circuit



and second qubit excitation operator through



where the left is the proposed method and the right is through Pauli evolution of all non-zero Pauli strings.

4.3 Trace gate for lattice gauge theory

Here we want to construct quantum circuits realizing the trace gate proposed in [1] for the Dihedral group D_N when $N = 2^n$. The trace gate operator is a diagonal operator of the following form :

$$H_{\text{Tr}}^{2^n} = |0\rangle \langle 0| \otimes \sum_{k=0}^{2^n-1} \cos\left(\frac{2\pi k}{2^n}\right) |k\rangle \langle k|.$$

Observe, that for $k > 0$ the two states $\{k, 2^{n-1} + k\}$ as well as the two states $\{2^n - k, 2^{n-1} - k\}$ are connected with a flip of the first bit. Hence, the subspace can be generated by

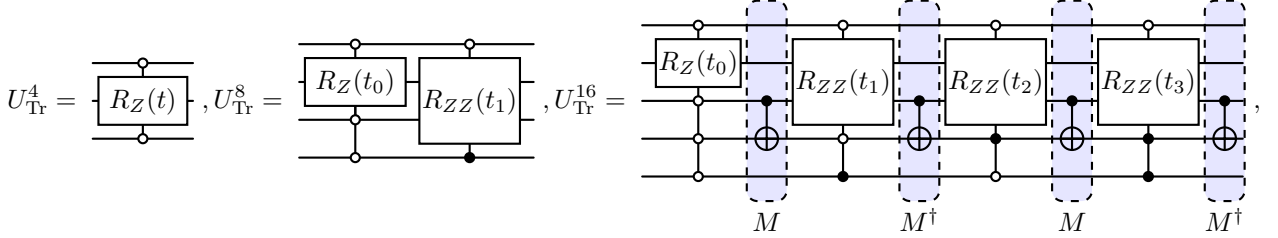
$$P_k = \langle X_1, X^{\text{bin}(k) \oplus \text{bin}(2^n - k)} \rangle |k\rangle.$$

Looking at the relative moduli of the four connected states, this allows us to rewrite

$$H_{\text{Tr}}^{2^n} = |0\rangle\langle 0| \otimes Z \otimes |0\rangle\langle 0|_{n-1} + |0\rangle\langle 0| \otimes \sum_{k=1}^{2^{n-2}-1} \cos\left(\frac{2\pi k}{2^n}\right) \sigma P_k$$

with $\sigma = Z \otimes Z \otimes I_{n-2}$ and $P_k = I_2 \otimes |k\rangle\langle k|_{n-2}$. It is important to notice that this implementation scales as 2^{n-2} but implements the exact operator reducing the complexity by a factor four applying phases for groups of 4 states.

As an example, the circuits for the trace gate for $n = 2, 3, 4$ can be realized with



where R_{ZZ} can be realized with two CX gates, see Figure 2.

4.4 Oracle for MAX k -CUT

Another example is the realization of the oracle operator for the MAX k -CUT problem. Given a weighted undirected graph $G = (V, E)$, the MAX k -CUT problem seeks a partition of the vertex set V into k subsets that maximizes the total weight of edges connecting vertices in different subsets. By assigning a label $x_i \in 1, \dots, k$ to each vertex $i \in V$, the MAX k -CUT cost function can be expressed as

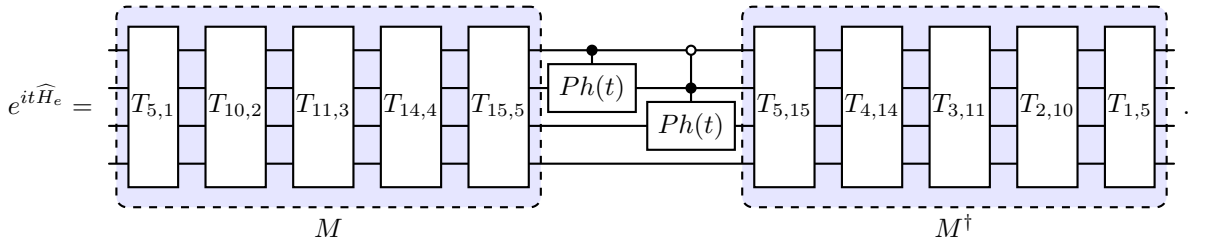
$$\max_{\mathbf{x} \in \{1, \dots, k\}^n} C(\mathbf{x}), \quad C(\mathbf{x}) = \sum_{(i,j) \in E} \begin{cases} w_{ij}, & \text{if } x_i \neq x_j \\ 0, & \text{otherwise,} \end{cases}$$

where $w_{ij} > 0$ is the weight of the edge $(i, j) \in E$. After encoding the labels into $L_k := \lceil \log_2(k) \rceil$ qubits the resulting oracle can be written as the sum of local diagonal projectors, i.e. $H_P = \sum_{e \in E} w_e H_e$, where each local term can be expressed as

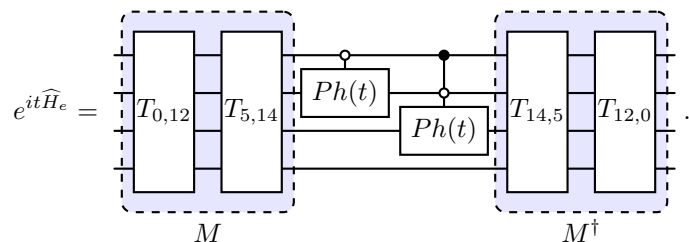
$$H_e = 2\widehat{H}_e - I, \quad \widehat{H}_e = \sum_{(i,j) \in \text{clr}} |i\rangle\langle i| \otimes |j\rangle\langle j|,$$

where $\text{clr} = \{\text{sets of equivalent colors}\}$. Notice that $\widehat{H}_e$ contains $\mathcal{O}(k \max_k(|\text{clr}_k|))$ diagonal projection operators of the form $|ij\rangle\langle ij|$ and can be implemented using the Theorem 2 for $\sigma = I$ resulting in a scaling of $\mathcal{O}(nk \max_k(|\text{clr}_k|))$ X, and CX gates and $\mathcal{O}(nk \max_k(|\text{clr}_k|) + \log(k \max_k(|\text{clr}_k|) \log(1/\epsilon)))$ T gates.

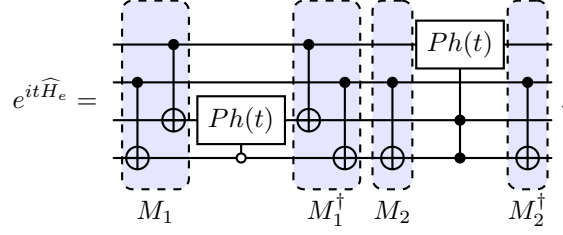
We provide an example we construct the $\widehat{H}_e$ for the MAX 3-CUT problem. Defining $\text{clr} = \{\{0, 0\}, \{1, 1\}, \{2, 2\}, \{3, 3\}, \{2, 3\}, \{3, 2\}\}$ we see that $\widehat{H}_e = P_B$ for $B = \{|0000\rangle, |0101\rangle, |1010\rangle, |1111\rangle, |1011\rangle, |1110\rangle\}$ or in integer representation $B = \{|0\rangle, |5\rangle, |10\rangle, |15\rangle, |11\rangle, |14\rangle\}$. Applying Theorem 2 directly gives the following circuit



By looking at the states carefully, we can apply transpose the states $|0\rangle$ to $|12\rangle$ and $|5\rangle$ to $|14\rangle$ to arrive the circuit



Following Remark 3 an alternative is to divide B into two sets that are generated by Pauli X-orbits, e.g. $B_2 = \langle X_2 X_4 \rangle |1011\rangle$ and $B_1 = \langle X_1 X_3, X_2 X_4 \rangle |0000\rangle$. We can then realize the oracle equivalently with the circuit



Comparing this with the circuit from [8] for $k = 3$ shows a drastic improvement in usage of ancilla qubits and gate counts.

4.5 Mixers for Constrained Optimization

Here, we want to show some examples of how to use the formalism introduce previously to construct constrained LX-mixers [7]. In this case we want construct mixing unitaries of the form

$$U_M(t) = e^{-itH_M}, \quad H_M = \sum_{j < k} (T)_{j,k} H_{z_j \leftrightarrow z_k}, \quad H_{z_j \leftrightarrow z_k} = |z_j\rangle\langle z_k| + |z_k\rangle\langle z_j|.$$

The Hamiltonian H_M is called valid [9] if the graph G_T of the adjacency matrix T is undirected and connected. Given a feasible set B we define the family of graphs $(G_{\bar{X}})_{\bar{X} \in \{I, X\}^n \setminus \{I\}^n}$ where $G_{\bar{X}} = (B, E_{\bar{X}})$. This gives rise to a family of mixers $(H_{\bar{X}})_{\bar{X} \in \{I, X\}^n \setminus \{I\}^n}$

$$H_{\bar{X}} = \sum_{j < k} (T_{\bar{X}})_{j,k} H_{z_j \leftrightarrow z_k} = \sum_{\{|x\rangle, |y\rangle\} \in E_{\bar{X}}} H_{x \leftrightarrow y} = \bar{X} P_{V_{\bar{X}}},$$

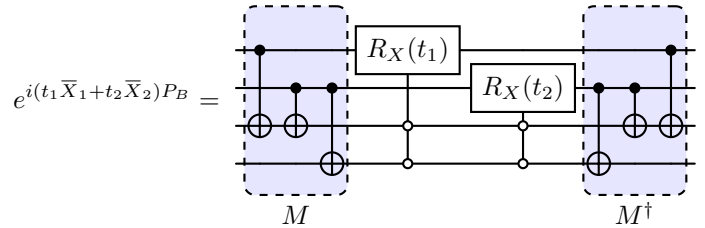
where $T_{\bar{X}}$ is the adjacency matrix of the graph $G_{\bar{X}}$. As we can see each term in $H_{\bar{X}}$ correspond to an Hamiltonian of the form σP and so we can use the main result in the case $\sigma = X$ for implementing the real time evolution.

4.5.1 Mixer for subspace generated by Pauli X-orbits

In the specific case where the subspace generated by a Pauli X-orbit, i.e. $B = \langle \bar{X}_1, \dots, \bar{X}_k \rangle |z\rangle$, for some reference state $|z\rangle$, it is interesting to notice that the graph $G = \bigcup_{i=1}^k G_{\bar{X}_i}$ is related to the valid adjacency of a k -regular graph with 2^k vertices. In fact the graph is has a number of vertices equal to the dimension of the generated subspace 2^k and each vertex is connected to exactly k other vertices by the generators of the subgroup $\bar{X}_i$. In particular, since the subspace is generated by a Pauli X-orbit, each generator can be propagated through the basis change M resulting in the following mixer:

$$U_M(t) = e^{it \sum_{j=1}^k \bar{X}_j P_B} = e^{it \sum_{j=1}^k M \bar{X}_j \otimes |0\rangle\langle 0|_{n-k} M^\dagger} = M \prod_{j=1}^k C_0^c e^{it \bar{X}_j} M^\dagger, \quad \mathbf{c} = (n, \dots, n-k+1)$$

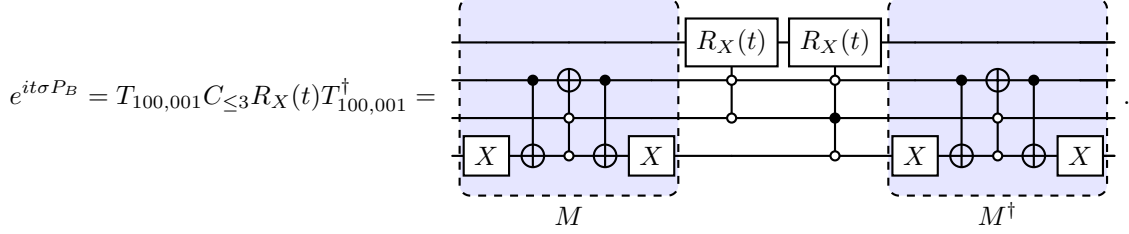
As an example we want to realize a mixing operator for the subspace given by $B = \{|0000\rangle, |1010\rangle, |0111\rangle, |1101\rangle\} = \langle X_1 X_3, X_2 X_3 X_4 \rangle |0000\rangle$. We note that $P_B = 1/4 \langle Z_2 Z_4, Z_1 Z_3 Z_4 \rangle$. Defining $\bar{X}_1 = X_1 X_3$, $\bar{X}_2 = X_2 X_3 X_4$ and following the algorithm in the proof of Theorem 3, we can realize



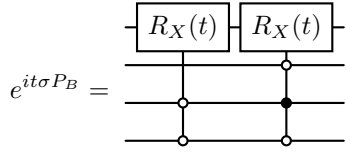
If we on the other hand apply the LX-mixer [7] to the problem, we realize $\bar{X}_1$ and $\bar{X}_2$ with the time evolution of four Pauli terms each, which leads to 8 R_Z gates.

4.5.2 LX-mixer

Next, we want to create a mixer for the subspace $B = \{|0000\rangle, |1000\rangle, |0100\rangle, |1100\rangle, |0010\rangle, |1010\rangle\}$ mixing the pairs connected by $\sigma = X_1$. Let's start by noticing that $P_B = \mathbb{I} \otimes \sum_{z' \in B'} |z'\rangle \langle z'|$ where $B' = \{|000\rangle, |100\rangle, |010\rangle\}$ since we have all pairs of states that differ only on the first bit. We want to map this 3 quantum states to be the first 3 states of the computational basis so we need then to apply the transposition gate $T_{100,001}$. In this new basis we can implement the evolution circuit for e^{-itH} using the low-pass control operator resulting in the following circuit



An alternative is to divide B into subspaces that are Pauli X-orbit generated, e.g. into $B_1 = \langle X_1, X_2 \rangle |0000\rangle$ and $B_2 = \langle X_1 \rangle |0010\rangle$, resulting in $P_{B_1} = \langle Z_3, Z_4 \rangle$, and $P_{B_2} = \langle Z_2, -Z_3, Z_4 \rangle$. Therefore, the Hamiltonian can also be generated with



On the other hand, using the method from [7], we can realize $\bar{X}$ with the time evolution of 12 Pauli terms and equally many R_Z gates.

5 Conclusion

We have presented a constructive and resource-efficient method to implement the real-time evolution of Hamiltonians of the form σP_B , emphasizing compactness with respect to non-transversal gates. Our approach provides significant improvements for both general and Pauli X-orbit generated subspaces, leading to practical circuits applicable across variational quantum algorithms. Notably, we demonstrate how standard quantum operations, such as fermionic excitations and constrained mixers, can be realized with reduced T-gate and ancilla requirements. These findings contribute towards more scalable implementations on fault-tolerant quantum architectures. In future work we plan to apply these methods to suitable problems.

6 Author contributions

Franz G. Fuchs and Ruben Pariente Bassa formulated the concept, developed the methodology, made the formal analysis and investigation, wrote the article, made the visualizations in equal parts.

7 Funding

This work has been funded by the Research Council of Norway through project number 332023.

8 Declarations

The authors declare no competing interests.

References

- [1] M. Sohaib Alam, Stuart Hadfield, Henry Lamm, and Andy C. Y. Li. Primitive quantum gates for dihedral gauge theories. *Physical Review D*, 105(11), 06 2022. ISSN 2470-0029. doi:10.1103/physrevd.105.114501.
- [2] Baptiste Claudon, Julien Zylberman, César Feniou, Fabrice Debbaesch, Alberto Peruzzo, and Jean-Philip Piquemal. Polylogarithmic-depth controlled-not gates without ancilla qubits. *Nature Communications*, 15(1), July 2024. ISSN 2041-1723. doi:10.1038/s41467-024-50065-x.

- [3] Alexander Cowtan, Silas Dilkes, Ross Duncan, Will Simmons, and Seyon Sivarajah. Phase gadget synthesis for shallow circuits. In Bob Coecke and Matthew Leifer, editors, *Proceedings 16th International Conference on Quantum Physics and Logic*, volume 318 of *Electronic Proceedings in Theoretical Computer Science*, page 213–228. Open Publishing Association, 06 2019. doi:10.4204/eptcs.318.13.
- [4] Diogo Cruz, Romain Fournier, Fabien Gremion, Alix Jeannerot, Kenichi Komagata, Tara Tosić, Jarla Thiesbrummel, Chun Lam Chan, Nicolas Macris, Marc-André Dupertuis, and Clément Javerzac-Galy. Efficient quantum algorithms for ghz and w states, and implementation on the ibm quantum computer. *Advanced Quantum Technologies*, 2(5–6), 04 2019. ISSN 2511-9044. doi:10.1002/qute.201900015.
- [5] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv*, 2014. doi:10.48550/arXiv.1411.4028.
- [6] Austin G Fowler, Simon J Devitt, and Cody Jones. Surface code implementation of block code state distillation. *Scientific reports*, 3(1):1939, 2013. doi:10.1038/srep01939.
- [7] Franz G. Fuchs and Ruben Pariente Bassa. Lx-mixers for qaoa: Optimal mixers restricted to subspaces and the stabilizer formalism. *Quantum*, 8:1535, 11 2024. ISSN 2521-327X. doi:10.22331/q-2024-11-25-1535.
- [8] Franz G Fuchs, Herman Øie Kolden, Niels Henrik Aase, and Giorgio Sartor. Efficient encoding of the weighted max k-cut on a quantum computer using qaoa. *SN Computer Science*, 2(2):89, 2021. doi:10.1007/s42979-020-00437-z.
- [9] Stuart Hadfield, Zhihui Wang, Bryan O’Gorman, Eleanor G Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019. doi:10.3390/a12020034.
- [10] Yong He, Mingxing Luo, E. Zhang, Hong-Ke Wang, and Xiao-Feng Wang. Decompositions of n-qubit toffoli gates with linear circuit complexity. *International Journal of Theoretical Physics*, 56, 07 2017. doi:10.1007/s10773-017-3389-4.
- [11] Steven Herbert, Julien Sorci, and Yao Tang. Almost-optimal computational-basis-state transpositions. *Physical Review A*, 110(1):012437, 2024. doi:10.1103/PhysRevA.110.012437.
- [12] Mark R. Hoffmann and Jack Simons. A unitary multiconfigurational coupled-cluster method: Theory and applications. *The Journal of Chemical Physics*, 88(2):993–1002, 01 1988. ISSN 1089-7690. doi:10.1063/1.454125.
- [13] Vadym Kliuchnikov, Dmitri Maslov, and Michele Mosca. Fast and efficient exact synthesis of single-qubit unitaries generated by clifford and t gates. *Quantum Info. Comput.*, 13(7–8):607–630, 07 2013. ISSN 1533-7146. doi:10.26421/QIC13.7-8-4.
- [14] Daniel Litinski. Magic state distillation: Not as costly as you think. *Quantum*, 3:205, 2019. doi:10.22331/q-2019-12-02-205.
- [15] Mikko Möttönen¹ and Juha J Vartiainen. Decompositions of general quantum gates. *Trends in quantum computing research*, page 149, 2006.
- [16] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O’Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5(1), 07 2014. ISSN 2041-1723. doi:10.1038/ncomms5213.
- [17] Debora Ramacciotti, Andreea I Lefterovici, and Antonio F Rotundo. Simple quantum algorithm to efficiently prepare sparse states. *Physical Review A*, 110(3):032609, 2024. doi:10.1103/PhysRevA.110.032609.
- [18] Neil J. Ross and Peter Selinger. Optimal ancilla-free clifford+t approximation of z-rotations. *Quantum Info. Comput.*, 16(11–12):901–953, 09 2016. ISSN 1533-7146. doi:10.26421/QIC15.11-12-4.
- [19] Rafaella Vale, Thiago Melo D Azevedo, Ismael Araújo, Israel F Araujo, and Adenilton J da Silva. Decomposition of multi-controlled special unitary single-qubit gates. *arXiv*, 2023. doi:10.48550/arXiv.2302.06377.
- [20] David Wierichs, Maxwell West, Roy T Forestano, M Cerezo, and Nathan Killoran. Recursive cartan decompositions for unitary synthesis. *arXiv*, 2025. doi:10.48550/arXiv.2503.19014.
- [21] Yordan S Yordanov, David RM Arvidsson-Shukur, and Crispin HW Barnes. Efficient quantum circuits for quantum computational chemistry. *Physical Review A*, 102(6):062612, 2020. doi:10.1103/physreva.102.062612.

Paper III



Encodings of the Weighted Max- k -Cut Problem on Qubit Systems

Franz G. Fuchs
Ruben Pariente Bassa
Frida Lien

Abstract: The weighted Max- k -Cut problem involves partitioning a weighted undirected graph into k subsets, or colors, to maximize the sum of the weights of edges between vertices in different subsets. This paper explores encoding methods for Max- k -Cut on qubit systems, utilizing quantum approximate optimization algorithms (QAOA) and addressing the challenge of encoding integer values on quantum devices with binary variables. We examine various encoding schemes and evaluate the efficiency of these approaches. The paper presents a systematic and resource-efficient method to implement the phase separation operator for the cost function of the Max- k -Cut problem. When encoding the problem into the full Hilbert space, we show the importance of encoding the colors in a balanced way. We also explore the option to encode the problem into a suitable subspace, by designing suitable state preparations and constrained mixers (LX- and Grover-mixer). Numerical simulations on weighted and unweighted graph instances demonstrate the effectiveness of these encoding schemes.

Preprint



OPEN ACCESS

EDITED BY

Jaewoo Joo,
University of Portsmouth, United Kingdom

REVIEWED BY

Saravana Prakash Thirumuruganandham,
SIT Health, Ecuador
Bikas K Chakrabarti,
Saha Institute of Nuclear Physics (SINP), India

*CORRESPONDENCE

Ruben Pariente Bassa,
✉ scoomens@gmail.com

RECEIVED 27 May 2025

REVISED 05 August 2025

ACCEPTED 15 August 2025

PUBLISHED 04 December 2025

CITATION

Fuchs FG, Pariente Bassa R and Lien F (2025)
Encodings of the weighted MAX k -CUT
problem on qubit systems.
Front. Quantum Sci. Technol. 4:1636042.
doi: 10.3389/frqst.2025.1636042

COPYRIGHT

© 2025 Fuchs, Pariente Bassa and Lien. This is an open-access article distributed under the terms of the [Creative Commons Attribution License \(CC BY\)](#). The use, distribution or reproduction in other forums is permitted, provided the original author(s) and the copyright owner(s) are credited and that the original publication in this journal is cited, in accordance with accepted academic practice. No use, distribution or reproduction is permitted which does not comply with these terms.

Encodings of the weighted MAX k -CUT problem on qubit systems

Franz G. Fuchs¹, Ruben Pariente Bassa^{1*} and Frida Lien²

¹Department of Mathematics and Cybernetics, SINTEF AS, Oslo, Norway, ²Department of Mathematics, University of Oslo, Oslo, Norway

The weighted MAX k -CUT problem involves partitioning a weighted undirected graph into k subsets, or colors, to maximize the sum of the weights of edges between vertices in different subsets. This problem has significant applications across multiple domains. This study explores encoding methods for MAX k -CUT on qubit systems by utilizing quantum approximate optimization algorithms (QAOA) and addressing the challenge of encoding integer values on quantum devices with binary variables. We examine various encoding schemes and evaluate the efficiency of these approaches. The study presents a systematic and resource-efficient method to implement the phase separation operator for the cost function of the MAX k -CUT problem. When encoding the problem into the full Hilbert space, we show the importance of encoding the colors in a balanced way. We also explore the option of encoding the problem into a suitable subspace by designing suitable state preparations and constrained mixers (LX- and Grover-mixer). Numerical simulations on weighted and unweighted graph instances demonstrate the effectiveness of these encoding schemes, particularly in optimizing circuit depth, approximation ratios, and computational efficiency.

KEYWORDS

quantum optimization algorithm, quantum circuit compilation, quantum approximate optimization algorithm, quantum technologies, variational quantum circuits

1 Introduction and related work

The quantum approximate optimization algorithm (Farhi et al., 2014)/quantum alternating operator ansatz (Hadfield et al., 2019) (QAOA) is a widely studied hybrid algorithm for approximately solving combinatorial optimization problems encoded into the ground state of a Hamiltonian. Given an objective function $f: \{0, 1\}^n \rightarrow \mathbb{R}$, the problem Hamiltonian is defined as $H_P|x\rangle = f(x)|x\rangle$. Thus, the ground states of H_P correspond to the minima of the objective function. In the general framework of QAOA, a parameterized quantum state is prepared as follows:

$$|\gamma, \beta\rangle = U_M(\beta_p)U_P(\gamma_p)\cdots U_M(\beta_1)U_P(\gamma_1)|\phi_0\rangle.$$

Starting from an initial state $|\phi_0\rangle$, the algorithm alternates between applying the phase-separation operator $U_P(\gamma)$ and the mixing operator $U_M(\beta)$. The goal is to optimize the parameters $\gamma, \beta \in \mathbb{R}$ such that $\langle \gamma, \beta | H_P | \gamma, \beta \rangle$ is minimized.

Since the original proposal of QAOA, several extensions and variants have been introduced. One such extension is ADAPT-QAOA (Zhu et al., 2022), which iteratively adds only the terms that most contribute to lowering the expectation value. Another variant is R-QAOA (Bravyi et al., 2020), which progressively removes variables from the Hamiltonian until the reduced instance can be classically solvable. A third extension, WS-QAOA (Egger et al., 2021), incorporates solutions from classical algorithms to warm-

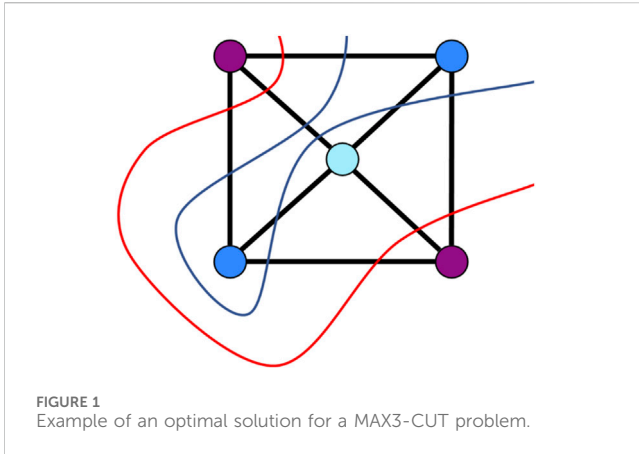


FIGURE 1
Example of an optimal solution for a MAX3-CUT problem.

start the QAOA process. When a given problem has constraints, the design of mixers is vital. For constrained problems, the design of mixers is crucial. A well-known example is the XY-mixer (Hadfield et al., 2017; Wang et al., 2020), which restricts evolution to “k-hot” states. GM-QAOA (Bartschi and Eidenbenz, 2020) uses Grover-like operators for all-to-all mixing of more general feasible subsets. The mixing operator of GM comprised a circuit U_S that prepares the uniform superposition of all feasible states, its inverse $U_S^\dagger$, and a multi-controlled rotational phase shift gate. Inspired by the stabilizer formalism, the LX-Mixer (Fuchs and Bassa, 2024; Fuchs et al., 2022) is a generalization of the above methods that provides a flexible framework that allows for tailored mixing within the feasible subspace.

To increase the practical relevance of QAOA, encoding problems with integer values, rather than binary variables, into qubit systems is an important area of research. This process generally introduces considerable flexibility in encoding choices, affecting not only the number of qubits and controlled operations needed but also the complexity of the optimization landscape and the quality of the approximation ratios that can be achieved. In this study, we explore various encoding schemes for the MAX k -CUT problem into qubit systems, building on and improving earlier research (Fuchs et al., 2021). This problem has numerous applications, including the placement of television commercials in program breaks, arrangement of containers on a ship with k bays, partitioning a set of items (e.g., books sold by an online shop) into k subsets, product module design, frequency assignment, scheduling, and pattern matching (Aardal et al., 2007; Ram Gaur et al., 2008). Given a weighted undirected graph $G = (V, E)$, the MAX k -CUT problem consists of finding a maximum-weight k -cut. This involves partitioning the vertices into k subsets such that the sum of the weights of the edges between vertices in different subsets is maximized (Figure 1). By assigning a label $x_i \in \{1, \dots, k\}$ to each vertex $i \in V$ and defining $\mathbf{x} = (x_1, \dots, x_{|V|})$, the optimization problem for MAX k -CUT can be formulated as

$$\max_{\mathbf{x} \in \{1, \dots, k\}^n} C(\mathbf{x}), \quad C(\mathbf{x}) = \sum_{(i,j) \in E} \begin{cases} w_{ij}, & \text{if } x_i \neq x_j \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

Here, $w_{ij} > 0$ is the weight of edge $(i, j) \in E$. The function $C(\mathbf{x})$ is commonly referred to as the “cost function.” For $k = 2$, the MAX k -CUT problem becomes the well-known MAX CUT problem. The

MAX k -CUT problem is NP-complete, and it has been demonstrated that no polynomial-time approximation scheme exists for $k \geq 2$ unless $P = NP$ (Alan and Jerrum, 1997). A randomized approximation algorithm for (1) has an approximation ratio α if

$$\mathbb{E}[C(\mathbf{x})] \geq \alpha C(\mathbf{x}^*),$$

where $\mathbf{x}^*$ denotes an optimal solution.

Expressing the solutions as strings of k -dits as in Equation 1 is a natural extension of the MAX (2-)CUT problem to $k > 2$. However, most quantum devices work with a two-level system, so we need to make a suitable encoding on a qubits system (Fuchs et al., 2021; Hadfield et al., 2019). One-hot encoding uses k bits for each vertex, where the single bit that is 1 encodes which set/color the vertex belongs to. Using this encoding requires $k|V|$ qubits. However, k not being a power of 2 results in more computational states per vertex than there are colors. The corresponding subspaces need to be dealt with in a suitable fashion. The Grover or the LX-mixer (which becomes the XY-mixer in this case) can be readily applied. However, the ratio of the dimension of the feasible states to the system size is $k/2^k$, which becomes exponentially small for increasing k . Therefore, it is advantageous to use a binary encoding, which does not have this problem. For a given k , we encode the information of a vertex belonging to one of the sets by $|i\rangle_{n_k}$, which requires

$$n_k := \lceil \log_2(k) \rceil$$

qubits. Here, $\lceil \cdot \rceil$ means rounding up to the nearest integer. Binary encoding requires $n_k|V|$ qubits, which is exponentially fewer than required by one-hot encoding. The resulting problem Hamiltonian can be written as the sum of local terms, that is,

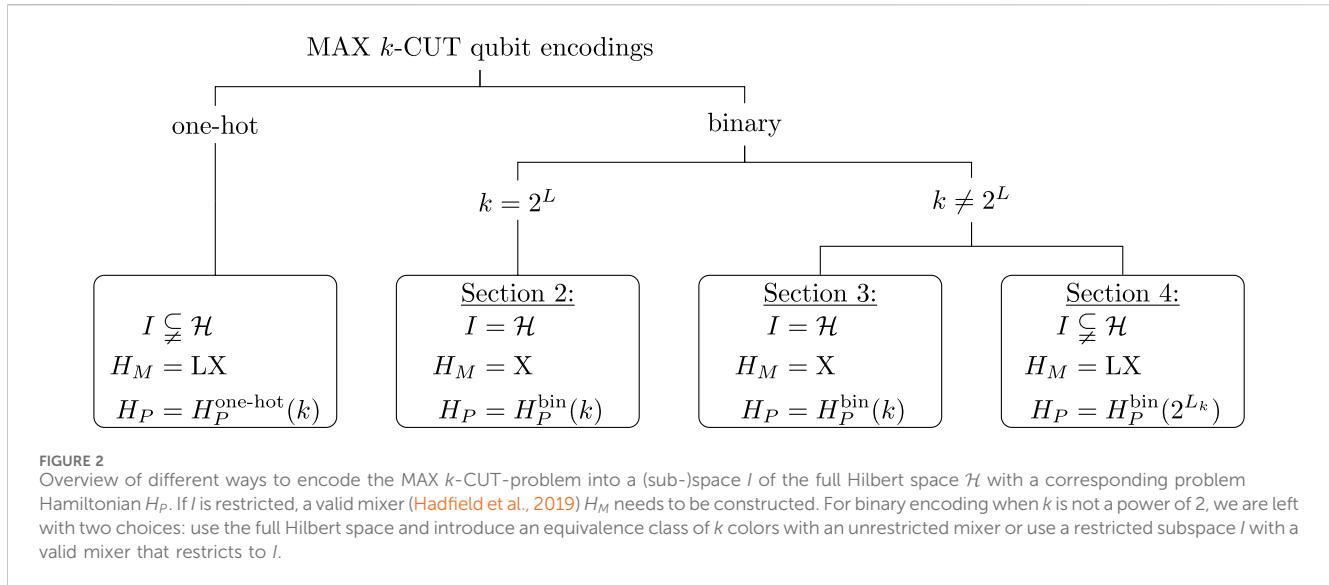
$$H_P = \sum_{e \in E} w_e H_e,$$

where $w_{i,j}$ is the weight of the edge between vertices i and j . Furthermore, the local term can be expressed as

$$H_e = 2\hat{H}_e - I, \quad \hat{H}_e = \sum_{(i,j) \in \text{clr}} |i\rangle\langle i| \otimes |j\rangle\langle j|. \quad (2)$$

where clr is a set of index pairs (i, j) representing equivalent colors under an equivalence relation. From the definition, H_e has eigenvalue +1 for a state $|i\rangle|j\rangle$ if $i \sim_{\text{clr}} j$ and -1 if not. The phase separating operator e^{-itH_e} differs from $e^{-it\hat{H}_e}$ only by a scalar factor and a global phase, so it is sufficient to realize the latter.

More formally, an equivalence relation $\sim_{\text{clr}}$ is defined on $X = \{0, \dots, 2^{n_k} - 1\}$, where clr consists of all pairs (i, j) such that $i \sim_{\text{clr}} j$. For example, if $\text{clr} = \{(0, 0), (1, 1), (2, 2), (3, 3), (2, 3), (3, 2)\}$, the equivalence relation defines the following equivalence classes: $[0] = \{0\}$, $[1] = \{1\}$, $[2] = \{2, 3\}$. For convenience, we can work with a generating set. Suppose R is a relation on the set X . The equivalence relation on X generated by a R , denoted $\bar{R}$, is then the smallest equivalence relation on X that contains R , which can be obtained by the reflexive, symmetric, and transitive closure (in that order). Hence, only the non-trivial relations must be specified, and the above example can be generated by the closure of the relation $R = \{(2, 3)\}$ which generates $\bar{R} = \{(0, 0), (1, 1), (2, 2), (3, 3), (2, 3), (3, 2)\}$. The set of



colors in the context of MAX k -CUT is then the set of equivalence classes induced by $\sim_{\text{clr}}$, denoted

$$X/\sim_{\text{clr}} := \{[x]: x \in X\},$$

with each equivalence class given by

$$[c] := \{x \in X: c \sim_{\text{clr}} x\}.$$

We recognize that the phase separating operator—the real time evolution of $\hat{H}_e$ —acts by applying a phase on a specific subsets of all computational basis states. When these subsets are sparse, such unitaries can be efficiently realized (Fuchs and Bassa, 2025): let P_B be the projector onto a subspace given by set of computational basis states B . The unitary operator e^{itP_B} can be realized by a circuit for mapping the (indexed) states of B to the first $|B|$ computational basis states, which can be achieved by $M = \prod_{j=0}^{|B|-1} T_{j,z_j}$, where $T_{i,j}$ is a transposition gate. It follows directly that

$$MP_B M^\dagger |j\rangle = MP_B |z_j\rangle = \begin{cases} M|z_j\rangle = |j\rangle, & \text{for } j < |B|, \\ \mathbf{0}, & \text{otherwise.} \end{cases}$$

Therefore, we find that $MP_B M^\dagger = \sum_{j < |B|} |j\rangle\langle j|$, from which it follows that the exponential has the form

$$e^{itP_B} = M^\dagger e^{it \sum_{j < |B|} |j\rangle\langle j|} M = M^\dagger C_{\leq |B|} Ph(t) M,$$

where $C_{\leq |B|} Ph(t)$ is a low-pass controlled phase shift gate as defined in Fuchs and Bassa (2025). For a general subspace, the overall circuit requires $\mathcal{O}(n|B|)$ CX gates and $\mathcal{O}(n|B| + \log(|B|)\log(\frac{1}{\epsilon}))$ T gates.

When the subspace is given by the span of a Pauli X-orbit as defined in Fuchs and Bassa (2025), the overall circuit requires only $\mathcal{O}(n \log(|B|))$ CX gates and $\mathcal{O}(n + \log(\frac{1}{\epsilon}))$ T gates. Here, ϵ is a given tolerance for approximating rotational gates. Even though span (B) might not be a Pauli X-orbit subspace, we can always decompose into a direct sum of Pauli X-orbit subspaces and apply the construction individually on the subspaces of the decomposition. We will therefore here aim to divide the subspaces such that they are Pauli X-orbit subspaces if it leads to an efficient circuit.

An overview of possible encoding techniques is provided in Figure 2. When k is not a power of 2, we must deal with the fact that we have non-trivial equivalence classes clr . There are two ways to deal with this. The first is to directly implement the phase separating Hamiltonian on the full Hilbert space (Fuchs et al., 2021) or restrict the evolution onto the relevant subspace. The main contributions of this study are for the cases when k is not a power of 2. We introduce:

- A systematic way to implement the phase separating Hamiltonian, resulting in a reduction of the complexity of the resulting circuit using the theorems from Fuchs and Bassa (2025).
- A method to directly work with a feasible subspace with a suitable state preparation and constrained mixer in Section 4.
- More balanced sets of equivalent colors, which help improve the resulting cost landscape, in Section 3.

We also employ the Grover mixer to all cases, indicating that the resulting landscape might be favorable. Throughout this study, the notation $C^l U$ means an l -controlled U gate. For example, a Toffoli gate is $C^2 X$, and $CPh = CPh(t)$ is the controlled phase shift gate, where we mostly omit the parameter. We will make use of the following notation for *multi-controlled unitary* (MCU) gates

$$C_{\mathbf{b}}^{\mathbf{c}} U_{\mathbf{t}} = (I_{\mathbf{c}} - |\mathbf{b}\rangle\langle \mathbf{b}|_{\mathbf{c}}) \otimes I_{\mathbf{t}} + |\mathbf{b}\rangle\langle \mathbf{b}|_{\mathbf{c}} \otimes U_{\mathbf{t}}, \quad (3)$$

where $\mathbf{c}, \mathbf{t}$ is index control and target qubits, and $\mathbf{b}$ specifies the control condition.

2 Binary encodings when k is a power of 2

When $k = 2^{n_k}$, the equivalence relation for the problem Hamiltonian in Equation 2 becomes trivial—that is, $\text{clr} = \{\} = \{(0, 0), (1, 1), \dots, (k-1, k-1)\}$ —and hence

TABLE 1 Approximation ratios achieved for the case when $k = 2^{n_k}$.

Graph	Mixer	$k = 2$	$k = 4$	$k = 8$
Erdős-Rényi	X	0.87	0.89	0.97
	Grover [□]	0.87	0.90	0.98
	Grover	0.79	0.83	0.93
Barabási-Albert	X	0.80	0.87	0.96
	Grover [□]	0.80	0.88	0.96
	Grover	0.74	0.82	0.92

$$\hat{H}_e = \sum_{i=0}^{k-1} |i\rangle\langle i| \otimes |i\rangle\langle i|. \quad (4)$$

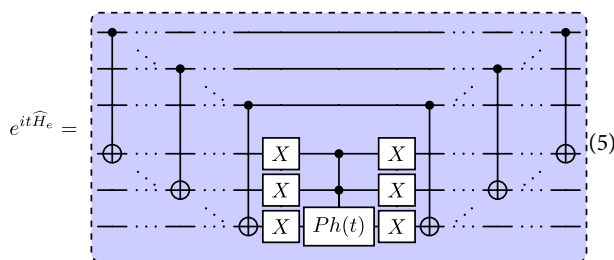
By definition, $\hat{H}_e$ is a projection operator P_B of rank k onto the subspace spanned by $B = \{|j\rangle \otimes |j\rangle, 0 \leq j < k\}$, consisting of all states with identical basis vectors in both registers. We notice that the set B is Pauli X-orbit generated as defined in Fuchs and Bassa (2025) since it can be written as

$$B = G_{n_k} |0\rangle^{\otimes 2n_k},$$

that is, the orbit of the computational basis state $|0\rangle^{\otimes 2n_k}$ under the group action of $G_{n_k} = \langle X_i \otimes X_i \rangle_{i=0}^{n_k-1}$. Here, $\langle S \rangle$ denotes the subgroup generated by a subset S of the Pauli group. To construct the phase separating operator for our concrete case, we follow the proof of Theorem 3 in Fuchs and Bassa (2025) which recursively gives us the following permutation matrix in each step, $M_j = C_1^j X_{j+n_k}$, which leaves the all-zero state unchanged and maps the state $|j\rangle \otimes |j\rangle$ to $|j\rangle \otimes |0\rangle$. Overall, this leads to

$$MP_B M^\dagger = I^{n_k} \otimes |0\rangle\langle 0|^{n_k}, \quad M = \prod_{i=0}^{n_k-1} C_1^i X_{i+n_k},$$

resulting in the following circuit:



The number of controlled gates required for realizing the mixer and phase separating operators are as follows.

- Realizing $e^{-it\hat{H}_e}$ through Equation 5 requires $2n_k$ CX gates and 1 $C^{n_k-1}Ph$ gate. Alternatively, we can also do a direct decomposition of $\hat{H}_e$ in the Pauli basis and perform Pauli evolution gates with this. For $k = 2, 4, 8$, the first approach needs $2CX$, $(4CX$ and $1CPh)$ and $(6CX$ and $1C^2Ph)$, whereas the Pauli evolution needs $2CX$, $10CX$, and $34CX$ per gate, respectively. Thus, it is better for these cases to realize the phase separation with the circuit given in Equation 5 and not through Pauli evolution.

- The X mixer does not require any controlled gates, the Grover mixer needs one $C^{|V|n_k-1}P$ gate, and the Grover[□] needs $|V| C^{n_k-1}P$ gates, where the box product is defined in Equation 11.

We numerically test the performance of the resulting QAOA algorithm on the same instances as in Fuchs et al. (2021), given in Supplementary Appendix A. Table 1 shows that the approximation ratios achieved are very good for all cases. An overview of the energy/approximation ratio landscapes are provided in Supplementary Appendix Table A3.

3 Binary encodings for $k \neq 2^{n_k}$ using the full Hilbert space

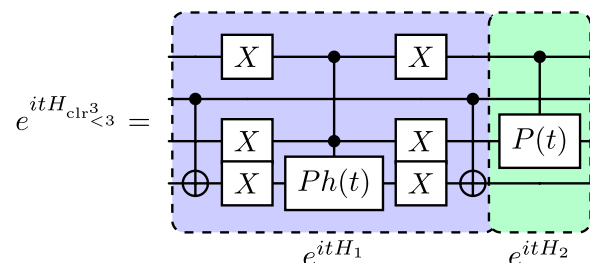
When k is not a power of 2, the equivalence relation for the problem Hamiltonian in Equation 2 becomes non-trivial. One way to deal with this is what we will refer to as $\leq k$ -encoding, where we encode the color k into all states with $|i\rangle$ for $k \leq i \leq 2^{n_k} - 1$. This gives us

$$\text{clr}_{\leq k}^k = \{(k-1, k), (k, k+1), \dots, (2^{n_k}-2, 2^{n_k}-1)\}.$$

Of course, in general there are many ways to define k colors through an equivalence relation clr , which influences the cardinality of the equivalence classes. There is, however, always a way to define it such that the maximum cardinality of all equivalence classes is at most 2. In the following, we denote such a relation as $\text{clr}_{\text{bal}}^k$. This also influences the rank of the Hamiltonian $\hat{H}_e$. In the following, we make a concrete construction of the Hamiltonians for $k \in \{3, 5, 6, 7\}$.

3.1 The case $k = 3$

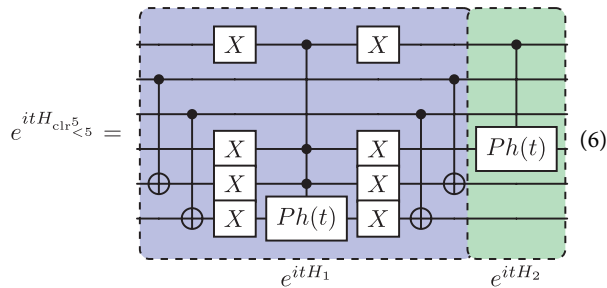
For $k = 3$, $\text{clr}_{\text{bal}}^3 = \text{clr}_{\leq 3}^3 = \{(2, 3)\}$, which means that the three colors are given by $[0] = \{0\}$, $[1] = \{1\}$, $[2] = \{2, 3\}$. The subspace of the projector is given by $B = \{|0000\rangle, |0101\rangle, |1010\rangle, |1111\rangle, |1011\rangle, |1110\rangle\}$. We can realize the phase separating operator using the low-pass controlled phase shift gate $C_{\leq 6}Ph$ in the permuted basis, as described in Fuchs and Bassa (2025). However, to reduce the circuit complexity, we can instead divide the subspace in two Pauli X-orbit generated subspaces, $B_1 = \langle X_1 X_3 \rangle |0000\rangle$ and $B_2 = \langle X_1, X_3 \rangle |1010\rangle$, which results in the circuit



3.2 The case $k = 5$

For $k = 5$, $\text{clr}_{\leq 5}^5 = \{(4, 5, 6, 7)\}$, which means that the five colors are given by $[0] = \{0\}$, $[1] = \{1\}$, $[2] = \{2\}$, $[3] = \{3\}$, $[4] = \{4, 5, 6, 7\}$.

We can divide B into two Pauli X-orbit generated subspaces. The first subspace containing four states is related to the first four colors $|000\rangle, |001\rangle, |010\rangle, |011\rangle$, forming the subset $B_1 = \langle X_1 X_4, X_2 X_5 \rangle |000000\rangle$. The second subspace contains all states related to the last degenerated color $|100\rangle, |101\rangle, |110\rangle, |111\rangle$, forming $B_2 = \langle X_1, X_2, X_4, X_5 \rangle |100100\rangle$. The resulting circuit is given by



which realizes the phase separating unitary with one single- and one triple-controlled phase shift gate.

Looking at the case for $k = 3$ and $k = 5$, it becomes clear that the structure of the circuit can be generalized for $\text{clr}_{<k}$ with $k = 2^l + 1$. The feasible subspace can always be divided into two Pauli X-orbit generated subspaces, $B_1 = \langle X_1 X_{n_k+1}, \dots, X_l X_{n_k+l} \rangle |0 \dots 0\rangle$, related to the first $k-1$ color states characterized by bitstrings $x = yy$, where the first digit of y is always 0. The second subspace related to the degenerate color is given by $B_2 = \langle X_1, \dots, X_{n_k-1}, X_{n_k+1}, \dots, X_{2n_k-1} \rangle |10 \dots 0\rangle |10 \dots 0\rangle$. From this, we can conclude that the resulting circuit is a generalization of Equation 6, where e^{itH_1} consists of the M_{GHZ} circuit where the CX between the first and n_k -th qubit is omitted and a phase shift gate the with target on the last qubit controlling the first and last n_k (without the last) qubits. The circuit for e^{itH_2} is a phase shift gate with control of the first and target on the $n_k + 1$ -th qubit.

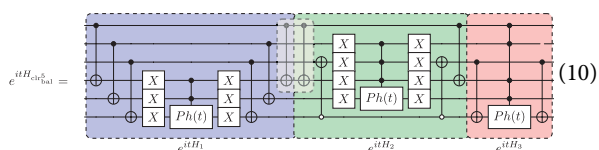
A second possibility is to encode the problem so that the cardinality of the equivalence classes is minimized. This can, for instance, be achieved by $\text{clr}_{\text{bal}}^5 = \{(0, 1), (4, 5), (6, 7)\}$. In this case, the five colors are given by $[0] = \{0, 1\}$, $[2] = \{2\}$, $[3] = \{3\}$, $[4] = \{4, 5\}$, $[6] = \{6, 7\}$ and the according Pauli X-orbit generated subspaces are

$$B_1 = \langle X_0 X_3, X_1 X_4, X_2 X_5 \rangle |000000\rangle, \quad (7)$$

$$B_2 = \langle X_0 X_3, X_2 X_5 \rangle |000001\rangle, \quad (8)$$

$$B_3 = \langle X_2 X_5 \rangle |111110\rangle. \quad (9)$$

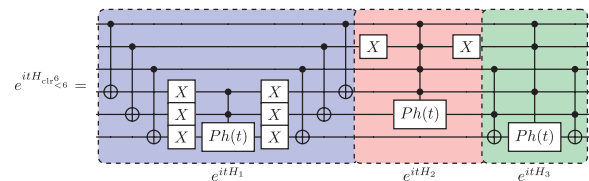
The resulting circuit for the phase separating operator is



where the gates in the gray box cancel to the identity.

3.3 The case $k = 6$

For $k = 6$, we have that $\text{clr}_{<6}^6 = \{(5, 6, 7)\}$, meaning that the six colors are given by $[0] = \{0\}$, $[1] = \{1\}$, $[2] = \{2\}$, $[3] = \{3\}$, $[4] = \{4\}$, $[5] = \{5, 6, 7\}$. We can divide B into the following two Pauli X-orbit generated subspaces $B_1 = \langle X_0 X_3, X_1 X_4, X_2 X_5 \rangle |000000\rangle$, $B_2 = \langle X_2 X_5 \rangle |110111\rangle$, together with $B_3 = \{|110101\rangle, |110111\rangle, |111101\rangle, |111110\rangle\}$, which cannot be generated by a Pauli X-orbit. The resulting circuit is given by



The circuit for the real time evolution of H_2 contains a Toffoli gate for realizing the basis change since the subspace is not Pauli X-orbit generated. In particular, the circuit for the second term H_2 can be derived considering that if we exchange the state $|111110\rangle$ and $|111111\rangle$ using a transposition gate (in this case a 5-controlled not-gate C^5X), the new set $\tilde{B}_2 = \langle X_2, X_4 \rangle |110101\rangle$ becomes Pauli X-orbit generated. Since all the states in the subset $B_2 \cup \tilde{B}_2$ have the bit value 1 in position 0,1,3,5 we can instead replace the use of C^5X by a Toffoli gate $CCX_{2,4 \rightarrow 5}$.

In order to encode the problem in a balanced way, we can use $\text{clr}_{\text{bal}}^6 = \{(0, 1), (4, 5)\}$. Since $\text{clr}_{\text{bal}}^6 \cup \{(6, 7)\} = \text{clr}_{\text{bal}}^5$, we can divide B into the subspace B_1 and B_2 shown in Equation 7; the resulting circuit is shown in Equation 10, but the circuit for H_3 is omitted.

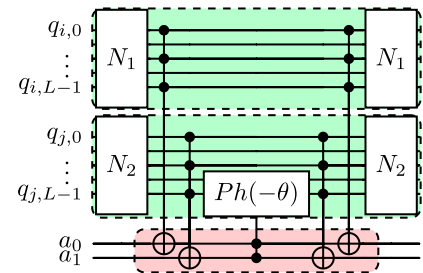
3.4 The case $k = 7$

For $k = 7$, $\text{clr}_{\text{bal}}^7 = \text{clr}_{<7}^7 = \{(6, 7)\}$, which means that the seven colors are given by $[0] = \{0\}$, $\dots$, $[5] = \{5\}$, $[6] = \{6, 7\}$. Since $\text{clr}_{\text{bal}}^7 \cup \{(0, 1), (4, 5)\} = \text{clr}_{\text{bal}}^5$, we can divide B into the subspace B_1 and B_3 shown in Equation 7, where the resulting circuit is shown in Equation 10, but the circuit for H_2 is omitted. Notice here too that the two adjacent C^2X_5 gates between the circuits for H_1 and H_3 cancel.

3.5 Resource analysis

When k is not a power of 2, the method presented in [9, Section 3.2.2] is not optimal in terms of resource efficiency; for each edge of the graph, it requires the circuit shown in Equation 5 and $a(a-1)$ times the circuit shown in Figure 3b, where $a = 2^{n_k} - (k-1)$. Each of these individual circuits uses four $C^{n_k}X$ -gates, and one C^2P gate and requires the use of two ancillary qubits. Concretely, for each edge, 2, 12, 6, 2, 56, $\dots$ of these circuits are needed for $k = 3, 5, 6, 7, 9, \dots$, respectively, with the cost shown in Figure 3a. The method derived from the theorem in Fuchs and Bassa (2025) on the other hand, requires much fewer entangling gates and does not require ancilla qubits. For a single edge, the size of the relevant

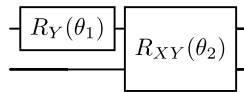
k	[9, Section 3.2.2]	this work
3	$1CPh, 2(4C^2X, 1C^2Ph), 4CX$	$1CPh, 1C^2Ph, 2CX$
5	$1C^2Ph, 12(4C^3X, 1C^2PhP), 6CX$	$1CPh, 1C^3Ph, 4CX$
6	$1C^2Ph, 6(4C^3X, 1C^2PhP), 6CX$	$1C^2Ph, 1C^3Ph, 8CX$
7	$1C^2Ph, 2(4C^3X, 1C^2PhP), 6CX$	$1C^2Ph, 1C^3Ph, 2C^2X, 6CX$



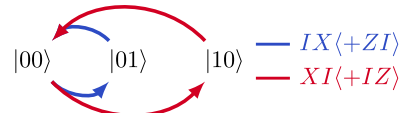
(a) Number of controlled gates to realize the phase shift gate for one edge of the graph. (b) Phase shift gate required in previous work [9].

FIGURE 3

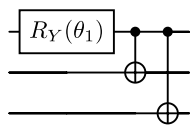
Resource comparison shows that the circuits for realizing the phase separating operator U_P as proposed here require fewer resources in terms of entangling gates as Fuchs et al. (2021). Another advantage of the proposed method is that it does not require ancilla qubits. Whereas we realize the diagonal operator directly, Fuchs et al. (2021) utilized the circuit shown in (b) between each edge (i, j) . For each element $\{y_1, y_2\} \in \text{clr}^k \setminus \{u_i, i, i\}$, one such circuit is necessary with $N_{y_i} = X^{\text{bin}(y_i)}$. (a) Number of controlled gates to realize the phase shift gate for one edge of the graph.



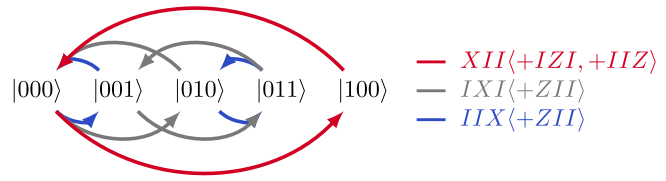
(a) $|\Phi_0^3\rangle$, $t_1 = 2 \arccos(1/\sqrt{3})$, $t_2 = \pi/2$.



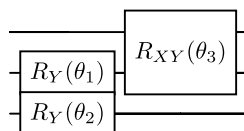
(b) One possible valid mixer for B_3 , using 4 CX gates.



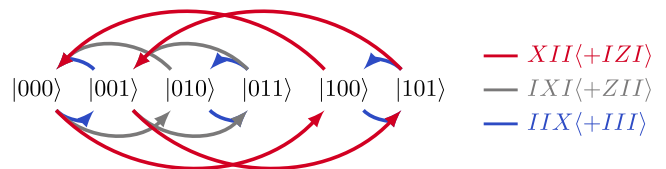
(c) $|\Phi_0^5\rangle$, $t_1 = 2 \arcsin(1/\sqrt{5})$.



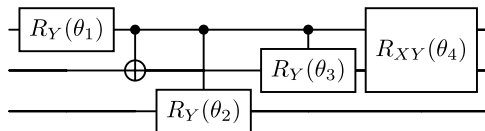
(d) One possible valid mixer for B_5 , using 12 CX gates.



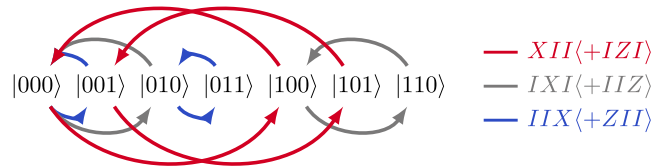
(e) $|\Phi_0^6\rangle$, $t_2 = 2 \arccos(1/\sqrt{3})$, $t_1 = t_3 = \pi/2$.



(f) One possible valid mixer for B_6 , using 4 CX gates.



(g) $|\Phi_0^7\rangle$, $t_1 = 2 \arcsin(1/\sqrt{7})$, $t_2 = t_4 = \pi/2$, $t_3 = 2 \arccos(1/\sqrt{3})$.



(h) One possible valid mixer for B_7 , using 6 CX gates.

FIGURE 4

Initial state preparation and mixer for $k = 3$ (a) and (b), $k = 5$ (c) and (d), $k = 6$ (e) and (f), $k = 7$ (g) and (h), when restricting to a suitably chosen subspace. We use the notation $R_P(t) = \exp(-i\frac{t}{2}P)$, $P \in \{X, Y, Z\}$, and $R_{XY}(t) = \exp(-i\frac{t}{4}(X \otimes X + Y \otimes Y))$.

subspace B scales as $\mathcal{O}(k^2)$ in the encoding using $\text{clr}_{\text{bal}}^k$ and as $\mathcal{O}(k^4)$ in the encoding using clr_{ck}^k . This quadratic scaling in the latter case is due to the degeneracy of the highest color level, which leads to

$(2^{2n} + 1 - k)^2$ valid computational basis states. From the theorem in Fuchs and Bassa (2025), the overall gate complexity for the two encoding strategies is as follows.

- Encoding using $\text{clr}_{\text{bal}}^k$: $\mathcal{O}(|E|k^2 \log_2(k))$ CNOTs and $\mathcal{O}(|E|k^2 \log_2(k) + 2 \log_2(k) \log_2(1/\epsilon))$ single-qubit rotations.
- Encoding using $\text{clr}_{\leq k}^k$: $\mathcal{O}(|E|k^4 \log_2(k))$ CNOTs and $\mathcal{O}(|E|k^4 \log_2(k) + 4 \log_2(k) \log_2(1/\epsilon))$ single-qubit rotations.

Here, we have explicitly constructed circuits for $k = 3, 5, 6, 7, 9, \dots$ using a decomposition similar to that in Fuchs and Bassa (2025), but with a key optimization: we partitioned subspace $|B|$ into ad hoc sums of Pauli X -orbits. This additional structure further reduces the implementation cost. See [Supplementary Appendix Table A3a](#) for a comparison.

4 Binary encodings for $k \neq 2^{n_k}$ using subspaces

When k is not a power of 2, there is an alternative to encoding the problem in the full Hilbert space as presented in [Section 3](#). Instead, the evolution of the QAOA can be restricted to a suitably chosen subspace spanned by a set B consisting of k computational basis states. Consequently, we need a way to prepare an initial state in a k -dimensional subspace for each vertex of the graph and a mixer constrained to this subspace. We observe that

$$H_p^{\text{bin}}(2^{n_k})|_{\text{span}(B)} = H_p^{\text{bin}}(k),$$

meaning we can use the circuit for the power-of-2 case. Overall, this approach shifts complexity from the phase separating Hamiltonian to the mixer.

4.1 Initial state

For the MAX k -CUT problem, one possibility is to define the subspace for each vertex through $B_k = \{|i\rangle \mid 0 \leq i \leq k-1\}$. Since the feasible bit-strings admit a Cartesian product structure, the feasible subspace becomes a tensor product of the form

$$B_k^1 \otimes \dots \otimes B_k^{|V|}.$$

An initial state can be prepared as the uniform superposition of all feasible basis states—

$$|\Phi_0^{\leq k}\rangle^{\otimes |V|}, \text{ where } |\Phi_0^{\leq k}\rangle = \frac{1}{\sqrt{k}} \sum_{i=0}^{k-1} |i\rangle.$$

This can, in general, be efficiently prepared with a gate complexity and circuit depth of only $\mathcal{O}(\log_2(k))$ (Shukla and Vedula, 2024). One way to realize the initial state for $k \in \{3, 5, 6, 7\}$ is shown in [Figures 4a–h](#).

4.2 Constrained mixer

Given a feasible subspace $\text{span}(B) \subset \mathcal{H}$, a mixer U_M is then called valid (Hadfield et al., 2019) if it *preserves the feasible subspace*, that is,

$$U_M(\beta)|v\rangle \in \text{span}(B), \quad \forall |v\rangle \in \text{span}(B), \forall \beta \in \mathbb{R},$$

and if it *provides transitions between all pairs of feasible states*, where for each pair of computational basis states $|x\rangle, |y\rangle \in B$ there exist $\beta^* \in \mathbb{R}$ and $r \in \mathbb{N} \cup \{0\}$, such that

$$|\langle x|U_M^r(\beta^*)|y\rangle| > 0.$$

Apart from the Grover mixer, we consider here constraint-preserving mixers in the stabilizer form (Fuchs and Bassa, 2024), in which case the Hamiltonian can be written as

$$H_M = \sum_{\alpha} c_{\alpha} \bar{X}^{\alpha} \prod_{V_{\alpha}},$$

where $\alpha \in \{0, 1\}^n \setminus \{0\}^n$ is a multi-index, $\bar{X}^{\alpha} = X_1^{\alpha_1} \dots X_n^{\alpha_n} \in \{I, X\}^n \setminus \{I\}^n$, and $\prod_{V_{\alpha}}$ is a projection onto V_{α} . Furthermore, V_{α} is a subspace fulfilling $|x\rangle \in V_{\alpha} \Rightarrow \bar{X}^{\alpha}|x\rangle \in V_{\alpha}$. The free parameter $c_{\alpha} \in \{0, 1\}$ must be chosen such that we have a valid mixer. Let $\{S_1, \dots, S_l\}$ be a minimal generating set of the stabilizer group $S = \langle S_1, \dots, S_l \rangle$ defining the code subspace $C(S)$. We can choose the projection operator to be of the form

$$\prod_{C(S)} = \frac{1}{2^k} \sum_{S \in \langle S_1, \dots, S_k \rangle} S.$$

For more details, see Fuchs and Bassa (2024). In the following, we will use the notation

$$\bar{X}\langle S_1, \dots, S_k \rangle := \frac{1}{2^k} \sum_{S \in \langle S_1, \dots, S_k \rangle} \bar{X}S.$$

Given the independence of the different color constraints for each vertex of the problem, we can then define the following LX-mixer Hamiltonian

$$H_M = G^{\square |V|} = G \square G \dots \square G,$$

where the Cartesian or Box product is defined as

$$G \square H = G \otimes \mathbb{I} + \mathbb{I} \otimes H. \quad (11)$$

It can be shown that if G is a valid mixer, H_M is constructed in this way (Fuchs and Bassa, 2024). Valid mixers with optimal cost in terms of CX gates are shown in [Figure 4](#) for $k \in \{3, 5, 6, 7\}$. We note that this choice is not unique and that other mixers are possible as well.

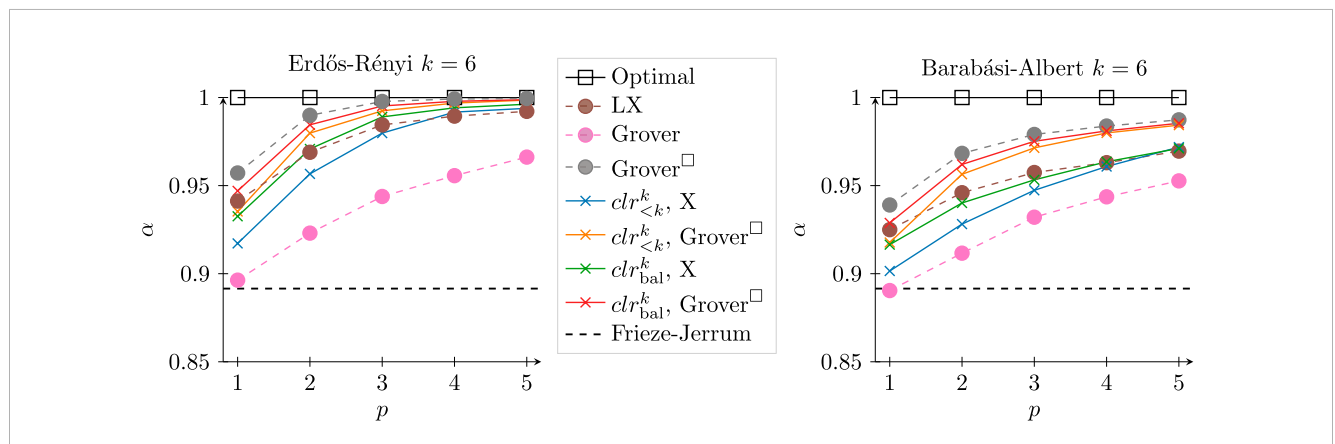
5 Simulations

We perform numerical simulations for an unweighted Erdős–Rényi graph and a weighted Barabási–Albert graph with ten vertices ([Supplementary Appendix A](#)). Since the case where k is a power of 2 is presented in [Section 2](#), we will focus on evaluating the methods presented in [Sections 3](#) and [4](#). Using an ideal simulator, we compare the results for the binary encoding with the standard X -mixer and different choices of equivalence relations to encode the colors, and we compare the subspace encoding with different choices of mixers. The resulting energy landscapes are provided in [Supplementary Appendix A](#). As expected, the landscapes are periodic in β when using the X - and Grover-mixer, whereas the landscape associated with the LX-mixer is generally not. The resulting approximation ratios are shown in [Table 2](#). Overall, the subspace-encoding using the Grover[□] mixer achieves the highest values for these two graphs, although with only a small distance to

TABLE 2 Comparison of different encoding schemes when k is not a power of 2. The X-mixer uses the encoding of the full Hilbert space presented in Section 3, whereas the LX- and Grover mixers encode the problem in a subspace as described in Section 4. Approximation ratios achieved for the graphs shown in Supplementary Appendix A for depth $p = 1$. The light gray color indicates methods using subspaces as described in Section 4, whereas the rest indicates methods using the full Hilbert space as described in Section 3.

Graph	Mixer	$ \phi_0\rangle$	clr	$k = 3$	$k = 5$	$k = 6$	$k = 7$
Erdős-Rényi	X	$ +\rangle$	$\text{clr}_{<k}^k$	0.80	0.86	0.92	0.95
	Grover $^\square$	$ +\rangle$	$\text{clr}_{<k}^k$	0.81	0.90	0.94	0.96
	X	$ +\rangle$	$\text{clr}_{\text{bal}}^k$	—	0.90	0.94	—
	Grover $^\square$	$ +\rangle$	$\text{clr}_{\text{bal}}^k$	—	0.93	0.95	—
	LX	$ \Phi_0^{<k}\rangle$	—	0.80	0.91	0.94	0.94
	Grover $^\square$	$ \Phi_0^{<k}\rangle$	—	0.83	0.94	0.96	0.97
	Grover	$ \Phi_0^{<k}\rangle$	—	0.75	0.87	0.90	0.92
Barabási-Albert	X	$ +\rangle$	$\text{clr}_{<k}^k$	0.79	0.84	0.90	0.93
	Grover $^\square$	$ +\rangle$	$\text{clr}_{<k}^k$	0.79	0.88	0.92	0.95
	X	$ +\rangle$	$\text{clr}_{\text{bal}}^k$	—	0.88	0.92	—
	Grover $^\square$	$ +\rangle$	$\text{clr}_{\text{bal}}^k$	—	0.91	0.93	—
	LX	$ \Phi_0^{<k}\rangle$	—	0.79	0.89	0.93	0.93
	Grover $^\square$	$ \Phi_0^{<k}\rangle$	—	0.82	0.92	0.94	0.95
	Grover	$ \Phi_0^{<k}\rangle$	—	0.75	0.86	0.89	0.91

TABLE 3 Approximation ratio for $k = 6$ for the different methods as the depth p is increased, employing the interpolation-based heuristic described in (Zhou et al., 2020) and compared with the Frieze–Jerrum semidefinite programming relaxation (Alan and Jerrum, 1997). The subspace methods constrained to subspaces use dashed lines with circles as markers, and the methods using the full Hilbert space use solid lines with “x” as markers.



the LX-mixer. We can also see that balancing the bin sizes for the different colors has a very positive effect on the approximation ratio when using the full Hilbert space and the X-mixer. The effect of increasing the depth is shown in Table 3, which is exemplified for $k = 6$. Note that the behavior for $k = 3, 5, 6$, and 7 were very similar. Overall, the Grover $^\square$ achieved the best convergence for these two graphs, irrespective of whether the full Hilbert space or the subspace method was used.

We note, however, that the averaged approximation ratio achieved should be carefully considered with respect to the required circuit depth. For encoding in the full Hilbert space, we can use the X-mixer, but the phase-separation operator requires a deeper circuit. For encoding into a subspace, we can use the phase-separation operator for the power-of-2 case (leading to a more shallow circuit), but the circuit for the constrained preserving mixer is deeper. An indication of the cost is given in Table 4, where we used

TABLE 4 Cost in terms of CX gates when using the qiskit compiler. Methods from previous research is marked with light gray background: The line with $H_P^{\text{old}}(k)$ is the Hamiltonian presented in Fuchs et al. (2021), and the $H_P^{\text{Pauli}}(k)$ decomposes the Hamiltonian in the Pauli basis. Terms related to the phase-separating Hamiltonian scale with the number of edges and terms related to the mixer scale with the number of vertices. Notice that by encoding the problem in a specific subspace rather than the entire Hilbert space, the complexity is shifted away from the phase-separating Hamiltonian and instead transferred to the mixer.

Mixer	Initial state	Cost Hamiltonian	$k = 3$	$k = 5$	$k = 6$	$k = 7$
X	$ +\rangle$	$H_P^{<k,\text{old}}(k)$	70 E	782 E	398 E	142 E
X	$ +\rangle$	$H_P^{<k,\text{Pauli}}(k)$	18 E	66 E	258 E	130 E
X	$ +\rangle$	$H_P^{<k}(k)$	12 E	26 E	90 E	58 E
X	$ +\rangle$	$H_P^{\text{bal},\text{old}}(k)$	—	398 E	270 E	-
X	$ +\rangle$	$H_P^{\text{bal},\text{Pauli}}(k)$	—	130 E	66 E	-
X	$ +\rangle$	$H_P^{\text{bal}}(k)$	—	82 E	36 E	-
LX	$ \Phi_0^{<k}\rangle$	$H_P(2^{n_k})$	$(2 + 4) V + 6 E $	$(2 + 12) V + 14 E $	$(2 + 4) V + 14 E $	$(5 + 6) V + 14 E $
Grover [□]	$ \Phi_0^{<k}\rangle$	$H_P(2^{n_k})$	$(2 + 6) V + 6 E $	$(2 + 12) V + 14 E $	$(2 + 12) V + 14 E $	$(5 + 18) V + 14 E $

the compiler from qiskit, assuming full connectivity. Note that for a sparse graph $|E| = \mathcal{O}(|V|)$ and for a fully connected graph $|E| = \Theta(|V|^2)$. Therefore, for graphs with sufficient connectivity, the subspace encoding uses fewer resources than the encoding into the full Hilbert space.

6 Conclusion

This study presents advances in encoding the MAX k -CUT problem into qubit systems, with a focus on cases where k is not a power of 2. We present new ways of encoding that significantly reduce the resource requirements compared to previous approaches. In our implementation, we compared two different encodings using the full Hilbert space. Empirically, we observed that the balanced encoding consistently produced more compact circuits and achieved higher approximation ratios across all tested circuit depths and for both graph instances. We believe that this improvement is linked to the structure of the cost Hamiltonian, specifically the reduced degeneracy induced by the balanced encoding, which may lead to a smoother optimization landscape. However, we acknowledge that we do not currently have a theoretical guarantee that the balanced encoding results in fewer barren plateaus than the $\leq k$ encoding. While the empirical results suggest an improvement in trainability, a rigorous analysis of the gradient landscape remains an open direction for future work. Balanced color sets and constrained mixers demonstrate notable improvements in optimization landscapes and approximation ratios.

While our method is designed for digital quantum architectures, extending it to quantum annealers is possible in principle but practically nontrivial. Quantum annealing hardware is restricted to 2-local Ising Hamiltonians, whereas our encoding yields k -local PUBO terms for $k \geq 3$. Mapping these higher-order terms to 2-local form would require ancillary qubits and penalty constraints, introducing significant overhead and embedding complexity (Nagies et al., 2025) or the use of hybrid techniques as exposed in (Wurtz et al., 2024). In the future, we

plan to do a careful analysis of which ansatz is preferable when k is not a power of 2, using a suite of graph instances. The analysis should take into account the overall circuit depth, the achieved approximation ratio for increasing depth size, as well as if the landscape has barren plateaus or many “bad” local minima.

Data availability statement

The raw data supporting the conclusions of this article will be made available by the authors, without undue reservation.

Author contributions

FF: Writing – review and editing, Investigation, Software, Conceptualization, Writing – original draft, Supervision, Visualization, Data curation, Project administration, Formal Analysis, Methodology. RP: Conceptualization, Investigation, Formal Analysis, Writing – review and editing, Methodology, Writing – original draft. FL: Writing – review and editing, Software, Data curation.

Funding

The authors declare that financial support was received for the research and/or publication of this article. We would like to thank for funding of the work by the Research Council of Norway through project number 332023.

Acknowledgements

The authors wish to thank Terje Nilsen at Kongsberg Discovery for the access to an H100 GPU for the simulations.

Conflict of interest

The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Generative AI statement

The authors declare that no Generative AI was used in the creation of this manuscript.

Any alternative text (alt text) provided alongside figures in this article has been generated by Frontiers with the support of artificial intelligence and reasonable efforts have been made to ensure accuracy, including review by the authors wherever possible. If you identify any issues, please contact us.

References

- Aardal, K. I., van Hoesel, S. P. M., Koster, A. M. C. A., Mannino, C., and Sassano, A. (2007). Models and solution techniques for frequency assignment problems. *Ann. Operations Res.* 153 (1), 79–129. doi:10.1007/s10479-007-0178-0
- Alan, F., and Jerrum, M. (1997). Improved approximation algorithms for max-cut and max bisection. *Algorithmica* 18 (1), 67–81. doi:10.1007/BF02523688
- Bartschi, A., and Eidenbenz, S. (2020). “Grover mixers for qaoa: shifting complexity from mixer design to state preparation,” in 2020 IEEE International Conference on Quantum Computing and Engineering (QCE) (IEEE), 72–82. doi:10.1109/qce49297.2020.00020
- Bravyi, S., Kliesch, A., Koenig, R., and Tang, E. (2020). Obstacles to variational quantum optimization from symmetry protection. *Phys. Rev. Lett.* 125 (26), 260505. doi:10.1103/PhysRevLett.125.260505
- Egger, D. J., Mareček, J., and Woerner, S. (2021). Warm-starting quantum optimization. *Quantum* 5, 479. doi:10.22331/q-2021-06-17-479
- Farhi, E., Goldstone, J., and Gutmann, S. (2014). A quantum approximate optimization algorithm. *arXiv Prepr. arXiv:1411.4028*. doi:10.48550/arXiv.1411.4028
- Fuchs, F. G., and Bassa, R. P. (2024). Lx-mixers for qaoa: optimal mixers restricted to subspaces and the stabilizer formalism. *Quantum* 8, 1535–11. doi:10.22331/q-2024-11-25-1535
- Fuchs, F. G., and Bassa, R. P. (2025). Compact circuits for constrained quantum evolutions of sparse operators. *arXiv*. doi:10.48550/arXiv.2504.09133
- Fuchs, F. G., Kolden, H. Ø., Henrik Aase, N., and Sartor, G. (2021). Efficient encoding of the weighted MAX $\mathbb{Z}_2$ -CUT on a quantum computer using QAOA. *SN Comput. Sci.* 2 (2), 89. doi:10.1007/s42979-020-00437-z
- Fuchs, F. G., Olsen Lye, K., Nilsen, H. M., Stasik, A. J., and Sartor, G. (2022). Constraint preserving mixers for the quantum approximate optimization algorithm. *Algorithms* 15 (6), 202. doi:10.3390/a15060202
- Hadfield, S., Wang, Z., Rieffel, E. G., O’Gorman, B., Venturelli, D., and Biswas, R. (2017). “Quantum approximate optimization with hard and soft constraints,” in *Proceedings of the second international workshop on post moore’s era supercomputing*, 15–21. doi:10.1145/3149526.3149530
- Hadfield, S., Wang, Z., O’Gorman, B., Rieffel, E. G., Venturelli, D., and Biswas, R. (2019). From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms* 12 (2), 34. doi:10.3390/a12020034
- Nagies, S., Geier, K. T., Akram, J., Bantounas, D., Johanning, M., and Hauke, P. (2025). Boosting quantum annealing performance through direct polynomial unconstrained binary optimization. *Quantum Sci. Technol.* 10 (3), 035008. doi:10.1088/2058-9565/adcae6
- Ram Gaur, D., Krishnamurti, R., and Kohli, R. (2008). The capacitated max k-cut problem. *Math. Program.* 115 (1), 65–72. doi:10.1007/s10107-007-0139-z
- Shukla, A., and Vedula, P. (2024). An efficient quantum algorithm for preparation of uniform quantum superposition states. *Quantum Inf. Process.* 23 (2), 38–1332. doi:10.1007/s11128-024-04258-4
- Wang, Z., Rubin, N. C., Dominy, J. M., and Rieffel, E. G. (2020). XY mixers: analytical and numerical results for the quantum alternating operator ansatz. *Phys. Rev. A* 101 (1), 012320. doi:10.1103/physreva.101.012320
- Wurtz, J., Sack, S. H., and Wang, S.-T. (2024). Solving non-native combinatorial optimization problems using hybrid quantum-classical algorithms. *IEEE Trans. Quantum Eng.* 5, 1–14. doi:10.1109/tqe.2024.3443660
- Zhou, L., Wang, S.-T., Choi, S., Pichler, H., and Lukin, M. D. (2020). Quantum approximate optimization algorithm: performance, mechanism, and implementation on near-term devices. *Phys. Rev. X* 10 (2), 021067. doi:10.1103/physrevx.10.021067
- Zhu, L., Tang, H. L., Barron, G. S., Calderon-Vargas, F. A., Mayhall, N. J., Barnes, E., et al. (2022). Adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer. *Phys. Rev. Res.* 4 (3), 033029. doi:10.1103/PhysRevResearch.4.033029

Publisher’s note

All claims expressed in this article are solely those of the authors and do not necessarily represent those of their affiliated organizations, or those of the publisher, the editors and the reviewers. Any product that may be evaluated in this article, or claim that may be made by its manufacturer, is not guaranteed or endorsed by the publisher.

Supplementary material

The Supplementary Material for this article can be found online at: <https://www.frontiersin.org/articles/10.3389/frqst.2025.1636042/full#supplementary-material>

Paper IV

Constraint Preserving Iterative Quantum Algorithms for Minimum Vertex Cover and Maximum Independent Set

IV

Ruben Pariente Bassa
Finley Quinton
Franz G. Fuchs
Pascal Halffmann

Iterative quantum algorithms for the minimum vertex cover problem based on continuous time quantum walks

Ruben Pariente Bassa,¹ Finley A. Quinton,² Franz G. Fuchs,^{1,3} and Pascal Halffmann⁴

¹*SINTEF AS, Department of Mathematics and Cybernetics, Oslo, Norway*

²*Department of Industrial Economics and Technology Management, NTNU, Trondheim*

³*University of Oslo, Department of Mathematics, Oslo, Norway*

⁴*Fraunhofer Institute for Industrial Mathematics ITWM, Department of Financial Mathematics, Kaiserslautern, Germany*

(Dated: May 28, 2026)

We introduce a family of constraint-preserving quantum-classical greedy algorithms for the Minimum Vertex Cover and Maximum Independent Set problems. The construction uses a mixer built from projected Pauli- X generators that confine the dynamics to the feasible vertex-cover subspace, yielding a Hamiltonian that acts as the adjacency matrix of a layered feasible-state graph $\mathcal{G}_{\text{VC}}(G)$. We interpret the resulting unitary evolution as a continuous-time quantum walk (CTQW) on this graph, and show that the walk spreads amplitude efficiently toward low-weight configurations, motivating an iterative greedy extraction of per-vertex statistics from the walk state. We develop three complementary quantum greedy strategies that rank vertices by, respectively, their marginal occupation probability, their conditional energy contribution, and an echo-fidelity measure of operator compatibility. Benchmarks across several random graph families show that all quantum variants exceed their classical greedy counterparts in mean approximation ratio and probability of recovering an optimal cover. The energy-based strategy is the strongest performer, capturing multi-vertex correlations via the full cost Hamiltonian and achieving near-optimal results already at shallow circuit depth. For bounded-degree graphs the state-preparation circuit requires only constant-depth Trotter layers, and the framework needs no penalty terms or variational training, providing a scalable and hardware-relevant approach to quantum combinatorial optimization.

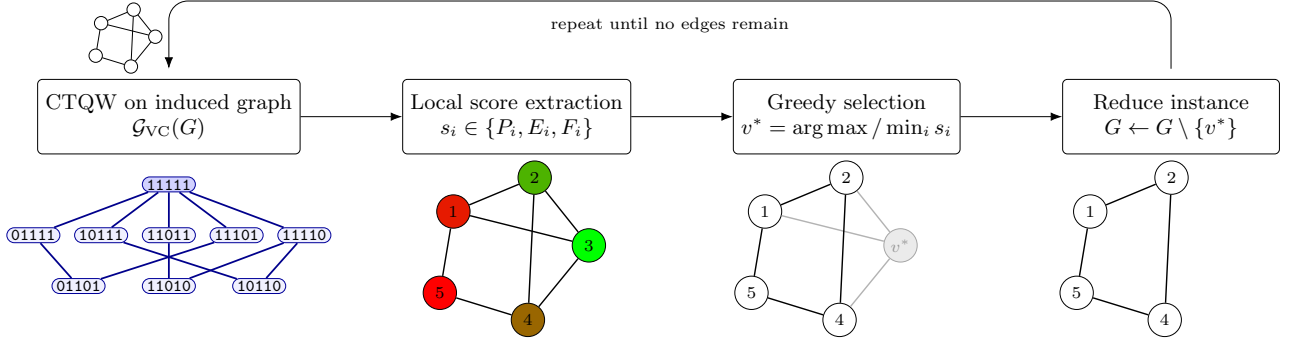


Figure 1: Schematic of the iterative quantum algorithm. The input graph G defines the induced feasible-state graph $\mathcal{G}_{\text{VC}}(G)$, on which a constraint-preserving quantum walk prepares a state supported entirely on valid vertex covers. Local observables of this state produce vertex scores $s_i \in \{P_i, E_i, F_i\}$ on the original graph, thereby converting global information in the feasible configuration space into locally accessible greedy decision signals. A vertex is then fixed according to the chosen rule, the graph is reduced accordingly, and the procedure is repeated on the reduced instance. In this way, the method combines constrained quantum exploration with recursive classical reduction, without penalty terms or variational optimization.

I. INTRODUCTION AND RELATED WORK

The Minimum Vertex Cover (MVC) and Maximum Independent Set (MIS) problems are two of the canonical NP-complete problems in combinatorial optimization, historically introduced in Karp’s seminal work on polynomial-time reductions [1]. Due to their fundamental role in graph theory and combinatorial optimization, these problems have long served as benchmarks for classical approximation algorithms, heuristic methods, and,

more recently, quantum optimization approaches. Classical polynomial-time algorithms achieve a tight factor-2 approximation for MVC via maximal matchings, linear programming relaxations, and primal-dual techniques [2], and under the Unique Games Conjecture, this approximation factor is optimal [3].

We consider the MVC problem on a graph $G = (V, E)$, where the goal is to select a minimum subset of vertices such that every edge is incident to at least one selected vertex. Introducing the binary variables $x_i \in \{0, 1\}$ indicating whether vertex i is included in the cover, the

problem can be written as

$$\min \sum_{i \in V} c_i x_i \quad \text{s.t.} \quad x_i + x_j \geq 1 \quad \forall (i, j) \in E. \quad (1)$$

The complementary Maximum Independent Set (MIS) problem is obtained via $y_i = 1 - x_i$, yielding the relation $|\text{MVC}| + |\text{MIS}| = |V|$, where $|\text{MVC}|$ and $|\text{MIS}|$ denote the sizes of a minimum vertex cover and a maximum independent set, respectively. This duality provides complementary perspectives on the same combinatorial structure, as selecting a vertex in the independent set corresponds to excluding it from the vertex cover.

Greedy algorithms constitute one of the simplest and most versatile paradigms in combinatorial optimization: iteratively make locally optimal choices with the aim of obtaining a globally good solution. They are among the earliest algorithmic strategies for combinatorial optimization, most prominently exemplified by Kruskal's and Prim's polynomial-time algorithms for the minimum spanning tree problem; and more generally for matroid problems [4]. Beyond exact optimization, greedy methods often yield provable approximation guarantees, for instance a 2-approximation (1/2-approximation, respectively) for matching and knapsack-type problems [2]. A similar result holds for MVC via a greedy-style 2-approximation based on maximal matchings. For MIS, in contrast, such approximation guarantees only hold for specific graph structures such as bounded degrees. Still, degree-based greedy strategies remain highly relevant in practice and have been extensively studied [5, 6]: Largest-degree-first rules delete many edges at once for MVC and preserves low-degree candidates for MIS. Smallest-degree-first (SDF) strategies are equally important, either by covering the neighbors of low-degree vertices in vertex cover or by directly selecting low-degree vertices and removing their neighborhoods in independent set. In heuristic optimization, greedy methods are widely used to construct initial feasible solutions, repair infeasible solutions, and drive local improvement steps; indeed, local search itself can be interpreted as a greedy strategy over a prescribed neighborhood. Greedy ideas also appear in modern mixed-integer optimization solvers as primal heuristics, rounding and repair procedures, and incumbent-generation mechanisms, and they form the basis of metaheuristic frameworks such as GRASP [7]. Overall, greedy optimization provides a simple, fast, and broadly applicable methodology that often yields high-quality polynomial-time solutions and, in structurally favorable cases, even approximation or optimality guarantees, while its limitations, such as only having a local view without considering the global consequences, motivate hybridization with other techniques [8].

In quantum optimization approaches, the constraints in equation (1) are often incorporated via penalty terms, leading to unconstrained QUBO or Ising formulations. Early work in this direction focused on adiabatic quantum computation and quantum annealing, where the ground state of a suitably constructed Hamiltonian en-

codes a valid vertex cover or independent set. General constructions for mapping NP-hard problems to Ising models are surveyed in [9], and such encodings have enabled both experimental and numerical studies on quantum annealers, including decomposition-based approaches for scaling beyond native hardware connectivity limits [10]. However, penalty-based formulations can be sensitive to parameter tuning and often lead to complex energy landscapes with many low-energy infeasible states [11, 12].

With the advent of gate-model quantum computing, MVC and MIS have been explored within variational frameworks such as the Quantum Approximate Optimization Algorithm (QAOA) [13] and the Variational Quantum Eigensolver (VQE) [14]. In these approaches, the constrained optimization problems are typically converted into unconstrained ones via penalty terms in the cost Hamiltonian, enabling the use of standard transverse-field mixers. Recent studies have investigated circuit design, scaling behavior, approximation performance, and practical limitations of such constructions. By exploiting the duality between vertex covers and independent sets, these algorithms can flexibly target either objective while leveraging shared circuit structures.

An alternative line of work emphasizes enforcing feasibility by construction rather than through penalties. The Quantum Alternating Operator Ansatz framework introduced in [15] enables the design of constraint-preserving mixers that restrict the dynamics to the feasible subspace. Within this paradigm, a variety of constructions have been proposed, including XY-type mixers for cardinality and covering constraints [16], Grover-style mixers that trade mixer complexity for state preparation [17], and general subspace-preserving constructions applicable to arbitrary constraint sets [11]. More recent work has connected constraint-preserving mixer design to stabilizer structure, yielding systematic methods for constructing optimal subspace-restricted mixers [18]. The MIS problem has been studied directly within this framework by Saleem [19], who analyses circuit depth dependence, initial state sensitivity, and approximation ratios on small graph instances, demonstrating that the constraint-preserving ansatz favours larger independent sets even at finite circuit depth.

Beyond mixer-based approaches, alternative frameworks have been proposed to enforce constraints directly at the level of the quantum dynamics. These include methods based on constrained evolution, such as quantum Zeno dynamics, which restrict the system to feasible subspaces without relying on energy penalties [12], as well as penalty-free formulations that avoid the introduction of auxiliary penalty terms altogether [20].

Recent work has established provable quantum speedups for exact combinatorial optimization through tree-search methods. One line of results shows that quantum-walk techniques can yield a near-quadratic speedup for branch-and-bound, and that this advantage can be generalized to a broad class of tree-search heuris-

tics, giving a universal near-quadratic improvement over classical branch-and-bound and branch-and-cut on every input instance within that framework [21, 22]. Related work introduces a quantum tree-generation approach for 0-1 knapsack, where all feasible solutions are constructed in superposition and amplitude amplification is then used to recover optimal solutions, with projected runtimes that are competitive with classical state-of-the-art solvers on instances containing up to 600 variables [23]. Taken together, these results mark substantial progress toward quantum advantage for exact solvers, although they are primarily formulated in the query-complexity or fault-tolerant setting and are therefore best viewed as complementary to near-term heuristic methods for gate-based quantum devices.

In parallel, hybrid quantum-classical strategies have been proposed to improve variational optimization performance. Warm-start techniques initialize quantum states or parameters using solutions of classical relaxations, such as linear or semi-definite programs, with the aim of inheriting classical approximation guarantees while refining solutions quantum mechanically [24]. More broadly, iterative quantum optimization algorithms combine shallow quantum circuits with classical reduction steps, where expectation values or correlations extracted from quantum states are used to guide variable fixing and recursive problem simplification. Notable examples include Recursive QAOA (RQAOA) [25] and Quantum-Informed Recursive Optimization (QIRO) [26], which interpolate between classical greedy heuristics and quantum-enhanced strategies. Other approaches, such as scalable quantum walk heuristics [27], encode the graph Laplacian of the original graph as a Hamiltonian and use iterative short-time quantum evolution to guide greedy vertex selection and dynamic variable freezing, enforcing feasibility through recursive classical graph reduction rather than by restricting the quantum dynamics to the feasible subspace; in contrast, the approach in this paper defines the walk on the induced feasible-state graph rather than on the original graph and directly addresses the realization and cost of the walk operator, including circuit-depth scaling.

All these methods fall within the broader class of hybrid quantum approaches known as Iterative Quantum Algorithms (IQA) [28, 29]. In this framework, quantum resources are used in a recursive loop with classical processing to progressively simplify the problem, offering a flexible paradigm that bridges purely classical heuristics and fully quantum optimization strategies.

Building on this perspective, we introduce a class of *constraint-preserving quantum greedy* algorithms for MVC that do not rely on penalty terms or variational optimization. In contrast to existing approaches based on penalty encodings or variational optimization, our method operates entirely within the feasible subspace, constructing quantum states that directly drive a greedy reduction procedure. Our approach employs a structured quantum ansatz that encodes only feasible vertex

covers and iteratively fixes variables based on quantum expectation values. This results in a hybrid algorithm that avoids penalty tuning, eliminates infeasible configurations, and directly integrates quantum information into classical decision-making, providing a new framework for quantum-assisted combinatorial optimization.

The **key idea** of this work is that a quantum continuous-time walk under a local mixer Hamiltonian, constrained to the feasible MVC subspace, converts non-local graph structure into local measurement signals. Starting from the all-in-the-cover reference state, we sample the walk at a chosen evolution time t and evaluate a per-vertex score for each vertex individually. Although each score is a single-qubit observable, its value reflects the surrounding graph structure through the constrained walk dynamics, providing non-local information accessible via a local measurement. The three algorithms proposed differ in which observable is used as the score (marginal inclusion probability, local energy, or fidelity with the reference state) but share the same underlying structure: at each step, the vertex with the most extreme score is fixed and removed, and the procedure repeats on the residual graph. The quantum part is therefore not used to output a full solution directly, but to construct per-vertex scores with non-local content that guide a recursive classical reduction.

Thanks to the duality between MVC and MIS, the same algorithmic framework can be directly applied to the Maximum Independent Set problem. The complementary nature of vertex covers and independent sets allows the sequential variable-fixing procedure developed for MVC to be translated to MIS without modifying the underlying quantum ansatz or measurement strategy, highlighting the generality of the approach. Importantly, the circuit depth of our ansatz scales polynomially with system size, making the approach well-suited for near-term noisy intermediate-scale quantum (NISQ) devices without requiring deep circuits.

The rest of the paper is organized as follows: In [section II](#) we introduce the feasible Hilbert subspace and the needed projected Pauli operators, followed by a detailed description of the continuous-time quantum walk (CTQW) and the proof of the reachability of the entire feasible solution space in [section III](#). In [section V](#) we present three iterative quantum optimization algorithms that leverage the quantum state to guide greedy vertex selection: the Quantum Probabilistic Greedy (QPG), Quantum Energy Greedy (QEG), and Quantum Fidelity Greedy (QFG) algorithms, each equipped with both largest-degree-first (LDF) and smallest-degree-first (SDF) selection strategies. Numerical results comparing these quantum algorithms against classical baselines on 3-regular, 4-regular, and Erdős-Rényi random graphs are reported in [section VI](#), followed by concluding remarks in [section VII](#). Technical details on the dynamical Lie algebra of the projected Pauli generators, entanglement analysis, gate and depth scaling properties, Monte Carlo simulations, and additional algorithmic variants are pro-

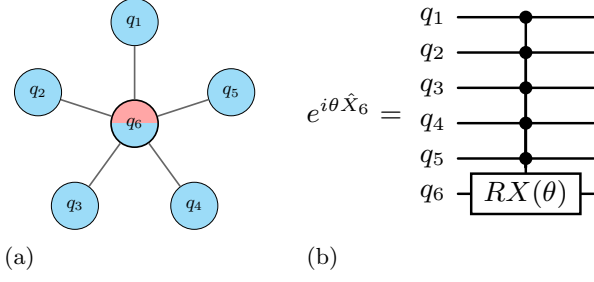


Figure 2: (a) Graph illustrating a vertex q_6 (red) and its neighborhood $\mathcal{N}(6)$ (blue). The neighbors may have additional edges among themselves or to other vertices not shown. (b) Circuit representation of the constraint preserving unitary $U_i(\alpha) = e^{i\alpha \hat{X}_6}$. A single-qubit rotation is applied to vertex 6, conditioned on all neighboring qubits being in the state $|\Omega\rangle$, thereby restricting the dynamics to the vertex-cover subspace.

vided in the appendices.

II. CONSTRAINT-PRESERVING OPERATORS FOR MVC

In this section, we fix the notation for the feasible Hilbert space and specialize the standard constraint-preserving mixer construction of Hadfield et al. [15] to the MVC problem. For MVC, a flip of vertex i is allowed exactly when all neighbors of i are already included in the cover.

Let $G = (V, E)$ be a graph with $|V| = n$ vertices. We represent a candidate vertex cover by a bit string $C \in \{0, 1\}^n$, where $C_i = 1$ indicates that vertex i is included in the cover. The feasible Hilbert space is

$$\mathcal{H}_{\text{VC}} := \text{span}(|C\rangle \mid C_i + C_j \geq 1 \ \forall (i, j) \in E) \subseteq (\mathbb{C}^2)^{\otimes n}.$$

Equivalently, $\mathcal{H}_{\text{VC}}$ is the image of the feasibility projector

$$\Pi_{\text{VC}} = \sum_{C \in \text{VC}(G)} |C\rangle\langle C| = \prod_{(i,j) \in E} \left(I - |00\rangle\langle 00|_{ij} \right),$$

where $\text{VC}(G)$ denotes the set of all vertex covers of G .

Let

$$\mathcal{N}(S) := \{j \in V \setminus S \mid \exists i \in S \text{ such that } \{i, j\} \in E\}$$

denote the open neighborhood of a set $S \subseteq V$. For notational simplicity, we omit set braces, e.g., $\mathcal{N}(i) := \mathcal{N}(\{i\})$. We define the projector onto the subspace where all neighbors of i belong to the cover by

$$\Pi_{\mathcal{N}(i)} := \bigotimes_{j \in \mathcal{N}(i)} |1\rangle\langle 1|_j \otimes I_{V \setminus \mathcal{N}(i)}.$$

We then define the neighborhood-controlled Pauli operators

$$\hat{\sigma}_i := \Pi_{\mathcal{N}(i)} \sigma_i, \quad \sigma \in \{X, Y\}.$$

Here $\Pi_{\mathcal{N}(i)}$ is understood as an operator on the full n -qubit Hilbert space, and σ_i acts nontrivially only on qubit i . Since $\Pi_{\mathcal{N}(i)}$ acts only on the neighbors of i , it commutes with σ_i .

The operators $\hat{X}_i$ and $\hat{Y}_i$ generate single-vertex transitions that preserve feasibility: if $|C\rangle \in \mathcal{H}_{\text{VC}}$, then $\hat{\sigma}_i |C\rangle$ is either zero or another state in $\mathcal{H}_{\text{VC}}$. Indeed, the only possible nontrivial action occurs when all neighbors of i are already in the cover, so flipping vertex i cannot leave any incident edge uncovered. This is the property used throughout the construction.

Exponentiating these generators gives the corresponding constraint-preserving unitaries,

$$U_i(\alpha) = e^{i\alpha \hat{\sigma}_i} = (I - \Pi_{\mathcal{N}(i)}) + \Pi_{\mathcal{N}(i)} e^{i\alpha \sigma_i}.$$

Thus, $U_i(\alpha)$ is a multi-controlled Pauli rotation on qubit i , conditioned on all neighboring qubits being in the state $|\Omega\rangle$. For the algorithms developed later, the generators are the projected Pauli- X operators

$$\hat{X}_i = \Pi_{\mathcal{N}(i)} X_i.$$

A completely analogous construction applies to the Maximum Independent Set (MIS) problem by exchanging occupied and unoccupied vertices. In the MIS convention, a bit value 1 denotes inclusion in the independent set, and feasibility requires that no adjacent vertices are both occupied. The MVC and MIS unitary dynamics $U_{\text{VC}}, U_{\text{MIS}}$ are related by the global Pauli- X transformation

$$U_{\text{MIS}} = X^{\otimes n} U_{\text{VC}} (X^{\otimes n})^\dagger, \quad |\psi_{\text{MIS}}\rangle = X^{\otimes n} |\psi_{\text{VC}}\rangle,$$

which maps vertex covers to their complementary independent sets. Under this transformation, the control condition $|\Omega\rangle\langle\Omega|$ on neighboring qubits is replaced by $|0\rangle\langle 0|$, so the MIS dynamics is generated by the same construction with the roles of 0 and 1 exchanged.

III. REACHABILITY AND CONNECTIVITY OF THE FEASIBLE GRAPH

In this section, we want to introduce the Hamiltonian that will characterize our algorithm and study in particular the reachability properties related to it. Once we have defined each single qubit operation that preserve the edge constraints we want to proceed defining a constraint preserving Hamiltonian to generate some specific dynamics in the vertex cover space:

$$H(\vec{\alpha}) = \sum_{i \in V} \alpha_i \hat{X}_i$$

The natural reference state is the all-in-the-cover configuration

$$|\Omega\rangle := |1\rangle^{\otimes |V|},$$

which is always feasible and corresponds to the maximal vertex cover.

We now show that every feasible vertex-cover configuration is reachable from $|\Omega\rangle$ by a sequence of constraint-preserving single-vertex moves. Equivalently, every vertex of the induced feasible-state graph lies in the same connected component as the root. Thus the projected Pauli generators are sufficient to connect the entire feasible graph to the all-in-the-cover state.

Theorem 1 (Reachability of feasible vertex-cover states). *Let $G = (V, E)$ and let*

$$\mathcal{H}_{\text{VC}} = \text{span}(|C\rangle \mid C \in \text{VC}(G))$$

be the feasible vertex-cover subspace. Then, for any vertex cover $C \in \text{VC}(G)$, there exists a parameter vector α^C with entries

$$\alpha_i^C \in \{0, \pi/2\},$$

such that

$$U(\alpha^C) |\Omega\rangle = e^{i \sum_{i \in V} \alpha_i^C \hat{X}_i} |\Omega\rangle = |C\rangle.$$

In particular, every feasible basis state is reachable from $|\Omega\rangle$ by a sequence of constraint-preserving local flips.

Proof. Let $C \in \text{VC}(G)$ be any feasible vertex cover, and define

$$S := \{i \in V \mid C_i = 0\},$$

that is, the set of vertices removed from the all-in-the-cover configuration.

Since C is a vertex cover, no edge of G can have both endpoints in S . Therefore S is an independent set. Hence, for any distinct $i, j \in S$, one has $(i, j) \notin E$. By the commutator relation for the projected generators, it follows that

$$[\hat{X}_i, \hat{X}_j] = 0 \quad \text{for all } i, j \in S.$$

Now choose the parameters

$$\alpha_i^C = \begin{cases} \pi/2, & i \in S, \\ 0, & i \notin S. \end{cases}$$

Since the generators with nonzero coefficients commute, the exponential factorizes:

$$e^{i \sum_{i \in V} \alpha_i^C \hat{X}_i} = \prod_{i \in S} e^{i \frac{\pi}{2} \hat{X}_i}.$$

Starting from the reference state $|\Omega\rangle = |1\rangle^{\otimes |V|}$, every neighborhood projector acts trivially,

$$\Pi_{\mathcal{N}(i)} |\Omega\rangle = |\Omega\rangle,$$

since all qubits are initially in the state $|\Omega\rangle$. Therefore, on $|\Omega\rangle$, each projected generator $\hat{X}_i = \Pi_{\mathcal{N}(i)} X_i$ acts as the ordinary Pauli- X operator on qubit i . Consequently,

$$e^{\sum_{i \in S} i \frac{\pi}{2} \hat{X}_i} |\Omega\rangle = \prod_{i \in S} R X_i \left(\frac{\pi}{2} \right) |\Omega\rangle = i^{|S|} |0\rangle_S |1\rangle_{V \setminus S}.$$

Consequently we have:

$$e^{i \sum_{i \in V} \alpha_i^C \hat{X}_i} |\Omega\rangle = i^{|C|} |C\rangle.$$

This proves the claim. $\square$

The theorem has an immediate graph-theoretic interpretation.

Corollary 1 (Connectivity from the root). *Every feasible configuration $C \in \text{VC}(G)$ is connected to the root configuration $\Omega = 1^{|V|}$ by a path in the induced feasible-state graph $\mathcal{G}_{\text{VC}}$. Equivalently, all vertices of $\mathcal{G}_{\text{VC}}$ lie in the same connected component as the root.*

Proof. The proof of [theorem 1](#) explicitly constructs a sequence of allowed one-bit flips taking Ω to C . Each such flip preserves feasibility and therefore corresponds to an edge of $\mathcal{G}_{\text{VC}}$. Hence these flips define a path from the root to C in the induced graph. $\square$

In fact, the argument reveals slightly more. Since every feasible configuration can be obtained from the all-in-the-cover state by removing a set of vertices that forms an independent set, the induced graph is naturally rooted at Ω and every feasible state admits at least one monotone descending path from the top layer to its own Hamming-weight layer. In the language of [section IV](#), this means that the full feasible subspace is dynamically accessible from the root, and the bottom layer $\mathcal{L}_{\tau(G)}$ containing the minimum vertex covers is never excluded by the constraint-preserving dynamics. This reachability property is the structural reason the induced-graph picture is useful for optimization. The constrained evolution does not merely remain inside the feasible subspace; it has access to the entire feasible graph.

IV. INDUCED LAYERED GRAPH AND CONTINUOUS-TIME WALK PERSPECTIVE

Having established that the Hamiltonian generated by the projected Pauli operators is fully controllable on the feasible subspace, we now turn to the quantum dynamics of

$$H_{\text{MVC}} = \sum_{i \in V} \hat{X}_i, \quad (2)$$

where $\hat{X}_i$ is the multi-controlled Pauli- X operator that flips bit i if and only if the resulting configuration remains a vertex cover of G .

A useful way to understand the constrained dynamics is to step back from the original graph G and instead analyze the geometry of the *feasible configurations* themselves. This perspective makes the support of the dynamics explicit and recasts the unitary evolution as CTQW on a layered state graph. [Figure 3](#) illustrates the correspondence on a small asymmetric instance.

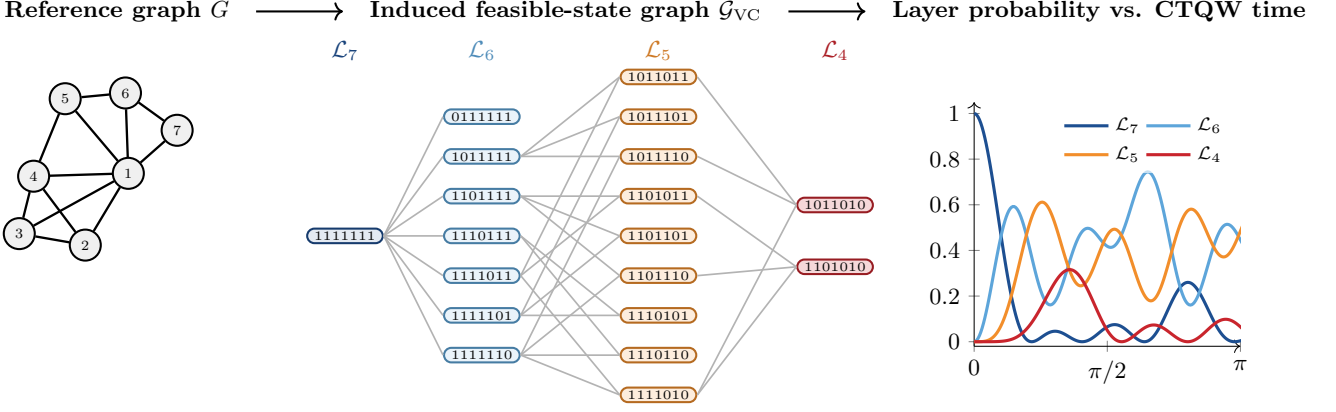


Figure 3: Example of the implicitly induced feasible-state graph underlying the constrained continuous-time quantum walk (CTQW). The input graph $G = (V, E)$ in the left panel defines the feasible-state graph $\mathcal{G}_{VC}$ in the middle panel, whose vertices are the feasible vertex covers of G ordered by Hamming weight and whose edges connect pairs of covers that differ by a single feasibility-preserving bit flip. The walk is initialized in the all-ones configuration $|\Omega\rangle = |1\rangle^{\otimes |V|}$ and propagates coherently down through layers of decreasing weight. The right panel shows the layer-resolved probabilities $P_k(t) = \sum_{C \in \mathcal{L}_k} |\langle C | \psi(t) \rangle|^2$: amplitude reaches the lowest-weight occupied layer before $t \approx \pi/2$ and subsequently flows back toward the root, with interference patterns emerging at later times. The induced graph is an explanatory object only and is not constructed explicitly by the algorithm.

A. The induced feasible-state graph

Let $VC(G)$ denote the set of all vertex covers of G . We define the *induced feasible-state graph* $\mathcal{G}_{VC} = (\mathcal{V}_{VC}, \mathcal{E}_{VC})$ by

$$\mathcal{V}_{VC} = VC(G),$$

with $(C, C') \in \mathcal{E}_{VC}$ if and only if $d_H(C, C') = 1$ and both C and C' are feasible, where d_H denotes Hamming distance. Two feasible configurations are therefore adjacent precisely when they differ by a single allowed local flip. The projected generators $\{\hat{X}_i\}_{i \in V}$ produce exactly these transitions, so the constrained evolution generated by equation (2) is supported entirely on $\mathcal{G}_{VC}$.

Layered structure by Hamming weight. The induced graph admits a canonical layering

$$\mathcal{L}_k = \{C \in VC(G) : |C| = k\}, \quad k = 0, 1, \dots, |V|,$$

where $|C| = \sum_i C_i$ is the Hamming weight of C . Because adjacent feasible configurations differ by a single bit, every edge of $\mathcal{G}_{VC}$ connects layers $\mathcal{L}_k$ and $\mathcal{L}_{k \pm 1}$. The graph is therefore *bipartite* with respect to the parity of the Hamming weight, with the immediate spectral consequence that the eigenvalues of the adjacency matrix $A = A(\mathcal{G}_{VC})$ come in pairs $\pm \lambda$.

The initial state $|\Omega\rangle$ sits in the unique top layer $\mathcal{L}_{|V|}$, and the lowest occupied layer is $\mathcal{L}_{\tau(G)}$, where $\tau(G)$ is the minimum vertex-cover number of G . Minimum vertex covers are thus the elements of $\mathcal{L}_{\tau(G)}$. The depth of $\mathcal{G}_{VC}$,

$$D(G) = |V| - \tau(G) = \alpha(G),$$

coincides with the Maximum Independent set dimension of G via the complement identity $\tau(G) + \alpha(G) = |V|$.

The layered structure is visible in figure 3: the root $|\Omega\rangle = |1111111\rangle$ occupies the top layer, and the minimum covers $|1011010\rangle, |1101010\rangle$ appears in the last layer $\mathcal{L}_{\tau(G)}$.

An analogous picture holds for the MIS problem. Under the global Pauli- X map $|C\rangle \mapsto X^{\otimes n} |C\rangle$, the induced feasible-state graph is identical in structure, with every configuration replaced by its bitwise complement. Consequently the Hamming-weight layering is inverted: the root becomes the empty set $|\Omega\rangle_{\text{MIS}} = |0\rangle^{\otimes |V|}$ in layer $\mathcal{L}_0$, and the deepest occupied layer $\mathcal{L}_{\alpha(G)}$ contains the maximum independent sets. For the graph in figure 3, this yields the same layered graph with root $|\Omega\rangle_{\text{MIS}} = |0000000\rangle$ and maximum independent sets $|0100101\rangle, |0010101\rangle$ in the bottom layer.

Constraint geometry and connectivity. The original graph G enters only through the feasibility constraint: G determines which one-bit moves preserve feasibility, and therefore determines both the size of each layer $|\mathcal{L}_k|$ and the layers connectivity. The continuous quantum walk itself, however, takes place on $\mathcal{G}_{VC}$.

B. Continuous-time dynamics on the induced graph

In the feasible computational basis $\{|C\rangle : C \in VC(G)\}$, the Hamiltonian in equation (2) has matrix elements

$$\langle C' | H_{\text{MVC}} | C \rangle = \begin{cases} 1 & \text{if } (C, C') \in \mathcal{E}_{VC}, \\ 0 & \text{otherwise,} \end{cases}$$

so that

$$H_{\text{MVC}}|_{\text{feasible}} = A(\mathcal{G}_{\text{VC}}). \quad (3)$$

The constrained unitary

$$U(t) = e^{itH_{\text{MVC}}}$$

therefore acts as a continuous-time quantum walk on $\mathcal{G}_{\text{VC}}$.

C. Layer probabilities and the walk-count expansion

Starting from $|\Omega\rangle$ at $t = 0$, the amplitude on a feasible cover C is given by the following path expansion

$$\langle C|U(t)|\Omega\rangle = \sum_{m=0}^{\infty} \frac{(it)^m}{m!} \langle C|H^m|\Omega\rangle = \sum_{m=0}^{\infty} \frac{(it)^m}{m!} \gamma_m^C,$$

where $\gamma_m^C = (A^m)_{C,\Omega}$ is the number of walks of length m from $|\Omega\rangle$ to $|C\rangle$ on $\mathcal{G}_{\text{VC}}$. Because $\mathcal{G}_{\text{VC}}$ is bipartite with respect to Hamming-weight parity, γ_m^C vanishes unless m has the same parity as the graph distance

$$d(C) = |V| - |C|$$

from the root to C . Setting $m = d + 2n$ and factoring out the overall phase, the amplitude is real up to i^d , and the single-cover probability admits the following expansion

$$P_C(t) = \left(\sum_{n=0}^{\infty} \frac{(-1)^n t^{d+2n}}{(d+2n)!} \gamma_{d+2n}^C \right)^2. \quad (4)$$

Layer probabilities follow by summation. Throughout this section we parameterize layers by the graph distance d from the root rather than by Hamming weight; the two are equivalent through $d = |V| - k$, with $d = 0$ corresponding to $|\Omega\rangle$ and $d = D(G)$ to the minimum-cover layer. The probability of finding the walker in the layer at distance d is

$$P_d(t) = \sum_{C \in \mathcal{L}_{|V|-d}} \left(\sum_{n=0}^{\infty} \frac{(-1)^n t^{d+2n}}{(d+2n)!} \gamma_{d+2n}^C \right)^2. \quad (5)$$

D. Transport and the Layers hitting time

The walk-count expansion of the previous subsection makes the short-time behaviour transparent. Retaining only the lowest-order term in Eq. equation (5) gives

$$P_d(t) \sim \frac{N_d}{(d!)^2} t^{2d}, \quad N_d = \sum_{C \in \mathcal{L}_{|V|-d}} (\gamma_d^C)^2, \quad (6)$$

where γ_d^C counts the shortest walks of length d from $|\Omega\rangle$ to a cover C at distance d from the root. The factor t^{2d}

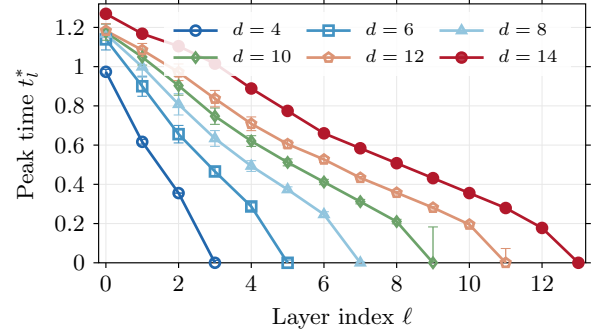


Figure 4: First-peak time t_ℓ^* as a function of layer index ℓ , measured from the minimum-cover layer, for several depths $D(G)$ of $\mathcal{G}_{\text{VC}}$. Markers show the mean over instances; error bars indicate the corresponding standard deviation.

is the operational signature of the layered geometry: the amplitude on a layer at distance d is suppressed as t^{2d} at short times, so deeper layers remain essentially unpopulated until the wavefront generated by H_{MVC} has had time to reach them. This is exactly the qualitative picture of Fig. figure 3: amplitude leaves the root, propagates coherently downward through layers of decreasing Hamming weight, and only later returns creating interference.

a. First-peak times. The location of the first maximum of $P_d(t)$ lies outside the regime of validity of Eq. equation (6): by the time a layer reaches its peak, many more terms of the walk-count series contribute, and a closed-form prediction would require being able to resum the full expansion or to properly truncate it. We therefore characterize the peak times numerically. Denote by t_ℓ^* the first time at which the layer at distance $d = D(G) - \ell$ from the root attains its maximal probability, so that ℓ measures the distance *upward* from the minimum-cover layer $L_{\tau(G)}$ ($\ell = 0$) toward the root.

Across all instances and depths considered, the first-peak times follow a clear linear trend in the layer index,

$$t_\ell^*(D(G)) \approx t_{D(G)}^0 - \frac{\ell}{v_{D(G)}}, \quad (7)$$

shown in figure 4. We notice first that, for fixed depth, the dependence on ℓ is approximately linear: the wavefront advances by approximately one layer per unit time, traversing successive layers at an approximately constant group velocity $v_{D(G)}$. Second, the intercept $t_{D(G)}^0$, which is the extrapolated time at which the wavefront reaches the minimum-cover layer, varies only weakly with the depth $D(G)$ of the induced graph; the slope, by contrast, flattens as $D(G)$ grows. In other words, deeper instances spread the same hitting time over more layers, so the per-layer velocity increase while the total time to reach $L_{\tau(G)}$ stays on a comparable scale.

b. From transport to the greedy criterion. The transport picture builds up the intuition behind the cen-

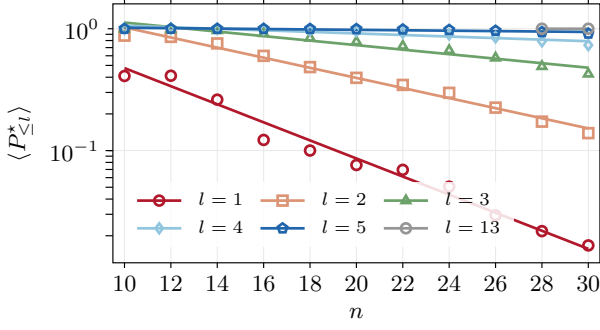


Figure 5: Cumulative bottom- l peak probability $P_{\leq l}^*(n) \equiv \max_t \sum_{j \leq l} P_{D(G)-j}(t)$ as a function of n at fixed offset l from the minimum-cover layer. Markers show numerical data; lines show exponential fits $A_l e^{-\alpha_l n}$. For $l \geq 5$, the fitted decay rate α_l is statistically indistinguishable from zero over the simulated range.

tral difficulty of any single-shot scheme. Equation (7) says that the wavefront does reach the minimum-cover layer $L_{\tau(G)}$, and does so at a time that is not too large. One might therefore hope to prepare the CTQW state at $t \approx t_{D(G)}^0$, measure, and read off a minimum cover directly with probability $P_{D(G)}(t_{D(G)}^0)$. As analyzed in Fig. figure 5, reaching a layer is not the same as concentrating probability on it: although the wavefront arrives at $L_{\tau(G)}$, the *single-layer* probability of being exactly on the minimum-cover layer is exponentially small in the system size. Rather than demanding the exact minimum-cover layer, one may accept any cover within the lowest few layers and ask for the cumulative probability of landing in the bottom $\ell + 1$ layers,

$$P_{\leq \ell}^*(n) \equiv \max_t \sum_{j \leq \ell} P_{D(G)-j}(t), \quad (8)$$

which measures the chance of producing a feasible cover of size at most $\tau(G) + \ell$. The numerical evidence of Fig. figure 5 indicates that, at fixed offset ℓ , the quantity $P_{\leq \ell}^*(n)$ decays exponentially in the graph size n , while at fixed n it grows rapidly with ℓ and saturates toward unity already for modest offsets ($\ell \approx 5$ for $n = 30$). The exact bottom layer is therefore exponentially hard to hit, but the bottom *few* layers are not.

The genuinely open question concerns the regime between these two extremes. Define

$$\ell^*(n; \epsilon) = \min \{ \ell : P_{\leq \ell}^*(n) \geq 1 - \epsilon \}, \quad (9)$$

the smallest offset at which the cumulative probability becomes appreciable. Over the simulated range $\ell^*(n; \epsilon)$ remains well below $D(G)/2$, but its asymptotic behaviour is unknown: we cannot determine from direct simulation whether $\ell^*(n; \epsilon)$ stays $O(1)$, grows only logarithmically, or eventually scales linearly in n . This equivalent to ask how deep into the layered graph the CTQW can

penetrate while retaining probability that is not exponentially suppressed. What survives the suppression of the deepest layers, in any case, is not the single-shot bottom-layer probability but the per-vertex statistics extracted from the walk state along the way. This is precisely the motivation for the iterative greedy procedures of section V: rather than insisting to measure directly a minimum cover, we extract from the walk state quantities that remain informative even when the bottom-layer probability is negligible, and use them to fix one or a few vertices per iteration on the residual graph.

V. ITERATIVE CONTINUOUS-TIME QUANTUM WALK OPTIMIZATION ALGORITHMS FOR MVC

We introduce a family of iterative quantum algorithms for the Minimum Vertex Cover (MVC) problem based on classical greedy vertex fixing heuristics.

Central to all three algorithms is the initial superposition state of all feasible solutions created by the continuous-time quantum walk (CTQW) on the feasibility graph G_{VC} , to which we refer as the CTQW-state $e^{itA_{G_{VC}}}|\Omega\rangle$. Throughout this work, this state is implemented using a first-order Suzuki–Trotter decomposition. As analyzed in Appendix, the resulting circuit admits a gate count scaling as $\mathcal{O}(np\Delta)$ section B 1, where Δ is the maximal degree, with a Trotter approximation error scaling as $\mathcal{O}(\frac{Et^2}{p})$ section B 2. Moreover, the multi-controlled structure of the walk induces strong multipartite entanglement and volume-law entropy scaling, allowing local observables to encode nonlocal graph information section B 3. This state serves as the starting point of each iteration for all our algorithms. At time t , the walk has spread amplitude across the feasible-cover configurations, and one of three greedy criteria is evaluated on this state. These are the marginal occupation probability, the expected Hamiltonian cost, and the echo fidelity. In the LDF approach, the vertex with the best criterion value is included in the vertex cover set and removed; in the SDF approach, the neighbours of the worst-scoring vertex are instead added to the cover. In both cases the selected vertices are removed and the process repeats on the residual graph. The residual graph defines a new feasibility graph, and the time evolution is performed again from the reduced induced feasible-state graph at the same t ; the procedure repeats until no edges remain. The choice of t is therefore a shared prerequisite for all three algorithms, and its effect is analyzed in each greedy method section. Moreover, in the special case of a single Trotter layer ($p=1$), the CTQW state admits an efficient classical representation in terms of a constrained quantum tree, allowing expectation values and marginal probabilities to be estimated through Monte Carlo sampling without explicit quantum-state simulation. This representation, together with the associated Monte Carlo algorithms used to study large graph instances, is presented in section C.

A. Quantum Probabilistic Greedy

The *Quantum Probabilistic Greedy* algorithm selects vertices according to their probability of belonging to the vertex cover as estimated from the quantum MVC state $|\psi(t)\rangle$.

The marginal probability that vertex v belongs to the cover is given by the expectation value of the projector onto $|1\rangle_v$,

$$P_v(t) = \frac{1 - \langle \psi(t) | Z_v | \psi(t) \rangle}{2}, \quad |\psi(t)\rangle = U(t) |\Omega\rangle, \quad (10)$$

where $U(t) = e^{it \sum_{i \in V} \hat{X}_i}$ is the CTQW unitary with time walk t . The vertex selected at each step is

$$v^* = \underset{v}{\operatorname{argmax}} P_v(t),$$

i.e. the qubit most likely to remain in $|1\rangle$, and is added to the vertex cover. We expect that, for specific time choices, the vertices participating in the optimal layers will get trapped resulting in a lower probability of being in the cover. Fixing a vertex to be in the cover corresponds to removing the related Hamiltonian term, leaving qubit v unchanged in state $|1\rangle$, removing from all the neighbors, since it is trivially satisfied. Using the Campbell identity, this probability can be rewritten in terms of nested commutators,

$$P_v(t) = \frac{1}{2} - \frac{1}{2} \sum_{n=0}^{\infty} \frac{(it)^n}{n!} \langle \Omega | [\sum_j \hat{X}_j, Z_v]_n | \Omega \rangle, \quad (11)$$

where $[A, B]_n$ denotes the n -fold nested commutator.

a. Non-local information structure. A crucial distinction between quantum and classical greedy algorithms emerges from the structure of the commutator expansion in equation (11). As shown in section A, the nested commutator $[\sum_j \hat{X}_j, Z_v]_n$ has support within the n -step neighborhood $\mathcal{N}(n(v))$ of vertex v (proposition 1). Moreover, due to the symmetry of the projected operators $\hat{X}_i$, only even-order terms contribute to the expectation value (corollary 3):

$$P_v(t) = \frac{1}{2} - \frac{1}{2} \sum_{k=0}^{\infty} \frac{(it)^{2k}}{(2k)!} \langle \Omega | [\sum_j \hat{X}_j, Z_v]_{2k} | \Omega \rangle.$$

This has profound implications: while classical greedy algorithms (LDF, SDF) make decisions based solely on local degree information $\deg(v)$ or at most the immediate neighborhood $\mathcal{N}(v)$, the quantum probability $P_v(t)$ incorporates contributions from progressively larger regions of the graph. As time t increases, and the continuous-time walk reaches deeper layers on the induced feasible-state graph, higher-order terms in the expansion become significant, and the effective information radius grows (corollary 2). This is the mechanism by which QPG encodes non-local structural information into local vertex selection.

At each iteration, the algorithm uses these marginal probabilities to guide a greedy decision. We distinguish two complementary strategies, inspired by classical largest-degree-first (LDF) and smallest-degree-first (SDF) heuristics.

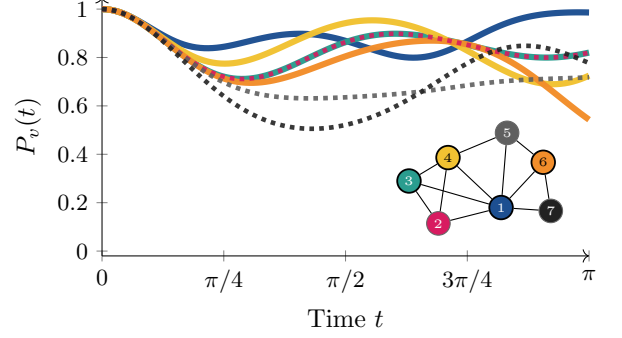


Figure 7: Vertex occupation probabilities $P_v(t)$ as a function of the continuous quantum walk time t for the example graph shown in figure 3. As t increases, higher-order terms in the commutator expansion become relevant and the quantum walk propagates in the layered graph, leading to a progressive separation of probability profiles and encoding increasingly non-local structural information, revealing how the quantum state induces a ranking over vertices, which is exploited by the greedy selection rules.

b. Largest-Probability (LDF-like) Strategy In the first approach, the algorithm selects the vertex with the largest occupation probability,

$$v^* = \underset{v \in V}{\operatorname{argmax}} P_v(t),$$

and fixes it in the vertex cover. The graph is then reduced by removing v^* and all edges incident to it.

This strategy mirrors the classical largest-degree greedy heuristic: vertices that are more likely to be in the cover (i.e., with large $P_v(t)$) tend to play a central role in covering many edges. From a quantum perspective, this corresponds to reinforcing configurations that already have significant weight in the quantum state.

We observe, for instance, in figure 7 that the ranking induced by the quantum probabilities is not necessarily aligned with the vertex degree ordering. In particular, for evolution times $t \in [3\pi/4, \pi]$, the QPG algorithm may favor vertices that are not those selected by the Largest Degree First (LDF) heuristic. In the example considered, LDF would select the vertex with the second-highest degree, whereas the quantum probabilities assign a different ordering. This illustrates how QPG departs from purely local degree-based criteria and incorporates non-local structural information arising from the quantum evolution.

c. Smallest-Probability (SDF-like) Strategy An alternative approach is to select the vertex with the small-

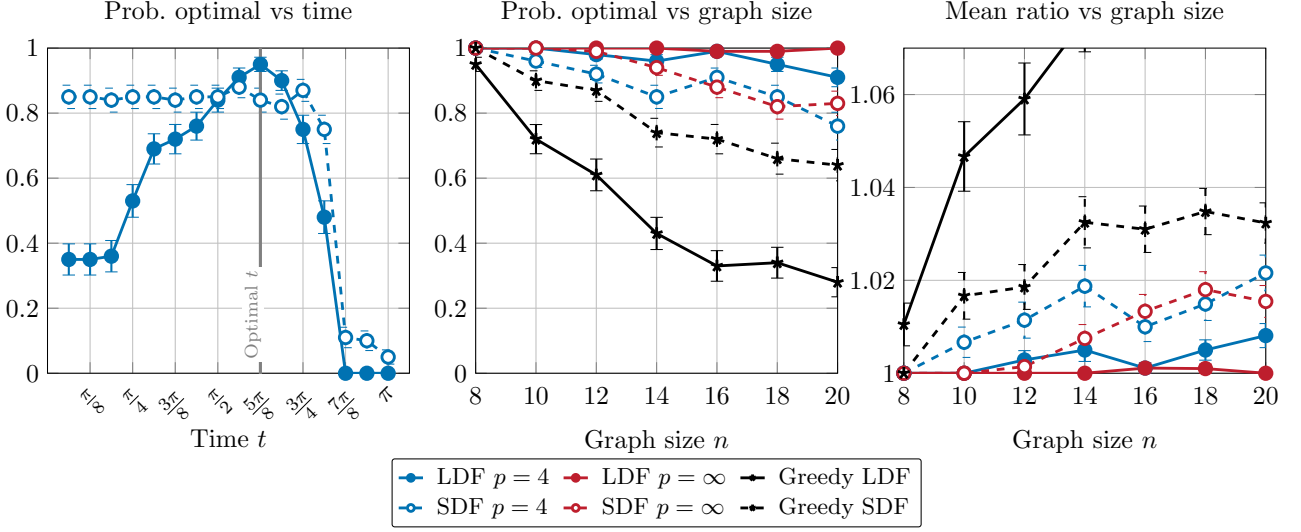


Figure 6: QPG performance on 100 3-regular graph instances for graphs of size $n = 18$. Left: probability of finding an optimal solution as a function of the continuous-time walk time t for $p = 4$; the vertical gray line marks the empirically selected optimal walk time used in the remaining panels. Middle and right: mean approximation ratio and probability of obtaining an optimal solution as functions of graph size n . Blue curves show the trotterized QPG algorithm with $p = 4$ layers, red curves show exact continuous-time simulations corresponding to $p = \infty$, and black curves show the classical greedy baselines. Solid lines with filled markers denote LDF, while dashed lines with open markers denote SDF. Error bars show the standard error of the mean.

Algorithm 1 QuantumProbabilisticGreedy-LDF

```

1: Input: Graph  $G = (V, E)$ , Cover set  $S = \emptyset$ , Trotter steps  $p$ , quantum walk-time  $t$ 
2: Prepare the CTQW state  $|\psi\rangle = e^{i\sum_{i \in V} t\hat{X}_i} |\Omega\rangle$ 
3: if  $E = \emptyset$  then
4:   return  $S$ 
5: end if
6: for each vertex  $v \in V$  do
7:   Compute  $P_v(t) = \frac{1 - \langle \psi | Z_v | \psi \rangle}{2}$ 
8: end for
9: Select  $v^* = \arg \max_{v \in V} P_v(t)$  and add to the Vertex Cover set  $S \leftarrow S \cup \{v^*\}$ 
10: Remove  $v^*$  and all incident edges:  $G \leftarrow (V \setminus \{v^*\}, E \setminus \{(v^*, u)\})$ 
11: return QuantumProbabilisticGreedy - LDF( $G, S, p, t$ )
  
```

est occupation probability,

$$v^* = \arg \min_{v \in V} P_v(t),$$

and instead fix all of its neighbors in the vertex cover. The graph is then reduced by removing the neighborhood $N(v^*)$ and all incident edges.

This rule is the quantum analogue of the classical minimum-degree heuristic. A small value of $P_v(t)$ indicates that the quantum state assigns low weight to configurations where v is in the cover. Consequently, to satisfy the edge constraints, its neighbors must be included instead. This leads to a more aggressive update step, where multiple vertices may be fixed simultaneously.

Algorithm 2 QuantumProbabilisticGreedy-SDF

```

1: Input: Graph  $G = (V, E)$ , Cover set  $S = \emptyset$ , Trotter steps  $p$ , quantum walk-time  $t$ 
2: Prepare the CTQW-state  $|\psi\rangle = e^{i\sum_{i \in V} t\hat{X}_i} |\Omega\rangle$ 
3: if  $E = \emptyset$  then
4:   return  $S$ 
5: end if
6: for each vertex  $v \in V$  do
7:   Compute  $P_v(t) = \frac{1 - \langle \psi | Z_v | \psi \rangle}{2}$ 
8: end for
9: Select  $v^* = \arg \min_{v \in V} P_v(t)$ 
10: Add the set of neighbors  $N(v^*)$  of  $v^*$  to the Cover set  $S \leftarrow S \cup N(v^*)$ 
11: Remove  $N(v^*)$  and all incident edges:
     $G \leftarrow (V \setminus N(v^*), E \setminus \{(u, w) \in E : u \in N(v^*) \text{ or } w \in N(v^*)\})$ 
12: return QuantumProbabilisticGreedy-SDF( $G, S, p, t$ )
  
```

The two strategies reflect different ways of extracting information from the quantum state:

- The LDF-like rule exploits *high-probability structure*, selecting vertices that are strongly supported by the quantum distribution.
- The SDF-like rule exploits *low-probability structure*, interpreting unlikely vertices as indicators that their neighbors should be chosen instead.

For each algorithm we must choose an initial walk time t , which sets the distribution over feasible covers induced

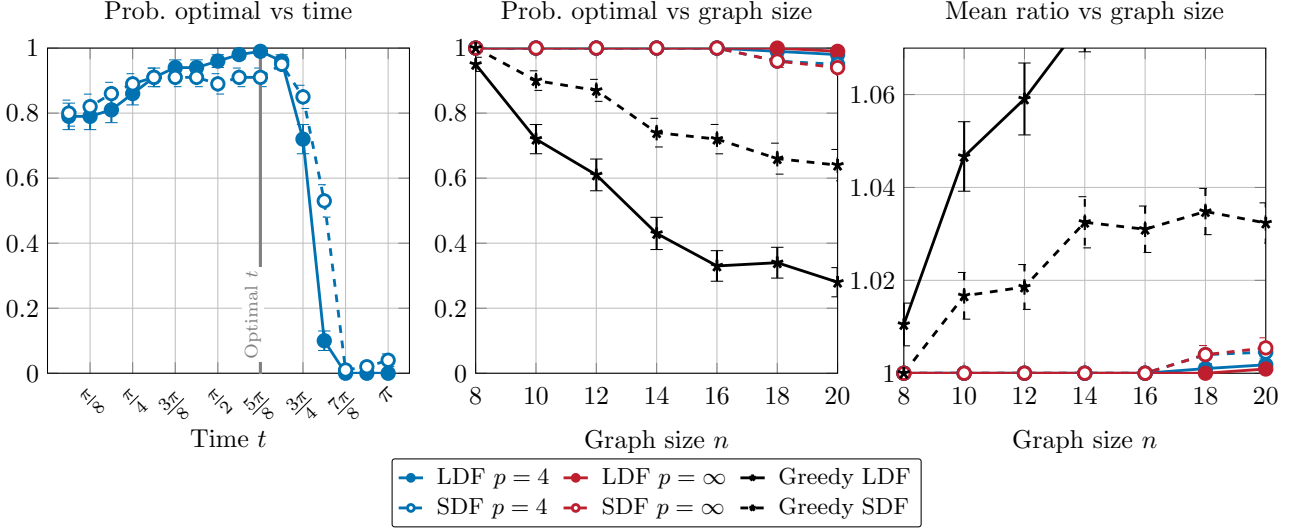


Figure 8: QEG performance on 100 3-regular graph instances for graphs of size $n = 18$. Left: probability of finding an optimal solution as a function of the continuous-time walk time t for $p = 4$; the vertical gray line marks the empirically selected optimal walk time used in the remaining panels. Middle and right: mean approximation ratio and probability of obtaining an optimal solution as functions of graph size n . Blue curves show the trotterized QEG algorithm with $p = 4$ layers, red curves show exact continuous-time simulations corresponding to $p = \infty$, and black curves show the classical greedy baselines. Solid lines with filled markers denote LDF, while dashed lines with open markers denote SDF. Error bars show the standard error of the mean.

by the CTQW. Figure 6 (left) reports the performance of the two strategies for $t \in [0, \pi]$ on 100 3-regular instances with $n = 18$ vertices. The strategies differ markedly in their sensitivity to t : SDF is broadly flat across $[0, 3\pi/4]$, likely because its greedy rule adds the entire neighborhood of the selected vertex at each step, whereas LDF peaks sharply around $t \approx 10\pi/16$.

The location of the LDF peak admits a natural interpretation in the layered-walk picture. The CTQW propagates a wavefront outward from $|\Omega\rangle$ through layers of decreasing Hamming weight, and the amplitude on the minimum-cover boundary $\mathcal{L}_{\tau(G)}$ peaks at a time $t_{\tau(G)}^*$ lying *before* $\pi/2$. One might expect this to be the optimal time for the greedy as well, but empirically the LDF peak $t_{\text{LDF}}^* \approx 5\pi/8$ occurs *after* the wavefront has departed the bottom layer, as amplitude is reflected back through the upper layers. The reason is that LDF exploits not the absolute occupation of the bottom layers but the *contrast* between per-vertex marginal occupations, i.e. the probability gap between always-in and never-in vertices. On the return trip the amplitude spreads across the upper layers $\mathcal{L}_\tau, \dots, \mathcal{L}_n$; since each contains the always-in vertices in all its states and excludes the never-in vertices from most, every layer contributes coherently to the same contrast direction, and the aggregate gap is maximal when amplitude is most uniformly spread. The irregularity of $\mathcal{G}_{\text{VC}}$ enhances this gap, as uneven layer connectivity traps a small fraction of amplitude near $\mathcal{L}_{\tau(G)}$ on states that sharply discriminate the two vertex classes.

Past t_{LDF}^* the same mechanism runs in reverse: the re-

turning wave pushes the always-in marginals back down and boosts the never-in ones, until around $t \gtrsim 3\pi/4$ the orderings invert and LDF preferentially selects the worst vertices. Performance then drops *below* the classical baseline. SDF is far less sensitive to this inversion, since its bulk update rule does not rely on the per-vertex ranking.

In the exact ($p = \infty$) case LDF outperforms SDF, reaching the optimal solution with probability $\approx 99\%$ across all instances; this advantage persists at finite Trotter depth $p = 4$, and both quantum strategies continue to outperform their classical greedy counterparts. The two variants thus offer complementary trade-offs: LDF makes conservative, fine-grained updates, while SDF enforces constraints more aggressively by fixing multiple vertices at once, affecting both the number of recursive steps and the structure of the intermediate quantum states.

B. Quantum Energy Greedy

While the previous strategy relies on marginal vertex probabilities, a more global selection criterion can be obtained by evaluating the expectation value of the cost Hamiltonian associated with the Minimum Vertex Cover problem. The *Quantum Energy Greedy* (QEG) algorithm measures the effect of fixing each vertex on the expected cost of the quantum state.

To determine which vertex should be fixed next, we evaluate the energy variation obtained by fixing each ver-

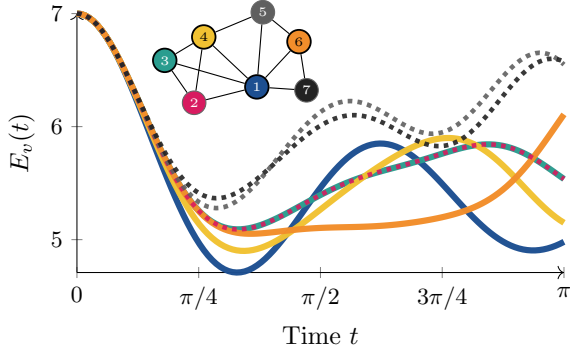


Figure 9: Vertex Energy $E_v(t)$ as a function of the continuous quantum walk time t for the example graph shown in figure 3. Similarly as for the probability case, as t increases, higher-order terms in the commutator expansion become relevant, the quantum walk spread in the layered graph, leading to a progressive separation of energies profiles encoding increasingly non-local structural information.

tex v to be in the cover. This is implemented by remove related term X_v from the Hamiltonian, leaving qubit v unchanged in $|\Omega\rangle$ state. The resulting energy is then computed as

$$E_v(t) = \langle \psi^v(t) | H_C | \psi^v(t) \rangle.$$

Similarly to the probability case, we can use the Baker-Campbell-Hausdorff identity to express the expectation value as:

$$E_v(t) = \sum_{n=0}^{\infty} \sum_k \frac{(it)^n}{n!} \langle \Omega | \left[\sum_{j \neq v} \hat{X}_j, Z_k \right]_n | \Omega \rangle. \quad (12)$$

Equivalently, it can be expressed in terms of the paths in the layered graph:

$$E_v(t) = \sum_{d=0}^{\tau(G_v)} (|V| - d) \sum_{C \in \mathcal{L}_{|V|-d}^v} \left(\sum_{n=0}^{\infty} \frac{(-1)^n t^{d+2n}}{(d+2n)!} \gamma_{d+2n}^C \right)^2,$$

where G_v is the graph obtained removing v and $\mathcal{L}_{|V|-d}^v$ are the layers related to $G_{V \setminus \{v\}}^v$

a. Global correlation structure. The energy-based selection criterion exhibits the same non-local information structure as the probability-based approach, but with enhanced sensitivity to correlations among vertices. As detailed in section A, the nested commutator structure in equation (12) inherits the same symmetry properties and light-cone behavior as the probability expansion. The symmetry of the $\hat{X}_i$ operators again enforces an even-order selection rule (corollary 3), ensuring that only terms with $n = 2k$ contribute to the expectation value.

However, the energy functional $E_v(t)$ differs from the probability $P_v(t)$ in a crucial way: rather than measuring a single-vertex marginal, it evaluates the expectation

value of the full cost Hamiltonian $H_C = \sum_{k \in V} Z_k$. This means each nested commutator term contains contributions from *all vertices simultaneously* resulting in a quantity more sensitive to multi-vertex correlations and global graph structure than $P_v(t)$.

The measurement of the Hamiltonian expectation value conditioned to each vertex in the graph rather than individual vertex marginals results in a number of circuit evaluations that scales as $\mathcal{O}(|V|^2)$.

b. Minimum Energy (LDF-like) Strategy In the first approach, the algorithm selects the vertex whose fixation produces the largest decrease in the expected cost, namely

$$v^* = \arg \min_{v \in V} E_v(t),$$

and fixes it in the vertex cover. The graph is then reduced by removing v^* and all edges incident to it.

This strategy can be interpreted as a quantum analogue of the classical largest-degree greedy heuristic. At each step, fixing each vertex to be in the cover results in a new layered graph with depth dependent on the specific vertex choice. We can now evolve the CTQW quantum state on the conditioned graph up for a certain time t and measure how much the expectation value of the cost Hamiltonian decreases when v is included in the cover set. The vertex with the largest energy reduction is selected and added to the cover. Similarly as for QPG, we observe in figure 9, that the ranking induced by the quantum conditional energies is not necessarily aligned with the vertex degree ordering.

From a quantum perspective, this rule exploits the *energy landscape* of the walk state. Rather than relying solely on local marginal information, it evaluates a quantity that captures correlations among multiple vertices through the continuous-time walk dynamics. A large energy reduction upon fixing v signals that many high-weight feasible configurations in $|\psi(t)\rangle$ already favour the inclusion of v , and fixing it reinforces these contributions.

Algorithm 3 QuantumEnergyGreedy-LDF

- 1: **Input:** Graph $G = (V, E)$, Cover Set $S = \emptyset$, Trotter steps p , quantum walk-time t
 - 2: **if** $E = \emptyset$ **then**
 - 3: **return** S
 - 4: **end if**
 - 5: **for** each vertex $v \in V$ **do**
 - 6: Prepare $|\psi^v(t)\rangle = e^{i \sum_{i \in V \setminus v} t \hat{X}_i} |\Omega\rangle$
 - 7: Compute $E_v(t) = \langle \psi^v(t) | H_C | \psi^v(t) \rangle$
 - 8: **end for**
 - 9: Select $v^* = \arg \min_{v \in V} E_v(t)$ and add to the Vertex Cover set $S \leftarrow S \cup \{v^*\}$
 - 10: Remove v^* and all incident edges: $G \leftarrow (V \setminus \{v^*\}, E \setminus \{(v^*, u)\})$
 - 11: **return** QuantumEnergyGreedy - LDF(G', S, p, t)
-

c. Maximum Energy (SDF-like) Strategy An alternative approach is to select the vertex whose fixation

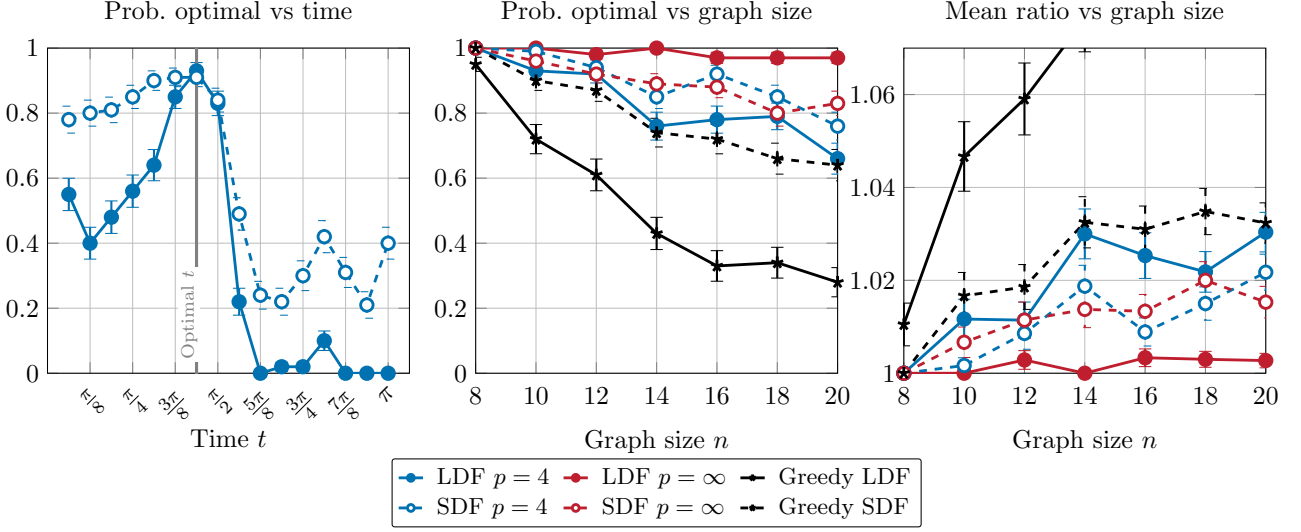


Figure 10: QFG performance on 100 3-regular graph instances for graphs of size $n = 18$. Left: probability of finding an optimal solution as a function of the continuous-time walk time t for $p = 4$; the vertical gray line marks the empirically selected optimal walk time used in the remaining panels. Middle and right: mean approximation ratio and probability of obtaining an optimal solution as functions of graph size n . Blue curves show the trotterized QFG algorithm with $p = 4$ layers, red curves show exact continuous-time simulations corresponding to $p = \infty$, and black curves show the classical greedy baselines. Solid lines with filled markers denote LDF, while dashed lines with open markers denote SDF. Error bars show the standard error of the mean.

produces the largest increase (or smallest decrease) in the expected energy,

$$v^* = \arg \max_{v \in V} E_v(t),$$

and instead fix all of its neighbors in the vertex cover. The graph is then reduced by removing the neighborhood $N(v^*)$ and all incident edges.

The intuition behind this rule is complementary to the previous one. If fixing v leads to a high expected cost, this indicates that configurations in which v belongs to the cover are energetically unfavorable in the current quantum state. In other words, the state assigns higher weight to configurations where v is excluded, and the edge constraints must therefore be satisfied by its neighbors.

The two energy-based strategies reflect complementary ways of exploiting the information encoded in the quantum state similarly to the Quantum vertex greedy approach:

- The LDF-like rule exploits *low-energy structure*, selecting vertices whose inclusion most effectively reduces the expected cost.
- The SDF-like rule exploits *high-energy structure*, interpreting energetically unfavorable vertices as indicators that their neighbors should be chosen instead.

As with QPG, the choice of walk time t sets the distribution over feasible covers used by the greedy criterion.

Algorithm 4 QuantumEnergyGreedy-SDF

- 1: **Input:** Graph $G = (V, E)$, Cover Set $S = \emptyset$, Trotter steps p , quantum walk-time t
 - 2: **if** $E = \emptyset$ **then**
 - 3: **return** S
 - 4: **end if**
 - 5: **for** each vertex $v \in V$ **do**
 - 6: Prepare $|\psi^v(t)\rangle = e^{i \sum_{i \in V \setminus v} t \hat{X}_i} |\Omega\rangle$
 - 7: Compute $E_v(t) = \langle \psi^v(t) | H_C | \psi^v(t) \rangle$
 - 8: **end for**
 - 9: Select $v^* = \arg \max_{v \in V} E_v(t)$
 - 10: Add the set of neighbors $N(v^*)$ of v^* to the Cover set $S \leftarrow S \cup N(v^*)$
 - 11: Remove $N(v^*)$ and all incident edges:
 - $G \leftarrow (V \setminus N(v^*), E \setminus \{(u, w) \in E : u \in N(v^*) \text{ or } w \in N(v^*)\})$
 - 12: **return** QuantumEnergyGreedy – SDF(G', S, p, t)
-

Figure 8 (left) reports the performance for $t \in [0, \pi]$ on 100 3-regular instances with $n = 18$ vertices, and the two strategies again differ in their sensitivity to t : SDF stays broadly flat, while LDF peaks around $t \approx 5\pi/8$, reflecting the same layered-walk mechanism described for QPG, the contrast between always-in and never-in marginals is maximal once the reflected wavefront has spread amplitude across the upper layers, and inverts at later times.

The main difference is the overall performance. Because $E_v(t)$ probes the full cost Hamiltonian rather than a single-vertex marginal, the selection criterion cap-

tures multi-vertex correlations and global graph structure, yielding a sharper and more reliable ranking than the marginal occupations used by QPG. As a result, both QEG variants reach near-optimal performance: in the exact ($p = \infty$) case the probability of finding the optimal solution is essentially 100% across all instances, and this advantage persists at finite Trotter depth $p = 4$, with both strategies clearly outperforming their classical greedy counterparts and improving on the corresponding QPG results. This gain comes at the cost of a heavier measurement overhead, since the conditional energies require $\mathcal{O}(|V|)$ circuit evaluations per greedy step rather than the $\mathcal{O}(1)$ of QPG.

C. Quantum Fidelity Greedy

The previous greedy strategies select vertices based on probabilities or energy variations. Here, we introduce an alternative approach that leverages the algebraic structure of the Projected Pauli Operators associated with the vertices. The key observation is that valid vertex-cover configurations correspond to sets of mutually commuting operators. In this perspective, the minimum vertex cover problem can be approached by greedily identifying a maximal subset of mixers that forms an approximately Abelian algebra. Importantly, the vertices that remain in this maximal commuting subset correspond to a maximum independent set, since removing vertices that do not commute effectively ensures that the remaining vertices do not share edges.

To identify such a subset, we employ an echo experiment that measures the degree of non-commutativity between a candidate vertex operator and the current active block. Consider the unitary operators

$$A_v(t_a) = \prod_{i \in S} e^{it_a \alpha_i X_i}, \quad B_v(t_b) = e^{it_b \alpha_v X_v},$$

where v is a trial vertex and S denotes the set of currently active vertices $V \setminus \{v\}$. If the operators commute, the echo sequence

$$U_v(t_a, t_b) = A_v(t_a) B_v(t_b) A_v(t_a)^\dagger B_v(t_b)^\dagger$$

acts trivially on the state. Deviations from the identity therefore quantify the non-commutativity between A and B . Starting from the initial state $|\Omega\rangle$, the fidelity

$$F_v = |\langle \Omega | U_v(t_a, t_b) | \Omega \rangle|^2 \quad (13)$$

is measured for each candidate vertex. Using the Campbell-Baker-Hausdorff identity we can express, similarly as before, the time evolution operator as:

$$U_v(t_a, t_b) = \left[\sum \frac{(it_a)^n}{n!} \left((\cos t_b - 1) \left[\sum_{i \in S} \hat{X}_i, \Pi_{\mathcal{N}(v)} \right]_n \right. \right. \\ \left. \left. + i \sin t_b \left[\sum_{i \in S} \hat{X}_i, \hat{X}_v \right]_n \right) \right] e^{it_b \hat{X}_v}.$$

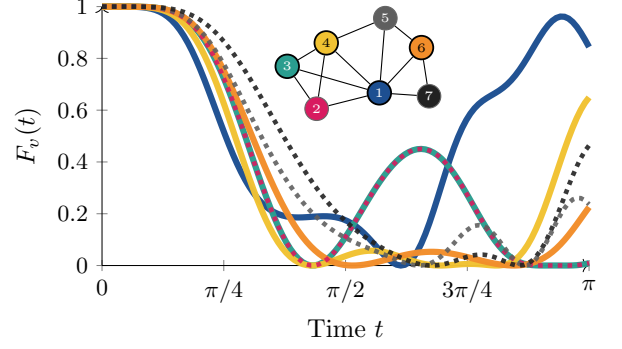


Figure 11: Echo Fidelity $F_v(t)$ as a function of the continuous quantum walk time t for the example graph shown in figure 3. Evolution of the Echo Fidelities for each vertex. Similarly as before, as t increases, higher-order terms in the commutator expansion become relevant, the quantum walk propagates in the layered graph, leading to a progressive separation revealing how the quantum echo fidelities induce a ranking over vertices, which is exploited by the greedy selection rules.

a. Algebraic structure and global information. The echo fidelity F_v similarly admits a nested commutator expansion, though with a more complex structure involving products of commutators between different operator sets. The same fundamental principles established in section A apply: the symmetry of the $\hat{X}_i$ operators leads to alternating symmetry properties in the nested commutators, and the light-cone structure ensures that the n -th order term probes correlations within an n -step neighborhood.

However, the echo protocol introduces an important distinction: it directly measures the *algebraic compatibility* between vertex v and the active set S , encoded through the commutators $[\sum_{i \in S} \hat{X}_i, \hat{X}_v]_n$ and $[\sum_{i \in S} \hat{X}_i, \Pi_{\mathcal{N}(v)}]_n$. This is fundamentally different from evaluating occupation probabilities or energies, instead it quantifies how much vertex v "conflicts" with the existing structure. Vertices with high fidelity exhibit near-commuting behavior, suggesting they can coexist in a mutually compatible (approximately Abelian) subset, while low-fidelity vertices strongly interfere with the active block and must be handled by including them or their neighbors in the cover.

The Quantum Fidelity Greedy approach has a scaling of the number of circuit evaluations similar to the QEG since at each recursive step we have to evaluate a number of conditioned fidelities that is equal to the number of vertices.

b. Smallest-Fidelity (SDF-like) Strategy In the first approach, the algorithm selects the vertex whose associated operator exhibits the strongest non-commutativity with respect to the rest of the vertices, namely

$$v^* = \arg \min_{v \in V} F_v,$$

and fixes it in the vertex cover. The graph is then reduced by removing v^* and all edges incident to it.

This strategy mirrors the classical smallest-degree heuristic. A low value of the fidelity F_v indicates that the operator associated with v strongly fails to commute with the active operators, signaling that v is highly constrained by its neighborhood. Fixing such vertices reduces this non-commutativity and drives the system toward a mutually commuting subset.

Algorithm 5 QuantumFidelityGreedy-SDF

```

1: Input: Graph  $G = (V, E)$ , Cover set  $S = \emptyset$ , Trotter
   depth  $p$ , quantum walk-times  $t_a, t_b$ 
2: if  $E = \emptyset$  then
3:   return  $S$ 
4: end if
5: for each vertex  $v \in V$  do
6:   Construct the echo circuit  $U_v = A_v B_v A_v^\dagger B_v^\dagger$   $\triangleright A_v$ 
     acts on  $U \setminus \{v\}$ ,  $B_v$  on vertex  $v$ 
7:   Measure the echo fidelity  $F_v = |\langle \Omega | U_v | \Omega \rangle|^2$ 
8: end for
9:  $v^* = \arg \min_{v \in V} F_v$  and add it to the Cover set  $S \leftarrow S \cup \{v^*\}$ 
10: Remove  $v^*$  and all incident edges:  $G \leftarrow (V \setminus \{v^*\}, E \setminus \{(v^*, u)\})$ 
11: return QuantumFidelityGreedy – SDF( $G', S, t_a, t_b, p$ )

```

c. Largest-Fidelity (LDF-like) Strategy An alternative approach is to select the vertex whose operator is most compatible with the active block, namely

$$v^* = \arg \max_{v \in V} F_v,$$

and instead fix all of its neighbors in the vertex cover. The graph is then reduced by removing the neighborhood $N(v^*)$ and all incident edges.

A large fidelity F_v indicates that the operator associated with v approximately commutes with the active operators, suggesting that v can remain in a mutually commuting subset. Consequently, to satisfy the vertex-cover constraints, its neighbors must be included instead.

From a quantum perspective, this strategy promotes the construction of a maximal commuting (approximately Abelian) subalgebra. The vertices that survive this process correspond to an independent set, while their neighbors form the vertex cover.

As with the previous algorithms, the echo times set the distribution probed by the fidelity criterion. Figure 10 (left) reports the performance for $t \in [0, \pi]$ on 100 3-regular instances with $n = 18$ vertices. In contrast to QPG and QEG, here *both* strategies are sensitive to the choice of t , and both peak *before* $\pi/2$. This earlier optimum is consistent with the algebraic nature of the criterion: the echo fidelity probes the non-commutativity between a vertex operator and the active block, a signal that is sharpest while the wavefront is still spreading outward and has not yet been reflected back through the upper layers.

Algorithm 6 QuantumFidelityGreedy-LDF

```

1: Input: Graph  $G = (V, E)$ , Cover set  $S = \emptyset$ , Trotter
   depth  $p$ , quantum walk-times  $t_a, t_b$ ,
2: if  $E = \emptyset$  then
3:   return  $S$ 
4: end if
5: for each vertex  $v \in V$  do
6:   Construct the echo circuit  $U_v = A_v B_v A_v^\dagger B_v^\dagger$   $\triangleright A_v$ 
     acts on  $U \setminus \{v\}$ ,  $B_v$  on vertex  $v$ 
7:   Measure the echo fidelity  $F_v = |\langle \Omega | U_v | \Omega \rangle|^2$ 
8: end for
9:  $v^* = \arg \max_{v \in V} F_v$ 
10: Add the set of neighbors  $\mathcal{N}(v^*)$  to the cover:  $S \leftarrow S \cup \mathcal{N}(v^*)$ 
11: Remove  $\mathcal{N}(v^*)$  and all incident edges:
      $G \leftarrow (V \setminus \mathcal{N}(v^*), E \setminus \{(u, w) \in E : u \in \mathcal{N}(v^*) \text{ or } w \in \mathcal{N}(v^*)\})$ 
12: return QuantumFidelityGreedy-LDF( $G', S, t_a, t_b, p$ )

```

In terms of solution quality, QFG underperforms the marginal and energy based criteria. Already in the exact ($p = \infty$) case both variants fall slightly below QPG, and the gap widens at finite Trotter depth $p = 4$, where the approximation of the echo sequence degrades the fidelity ranking more strongly than it does the marginal occupations or conditional energies. The two variants nonetheless remain competitive with, and for suitable t exceed their classical greedy counterparts, while sharing the $\mathcal{O}(|V|^2)$ circuit-evaluation overhead of QEG, since a conditional fidelity must be evaluated for every vertex at each recursive step.

VI. COMPARISON AND BENCHMARKING

We evaluate our algorithms on a benchmark of 100 graphs per graph size across three graph families: 3-regular, 4-regular and Erdős–Rényi random graphs with a connection probability of $p = 0.3$. Graphs are generated at sizes $n \in \{8, 10, 12, 14, 16, 18, 20\}$ nodes. The three previously presented quantum algorithms, namely QPG, QEG, and QFG, are each run at circuit depths $p \in \{1, 2, 3, 4\}$ and for the exact case, and for each method we consider both the LDF (largest-degree-first) and SDF (smallest-degree-first) greedy strategies. Classical baselines include the greedy LDF heuristic, the greedy SDF heuristic, and a greedy random-edge heuristic. Performance is assessed using two metrics: the mean approximation ratio averaged over all instances at each graph size, and the probability of recovering an optimal vertex cover.

A. Trotter effect

An important practical question is how the performance of the proposed algorithms changes when the

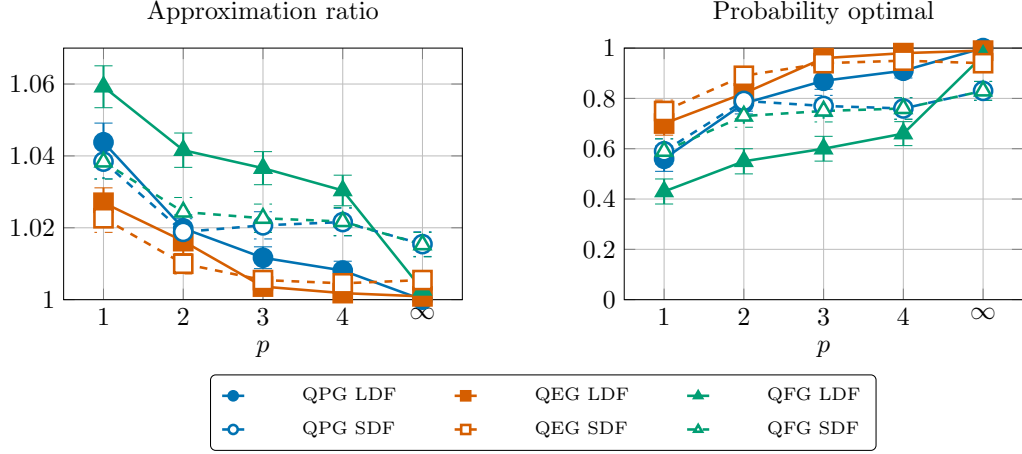


Figure 12: QPG, QEG and QFG performances as a function of the number of Trotter layers p on 100 3-regular graph instances of size $n = 20$. Left: mean approximation ratio. Right: probability of obtaining an optimal solution. $p = \infty$ shows the exact continuous-time evolution. Solid lines with filled markers denote LDF, while dashed lines with open markers denote SDF.

continuous-time walk is approximated by a finite-depth first-order Suzuki–Trotter expansion. In figure 12 we report the mean approximation ratio and the probability of recovering an optimal vertex cover as functions of the Trotter depth p for QPG, QEG and QFG on 100 random 3-regular instances with $n = 20$. The final point, $p = \infty$, corresponds to the exact continuous evolution.

For the LDF strategy, both metrics improve almost monotonically as the number of Trotter layers increases. Each additional layer reduces the difference between the trotterized dynamics and the exact walk, leading to a more accurate ordering of the vertex probabilities that drive the greedy selection. Consequently, the finite-depth curves progressively approach the exact limit. The improvement is most pronounced in the first few layers and rapidly saturates: already at $p = 4$ the performance is close to that of the exact evolution. This behavior agrees with the analysis of section B 2, where the approximation error scales as $\mathcal{O}(Et^2/p)$. For the moderate walk times considered here, only a small number of Trotter steps is therefore sufficient to suppress the error below the level at which it significantly affects the greedy decision process.

The SDF strategy exhibits a different behavior. The performance improves from $p = 1$ to $p = 2$, already surpassing the corresponding classical heuristics, but remains nearly constant for larger depths and up to the exact limit. A possible explanation is that the SDF update rule is more aggressive: low-scoring vertices trigger the inclusion of neighboring vertices into the cover, causing larger reductions of the instance at each step. As a consequence, the algorithm becomes less sensitive to the fine structure of the probability ranking generated by the quantum walk, and further improvements in the accuracy of the Trotter approximation have only a limited impact on the final solution quality.

The same qualitative behavior is observed across the full range of graph sizes considered in the benchmarks and is not specific to the $n = 20$ instances shown in figure 12. In particular, the rapid convergence of the finite-depth approximation toward the exact limit for the LDF variants, together with the earlier saturation observed for SDF, persists for different system sizes. Similar trends are also found for Erdős–Rényi graphs with connection probability $p = 0.3$, indicating that the effect of the Trotter depth is largely independent of the graph family and reflects a general feature of the constrained quantum dynamics underlying the algorithms.

B. Comparison across graph families

Across different graph families the LDF and SDF performance curves retain their characteristic dependence on the walk time, and the empirically optimal walk time lies on the same scale as for the regular instances, indicating that the time at which the marginal contrast is maximal is governed primarily by the layered geometry of $\mathcal{G}_{\text{VC}}$ rather than by the specific graph family.

Quantitatively, however, the Erdős–Rényi instances are easier for all methods: both the mean approximation ratio and the optimal-recovery probability are systematically higher than on 3 or 4 regular graphs of the same size, as reported in figure 13 for $p = 4$. A natural explanation can be found in the structure of the induced feasible-state graph. The denser connectivity of the $p = 0.3$ Erdős–Rényi ensemble leads to a larger minimum vertex-cover number $\tau(G)$ and therefore to a smaller depth of the induced graph. Consequently, the walker has fewer layers to traverse before reaching the minimum-cover boundary. At the same time, the stronger connectivity imposes more constraints, producing a more irregular feasible-

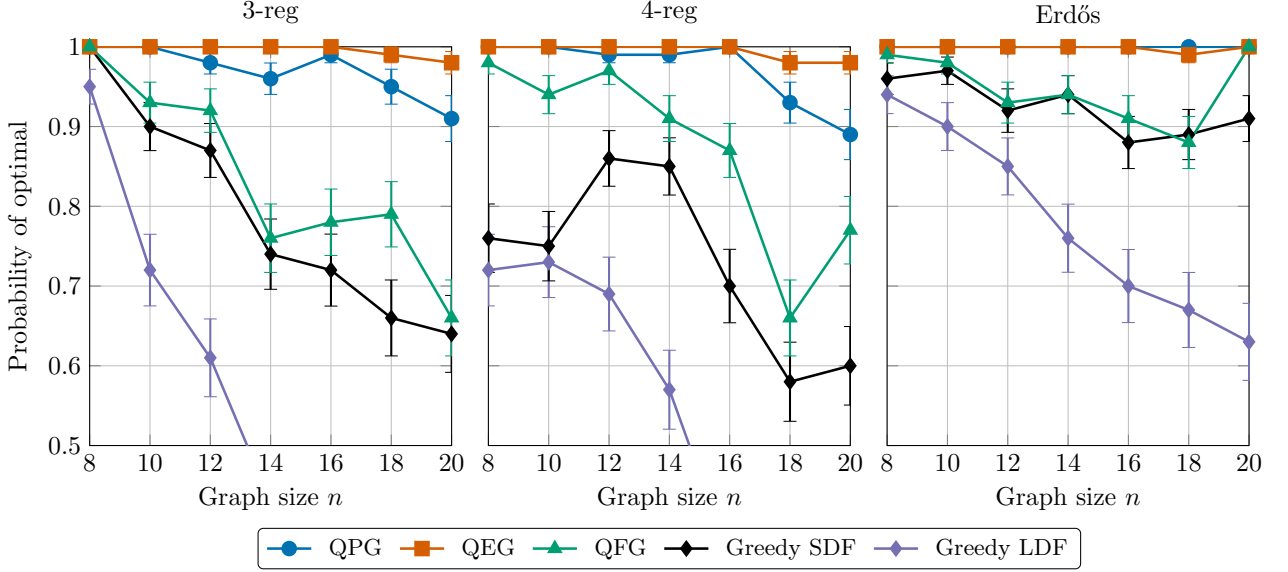


Figure 13: Probability of finding the optimal solution at circuit depth $p = 4$, across graph types and iterative quantum algorithms. Each panel corresponds to a different type of graph: 3 regular (left), 4 regular (centre), and Erdős-Rényi $p = 0.3$ (right). Shown are results for the three algorithms families: QPG (blue), QEG (orange), and QFG (green). Only the Largest-Degree-First (LDF) ordering is shown since is the one that performs best. Error bars show the standard error of the mean over 100 random graph instances per size.

state graph making it easier for the greedy criterion to succeed.

C. Choice of variant

Across all tested scoring rules and greedy strategies, the LDF variants provide the most direct vertex-cover interpretation: a high-scoring vertex is fixed into the cover and the instance is reduced. Among these, QEG-LDF gives the strongest and the most promising performance, making it the strongest and promising of the algorithms tested in our benchmarks. In particular as reported in [figure 13](#), the QEG is performing way better at low number of trotter steps with performance very close to the exact ones for every type of graph studied. This is consistent with the fact that the energy score evaluates the effect of fixing a vertex on the global cost expectation, whereas QPG uses only a marginal inclusion probability and QFG uses an echo-fidelity signal that probes the structure of the constrained dynamics more indirectly.

The SDF variants reduce the number of recursive steps, but it also produces more aggressive updates and is therefore less fine-grained optimization. In the present benchmarks, the SDF variants are useful for comparison and for illustrating the flexibility of the framework.

VII. CONCLUSION

We have introduced and analyzed a family of constraint-preserving quantum-classical greedy algorithms for the Minimum Vertex Cover and Maximum Independent Set problems. The construction rests on a mixer built from projected Pauli- X generators $\hat{X}_i = \Pi_{N(i)} X_i$, which confine the dynamics to the feasible vertex-cover subspace without requiring the introduction of penalty terms or variational training. Restricted to this subspace, the resulting Hamiltonian $H_{\text{MVC}} = \sum_i \hat{X}_i$ acts as the adjacency matrix of an induced feasible-state graph G_{VC} , whose vertices are valid covers and whose edges connect covers related by a single feasibility-preserving bit flip. This graph carries a natural layering by Hamming weight, with the all-in-the-cover state $|\Omega\rangle$ occupying the top layer and the minimum vertex covers populating the bottom layer $L_{\tau(G)}$.

We proved that every feasible configuration is reachable from $|\Omega\rangle$ through the constrained dynamics, so that G_{VC} is fully connected from the root, and we interpreted the unitary $e^{itH_{\text{MVC}}}$ as a continuous-time quantum walk on this layered graph. A walk-count expansion linked transition amplitudes to the number of paths on G_{VC} . The cumulative bottom-layer probability decays approximately exponentially with distance from the minimum-cover layer: the walk spreads amplitude efficiently toward low-weight configurations, yet the probability of landing exactly on a minimum cover is exponentially suppressed. This observation motivates the iterative quantum algo-

gorithms where one extracts per-vertex statistics from the walk state, including vertices into the vertex cover recursively.

We consequently developed three complementary greedy strategies, Quantum Probabilistic Greedy (QPG), Quantum Energy Greedy (QEG), and Quantum Fidelity Greedy (QFG), which select vertices according to marginal occupation probabilities, conditional energy variations, and echo-fidelity measures of operator compatibility, respectively. Despite their distinct operational definitions, all three exploit the same underlying capacity of the constrained walk to convert global graph information into locally measurable signals, and each admits both a largest-degree-first (LDF) variant that fixes a high-scoring vertex into the cover and a smallest-degree-first (SDF) variant that instead commits the neighbors of a low-scoring vertex.

Our benchmarks span 3-regular, 4-regular, and Erdős-Rényi random graphs. Across all three families, and at a suitably chosen walk time, every quantum variant exceeds its classical greedy counterpart in both the mean approximation ratio and the probability of recovering an optimal cover. The performance is therefore conditional on the walk time, whose optimal value we find to be governed primarily by the layered geometry of G_{VC} . Among the variants, QEG-LDF is the strongest and most promising: because the conditional energy $E_v(t)$ probes the full cost Hamiltonian rather than a single-vertex marginal, it captures multi-vertex correlations and yields a sharper, more reliable ranking than QPG, while avoiding the more indirect echo-fidelity signal of QFG. QEG attains near-optimal performance already at shallow circuit depth ($p \leq 4$), with the finite-depth results closely tracking the exact continuous-time limit. This accuracy comes at a measurement cost of $O(|V|^2)$ circuit evaluations per run, against the $O(|V|)$ of QPG; QFG shares the heavier $O(|V|^2)$ overhead of QEG while delivering the weakest of the three rankings. The SDF variants, though less fine-grained, reduce the number of recursive steps and demonstrate the flexibility of the dual framework.

Two practical features make the approach attractive for near-term hardware. First, the state-preparation circuit has polynomial gate count and, for bounded-degree graphs, constant-depth Trotter layers, so the quantum resource requirement is mild. Second, and independently, in the single-layer case ($p = 1$) the CTQW state admits an efficient classical surrogate via the constrained quantum tree; we exploited this surrogate purely as a simulation tool to probe the scaling of solution quality, reaching instances of up to 300 vertices, well beyond the reach of exact state-vector simulation. We emphasize that the 300-vertex results validate the *algorithmic* scaling at $p = 1$ and do not by themselves constitute a quantum demonstration; the case for quantum relevance rests on the circuit-depth scaling and on the role of deeper ($p > 1$) walks, which our finite- p benchmarks indicate continue to improve the ranking quality.

Several open questions follow naturally, the most consequential being whether the transport properties of the constrained walk admit a provable approximation guarantee. Our numerics indicate that the cumulative bottom- ℓ peak probability $P_{\leq \ell}^*(n)$ decays exponentially in n at fixed offset ℓ , but that the fitted decay rate becomes statistically indistinguishable from zero for small offsets across the simulated range, so that every benchmark instance reaches the lowest few layers with near-unit probability. Whether this favorable behavior reflects a genuine asymptotic guarantee, or whether the offset $\ell^*(n; \epsilon)$ at which the cumulative probability saturates eventually grows with n , is the central theoretical question raised by our analysis and cannot be settled by direct simulation alone. A resolution in which $\ell^*(n; \epsilon)$ grows slowly would imply that a single measurement yields a cover within an additive $o(n)$ of optimal with high probability; settling its scaling is, in our view, the most important next step. On the algorithmic side, a systematic comparison against other iterative quantum methods and state-of-the-art classical heuristics is needed to delineate the true scope of the method. The framework also suggests extensions inspired by other classical greedy strategies: edge-based or higher-order selection criteria could enhance expressivity, and problem-dependent, structure-informed choices of the walk time may offer a further lever on performance.

VIII. AVAILABILITY OF DATA AND CODE

To support reproducibility and independent verification of our results, we have made all relevant data and source code publicly available at <https://github.com/OpenQuantumComputing/>.

IX. FUNDING

This work has been funded by the Research Council of Norway through project number 332023.

X. AUTHOR CONTRIBUTIONS

Ruben Pariente Bassa contributed to the conceptualization, methodology, software development, formal analysis, investigation, visualization and manuscript writing. Finley A. Quinton contributed to the conceptualization, methodology, software development, formal analysis, investigation, visualization and manuscript writing. Franz G. Fuchs contributed to the conceptualization, scientific guidance, supervision, funding acquisition, visualization, and manuscript writing. Pascal Halffmann has contributed to the scientific guidance, supervision, and manuscript writing.

- [1] R. M. Karp, Reducibility among combinatorial problems, in *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art* (Springer, 2009) pp. 219–241.
- [2] V. V. Vazirani, *Approximation algorithms*, Vol. 1 (Springer, 2001).
- [3] S. Khot, On the power of unique 2-prover 1-round games, in *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing* (2002) pp. 767–775.
- [4] D. Jungnickel, The greedy algorithm, in *Graphs, Networks and Algorithms* (Springer Berlin Heidelberg, 2013) pp. 135–161.
- [5] M. Halldórsson and J. Radhakrishnan, Greed is good: Approximating independent sets in sparse and bounded-degree graphs, *Algorithmica (New York)* **18**, 145 (1997).
- [6] P. Krysta, M. Mari, and N. Zhi, Ultimate greedy approximation of independent sets in subcubic graphs, *Algorithmica* **86**, 3518 (2024).
- [7] T. A. Feo and M. G. C. Resende, Greedy randomized adaptive search procedures, *Journal of Global Optimization* **6**, 109 (1995).
- [8] J. Bang-Jensen, G. Gutin, and A. Yeo, When the greedy algorithm fails, *Discrete Optimization* **1**, 121 (2004).
- [9] A. Lucas, Ising formulations of many np problems, *Frontiers in Physics* **2**, 5 (2014).
- [10] E. Pelofske, G. Hahn, and H. Djidjev, Solving large minimum vertex cover problems on a quantum annealer, arXiv:1904.00051 (2019).
- [11] F. G. Fuchs, K. O. Lye, H. Møll Nilsen, A. J. Stasik, and G. Sartor, Constraint preserving mixers for the quantum approximate optimization algorithm, *Algorithms* **15**, 202 (2022).
- [12] D. Herman, R. Shaydulin, Y. Sun, S. Chakrabarti, S. Hu, P. Minssen, A. Rattew, R. Yalovetzky, and M. Pistoia, Constrained optimization via quantum zeno dynamics, *Communications Physics* **6**, 219 (2023).
- [13] E. Farhi, J. Goldstone, and S. Gutmann, A quantum approximate optimization algorithm, arXiv (2014), arXiv:1411.4028 [quant-ph].
- [14] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O’Brien, A variational eigenvalue solver on a photonic quantum processor, *Nature communications* **5**, 4213 (2014).
- [15] S. Hadfield, Z. Wang, B. O’Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, From the quantum approximate optimization algorithm to a quantum alternating operator ansatz, *Algorithms* **12**, 10.3390/a12020034 (2019).
- [16] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, XY mixers: Analytical and numerical results for the quantum alternating operator ansatz, *Phys. Rev. A* **101**, 012320 (2020).
- [17] A. Bärttschi and S. Eidenbenz, Grover mixers for qaoa: Shifting complexity from mixer design to state preparation, arXiv:2006.00354 (2020).
- [18] F. G. Fuchs and R. P. Bassa, Lx-mixers for qaoa: Optimal mixers restricted to subspaces and the stabilizer formalism, *Quantum* **8**, 1535 (2024).
- [19] Z. H. Saleem, Max-independent set and the quantum alternating operator ansatz, *International Journal of Quantum Information* **18**, 2050011 (2020).
- [20] D. Bucher, J. Stein, S. Feld, and C. Linnhoff-Popien, Penalty-free approach to accelerating constrained quantum optimization, *Physical Review A* **112**, 062605 (2025).
- [21] A. Montanaro, Quantum speedup of branch-and-bound algorithms, *Physical Review Research* **2**, 013056 (2020).
- [22] S. Chakrabarti, P. Minssen, R. Yalovetzky, and M. Pistoia, Universal quantum speedup for branch-and-bound, branch-and-cut, and tree-search algorithms, arXiv preprint arXiv:2210.03210 (2022).
- [23] S. Wilkening, A.-I. Lefterovici, L. Binkowski, M. Perk, S. P. Fekete, and T. J. Osborne, A quantum algorithm for solving 0-1 knapsack problems, *npj Quantum Information* **11**, 10.1038/s41534-025-01097-8 (2025).
- [24] D. J. Egger, J. Mareček, and S. Woerner, Warm-starting quantum optimization, *Quantum* **5**, 479 (2021).
- [25] S. Bravyi, A. Kliesch, R. Koenig, and E. Tang, Hybrid quantum-classical algorithms for approximate graph coloring, *Quantum* **6**, 678 (2022).
- [26] J. R. Finžgar, A. Kerschbaumer, M. J. Schuetz, C. B. Mendl, and H. G. Katzgraber, Quantum-informed recursive optimization algorithms, *PRX Quantum* **5**, 020327 (2024).
- [27] F. Luiz, A. Iwakami, D. Moraes, and M. de Oliveira, Scalable quantum walk-based heuristics for the minimum vertex cover problem, arXiv preprint arXiv:2512.02940 (2025).
- [28] L. T. Brady and S. Hadfield, Iterative quantum algorithms for maximum independent set, *Phys. Rev. A* **110**, 052435 (2024).
- [29] M. Dupont, B. Evert, M. J. Hodson, B. Sundar, S. Jeffrey, Y. Yamaguchi, D. Feng, F. B. Maciejewski, S. Hadfield, M. S. Alam, Z. Wang, S. Grabbe, P. A. Lott, E. G. Rieffel, D. Venturelli, and M. J. Reagor, Quantum-enhanced greedy combinatorial optimization solver, *Science Advances* **9**, 10.1126/sciadv.adi0487 (2023).
- [30] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, Elementary gates for quantum computation, *Physical Review A* **52**, 3457–3467 (1995).
- [31] R. Vale, T. M. D. Azevedo, I. C. S. Araújo, I. F. Araujo, and A. J. da Silva, Decomposition of multi-controlled special unitary single-qubit gates (2023), arXiv:2302.06377 [quant-ph].
- [32] P. J. Christiansen, L. Binkowski, D. Ramacciotti, and S. Wilkening, Quantum tree generator improves qaoa state-of-the-art for the knapsack problem, in *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2025) p. 1–10.
- [33] S. Wilkening, Constraint-oriented biased quantum search for general constrained combinatorial optimization problems (2025), arXiv:2512.08384 [quant-ph].

Appendix A: Nested Commutators of Projected Pauli X Generators

In this appendix, we analyze the algebraic structure generated by projected single-qubit Pauli X operators acting within the vertex-cover subspace $\mathcal{H}_{\text{VC}}$. Rather than attempting a full characterization of the resulting dynamical Lie algebra, we focus on structural properties that directly govern the behavior of the quantum algorithms introduced in the main text.

Our approach is guided by the observation that all relevant observables, marginal selection probabilities, conditional energy estimates, and fidelity measures, can be expressed in terms of the diagonal contributions of nested commutators of the projected generators $\{\hat{X}_i\}_{i \in V}$ with respect to Z_v or other Projected Paulis. The key question is therefore not the full Lie closure, but how operator support, symmetry, and constraint structure propagate under repeated commutation.

To this end, we proceed constructively. We first define the constraint-preserving projected operators and identify the graph-theoretic conditions under which projected Pauli strings are nonvanishing. We then establish fundamental symmetry properties of the generators, showing that they form a real antisymmetric algebra when restricted to $\mathcal{H}_{\text{VC}}$.

Building on these results, we analyze the structure of nested commutators, deriving an operator light-cone that bounds the growth of support and proving selection rules that restrict which terms contribute to expectation values. These properties provide an understanding how local quantum measurements encode increasingly non local graph information.

1. Constraint-Preserving Operators

We begin by reviewing the projection formalism that ensures all operators respect the vertex-cover constraint. The projector onto the valid vertex-cover subspace is

$$\Pi_{\text{VC}} = \sum_{C \in \text{VC}} |C\rangle\langle C| = \prod_{(i,j) \in E} (\mathbb{I} - |00\rangle\langle 00|_{ij}),$$

where the product form explicitly enforces the edge constraint $C_i + C_j \geq 1$ for every edge $(i, j) \in E$.

To implement Pauli rotations that preserve the vertex-cover constraint, we sandwich single-qubit operators between projectors. For a vertex $i \in V$ with open neighborhood $\mathcal{N}(i) = \{j \in V : (i, j) \in E\}$, the projected Pauli X operator takes the form

$$\Pi_{\text{VC}} X_i \Pi_{\text{VC}} = \Pi_{\text{VC}_{G \setminus \mathcal{N}(i)}} \otimes \left(\Pi_{\mathcal{N}(i)} \otimes X_i \right) = \Pi_{\text{VC}_{G \setminus \mathcal{N}(i)}} \otimes \hat{X}_i,$$

where $\Pi_{\mathcal{N}(i)} = \bigotimes_{j \in \mathcal{N}(i)} |1\rangle\langle 1|_j$ projects all neighbors of i onto the occupied state $|1\rangle$, and $\Pi_{\text{VC}_{G \setminus \mathcal{N}(i)}}$ maintains the vertex-cover constraint on the remaining vertices.

This construction ensures that $\hat{X}_i$ acts nontrivially only when all neighbors of vertex i are in the cover. Outside this subspace, $\hat{X}_i$ vanishes since we have trivially $[\hat{X}_i, \Pi_{\text{VC}}] = 0$.

Not all projected Pauli operators are nonzero; their support must satisfy a graph-theoretic constraint tied to the bipartite structure of vertex covers. Let $P = \otimes_{i \in B} \sigma_i^{X/Y} \otimes_{j \notin B} \sigma_j^{I/Z}$ be a Pauli strings that acts as X or Y on some set of qubits B , and as I or Z on the complement $V \setminus B$. We denote as $\Pi_{\text{VC}} P \Pi_{\text{VC}}$ be the related Projected Pauli operator. Expanding the projector in its product form,

$$\Pi_{\text{VC}} P \Pi_{\text{VC}} = \prod_{(i,j) \in E} (\mathbb{I} - |00\rangle\langle 00|_{ij}) P \prod_{(k,\ell) \in E} (\mathbb{I} - |00\rangle\langle 00|_{k\ell}).$$

We analyze how each edge projector interacts with the Pauli operators, distinguishing three cases based on whether the edge endpoints lie in the flipping set B :

1. **Both endpoints outside B** ($i, j \notin B$): The operator acts as $\sigma_i^{I/Z} \otimes \sigma_j^{I/Z}$, which commutes with $|00\rangle\langle 00|_{ij}$. Thus

$$(\mathbb{I} - |00\rangle\langle 00|_{ij}) (\sigma_i^{I/Z} \otimes \sigma_j^{I/Z}) (\mathbb{I} - |00\rangle\langle 00|_{ij}) = (\sigma_i^{I/Z} \otimes \sigma_j^{I/Z}) (\mathbb{I} - |00\rangle\langle 00|_{ij}).$$

2. **One endpoint in B** ($i \in B, j \notin B$): Here the operator acts as $\sigma_i^{X/Y} \otimes \sigma_j^{I/Z}$. Using $\sigma^{X/Y} |0\rangle = \pm |1\rangle$ and $\sigma^{X/Y} |1\rangle = \pm |0\rangle$, we compute

$$(\mathbb{I} - |00\rangle\langle 00|_{ij}) (\sigma_i^{X/Y} \otimes \sigma_j^{I/Z}) (\mathbb{I} - |00\rangle\langle 00|_{ij}) = \pm \sigma_i^{X/Y} \otimes |1\rangle\langle 1|_j$$

where the projector $|1\rangle\langle 1|_j$ enforces that neighbor j must be occupied.

3. **Both endpoints in B** ($i, j \in B$): The operator acts as $\sigma_i^{X/Y} \otimes \sigma_j^{X/Y}$. Both qubits flip simultaneously, and the edge constraint is preserved only if they have opposite Z eigenvalues:

$$(\mathbb{I} - |00\rangle\langle 00|_{ij}) (\sigma_i^{X/Y} \otimes \sigma_j^{X/Y}) (\mathbb{I} - |00\rangle\langle 00|_{ij}) = (\sigma_i^{X/Y} \otimes \sigma_j^{X/Y}) \frac{\mathbb{I} - Z_i Z_j}{2}$$

The factor $\frac{\mathbb{I} - Z_i Z_j}{2}$ projects onto the symmetric sector $Z_i Z_j = +1$.

Combining these results over all edges and collecting the neighbor projectors $|1\rangle\langle 1|_j$ for $j \in \mathcal{N}(B) = \{k \notin B : (k, \ell) \in E \text{ for some } \ell \in B\}$, we obtain

$$\Pi_{VC} P \Pi_{VC} = \eta(B) \left(\bigotimes_{i \in B} \sigma_i^{X/Y} \right) \left(\prod_{(a,b) \in E(G[B])} \frac{\mathbb{I} - Z_a Z_b}{2} \right) \otimes \Pi_{\mathcal{N}(B)} \otimes (\Pi_{VC_{G \setminus \mathcal{N}(B) \cup B}} \times \bigotimes_{j \notin B \cup \mathcal{N}(B)} \sigma_j^{I/Z}),$$

where $\eta(B) = (-1)^{|\{(i,j) \in E : i \in B, j \in \mathcal{N}(B), \sigma_j = Z\}|}$ is a phase factor arising from the anticommutation of Z operators with X/Y flips, and $E(G[B])$ denotes the edge set of the induced subgraph on B .

The bipartite constraint. The product of symmetric projectors $\prod_{(a,b) \in E(G[B])} \frac{\mathbb{I} - Z_a Z_b}{2}$ encodes the crucial structural requirement. Each factor projects onto the subspace where Z_a and Z_b have the same eigenvalue. For the entire product to be nonzero, there must exist a simultaneous eigenstate where all pairs of adjacent vertices in B share the same Z eigenvalue, equivalently, a proper 2-coloring of $G[B]$.

Such a coloring exists if and only if $G[B]$ is bipartite. If $G[B]$ contains an odd cycle $(v_1, v_2, \dots, v_{2k+1}, v_1)$, the constraints $Z_{v_1} = Z_{v_2} = \dots = Z_{v_{2k+1}} = Z_{v_1}$ with an odd number of equalities form a contradiction. Therefore

$$\prod_{(a,b) \in E(G[B])} \frac{\mathbb{I} - Z_a Z_b}{2} = 0 \iff G[B] \text{ is non-bipartite,}$$

which completes the proof of [lemma 1](#) and establishes the general criterion for which Pauli operators survive projection.

Summary. The constraint-preserving property of projected Pauli operators is encoded in two complementary requirements: (i) all neighbors of the flipping set B must be occupied (enforced by $\Pi_{\mathcal{N}(B)}$), and (ii) the induced subgraph $G[B]$ must be bipartite (enforced by the vanishing of $\prod_{(a,b) \in E(G[B])} \frac{\mathbb{I} - Z_a Z_b}{2}$ for non-bipartite graphs). These structural constraints significantly restrict the space of available quantum moves and shape the dynamics within the vertex-cover subspace.

Lemma 1 (Bipartiteness of the Flipping Support). Let $P = (\bigotimes_{i \in B} \sigma_i^{X/Y}) \otimes (\bigotimes_{j \notin B} \sigma_j^{I/Z})$ be a Pauli operator, where $B \subseteq V$ is the set of qubits on which P acts as X or Y . Then

$$\Pi_{VC} P \Pi_{VC} \neq 0 \implies \text{the induced subgraph } G[B] \text{ is bipartite.}$$

Proof. The operator P flips the computational basis state of every qubit in B simultaneously. For the projected operator $\Pi_{VC} P \Pi_{VC}$ to be nonzero, there must exist at least one valid vertex-cover configuration $|C\rangle$ such that $P|C\rangle$ is also a valid vertex cover.

Consider any edge $(u, v) \in E$ with both endpoints in B . The simultaneous flip changes both C_u and C_v . For the edge constraint $C_u + C_v \geq 1$ to remain satisfied before and after the flip, the two vertices must have had complementary initial occupations: one occupied ($C = 1$) and one unoccupied ($C = 0$). This complementarity condition must hold for every edge within B .

This is precisely the requirement for a proper 2-coloring of the induced subgraph $G[B]$, which exists if and only if $G[B]$ is bipartite. If $G[B]$ contains an odd cycle, no valid 2-coloring exists, meaning no vertex-cover configuration can be mapped to another valid cover by the simultaneous flip. Therefore $\Pi_{VC} P \Pi_{VC} = 0$ whenever $G[B]$ is non-bipartite. $\square$

2. Nested Commutators

Both the QPG selection probability $P_v(t)$ in [equation \(11\)](#) and the QEG energy estimate $E_v(t)$ in [equation \(12\)](#) are governed by nested commutators of the transverse-field Hamiltonian $H = \sum_i \hat{X}_i$ with Pauli Z operators that compose the cost Hamiltonian $H_C = \sum_i Z_i$. For the QFG in ?? instance, we are interested in the nested commutator between a specific vertex operator $\hat{X}_v$ and the rest of the graph $\sum_{i \in S} \hat{X}_i$.

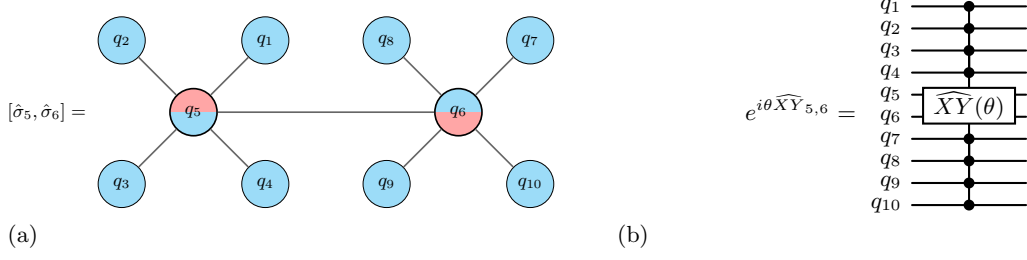


Figure 14: (a) Graphical representation of two adjacent vertices, q_5 and q_6 , and their respective neighborhoods. (b) Corresponding quantum circuit for the higher-order multi-controlled operation generated by the nested commutator $[\hat{\sigma}_5, \hat{\sigma}_6]$, implementing a rotation $\widehat{XY}_{5,6}(\theta)$ conditioned on all neighboring qubits being in the state $|1\rangle$.

Their first nontrivial commutators already reveal the emergence of higher-order, yet still constraint-preserving, transitions. For adjacent vertices i and j , one obtains

$$[\hat{X}_i, \hat{X}_j] = \Pi_{\mathcal{N}(i,j)} \otimes (X_i X_j + Y_i Y_j) \delta_E(i, j) = \widehat{XY}_{ij} \delta_E(i, j),$$

and $\delta_E(i, j) = 1$ if $(i, j) \in E$ and 0 otherwise. Thus, if two vertices are not connected by an edge, the corresponding generators commute. If they are connected, their commutator generates a two-body operator supported on that edge and conditioned on the surrounding neighborhood, as illustrated in figure 14.

This pattern continues under further nested commutators: starting from single-vertex generators $\hat{X}_i$, one generates increasingly nonlocal operators supported on connected subgraphs, while still preserving feasibility at every step. In this sense, the algebra generated by the projected Pauli operators provides a hierarchy of allowed transitions inside $\mathcal{H}_{VC}$, ranging from local conditional flips to collective multi-qubit moves. This higher-order structure is the algebraic motivation of the nonlocal information extracted later by the greedy selection rules.

In particular, we want to understand how the nested commutators behaves:

$$[\sum_i \hat{X}_i, Z_v]_n = \sum_{i_n} \cdots \sum_{i_1} [\hat{X}_{i_n}, [\dots, [\hat{X}_{i_1}, Z_v]]].$$

$$[\sum_{i \in S} \hat{X}_i, \hat{X}_v]_n = \sum_{i_n \in S} \cdots \sum_{i_1 \in S} [\hat{X}_{i_n}, [\dots, [\hat{X}_{i_1}, \hat{X}_v]]].$$

$$[\sum_i \hat{X}_i, \Pi_{\mathcal{N}(v)}]_n = \sum_{i_n} \cdots \sum_{i_1} [\hat{X}_{i_n}, [\dots, [\hat{X}_{i_1}, \Pi_{\mathcal{N}(v)}]]].$$

We define the n -nested commutator recursively as

$$C_n := [\sum_{i \in V} \hat{X}_i, C_{n-1}], \quad C_0 = Z_v, X_v, \Pi_{\mathcal{N}(v)}.$$

Note that projectors commute with all Z operators, $[\Pi_{\mathcal{N}(i)}, Z_v] = 0 \quad \forall i, v$ but not with X or Y : $[\Pi_{\mathcal{N}(i)}, X_j] = -iY_j$ and $[\Pi_{\mathcal{N}(i)}, Y_j] = iX_j$ for $j \in \mathcal{N}(i)$.

a. Single-step commutation rule. For a Projected Pauli string $\hat{\mathcal{O}}_S = \mathcal{O}_S \Pi_{\mathcal{N}(S)}$ with support S and $\Pi_{\mathcal{N}(S)} = \prod_{j \in \mathcal{N}(S)} \frac{I - Z_j}{2}$, a direct application of the Leibniz rule yields

$$[\hat{X}_i, \hat{\mathcal{O}}_S] = X_i [\Pi_{\mathcal{N}(i)}, \mathcal{O}_S] \Pi_{\mathcal{N}(S)} + [X_i, \mathcal{O}_S] \Pi_{\mathcal{N}(i)} \Pi_{\mathcal{N}(S)} + \mathcal{O}_S [X_i, \Pi_{\mathcal{N}(S)}] \Pi_{\mathcal{N}(i)}. \quad (\text{A1})$$

since $[\Pi_{\mathcal{N}(i)}, \Pi_{\mathcal{N}(S)}] = 0$ The three terms are generically nonzero when $\mathcal{N}(i) \cap S \neq \emptyset$, $\{i\} \cap S \neq \emptyset$, and $\{i\} \cap \mathcal{N}(S) \neq \emptyset$, respectively. The result is a sum of Projected Pauli operators supported on $S \cup \mathcal{N}(i) \cup \{i\}$: support grows by at most one graph-distance shell per commutation step.

b. Projected Pauli decomposition. Since the commutation between Projected Pauli produce projected Paulis we can express C_n as a sum of Projected Pauli operators. Let's start defining the closed n -step neighborhood of v as

$$\mathcal{N}_n(v) = \{u \in V \mid \text{dist}(u, v) \leq n\},$$

with $\mathcal{N}_0(v) = \{v\}$. We write

$$C_n = \sum_{S \subseteq \mathcal{N}_n(v)} f_n(S) \hat{\mathcal{O}}_S,$$

where $\mathcal{O}_S$ is a Pauli string supported on S and $f_n(S) \in \mathbb{C}$ are coefficients determined by the recursion below.

Proposition 1 (Operator Light Cone). *For all $n \geq 0$, $\text{supp}(C_n) \subseteq \mathcal{N}_n(v)$.*

Proof. By induction on n .

Base case ($n = 0$). $C_0 = Z_v$ has support $\{v\} = \mathcal{N}_0(v)$.

Inductive step. Suppose $\text{supp}(C_{n-1}) \subseteq \mathcal{N}_{n-1}(v)$, so that $C_{n-1} = \sum_{S \subseteq \mathcal{N}_{n-1}(v)} f_{n-1}(S) \mathcal{O}_S$. By linearity,

$$C_n = \sum_{S \subseteq \mathcal{N}_{n-1}(v)} f_{n-1}(S) \sum_{i \in V} [\hat{X}_i, \hat{\mathcal{O}}_S].$$

By [equation \(A1\)](#), $[\hat{X}_i, \mathcal{O}_S]$ has support in $S \cup \mathcal{N}(i) \cup \{i\}$, and is identically zero unless $i \in S \cup \mathcal{N}(S) \subseteq \mathcal{N}_{n-1}(v) \cup \mathcal{N}(\mathcal{N}_{n-1}(v)) = \mathcal{N}_n(v)$. Hence every nonzero term in C_n is supported within $\mathcal{N}_n(v)$. $\square$

Corollary 2 (Global Saturation). *For $n \geq \text{diam}(G)$, the support of C_n saturates to the full vertex set: $\text{supp}(C_n) = V$.*

This means that beyond depth $\text{diam}(G)$, every qubit participates in the commutator expansion.

3. Trotter Expansion for Product Formulas

Having established the nested commutator structure, we now analyze how Trotterized time evolution encodes similar algebraic properties. Understanding the commutator structure of these Trotterized unitaries is essential for analyzing their performance and convergence.

a. Product Formula Decomposition. We are interested in how the conjugation $U_{\text{Trotter}}^p(t) Z_v U_{\text{Trotter}}^p(t)^\dagger$ differs from the exact Baker–Campbell–Hausdorff (BCH) expansion, where $U_{\text{Trotter}}^p(t) = \prod_p \prod_i e^{i \frac{t}{p} \hat{X}_i}$

Let's start with a two terms example:

$$\begin{aligned} e^{it\hat{X}_2} e^{it\hat{X}_1} Z_v e^{-it\hat{X}_1} e^{-it\hat{X}_2} &= e^{it\hat{X}_2} \left(\sum_{n_1=0}^{\infty} \frac{(it)^{n_1}}{n_1!} [\hat{X}_1, Z_v]_{n_1} \right) e^{-it\hat{X}_2} \\ &= \sum_{n_2=0}^{\infty} \sum_{n_1=0}^{\infty} \frac{(it)^{n_1+n_2}}{n_1! n_2!} [\hat{X}_2, [\hat{X}_1, Z_v]_{n_1}]_{n_2}. \end{aligned}$$

Each unitary $e^{\pm it\hat{X}_i}$ generates a nested commutator expansion, and the full product accumulates contributions from all possible commutator paths.

Generalizing to p layers and $N = |V|$ qubits, we obtain

$$\begin{aligned} U_{\text{Trotter}}^p(t) Z_v U_{\text{Trotter}}^p(t)^\dagger &= \sum_{\{n_i^r\}} \frac{1}{\prod_{i,r} n_i^r!} \left(i \frac{t}{p} \right)^{\sum_{i,r} n_i^r} \\ &\quad \times \left[\hat{X}_N, \dots [\hat{X}_1, [\dots [\hat{X}_N, \dots [\hat{X}_1, Z_v]_{n_1^1} \dots]_{n_N^1} \dots]_{n_1^p} \right]_{n_N^p}, \end{aligned}$$

where the sum runs over all non-negative integers n_i^r for $i \in V$ and $r = 1, \dots, p$.

For the First order ($\mathcal{O}(t)$) each layer contributes $\frac{it}{p} \sum_{i \in V} [\hat{X}_i, Z_v]$, and summing over p layers gives

$$it \sum_{i \in V} [\hat{X}_i, Z_v],$$

which matches the exact BCH expansion.

At the second order ($\mathcal{O}(t^2)$) two types of contributions arise:

Intra-layer terms:

$$-\frac{t^2}{2p} \sum_{i \in V} [\hat{X}_i, Z_v]_2.$$

Inter-layer terms:

$$-\frac{t^2}{2p^2} \mathcal{D}(p, N) \sum_{i, j \in V} [\hat{X}_i, [\hat{X}_j, Z_v]],$$

where the degeneracy factor is $\mathcal{D}(p) = p(p-1)$.

4. Symmetry Selection Rules and Diagonal Survival

We now exploit the symmetry property of the Hamiltonian to derive selection rules that dramatically simplify the nested commutator expansions appearing in $P_v(t)$, $E_v(t)$, and $F_v(t)$. These selection rules reveal that only even-order terms contribute to the expectation values, and provide the mechanism by which quantum dynamics encode global graph information into local observables.

Lemma 2 (Alternating Symmetry of Nested Commutators). In the real basis $\{|C\rangle\}_{C \in \text{VC}(G)}$ of $\mathcal{H}_{\text{VC}}$, nested commutators of the form

$$C_n = \left[\sum_{i \in V} \hat{X}_i, C_{n-1} \right], \quad C_0 \in \{Z_v, \Pi_{\mathcal{N}(v)}, \hat{X}_v\},$$

satisfy the alternating symmetry relation

$$C_n^T = (-1)^n C_n,$$

where T denotes the matrix transpose. Equivalently, C_n is symmetric for even n and antisymmetric for odd n .

Proof. We proceed by induction on n .

Base case ($n = 0$). Seed operators are symmetric: Z_v is diagonal, and $\Pi_{\mathcal{N}(v)} = \prod_{j \in \mathcal{N}(v)} \frac{I - Z_j}{2}$ is a product of diagonal matrices, and $\hat{X}_v$ is symmetric by construction. Thus $C_0^T = C_0$.

Inductive step. Assume $C_{n-1}^T = (-1)^{n-1} C_{n-1}$. Since every generator satisfies $\hat{X}_i^T = \hat{X}_i$, taking the transpose of the recursion relation and applying the antilinearity $(AB)^T = B^T A^T$,

$$\begin{aligned} C_n^T &= \left[\sum_{i \in V} \hat{X}_i, C_{n-1} \right]^T \\ &= \sum_i (\hat{X}_i C_{n-1} - C_{n-1} \hat{X}_i)^T \\ &= \sum_i (C_{n-1}^T \hat{X}_i^T - \hat{X}_i^T C_{n-1}^T) \\ &= \sum_i ((-1)^{n-1} C_{n-1}) \hat{X}_i - \hat{X}_i ((-1)^{n-1} C_{n-1}) \\ &= (-1)^n \sum_i (C_{n-1} \hat{X}_i - \hat{X}_i C_{n-1}) \\ &= (-1)^n \left[C_{n-1}, \sum_i \hat{X}_i \right] \\ &= (-1)^n \left[\sum_i \hat{X}_i, C_{n-1} \right] = (-1)^n C_n, \end{aligned}$$

□

Lemma 3 (Diagonal Contributions). For the all-ones computational basis state $|\Omega\rangle \equiv |1\rangle^{\otimes |V|}$, antisymmetric operators have vanishing expectation value:

$$A^T = -A \implies \langle \Omega | A | \Omega \rangle = 0.$$

Proof. The state $|\Omega\rangle$ is a real vector in the computational basis. For any operator A , the expectation value is a scalar, hence invariant under transpose:

$$\langle \Omega | A | \Omega \rangle = (\langle \Omega | A | \Omega \rangle)^T = \langle \Omega | A^T | \Omega \rangle.$$

If $A^T = -A$, then

$$\langle \Omega | A | \Omega \rangle = \langle \Omega | (-A) | \Omega \rangle = -\langle \Omega | A | \Omega \rangle,$$

which implies $\langle \Omega | A | \Omega \rangle = 0$. $\square$

Combining lemma 2 and lemma 3 immediately yields a powerful simplification of all three quantum metrics.

Corollary 3 (Odd-Order Selection Rule). *Odd-order nested commutators make no contribution to expectation values with respect to the all-ones state:*

$$\langle \Omega | C_{2k+1} | \Omega \rangle = 0 \quad \forall k \geq 0.$$

Consequently, the series expansions reduce to even orders only:

$$P_v(t) = \frac{1}{2} - \frac{1}{2} \sum_{k=0}^{\infty} \frac{(it)^{2k}}{(2k)!} \langle \Omega | \left[\sum_j \hat{X}_j, Z_v \right]_{2k} | \Omega \rangle, \quad (\text{A2})$$

$$E_v(t) = \sum_{k=0}^{\infty} \sum_{\ell} \frac{(it)^{2k}}{(2k)!} \langle \Omega | \left[\sum_{j \neq v} \hat{X}_j, Z_{\ell} \right]_{2k} | \Omega \rangle. \quad (\text{A3})$$

Proof. By lemma 2, C_{2k+1} is antisymmetric for all $k \geq 0$. By lemma 3, $\langle \Omega | C_{2k+1} | \Omega \rangle = 0$. The truncated series follow immediately by dropping all vanishing even-order terms from the original expansions. $\square$

a. Selection rules for quantum echo fidelity. The fidelity expansion involves three distinct nested commutators:

$$\begin{aligned} F_v(t) = & \left| \sum_{n=0}^{\infty} \frac{(it_a)^n}{n!} \left[(\cos t_b - 1) \cos t_b \langle \Omega | \left[\sum_{i \in S} \hat{X}_i, \Pi_{\mathcal{N}(v)} \right]_n | \Omega \rangle \right. \right. \\ & + i \sin^2 t_b \langle \Omega | \left[\sum_{i \in S} \hat{X}_i, \hat{X}_v \right]_n \hat{X}_v | \Omega \rangle \\ & \left. \left. + i(\cos t_b - 1) \sin t_b \langle \Omega | \left[\sum_{i \in S} \hat{X}_i, \Pi_{\mathcal{N}(v)} \right]_n \hat{X}_v | \Omega \rangle \right] \right|^2. \end{aligned} \quad (\text{A4})$$

The first is related directly corollary 3, so only even terms are expected to contribute. For the last two terms instead are the odd terms to contribute since we multiplie the nested commutator with the symmetric operator $\hat{X}_v$ flipping the the parity. In summary: the first term in equation (A4) receive contributions only from even orders, while the second term and the third receives contributions only from odd orders.

b. Encoding global information in local observables. The selection rules reveal a striking contrast with classical greedy algorithms. A classical heuristic selects vertices using purely local information, typically the degree $\deg(v)$ or a score depending only on the immediate neighborhood $\mathcal{N}(v)$. By contrast, the quantum selection probability $P_v(t)$ and energy estimate $E_v(t)$ incorporate contributions from arbitrarily distant vertices through the nested commutator hierarchy.

To see this quantitatively, observe that the k -th term in equation (A2) involves the matrix element $\langle \Omega | C_{2k} | \Omega \rangle$. By proposition 1, the operator C_{2k} has support confined to the closed $(2k)$ -neighborhood

$$\mathcal{N}_{2k}(v) = \{u \in V : \text{dist}(u, v) \leq 2k\}. \quad (\text{A5})$$

Thus the contribution of the k -th term encodes information from all vertices within graph distance $2k$ of v . As the quantum walk time t increases, higher-order terms become significant, effectively broadening the information radius. Once t is large enough that terms with $2k \geq \text{diam}(G)$ contribute non-negligibly, the selection probability receives input from the *entire graph* (corollary 2).

Appendix B: The CTQW State Analysis

In this section we analyze different properties of the continuous quantum walk of the Vertex cover state graph initialize in the root state $|\Omega\rangle$. In particular we will focus on three different aspects that are relevant for understanding its performance: (i) the scaling of gate counts and circuit depth required to implement the CTQW respect to the number of vertices and trotter steps, (ii) The trotter error analysis, and (iii) the entanglement scaling induced by its multi-controlled structure.

As we show, while the circuit can remain shallow for bounded-degree graphs due to parallelization, the generated quantum state exhibits strong entanglement and effectively encodes global graph structure.

We first quantify the gate complexity and parallelization scaling of the state preparation.

1. Scaling

The cost in terms of gates and depth of implementing the one trotter step of the quantum walk unitary

$$U(t) = \prod_i C_{\mathcal{N}(i)} R X_i(t)$$

is determined by the vertex degrees of the graph $G(V, E)$. For each vertex i , the operator $C_{\mathcal{N}(i)} R X_i(t)$ is a multi-controlled single-qubit rotation acting on qubit i , with one control qubit for each neighbor in the set $\mathcal{N}(i)$. Defining the degree of vertex i as

$$d_i := |\mathcal{N}(i)|,$$

this operation corresponds to a d_i -controlled X -rotation.

A d_i -controlled single-qubit rotation can be decomposed into one and two qubit gates with a cost that depends on the number of ancillary qubits:

- **No ancilla qubits.** In the absence of ancillas, the decomposition of a d_i -controlled single-qubit rotation requires $O(d_i^2)$ CNOT gates and $\mathcal{O}(d_i^2) + \mathcal{O}(\log(1/\varepsilon))$ T gates. The corresponding circuit depth also scales as $O(d_i^2)$. [30]
- **One clean ancilla qubit.** If a single clean ancilla qubit is available, the same operation can be implemented using $O(d_i)$ CNOT and $\mathcal{O}(d_i^2) + \mathcal{O}(\log(1/\varepsilon))$, with a circuit depth scaling linearly as $O(d_i)$ [31].
- **$O(\log d_i)$ ancilla qubits.** Allowing for $O(\log d_i)$ ancilla qubits enables a further reduction of the circuit depth to $O(\log d_i)$, while maintaining a linear CNOT count of $O(d_i)$ [31].

Summing over all vertices, the total gate count of the trotter step scales as $\sum_i O(d_i^m)$ where $m = 2$ in the ancilla-free case and $m = 1$ when ancillas are available. Let

$$\Delta := \max_i d_i.$$

The total gate count scales as $O(n\Delta)$ or, since $\sum_i d_i = 2|E|$, so is linear in the number of edges for ancilla-assisted implementations, and is $O(n\Delta^2)$ in the ancilla-free case.

The depth of the full unitary depends both on the depth of the individual multi-controlled rotations and on the extent to which gates acting on different vertices can be parallelized. Ignoring parallelization across vertices, one trotter step scales in depth as $O(n\Delta^2)$ without ancillas, $O(n\Delta)$ with one ancilla, and $O(n \log \Delta)$ with $O(\log \Delta)$ ancillas. Two multicontrol rotations operators can be executed in parallel if and only if their supports are disjoint.

This condition is violated whenever the corresponding vertices i and j are adjacent or share a common neighbor in the interaction graph $G^2 = (V, E)$ that correspond to take the *square* of the problem graph, which has the same vertex set as G and an edge between two vertices whenever their graph distance in G is at most two. We can approximate vertex coloring of G^2 assigns colors to vertices such that no two vertices within distance two in G share the same color. Vertices of the same color correspond to mixer terms whose supports are disjoint and can therefore be applied simultaneously.

Let $\chi(G^2)$ denote the chromatic number of the squared graph. Then one trotter step can be scheduled in at most $\chi(G^2)$ sequential layers, each layer consisting of mutually commuting mixer terms. The total circuit depth of a quantum walk trotter step is therefore bounded by

$$\text{Depth}(U_M) \leq \chi(G^2) d_{\text{gate}},$$

where d_{gate} is the depth required to implement a single d_i -controlled rotation.

For a graph with maximum degree Δ , each vertex in G^2 has degree at most $O(\Delta^2)$, implying the standard bound

$$\chi(G^2) \leq \Delta^2 + 1.$$

In the important case of bounded-degree graphs, where $\Delta = O(1)$, the chromatic number $\chi(G^2)$ is therefore constant. As a consequence, the depth of the trotter step circuit does not scale with the system size n and is determined entirely by the cost of implementing an individual multi-controlled rotation.

2. Trotter Error Analysis

We now analyze the error introduced by the Trotterization of the continuous-time quantum walk evolution. Given the exact continuous evolution is

$$U(t) = e^{itH_M}.$$

Since the local terms $\hat{X}_i$ do not generally commute, the exact evolution is approximated through a first-order Suzuki–Trotter decomposition:

$$U_p(t) = \left(\prod_{i \in V} e^{i \frac{t}{p} \hat{X}_i} \right)^p,$$

where p denotes the number of Trotter steps.

To quantify the approximation quality, we evaluate the state error with respect to the exact evolved state initialized in the root configuration $|\Omega\rangle$:

$$\epsilon_p(t) = \|U_p(t) |\Omega\rangle - e^{itH_M} |\Omega\rangle\|_2,$$

For a first-order product formula, the operator norm error satisfies the standard bound

$$\|U_p(t) - e^{itH_M}\| = O\left(\frac{t^2}{p} \sum_{i < j} \|\llbracket \hat{X}_i, \hat{X}_j \rrbracket\|\right).$$

The scaling of the Trotter error is therefore controlled by the commutator structure of the constrained mixer terms.

Two terms $\hat{X}_i$ and $\hat{X}_j$ commute whenever their supports are disjoint. Non-commuting contributions arise only when the corresponding vertices are adjacent or share neighboring constraints. Consequently, the number of non-vanishing commutators is determined by the number of edge $|E|$. Consequently we have:

$$\epsilon_p(t) = O\left(\frac{Et^2}{p}\right).$$

This scaling is consistent with the numerical results shown in [figure 15](#). Figure (a) shows the dependence of the state error on the number of Trotter steps p for a fixed 3-regular graph with $n = 18$ vertices. The observed behavior follows the expected power-law decay

$$\epsilon_p(t) \sim p^{-1},$$

characteristic of first-order Trotter formulas.

Figure (b) reports the dependence of the error on the evolution time t for different values of p . The growth is approximately quadratic in time for sufficiently small t , in agreement with the perturbative prediction

$$\epsilon_p(t) \propto \frac{t^2}{p}.$$

Finally, Figure (c) analyzes the scaling with graph size for ensembles of randomly generated 3-regular graphs at fixed evolution time and fixed Trotter depth. The error exhibits an approximately linear growth with the number of vertices,

$$\epsilon_p(t) \propto n,$$

consistent with the number of edges in the 3 regular case.

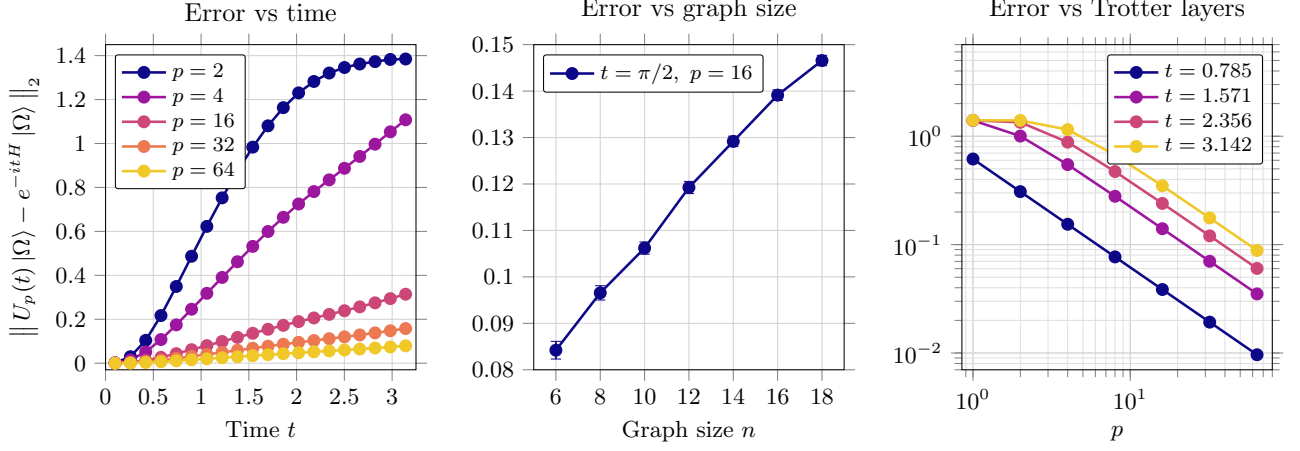


Figure 15: Trotter error analysis for the constrained continuous-time quantum walk. (a) Norm-2 distance between the exact CTQW state and the Trotterized state as a function of the number of Trotter steps p for a fixed 3-regular graph with $n = 18$ vertices. (b) Dependence of the state error on the evolution time t for different Trotter depths p on the same graph. (c) Scaling of the Trotter error with graph size for ensembles of random 3-regular graphs at fixed evolution time and fixed Trotter depth.

3. Entanglement Analysis

As shown in [section B 3](#), the mean subsystem entanglement entropy S_{blocks} for connected subgraphs of increasing size grows approximately linearly with subsystem size, indicating a clear volume-law scaling. Subsystems of size k approach entropies close to the maximal value expected for half of the system, demonstrating that correlations are highly delocalized across the entire register.

This behavior can be understood as a direct consequence of the structure of the quantum walk unitary. Each term $C_{\mathcal{N}(i)}RX_i(\beta_i)$ is a multi-controlled rotation that conditionally entangles qubit i with all qubits in its neighborhood $\mathcal{N}(i)$. While each individual operation is local in the graph, repeated application across vertices induces a rapid growth of operator support. This mechanism is formally captured by the light-cone result proven in [proposition 1](#).

From the Heisenberg perspective, this corresponds to the spreading of initially local operators under conjugation by the circuit. As already suggested by the nested commutator expansion introduced below, successive commutations with the transverse-field generator $H_X = \sum_i \hat{X}_i$ enlarge the support of observables by at most one graph-distance shell per step.

This extensive operator spreading has two important consequences:

- **Volume-law entanglement.** Since local degrees of freedom become correlated with an extensive portion of the system, reduced density matrices rapidly approach high entropy. This explains the observed linear scaling of S_{blocks} with subsystem size.
- **Global correlation structure.** Observables such as $\langle Z_i \rangle$ depend on contributions from nested commutators involving increasingly distant vertices. As a result, even single-qubit expectation values encode global information about the graph.

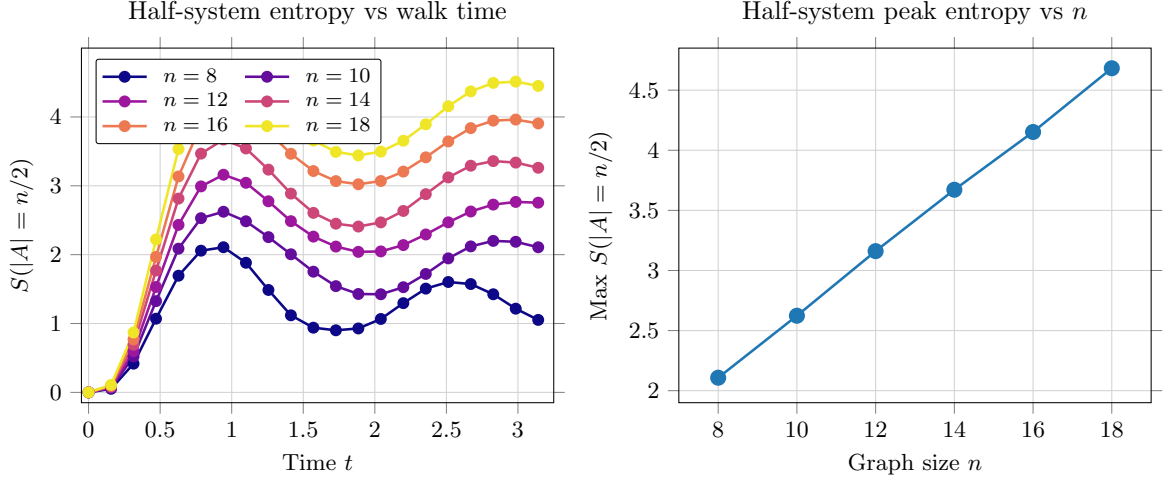


Figure 16: Mean von Neumann entropy S_{blocks} calculated over 40 connected subgraphs. (a) S_{blocks} versus walk time t with connected block size $k = n/2$. (b) Scaling of $S_{\text{blocks}}^{\text{max}}$ with total system size n for $t = \pi/4$.

Appendix C: Monte Carlo Simulation for $p = 1$

While the quantum walk state $|\psi(t)\rangle$ exhibits volume-law entanglement and encodes global graph structure through nested commutator dynamics, a remarkable simplification occurs in the single-layer case ($p = 1$). At this depth, the quantum circuit admits an efficient classical representation that enables Monte Carlo estimation of expectation values without requiring explicit quantum state preparation or measurement.

In what follows, we formalize the quantum tree representation underlying the $p = 1$ state, derive the pseudo-Boolean form of the cost function, and describe the Monte Carlo sampling procedure. We then present numerical results for the performance of our QPG and QEG algorithms using the Monte Carlo estimator for regular graphs and Erdos-Renyi graphs up to size 300.

1. Quantum Tree Representation of the Feasible Subspace

For $p = 1$, the evolved quantum state admits a natural interpretation as a weighted superposition over feasible vertex covers.

The final quantum state can be written as

$$|\psi(t)\rangle = \prod_{i \in V} C_{\mathcal{N}(i)} R X_i(t) |\Omega\rangle,$$

where the ordering of vertices induces a sequential structure in the circuit.

a. Quantum Tree Construction. The circuit naturally induces a rooted tree structure, which we refer to as the *quantum state tree*. Each level of the tree corresponds to a vertex $i \in V$ in the prescribed ordering, and each node represents a partial assignment consistent with the vertex-cover constraint.

At level i , the controlled rotation $C_{\mathcal{N}(i)} R X(t)$ generates a binary branching:

- with amplitude $\cos(t)$, the qubit remains in state $|\Omega\rangle$ (the vertex remains in the cover), corresponding to the left branch;
- with amplitude $i \sin(t)$, the qubit flips to $|0\rangle$ (the vertex is removed from the cover), corresponding to the right branch.

Due to the vertex-cover constraints, not all branches correspond to valid configurations. Branches that would violate feasibility are effectively *pruned* by the projectors implicit in the controlled rotations. As a result, the quantum tree is generally *unbalanced*, with branching occurring only along paths corresponding to valid vertex-cover assignments. This construction is closely related to the class of *quantum tree generators* introduced in the context of Quantum

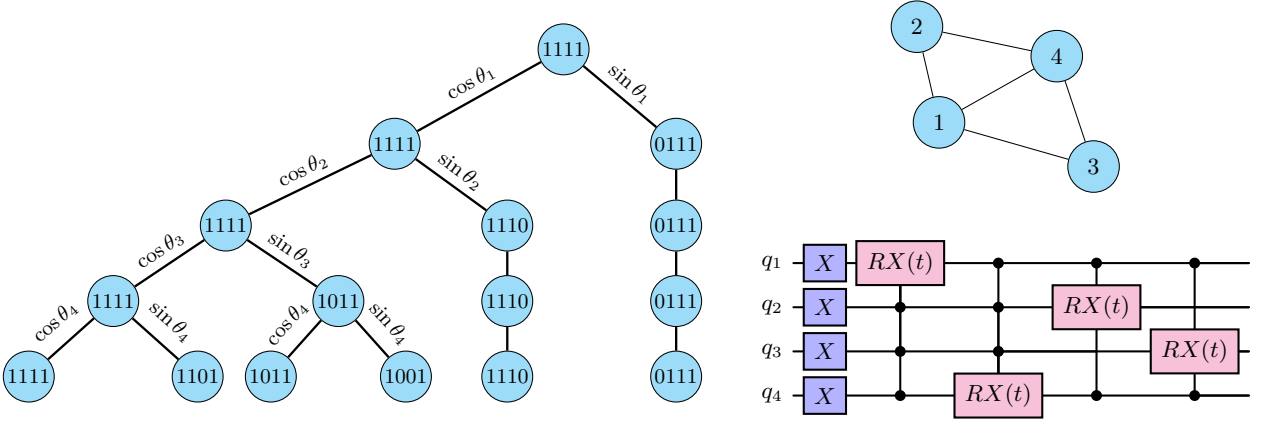


Figure 17: On the left we have the quantum tree obtained from the 4 vertex graph and its corresponding quantum circuit on the right. We can notice that resulting graph is an unbalanced tree, as expected due to the cover constraints.

Branch-and-Bound (QBNB) and constrained variants of QAOA, where feasible solutions are encoded as amplitudes along root-to-leaf paths of a decision tree [32, 33]. In those approaches, soft linear constraints are typically enforced through the use of additional ancilla registers and penalty mechanisms, which guide the evolution toward feasible regions of the search space.

In contrast, the present construction implements a *hard constraint embedding* directly at the circuit level: feasibility is enforced locally by the multi-controlled rotations, and invalid branches are naturally pruned without requiring auxiliary qubits or penalty terms. As a result, the quantum tree is generated intrinsically within the computational register, leading to a more resource-efficient representation of the feasible subspace while preserving the same path-amplitude interpretation exploited in QBNB frameworks.

Each leaf of the quantum tree corresponds to a feasible vertex cover $C \in \text{VC}(G)$. The amplitude associated with the computational basis state $|C\rangle$ is uniquely determined by the sequence of left and right branches along the corresponding path γ_C :

$$|\psi(t)\rangle = \sum_{C \in \text{VC}(G)} \left(\prod_{j \in R_{\gamma_C}} i \sin(t) \prod_{k \in L_{\gamma_C}} \cos(t) \right) |C\rangle,$$

where R_{γ_C} (resp. L_{γ_C}) denotes the set of indices at which the path branches right (resp. left).

This representation follows from the reachability properties of the constrained mixer: every feasible vertex cover can be obtained from the reference configuration $|\Omega\rangle$ through a sequence of admissible flips, and the circuit assigns a coherent amplitude to each such path.

2. Cost Function as Pseudo Boolean Function

For $p = 1$, since the cost Hamiltonian $H_C = \frac{1}{2} \sum_{i \in V} c_i Z_i$ is diagonal in the computational basis, the expectation value with respect to the quantum state $|\psi(t)\rangle$ can be expressed as follows:

$$\langle \psi(t) | H_C | \psi(t) \rangle = \sum_{C \in \text{VC}(G)} C_C \prod_{j \in R_{\gamma_C}} \sin^2(t) \prod_{k \in L_{\gamma_C}} \cos^2(t),$$

where $C_C = \sum_{i \in C} c_i$ denotes the cost of the vertex cover C , and R_{γ_C} , L_{γ_C} correspond to the sets of qubits that can be flipped along the path γ_C in the quantum tree.

Introducing the change of variables $x := \cos^2(t)$, $x_i \in [0, 1]$, the expectation value can be expressed as a relaxed pseudo-Boolean function.

$$F(x) = \sum_{C \in \text{VC}(G)} C_C \prod_{j \in R_{\gamma_C}} (1 - x) \prod_{k \in L_{\gamma_C}} x.$$

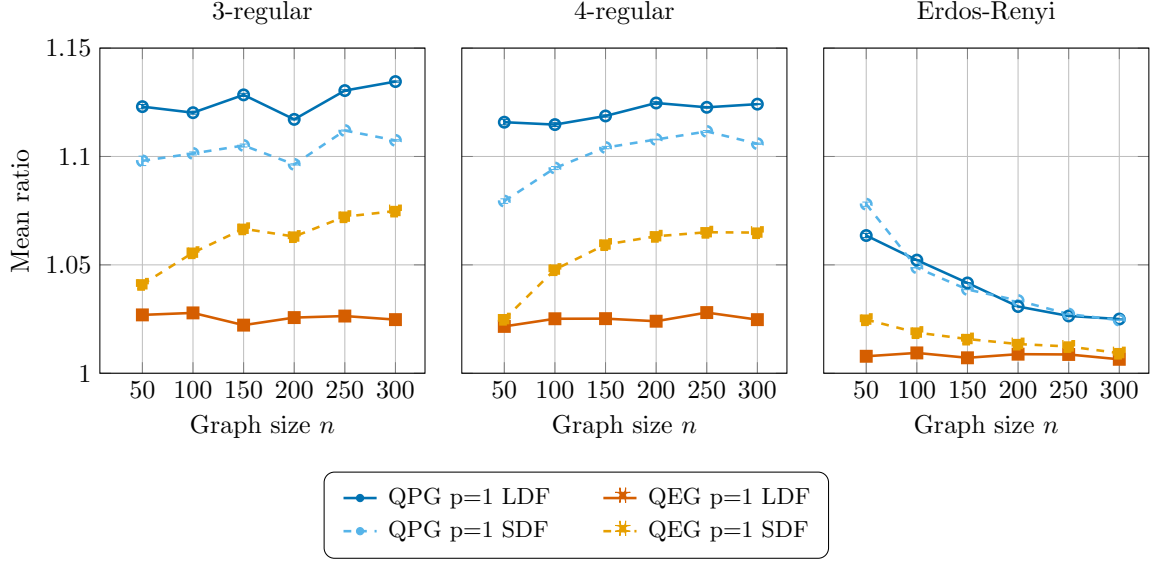


Figure 18: Mean approximation ratio of Monte Carlo simulation of the $p=1$ case on 20 graphs for each size and type. Each panel corresponds to a different type of graph: 3 regular (left), 4 regular (center), and Erdos-Renyi $p = 0.3$ (right). QPG and QEG performances are analyzed to dimension up to 300 for both strategies LDF and SDF.

This form admits a natural reinterpretation as a *path integral over the quantum tree*:

$$F(x) = \sum_{\gamma \in \Gamma} C_{\gamma} \prod_{i \in V} f_i(\gamma_i; x),$$

where Γ is the set of all feasible root-to-leaf paths in the quantum tree, C_{γ} is the cost associated with the leaf reached by γ , and

$$f_i(\gamma_i; x) = \begin{cases} x, & \text{if vertex } i \text{ is retained along path } \gamma, \\ 1 - x, & \text{if vertex } i \text{ is flipped along path } \gamma. \end{cases}$$

defining the path probabilities $w_{\gamma}(x) = \prod_{i \in V} f_i(\gamma_i; x)$ we obtain

$$F(x) = \sum_{\gamma \in \Gamma} C_{\gamma} w_{\gamma}(x), \quad \sum_{\gamma \in \Gamma} w_{\gamma}(x) = 1, \quad (\text{C1})$$

with normalization $\sum_{\gamma} w_{\gamma} = 1$.

Hence $\{w_{\gamma}(x)\}_{\gamma \in \Gamma}$ is a probability distribution over paths, and $F(x) = \mathbb{E}_{\gamma \sim w} [C_{\gamma}]$ is a genuine expectation, with *no* hidden normalization constant. The per-vertex marginal probability that vertex v lies in the cover is likewise

$$P_v(x) = \sum_{\gamma \in \Gamma} w_{\gamma}(x) \mathbb{I}[v \in \gamma], \quad (\text{C2})$$

which coincides with the QPG score evaluated at $p = 1$.

3. Sequential Monte Carlo sampler

Equations (C1)–(C2) show that both QEG and QPG scores at $p = 1$ are expectations under the path measure $w_{\gamma}(x)$. Because w_{γ} factorizes over the tree, a path can be drawn by a single top-down traversal that visits vertices in the fixed order and respects feasibility, with no global summation over the exponentially many covers.

A vertex is removed only when both (i) a uniform draw exceeds x and (ii) the removal preserves feasibility; the feasibility test at line 4 is exactly the pruning of forced nodes, so the leaf returned is sampled with probability $w_{\gamma_C}(x) = |a_C(t)|^2$. The traversal touches each vertex once and the feasibility check inspects only the neighbors of i , giving $O(n + |E|)$ time per sample.

Algorithm 7 Single-sample tree traversal for the $p = 1$ surrogate

```

1: Input: graph  $G = (V, E)$ , parameter  $x = \cos^2 t$ , vertex order  $1, \dots, n$ 
2: Initialize  $C \leftarrow V$  ▷ all vertices in the cover
3: for  $i = 1$  to  $n$  do
4:   if removing  $i$  keeps  $C \setminus \{i\}$  a valid cover then ▷ branching node
5:     draw  $u \sim \mathcal{U}(0, 1)$ 
6:     if  $u \geq x$  then  $C \leftarrow C \setminus \{i\}$  ▷ remove (prob.  $1 - x$ )
7:   end if
8:   end if ▷ otherwise  $i$  is retained deterministically
9: end for
10: return feasible cover  $C$  and its cost  $\mathcal{C}_C = \sum_{i \in C} c_i$ 

```

a. Estimators. From N independent samples $C^{(1)}, \dots, C^{(N)}$ drawn by Algorithm [algorithm 7](#), the cost expectation and the per-vertex marginals are estimated by

$$\hat{F} = \frac{1}{N} \sum_{s=1}^N \mathcal{C}_{C^{(s)}}, \quad \hat{P}_v = \frac{1}{N} \sum_{s=1}^N \mathbb{I}[v \in C^{(s)}].$$

Since each $C^{(s)} \sim w(x)$, both estimators are unbiased, $\mathbb{E}[\hat{F}] = F(x)$ and $\mathbb{E}[\hat{P}_v] = P_v(x)$, with variance $O(1/N)$. Crucially the procedure scales linearly in n and in N , making it tractable for graphs with hundreds of vertices, where exact preparation of $|\psi(t)\rangle$ is infeasible.

b. Greedy scores at $p = 1$. The QPG-LDF/SDF rules use $\hat{P}_v$ directly in place of the measured marginals. The QEG rules require the conditional cost obtained by fixing a vertex; at $p = 1$ this is implemented by running Algorithm [algorithm 7](#) on the graph with the corresponding mixer term removed (vertex v pinned to the cover), and estimating $\hat{F}$ on the resulting tree. As with the full algorithm, this incurs one estimator per candidate vertex per greedy step, i.e. $O(n)$ traversals per step and $O(n^2)$ per run.

4. Numerical results up to $n = 300$

Figure [figure 18](#) reports the mean approximation ratio of the $p = 1$ surrogate for QPG and QEG, in both LDF and SDF variants, on 20 instances per size for 3-regular, 4-regular, and Erdős-Rényi ($\rho = 0.3$) graphs, with sizes up to $n = 300$. The qualitative ordering established in the main text persists deep into this regime: QEG-LDF remains the strongest variant across all three families with no sign of degradation over the simulated range. These results validate the algorithmic scaling at $p = 1$; they do not, on their own, establish a quantum advantage.

Conclusion

Bibliography

- [1] Emile Aarts and Jan Karel Lenstra, eds. *Local Search in Combinatorial Optimization*. Princeton University Press, 2003.
- [2] Yakir Aharonov, Luiz Davidovich and Nicim Zagury. ‘Quantum Random Walks’. In: *Physical Review A* 48.2 (1993), pp. 1687–1690. DOI: 10.1103/PhysRevA.48.1687.
- [3] Andris Ambainis and Martins Kokainis. ‘Quantum Algorithm for Tree Size Estimation, with Applications to Backtracking and 2-Player Games’. In: (2017), pp. 989–1002. DOI: 10.1145/3055399.3055449.
- [4] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge: Cambridge University Press, 2009.
- [5] Adriano Barenco et al. ‘Elementary Gates for Quantum Computation’. In: *Physical Review A* 52.5 (1995). Preprint at <https://arxiv.org/abs/quant-ph/9503016>, pp. 3457–3467. DOI: 10.1103/PhysRevA.52.3457.
- [6] Andreas Bärtschi and Stephan Eidenbenz. ‘Grover Mixers for QAOA: Shifting Complexity from Mixer Design to State Preparation’. In: *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Preprint at <https://arxiv.org/abs/2006.00354>. 2020, pp. 72–82. DOI: 10.1109/QCE49297.2020.00020.
- [7] Andreas Bärtschi and Stephan Eidenbenz. ‘Grover Mixers for QAOA: Shifting Complexity from Mixer Design to State Preparation’. In: *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Preprint at <https://arxiv.org/abs/2006.00354>. 2020, pp. 72–82. DOI: 10.1109/QCE49297.2020.00020.
- [8] Richard Bellman. *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.
- [9] Armin Biere et al., eds. *Handbook of Satisfiability*. IOS Press, 2009.
- [10] Hans L. Bodlaender. ‘A Linear-Time Algorithm for Finding Tree-Decompositions of Small Treewidth’. In: *SIAM Journal on Computing* 25.6 (1996), pp. 1305–1317. DOI: 10.1137/S0097539793251219.
- [11] Lucas T. Brady and Stuart Hadfield. ‘Iterative Quantum Algorithms for Maximum Independent Set: A Tale of Low-Depth Quantum Algorithms’. In: *Physical Review A* 110.5 (2024). Preprint at <https://arxiv.org/abs/2309.13110>, p. 052435. DOI: 10.1103/PhysRevA.110.052435.

- [12] Sergey Bravyi et al. ‘Obstacles to Variational Quantum Optimization from Symmetry Protection’. In: *Physical Review Letters* 125.26 (2020). Preprint at <https://arxiv.org/abs/1910.08980>, p. 260505. DOI: 10.1103/PhysRevLett.125.260505.
- [13] David Bucher et al. ‘IF-QAOA: A Penalty-Free Approach to Accelerating Constrained Quantum Optimization’. In: *arXiv preprint* (2025). Published in *Phys. Rev. A* (2025), DOI 10.1103/fb5m-cl9m. <https://arxiv.org/abs/2504.08663>. arXiv: 2504.08663 [quant-ph].
- [14] David Bucher et al. ‘IF-QAOA: A Penalty-Free Approach to Accelerating Constrained Quantum Optimization’. In: *arXiv preprint* (2025). Key says "2023" but paper is 2025. Published in *Phys. Rev. A* (2025). <https://arxiv.org/abs/2504.08663>. arXiv: 2504.08663 [quant-ph].
- [15] Earl Campbell. ‘Random Compiler for Fast Hamiltonian Simulation’. In: *Physical Review Letters* 123.7 (2019). Preprint at <https://arxiv.org/abs/1811.08017>, p. 070503. DOI: 10.1103/PhysRevLett.123.070503.
- [16] M. Cerezo et al. ‘Does Provable Absence of Barren Plateaus Imply Classical Simulability? Or, Why We Need to Rethink Variational Quantum Computing’. In: *arXiv preprint* (2023). <https://arxiv.org/abs/2312.09121>. arXiv: 2312.09121 [quant-ph].
- [17] Shouvanik Chakrabarti et al. ‘Universal Quantum Speedup for Branch-and-Bound, Branch-and-Cut, and Tree-Search Algorithms’. In: *arXiv preprint* (2022). <https://arxiv.org/abs/2210.03210>. arXiv: 2210.03210 [quant-ph].
- [18] Jianer Chen, Iyad A. Kanj and Ge Xia. ‘Improved Upper Bounds for Vertex Cover’. In: *Theoretical Computer Science* 411.40–42 (2010), pp. 3736–3756. DOI: 10.1016/j.tcs.2010.06.026.
- [19] Andrew M. Childs and Jeffrey Goldstone. ‘Spatial Search by Quantum Walk’. In: *Physical Review A* 70.2 (2004). Preprint at <https://arxiv.org/abs/quant-ph/0306054>, p. 022314. DOI: 10.1103/PhysRevA.70.022314.
- [20] Andrew M. Childs et al. ‘Exponential Algorithmic Speedup by a Quantum Walk’. In: *Proceedings of the 35th Annual ACM Symposium on Theory of Computing (STOC '03)*. Preprint at <https://arxiv.org/abs/quant-ph/0209131>. 2003, pp. 59–68. DOI: 10.1145/780542.780552.
- [21] Paul Christiansen et al. ‘Quantum Tree Generator Improves QAOA State-of-the-Art for the Knapsack Problem’. In: *arXiv preprint* (2024). <https://arxiv.org/abs/2411.00518>. arXiv: 2411.00518 [quant-ph].
- [22] Bob Coecke and Ross Duncan. ‘Interacting Quantum Observables: Categorical Algebra and Diagrammatics’. In: *New Journal of Physics* 13.4 (2011). Preprint at <https://arxiv.org/abs/0906.4725>, p. 043016. DOI: 10.1088/1367-2630/13/4/043016.
- [23] Stephen A. Cook. ‘The Complexity of Theorem-Proving Procedures’. In: *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC '71)*. ACM, 1971, pp. 151–158. DOI: 10.1145/800157.805047.

- [24] Stephen A. Cook. *The P versus NP Problem*. Clay Mathematics Institute Millennium Prize Problem description. Available at <https://www.claymath.org/millennium-problems/>. 2000.
- [25] Gérard Cornuéjols. *Valid Inequalities for Mixed Integer Linear Programs*. Springer, 2008.
- [26] Bruno Courcelle. ‘The Monadic Second-Order Logic of Graphs. I. Recognizable Sets of Finite Graphs’. In: *Information and Computation* 85.1 (1990), pp. 12–75. DOI: 10.1016/0890-5401(90)90043-H.
- [27] Alexander Cowtan et al. ‘Phase Gadget Synthesis for Shallow Circuits’. In: *Proceedings of the 16th International Conference on Quantum Physics and Logic (QPL 2019)*. Vol. 318. Electronic Proceedings in Theoretical Computer Science. Preprint at <https://arxiv.org/abs/1906.01734>. Open Publishing Association, 2019, pp. 213–228. DOI: 10.4204/EPTCS.318.13.
- [28] George B. Dantzig. *Linear Programming and Extensions*. Princeton, NJ: Princeton University Press, 1963.
- [29] N. L. Diaz et al. ‘Showcasing a Barren Plateau Theory Beyond the Dynamical Lie Algebra’. In: *arXiv preprint* (2023). <https://arxiv.org/abs/2310.11505>. arXiv: 2310.11505 [quant-ph].
- [30] Rodney G. Downey and Michael R. Fellows. *Fundamentals of Parameterized Complexity*. London: Springer, 2013.
- [31] Ross Duncan et al. ‘Graph-Theoretic Simplification of Quantum Circuits with the ZX-Calculus’. In: *Quantum* 4 (2020). Preprint at <https://arxiv.org/abs/1902.03178>, p. 279. DOI: 10.22331/q-2020-06-04-279.
- [32] Daniel J. Egger, Jakub Mareček and Stefan Woerner. ‘Warm-Starting Quantum Optimization’. In: *Quantum* 5 (2021). Preprint at <https://arxiv.org/abs/2009.10095>, p. 479. DOI: 10.22331/q-2021-06-17-479.
- [33] Edward Farhi, Jeffrey Goldstone and Sam Gutmann. ‘A Quantum Approximate Optimization Algorithm’. In: *arXiv preprint arXiv:1411.4028* (2014). arXiv: 1411.4028 [quant-ph].
- [34] Edward Farhi and Sam Gutmann. ‘Quantum Computation and Decision Trees’. In: *Physical Review A* 58.2 (1998). Preprint at <https://arxiv.org/abs/quant-ph/9706062>, pp. 915–928. DOI: 10.1103/PhysRevA.58.915.
- [35] Edward Farhi et al. ‘Quantum Computation by Adiabatic Evolution’. In: *arXiv preprint* (2000). <https://arxiv.org/abs/quant-ph/0001106>. arXiv: quant-ph/0001106 [quant-ph].
- [36] Jernej Rudi Finžgar et al. ‘Quantum-Informed Recursive Optimization Algorithms’. In: *PRX Quantum* 5.2 (2024). Preprint at <https://arxiv.org/abs/2308.13607>, p. 020327. DOI: 10.1103/PRXQuantum.5.020327.
- [37] Alan Frieze and Mark Jerrum. ‘Improved Approximation Algorithms for MAX k -CUT and MAX BISECTION’. In: *Algorithmica* 18.1 (1997), pp. 67–81. DOI: 10.1007/BF02523688.

- [38] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.
- [39] Fred Glover. ‘Tabu Search—Part I’. In: *ORSA Journal on Computing* 1.3 (1989), pp. 190–206. DOI: [10.1287/ijoc.1.3.190](https://doi.org/10.1287/ijoc.1.3.190).
- [40] Fred Glover. ‘Tabu Search—Part II’. In: *ORSA Journal on Computing* 2.1 (1990), pp. 4–32. DOI: [10.1287/ijoc.2.1.4](https://doi.org/10.1287/ijoc.2.1.4).
- [41] Michel X. Goemans and David P. Williamson. ‘Improved Approximation Algorithms for Maximum Cut and Satisfiability Problems Using Semidefinite Programming’. In: *Journal of the ACM* 42.6 (1995), pp. 1115–1145. DOI: [10.1145/227683.227684](https://doi.org/10.1145/227683.227684).
- [42] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.
- [43] Daniel Gottesman. ‘Stabilizer Codes and Quantum Error Correction’. Preprint at <https://arxiv.org/abs/quant-ph/9705052>. PhD thesis. California Institute of Technology, 1997. DOI: [10.7907/rzr7-dt72](https://doi.org/10.7907/rzr7-dt72).
- [44] Lov K. Grover. ‘A Fast Quantum Mechanical Algorithm for Database Search’. In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC ’96)*. Preprint at <https://arxiv.org/abs/quant-ph/9605043>. Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 212–219. DOI: [10.1145/237814.237866](https://doi.org/10.1145/237814.237866).
- [45] Johan Håstad. ‘Clique is Hard to Approximate Within $n^{1-\varepsilon}$ ’. In: *Acta Mathematica* 182.1 (1999), pp. 105–142. DOI: [10.1007/BF02392825](https://doi.org/10.1007/BF02392825).
- [46] Johan Håstad. ‘Some Optimal Inapproximability Results’. In: *Journal of the ACM* 48.4 (2001), pp. 798–859. DOI: [10.1145/502090.502098](https://doi.org/10.1145/502090.502098).
- [47] Dylan Herman et al. ‘Constrained Optimization via Quantum Zeno Dynamics’. In: *Communications Physics* 6 (2023). Preprint at <https://arxiv.org/abs/2209.15024>, p. 219. DOI: [10.1038/s42005-023-01331-9](https://doi.org/10.1038/s42005-023-01331-9).
- [48] Dorit S. Hochbaum. ‘Approximation Algorithms for the Set Covering and Vertex Cover Problems’. In: *SIAM Journal on Computing* 11.3 (1982), pp. 555–556. DOI: [10.1137/0211045](https://doi.org/10.1137/0211045).
- [49] John H. Holland. *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [50] David S. Johnson. ‘Approximation Algorithms for Combinatorial Problems’. In: *Journal of Computer and System Sciences* 9.3 (1974), pp. 256–278. DOI: [10.1016/S0022-0000\(74\)80044-9](https://doi.org/10.1016/S0022-0000(74)80044-9).
- [51] Tadashi Kadowaki and Hidetoshi Nishimori. ‘Quantum Annealing in the Transverse Ising Model’. In: *Physical Review E* 58.5 (1998). Preprint at <https://arxiv.org/abs/cond-mat/9804280>, pp. 5355–5363. DOI: [10.1103/PhysRevE.58.5355](https://doi.org/10.1103/PhysRevE.58.5355).
- [52] Narendra Karmarkar. ‘A New Polynomial-Time Algorithm for Linear Programming’. In: *Combinatorica* 4.4 (1984), pp. 373–395. DOI: [10.1007/BF02579150](https://doi.org/10.1007/BF02579150).

- [53] Richard M. Karp. ‘Reducibility Among Combinatorial Problems’. In: *Complexity of Computer Computations*. Ed. by Raymond E. Miller and James W. Thatcher. New York: Plenum Press, 1972, pp. 85–103.
- [54] Tosio Kato. ‘On the Adiabatic Theorem of Quantum Mechanics’. In: *Journal of the Physical Society of Japan* 5.6 (1950), pp. 435–439. DOI: 10.1143/JPSJ.5.435.
- [55] Subhash Khot. ‘On the Power of Unique 2-Prover 1-Round Games’. In: *Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing (STOC '02)*. ACM, 2002, pp. 767–775. DOI: 10.1145/509907.510017.
- [56] Subhash Khot and Oded Regev. ‘Vertex Cover Might Be Hard to Approximate to Within $2 - \varepsilon$ ’. In: *Journal of Computer and System Sciences* 74.3 (2008), pp. 335–349. DOI: 10.1016/j.jcss.2007.06.019.
- [57] Subhash Khot et al. ‘Optimal Inapproximability Results for MAX-CUT and Other 2-Variable CSPs?’ In: *SIAM Journal on Computing* 37.1 (2007), pp. 319–357. DOI: 10.1137/S0097539705447372.
- [58] Scott Kirkpatrick, C. Daniel Gelatt and Mario P. Vecchi. ‘Optimization by Simulated Annealing’. In: *Science* 220.4598 (1983), pp. 671–680. DOI: 10.1126/science.220.4598.671.
- [59] Aleks Kissinger and John van de Wetering. ‘Reducing the Number of Non-Clifford Gates in Quantum Circuits’. In: *Physical Review A* 102.2 (2020). Preprint at <https://arxiv.org/abs/1903.10477>, p. 022406. DOI: 10.1103/PhysRevA.102.022406.
- [60] A. Yu. Kitaev, A. H. Shen and M. N. Vyalyi. *Classical and Quantum Computation*. Vol. 47. Graduate Studies in Mathematics. Providence, RI: American Mathematical Society, 2002. ISBN: 978-0-8218-3229-5. DOI: 10.1090/gsm/047.
- [61] A. H. Land and A. G. Doig. ‘An Automatic Method of Solving Discrete Programming Problems’. In: *Econometrica* 28.3 (1960), pp. 497–520. DOI: 10.2307/1910129.
- [62] Martín Larocca et al. ‘Barren Plateaus in Variational Quantum Computing’. In: *Nature Reviews Physics* (2025). Preprint (2024) at <https://arxiv.org/abs/2405.00781>. DOI: 10.1038/s42254-025-00813-9.
- [63] Martín Larocca et al. ‘Diagnosing Barren Plateaus with Tools from Quantum Optimal Control’. In: *Quantum* 6 (2022). Preprint at <https://arxiv.org/abs/2105.14377>, p. 824. DOI: 10.22331/q-2022-09-29-824.
- [64] E. L. Lawler and D. E. Wood. ‘Branch-and-Bound Methods: A Survey’. In: *Operations Research* 14.4 (1966), pp. 699–719. DOI: 10.1287/opre.14.4.699.
- [65] Leonid A. Levin. ‘Universal Sequential Search Problems’. In: *Problems of Information Transmission* 9.3 (1973), pp. 265–266.
- [66] Andrew Lucas. ‘Ising Formulations of Many NP Problems’. In: *Frontiers in Physics* 2 (2014). Preprint at <https://arxiv.org/abs/1302.5843>, p. 5. DOI: 10.3389/fphy.2014.00005.

- [67] F. S. Luiz et al. ‘Scalable Quantum Walk-Based Heuristics for the Minimum Vertex Cover Problem’. In: *arXiv preprint* (2025). VERIFY – method/details may not match your description. <https://arxiv.org/abs/2512.02940>. arXiv: 2512.02940 [quant-ph].
- [68] João P. Marques-Silva and Karem A. Sakallah. ‘GRASP: A Search Algorithm for Propositional Satisfiability’. In: *IEEE Transactions on Computers* 48.5 (1999), pp. 506–521. DOI: 10.1109/12.769433.
- [69] Jarrod R. McClean et al. ‘Barren Plateaus in Quantum Neural Network Training Landscapes’. In: *Nature Communications* 9 (2018), p. 4812. DOI: 10.1038/s41467-018-07090-4.
- [70] Kosuke Mitarai et al. ‘Quantum Circuit Learning’. In: *Physical Review A* 98.3 (2018). Preprint at <https://arxiv.org/abs/1803.00745>, p. 032309. DOI: 10.1103/PhysRevA.98.032309.
- [71] Ashley Montanaro. ‘Quantum Speedup of Branch-and-Bound Algorithms’. In: *Physical Review Research* 2.1 (2020), p. 013056. DOI: 10.1103/PhysRevResearch.2.013056.
- [72] Ashley Montanaro. ‘Quantum Speedup of Branch-and-Bound Algorithms’. In: *Physical Review Research* 2.1 (2020). Preprint at <https://arxiv.org/abs/1906.10375>, p. 013056. DOI: 10.1103/PhysRevResearch.2.013056.
- [73] George L. Nemhauser and Leslie E. Trotter. ‘Vertex Packings: Structural Properties and Algorithms’. In: *Mathematical Programming* 8.1 (1975), pp. 232–248. DOI: 10.1007/BF01580444.
- [74] Manfred Padberg and Giovanni Rinaldi. ‘A Branch-and-Cut Algorithm for the Resolution of Large-Scale Symmetric Traveling Salesman Problems’. In: *SIAM Review* 33.1 (1991), pp. 60–100. DOI: 10.1137/1033004.
- [75] Alberto Peruzzo et al. ‘A Variational Eigenvalue Solver on a Photonic Quantum Processor’. In: *Nature Communications* 5 (2014), p. 4213. DOI: 10.1038/ncomms5213.
- [76] Michael Ragone et al. ‘A Lie Algebraic Theory of Barren Plateaus for Deep Parameterized Quantum Circuits’. In: *Nature Communications* 15 (2024). Preprint at <https://arxiv.org/abs/2309.09342>, p. 7172. DOI: 10.1038/s41467-024-49909-3.
- [77] Sartaj Sahni and Teofilo Gonzalez. ‘P-Complete Approximation Problems’. In: *Journal of the ACM* 23.3 (1976), pp. 555–565. DOI: 10.1145/321958.321975.
- [78] Maria Schuld et al. ‘Evaluating Analytic Gradients on Quantum Hardware’. In: *Physical Review A* 99.3 (2019). Preprint at <https://arxiv.org/abs/1811.11184>, p. 032331. DOI: 10.1103/PhysRevA.99.032331.
- [79] Vivek V. Shende, Stephen S. Bullock and Igor L. Markov. ‘Synthesis of Quantum-Logic Circuits’. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25.6 (2006). Preprint at <https://arxiv.org/abs/quant-ph/0406176>, pp. 1000–1010. DOI: 10.1109/TCAD.2005.855930.

- [80] Anders Sørensen and Klaus Mølmer. ‘Quantum Computation with Ions in Thermal Motion’. In: *Physical Review Letters* 82.9 (1999), pp. 1971–1974. DOI: 10.1103/PhysRevLett.82.1971.
- [81] James C. Spall. ‘Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation’. In: *IEEE Transactions on Automatic Control* 37.3 (1992), pp. 332–341. DOI: 10.1109/9.119632.
- [82] Mario Szegedy. ‘Quantum Speed-Up of Markov Chain Based Algorithms’. In: *Proceedings of the 45th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2004)*. 2004, pp. 32–41. DOI: 10.1109/FOCS.2004.53.
- [83] Luca Trevisan et al. ‘Gadgets, Approximation, and Linear Programming’. In: *SIAM Journal on Computing* 29.6 (2000), pp. 2074–2097. DOI: 10.1137/S0097539797328847.
- [84] H. F. Trotter. ‘On the Product of Semi-Groups of Operators’. In: *Proceedings of the American Mathematical Society* 10.4 (1959), pp. 545–551. DOI: 10.1090/S0002-9939-1959-0108732-6.
- [85] Vijay V. Vazirani. *Approximation Algorithms*. Berlin: Springer, 2001.
- [86] Zhihui Wang et al. ‘<mml:math xmlns:mml=“http://www.w3.org/1998/Math/MathML”><mml:math>mixers: Analytical and numerical results for the quantum alternating operator ansatz’. In: *Physical Review A* 101.1 (Jan. 2020). ISSN: 2469-9934. DOI: 10.1103/physreva.101.012320. URL: <http://dx.doi.org/10.1103/PhysRevA.101.012320>.
- [87] David Wierichs et al. ‘Recursive Cartan Decompositions for Unitary Synthesis’. In: *arXiv preprint* (2025). <https://arxiv.org/abs/2503.19014>. arXiv: 2503.19014 [quant-ph].
- [88] Sören Wilkening et al. ‘A Quantum Algorithm for Solving 0-1 Knapsack Problems’. In: *npj Quantum Information* 11 (2025). Preprint at <https://arxiv.org/abs/2310.06623>, p. 146. DOI: 10.1038/s41534-025-01097-8.
- [89] David P. Williamson and David B. Shmoys. *The Design of Approximation Algorithms*. Cambridge: Cambridge University Press, 2011.
- [90] Yordan S. Yordanov, David R. M. Arvidsson-Shukur and Crispin H. W. Barnes. ‘Efficient Quantum Circuits for Quantum Computational Chemistry’. In: *Physical Review A* 102.6 (2020). Preprint at <https://arxiv.org/abs/2005.14475>, p. 062612. DOI: 10.1103/PhysRevA.102.062612.